GPUNTOBIT




# Il libro di *Triad*

*Imparare il linguaggio e il codice del progetto triad2.0, dai primi programmi al seed in codice macchina*

I Fondamenti · II Triad-Core · III Triad-Full · IV Dentro il progetto · V Sul campo

```
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
```

Prima edizione · Standard del linguaggio 0.2.0 · 2026

github.com/GPuntobit/triad2.0

# Prefazione

Triad è un linguaggio di programmazione autonomo: sintassi a blocchi e tipi dichiarati, multiple dispatch, programmazione logica integrata, codice come dato. La sua definizione è una specifica normativa indipendente dall'implementazione, accompagnata da una specifica eseguibile e da una suite di conformità con golden verificabili. Questo libro ne è il percorso di studio.

Il libro nasce da un'esigenza semplice: imparare Triad non significa solo leggere la specifica, ma costruire un modello mentale del linguaggio, dei suoi costrutti e del suo runtime. La specifica è normativa e precisa, ma è scritta per essere implementata, non per essere letta come un romanzo. Questo volume la traduce in un percorso: spiegazioni, esempi, esercizi, marcatori di profilo. Le spiegazioni, gli esempi e gli esercizi costruiscono passo dopo passo il modello mentale del linguaggio. La specifica resta la fonte normativa di riferimento, non un prerequisito di lettura.

Questo libro non presuppone che tu sappia già programmare. Chi ha mai scritto un programma in un altro linguaggio — e sa cosa sono una variabile, una funzione, un ciclo, una ricorsione — può saltare le *Fondamenta* e cominciare dal Manuale o dal Tutorial. Chi invece non ha mai programmato trova qui tutto il necessario: la parte *Fondamenta: programmare da zero con Triad* insegna la programmazione da capo, usando il linguaggio stesso, e costruisce uno alla volta i concetti che il resto del libro dà per noti. Le influenze del linguaggio (Julia, Prolog, Lisp) sono documentate nell'Appendice A della specifica: sono note informative, non definizioni. Triad non è un dialetto di nessuno dei tre.

Il volume riunisce tre testi complementari e indipendenti. Le *Fondamenta* introducono alla programmazione per chi parte da zero. Il *Manuale di Triad 2.0* spiega il linguaggio, propone esercizi e chiude con grammatica, schede rapide, tabella degli errori, glossario e appendici. Il *Tutorial di Triad 2.0* è diviso in cinque parti: primi passi, Triad-Core, Triad-Full, codice del progetto e riferimento. Si può leggere ciascun testo per intero senza gli altri: ognuno spiega ciò che gli serve. L'indice che segue raccoglie i capitoli di tutti e tre i testi, così da ritrovarli in un unico posto.

Se non hai mai programmato, comincia dalle Fondamenta e prosegui con il Tutorial, parti I e II. Se vuoi scrivere programmi eseguibili oggi e sai già programmare, comincia direttamente dal Tutorial, parti I e II; usa il Manuale per approfondire i concetti e fare gli esercizi. Per studiare lo standard completo, prosegui con la parte III del Tutorial e i capitoli corrispondenti del Manuale. La parte IV è pensata per chi vuole esplorare o modificare il repository.

Le appendici del Manuale orientano tra le influenze del linguaggio e le fonti del progetto. I testi normativi integrali non sono riprodotti nel volume: chi vuole il dettaglio formale li trova nei file `SPEC.md` e `SPEC-CORE.md` del repository, che restano le fonti definitive in caso di divergenza. Il volume contiene comunque le regole necessarie a capire e usare il linguaggio: il rinvio alla specifica è un approfondimento, non una dipendenza.

Due profili di conformità attraversano tutto il libro. Triad-Core è il sottoinsieme di bootstrap: valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O. Triad-Full è lo standard completo. Ogni programma Core è anche un programma Full valido e deve comportarsi allo stesso modo. I marcatori **[Core]** e **[Full]** indicano in quale profilo esiste ciascun costrutto; i blocchi di codice sono marcati `triad` per il codice Core e `triad-full` per il codice definito dallo standard Full, non ancora

eseguibile dal runtime corrente. Gli esempi Core si possono provare con l'interprete JavaScript di riferimento (`node seed/dev/triad_core.cjs`); i frammenti indicati come specifici del seed fanno eccezione.

Il seed S0 è un interprete Triad-Core scritto direttamente in codice macchina x86-64, senza un linguaggio ospite; oggi il seed eseguibile è quello per Windows. Non va confuso con l'interprete JavaScript usato per eseguire gli esempi Core e verificare la conformance. Il piano di bootstrap prevede S1 (un interprete Triad scritto in Triad-Core), S2 (compilatore nativo), S3 (self-hosting) e S4 (parità cross-platform). Dove il runtime non copre un costrutto, il testo lo dice esplicitamente.

Buona lettura, e buon codice.

— GPuntobit

# Indice

## Fondamenta: programmare da zero


1. Che cos'è un programma
    Esercizi
2. Il terminale e il primo programma
    2.1 Il terminale in tre comandi
    2.2 Il primo programma, passo per passo
    2.3 Il ciclo di lavoro
    Esercizi
3. Valori: i dati con cui si calcola
    Esercizi
4. Variabili: dare un nome a un valore
    Esercizi
5. Espressioni: combinare i valori
    Esercizi
6. Stringhe: lavorare col testo
    Esercizi
7. Decisioni: if, elseif, else
    Esercizi
8. Ripetizioni: while
    8.1 Esercizi per sottogenere
    8.2 Soluzioni
9. Liste: tenere insieme molti valori
    Esercizi
10. Funzioni: dare un nome a un'operazione
    Esercizi
11. Ricorsione: una funzione che chiama sé stessa

Esercizi

12. Errori e debug: cosa fare quando il programma non funziona

12.1 Le due famiglie di errori

12.2 I messaggi più comuni, tradotti in italiano

12.3 Il metodo in quattro passi

12.4 Gli errori dei principianti, per nome

Esercizi

13. Pensare da programmatore: scomporre un problema

13.1 Capire il problema, con esempi in mano

13.2 Scomporre in passi, in italiano

13.3 Prima la versione semplice, poi aggiungi

13.4 Scrivere poco, verificare spesso

13.5 Quando sei bloccato

Esercizi

14. Esercizi

Soluzioni

15. Glossario dei termini informatici

Esercizi

16. Una domanda che capita spesso: cosa sono «success» e «fail»?

15.1 Il riferimento: la verifica automatica di un task

15.2 I tentativi: pass@1 e pass@k

15.3 Le altre due facce di «successo e fallimento» in Triad

15.4 La domanda dietro la domanda

Esercizi

17. Domande da neofita

Esercizi

18. Come proseguire

Esercizi

Appendice A — Installazione guidata passo passo

A.1 Che cosa ti serve

A.2 Passo 1 — Installare Node.js

A.3 Passo 2 — Ottenere il repository del progetto

A.4 Passo 3 — Il terminale nella cartella giusta

A.5 Passo 4 — Il programma di prova

A.6 Se qualcosa non funziona

A.7 Windows, macOS e Linux: le differenze

Appendice B — Scheda rapida della sintassi (Triad-Core)

B.1 Il minimo indispensabile

B.2 Valori e tipi

B.3 Operatori

B.4 Stampa e stringhe

B.5 Decisioni

B.6 Cicli

B.7 Liste

B.8 Funzioni

B.9 I messaggi d'errore, in pillole

B.10 Che cosa NON c'è in Triad-Core

## Manuale di Triad 2.0

Come leggere questo manuale

A chi è indirizzato

Due profili, una sola lingua

Convenzioni di notazione

Come eseguire gli esempi

**Parte I — Il linguaggio**

1. Lessico

1.1 Identificatori

1.2 Numeri

1.3 Stringhe

1.4 Simboli

1.5 Commenti

1.6 Separatori e fine-riga

1.7 La regola dei due punti e del punto interrogativo

1.8 Parole chiave
1.9 Identificatori avanzati e convenzioni
1.10 Escape e stringhe avanzate
1.11 Esercizi sul lessico
2. Valori e tipi
2.1 I tipi
2.2 Gerarchia dei tipi
2.3 Struct
2.4 Conversioni
2.5 Asserzione di tipo
2.6 Il vincolo del dichiarato
2.7 Le condizioni accettano solo Bool
2.8 Come stampa un valore
Esercizi
3. Variabili e assegnamento
Esercizi
4. Operatori e precedenza
4.1 Aritmetica intera
4.2 Logica e confronti
Esercizi
5. Controllo di flusso
5.1 if / elseif / else [Core]
5.2 while, break, continue [Core]
5.3 let e begin [Core]
5.4 for e i range [Full]
5.5 Le comprehension [Full]
5.6 try / catch [Full]
Esercizi
6. Funzioni
6.1 Tre forme di definizione [Core]
6.2 return, ricorsione, arità
6.3 Funzioni di ordine superiore

6.4 L'annotazione di effetto !io

Esercizi

7. Tipi e multiple dispatch

7.1 Il tipo di ritorno

7.2 Funzioni locali

Esercizi

8. Lambda, chiusure e scope

8.1 Lambda [Core]

8.2 Chiusure

8.3 Le regole di scope

Esercizi

9. Liste

Esercizi

10. Stringhe, simboli e conversioni

Esercizi

11. Errori come dati

11.1 La forma Err

11.2 Produrre e propagare

11.3 Le categorie di errore

11.4 In Core: errori come liste taggate

Esercizi

12. Contratti

Esercizi

13. Budget

Esercizi

14. Tracce

Glossario degli eventi di traccia (porte di Byrd)

Esercizi

15. Effetti dichiarati

Esercizi

16. Logica

16.1 Fatti, regole, termini

16.2 I goal

16.3 solve è un valore

16.4 Un predicato usato come funzione

16.5 Esempi

Esercizi

17. Codice come dato

17.1 Quote, unquote, eval

Esercizi

18. Macro

18.1 Codice che genera codice

Esercizi

19. Builtin

Esercizi

20. Riga di comando e REPL

20.1 Il REPL

Esercizi

21. Primitivi di bootstrap

Esercizi

22. Stile e pattern idiomatici

22.1 Funzioni di ordine superiore scritte in Core

22.2 Record come liste taggate

22.3 Ambiente come lista di associazioni

22.4 Stato mutabile condiviso

22.5 Costruire stringhe

22.6 Regole di stile

Esercizi

**Parte II — Esercizi**

23. Esercizi guidati (Triad-Core)

Esercizio 1 — Potenza e somme

Esercizio 2 — Numeri primi

Esercizio 3 — Inverti una lista

Esercizio 4 — Ordinamento per inserzione
Esercizio 5 — Contatore di parole
Esercizio 6 — Un mini-analizzatore lessicale
Esercizio 7 — Valutare un'espressione
Esercizio 8 — Un ambiente a dizionario
Esercizi supplementari
24. Esercizi su Triad-Full
Esercizio 9 — Dispatch su struct
Esercizio 10 — Una base di conoscenza
Esercizio 11 — Contratti in pratica
Esercizio 12 — Una macro che misura
Esercizi supplementari

**Parte III — Riferimento**

25. Grammatica
25.1 Triad-Core (EBNF normativa)
25.2 Triad-Full (EBNF normativa)
Esercizi
26. Schede rapide
26.1 Valori
26.2 Controllo di flusso
26.3 Funzioni
Esercizi
27. Tabella degli errori
27.1 In Core
27.2 In Triad-Full
Esercizi
28. Glossario
Esercizi
29. Cosa gira oggi
Esercizi

**Appendice A — Influenze dichiarate**

**Appendice B — Dove guardare**

# Tutorial di Triad 2.0

Che cosa impari e come è organizzato

Come usare questo tutorial

**Parte I — Primi passi**

1. Cos'è Triad
   - 1.1 Due profili: Core e Full
   - 1.2 La catena di bootstrap
   - Esercizi
2. Installare ed eseguire
   - 2.1 Requisiti
   - 2.2 Eseguire un file
   - 2.3 Materializzare il seed
   - 2.4 Launcher bin/triad e bin/triad.cmd
   - 2.5 Codici di uscita
   - 2.6 La CLI di sviluppo in breve
   - Esercizi
3. Il primo programma
   - 3.1 Argomenti da riga di comando
   - 3.2 Il ciclo di lavoro consigliato
   - Esercizi

**Parte II — Triad-Core passo per passo (eseguibile oggi)**

4. Lessico
   - Esercizi
5. Valori, tipi e stampa
   - 5.1 Regole di stampa di print/println
   - 5.2 Uguaglianza
   - Esercizi
6. Variabili e assegnamento

Esercizi

7. Operatori ed espressioni

7.1 Precedenza (dalla più bassa alla più alta)

7.2 Aritmetica intera

7.3 Logica e condizioni

Esercizi

8. Controllo di flusso

8.1 if / elseif / else

8.2 while, break, continue

8.3 let e begin

Esercizi

9. Funzioni

9.1 Tre modi di definire una funzione

9.2 return, ricorsione, arità

9.3 Annotazione di effetto !io

Esercizi

10. Lambda, chiusure e scope

10.1 Lambda

10.2 Chiusure

10.3 Regole di scope

Esercizi

11. Liste

Esercizi

12. Stringhe, simboli e conversioni

Esercizi

13. Primitivi di bootstrap

Esercizi

14. Errori di esecuzione

14.1 Errori come valori, in Core

Esercizi

15. Interprete JS e seed Windows: differenze

Esercizi

16. Pattern idiomatici Triad-Core

16.1 Funzioni di ordine superiore scritte in Core

16.2 Record come liste taggate

16.3 Ambiente come lista di associazioni

16.4 Stato mutabile condiviso

16.5 Costruire stringhe

16.6 Stile

Esercizi

17. Esercizi guidati

Esercizio 1 — Potenza e somme

Esercizio 2 — Numeri primi

Esercizio 3 — Inverti una lista

Esercizio 4 — Ordinamento per inserzione

Esercizio 5 — Contatore di parole

Esercizio 6 — Un mini-tokenizer

Esercizio 7 — Valutare un'espressione

Esercizi supplementari

**Parte III — Triad-Full (lo standard completo)**

18. Lessico e tipi Full (SPEC §1, §4)

Esercizi

19. Funzioni tipate e multiple dispatch (SPEC §5)

Esercizi

20. Struct (SPEC §4)

Esercizi

21. Controllo di flusso Full: for, range, comprehension, try (SPEC §2)

Esercizi

22. Logica: fatti, regole, solve (SPEC §6)

Esercizi

23. Destrutturazione (SPEC §2)

Esercizi

24. Errori come dati (SPEC §8)
    Esercizi
25. Contratti (SPEC §9)
    Esercizi
26. Budget (SPEC §10)
    Esercizi
27. Tracce (SPEC §11)
    Esercizi
28. Effetti dichiarati (SPEC §12)
    Esercizi
29. Codice come dato, albero uniforme e macro (SPEC §3, §7)
    29.1 L'albero uniforme
    29.2 Quote, unquote, eval
    29.3 Macro
    29.4 Codice che genera codice
    Esercizi
30. Builtin Full (SPEC §13)
    Esercizi
31. Riga di comando e REPL (SPEC §14)
    Esercizi
32. Esempi Full con output golden
    32.1 assaggio.td
    32.2 famiglia.td
    32.3 garanzie.td
    32.4 macro.td
    32.5 Contratti e budget insieme
    Esercizi

**Parte IV — Il codice del progetto**

33. Mappa del repository
    Esercizi
34. L'interprete di riferimento JS (seed/dev/triad_core.cjs)

34.1 Architettura
34.2 Esempio: aggiungere un builtin
34.3 Esempio: aggiungere sintassi
Esercizi
35. Il seed S0 Windows e il suo generatore
35.1 Cos'è
35.2 Convenzioni interne
35.3 Il generatore seed/dev/gen_seed.js
35.4 Flusso per modificare il seed
Esercizi
36. Il seed Linux (E0)
Esercizi
37. Self-hosting: S1 in selfhost/
37.1 Come si scrive codice S1 oggi
37.2 Dove va S1
Esercizi
38. Conformance: leggere, scrivere e verificare casi
38.1 Formato
38.2 Il runner conformance/run.mjs
38.3 Aggiungere un caso Core
Esercizi
39. La CLI di sviluppo triad-cli
39.1 Uso
39.2 Struttura del codice
39.3 Aggiungere un comando
39.4 Eseguibile standalone
Esercizi
40. Benchmark AI (ai/)
Esercizi
41. Editor, launcher e sito
Esercizi
42. Integrazione continua

Esercizi

43. Flusso di contributo

Esercizi

**Parte V — Riferimento**

44. Grammatica Triad-Core (EBNF)

Esercizi

45. Schede rapide Core e Full

Esercizi

46. Debug e problemi comuni

Esercizi

47. Glossario

Esercizi

48. Stato, limiti noti e fix storici

Esercizi

49. FAQ

Esercizi

# Fondamenta: programmare da zero con Triad

> *Questa parte del libro non presuppone nulla: è pensata per chi non ha mai scritto un programma in vita sua. Spiega cosa fa un computer quando esegue un programma, come si scrive e si esegue il primo programma Triad, e costruisce uno alla volta i concetti — valori, variabili, espressioni, decisioni, ripetizioni, liste, funzioni, ricorsione, errori — che tutto il resto del libro dà per noti. Chi non ha mai installato uno strumento di sviluppo parte dall'Appendice A (installazione guidata); l'Appendice B raccoglie la scheda rapida della sintassi. Se sai già programmare, puoi saltare tutto e cominciare dal Manuale o dal Tutorial; se non sai da dove cominciare, comincia da qui.*

Questo testo nasce da una constatazione: il Manuale e il Tutorial spiegano **Triad**, ma danno per scontato che il lettore sappia già cosa sono una variabile, un ciclo o una funzione. Chi non ha mai programmato non ha questi concetti, e senza di essi nessun manuale di nessun linguaggio è leggibile. Le *Fondamenta* colmano la distanza: insegnano la programmazione **usando Triad**, così impari due cose insieme — a pensare come un programmatore e a scrivere i primi programmi in questo linguaggio.

Non serve nient'altro che un computer e la voglia di provare. Ogni esempio di questa parte è scritto in **Triad-Core**, il sottoinsieme del linguaggio che oggi è eseguibile: puoi digitarlo e vederlo funzionare. Alla fine della parte trovi esercizi con soluzioni, un glossario dei termini tecnici usati e una mappa per proseguire nel resto del libro.

## 1. Che cos'è un programma

Un computer è una macchina che esegue **istruzioni**. Non capisce né ragiona: legge un'istruzione, la esegue, passa alla successiva, con una velocità enorme ma con un'intelligenza nulla. Tutto ciò che un computer fa — mostrare una pagina web, salvare una fotografia, far muovere un robot — è il risultato di migliaia o milioni di istruzioni semplici eseguite una dopo l'altra.

Un **programma** è il testo di quelle istruzioni, scritto in una forma che il computer può leggere. Il testo di un programma si chiama anche **codice sorgente**, o semplicemente codice. Chi scrive programmi si chiama programmatore; scrivere un programma si chiama programmare.

Le istruzioni non si scrivono in italiano, perché l'italiano è ambiguo e pieno di eccezioni, e una macchina che deve eseguire letteralmente non può permettersi ambiguità. Si scrivono in un **linguaggio di programmazione**: una lingua con regole precise e pochissime eccezioni, fatta apposta per descrivere calcoli e azioni. Triad è uno di questi linguaggi. Come ogni linguaggio di programmazione, ha una grammatica (come si compongono le frasi), un vocabolario (le parole ammesse) e una semantica (che cosa significa ogni frase). La differenza rispetto alle lingue parlate è che le regole sono poche, assolute e vanno imparate una volta per tutte: se un programma non rispetta le regole, il computer non «ci prova lo stesso», si ferma e dice che cosa non ha capito.

Il programma scritto in un linguaggio di programmazione deve prima o poi essere tradotto in qualcosa che il computer esegue davvero. Ci sono due modi principali per farlo:

- un **compilatore** traduce tutto il programma in anticipo, in una forma che il computer esegue direttamente;

- un **interprete** legge ed esegue il programma istruzione per istruzione, al momento.

Triad si esegue oggi con un interprete: il programma resta testo leggibile, e l'interprete lo esegue mentre lo legge. Questo rende il ciclo di lavoro immediato: scrivi, esegui, vedi il risultato, correggi.

Vediamo subito un programma vero. Questo è un intero programma Triad:

```
println("ciao, mondo")
```

Output:

```
ciao, mondo
```

Il programma contiene una sola istruzione: `println("ciao, mondo")`. Quando l'interprete la esegue, il computer mostra la scritta `ciao, mondo`. La parola `println` significa «print line», stampa riga: mostra qualcosa a schermo e poi va a capo. Il testo tra le virgolette è una **stringa**, cioè una sequenza di caratteri che il programma tratta come testo.

Nota che il programma non *descrive* il risultato: lo *produce*. Non c'è un motore che «capisce» che vuoi dire ciao; c'è un'istruzione che, quando viene eseguita, fa apparire dei caratteri sullo schermo. Se scrivi `printn` invece di `println`, il programma non funziona: non perché il computer si offenda, ma perché `printn` non è un'istruzione del linguaggio, così come «acqua» non è un verbo all'infinito.

Tre parole chiave restino da questo capitolo, perché le useremo spesso: **programma** (il testo delle istruzioni), **interprete** (il programma che esegue il tuo programma), **stringa** (un testo trattato come dato). Il resto viene da sé.

## Esercizi

1. **Il programma più piccolo.** Scrivi un file `minimo.triad` con una sola istruzione che stampi `ok`, ed eseguilo. Poi togli una virgoletta e riesegui: che cosa ti dice il terminale?
2. **Due istruzioni, in ordine.** Scrivi un programma che stampi `prima` e poi `dopo`, su due righe. Scambia le due righe e riesegui: che cosa cambia nell'output?
3. **Tre righe.** Scrivi un programma che stampi su tre righe `Triad`, `è`, `un linguaggio`.
4. **Quante istruzioni?** Scrivi un programma con tre `println`, poi rileggilo e conta le istruzioni: quante sono davvero? Che cosa fa ciascuna?
5. **Refuso.** Scrivi `printn("ciao")` ed esegui: il programma funziona? Perché il computer si ferma invece di «capire» che intendevi `println`?
6. **Output previsto.** Senza eseguire, scrivi su un foglio l'output di un programma che stampi `uno`, `due`, `tre`; poi esegui e confronta: avevi previsto giusto?
7. **Stringa o istruzione?** Nel programma `println("ciao, mondo")` che cosa è istruzione e che cosa è dato? Prova a spiegarlo a voce alta.
8. **Interprete in azione.** Esegui un programma due volte di fila: l'output cambia? Che cosa succede al file del programma mentre lo esegui?
9. **Il tuo primo programma originale.** Scrivi un programma che presenti te stesso: nome, città e una cosa che ti piace, ciascuna su una riga.
10. **Ripasso a parole.** Rispondi senza guardare il libro: che differenza c'è tra programma e interprete? Tra compilatore e interprete? Che cos'è una stringa?

## 2. Il terminale e il primo programma

Per scrivere ed eseguire programmi non serve un ambiente speciale: servono tre cose. Un **editor di testo** per scrivere il programma (il Blocco note di Windows va benissimo; meglio ancora un editor come Visual Studio Code). Il **terminale**, ovvero la finestra in cui dai comandi al computer digitandoli (su Windows si chiama «Prompt dei comandi» o «PowerShell», su Linux e macOS «Terminale»). E un **runtime** per Triad, cioè il programma che esegue i tuoi programmi: in questo libro useremo l'interprete JavaScript di riferimento, che richiede Node.js (versione 18 o superiore) e il repository del progetto. Se non hai mai installato uno strumento di sviluppo, o se un passaggio non ti è chiaro, l'**Appendice A** riprende tutto da zero, con un controllo da fare dopo ogni tappa e la tabella dei guasti comuni.

### 2.1 Il terminale in tre comandi

Il terminale è un modo per parlare al computer scrivendo comandi invece di cliccare. Quando lo apri, vedi una scritta (il **prompt**) che ti dice in quale **cartella** stai lavorando: una cartella è una delle directory in cui sono organizzati i file del computer, la stessa che vedi nella finestra «Esplora file». I comandi che servono per questo libro sono pochissimi:

| comando | che cosa fa |
|---|---|
| `cd percorso` | entra nella cartella indicata (*change directory*) |
| `cd ..` | torna alla cartella superiore |
| `ls` (Linux/macOS) · `dir` (Windows) | elenca i file della cartella corrente |

| comando | che cosa fa |
|---|---|
| `node file.triad` | esegue un programma Triad con l'interprete JavaScript |

Il **percorso** è il «nome lungo» di una cartella, per esempio `C:\Users\mario\triad2.0` su Windows o `/home/mario/triad2.0` su Linux. Il repository del progetto si ottiene con un solo comando, da digitare nella cartella in cui vuoi tenerlo:

```
git clone https://github.com/GPuntobit/triad2.0.git
```

Poi si entra nella cartella creata (`cd triad2.0`) e il gioco è fatto: tutti gli esempi di questo libro si eseguono da lì. Su Windows il comando per eseguire un programma è `node seed\dev\triad_core.cjs file.triad`; su Linux e macOS `node seed/dev/triad_core.cjs file.triad`. Nel resto del libro mostreremo la forma Linux/macOS: su Windows cambia solo la forma delle barre nei percorsi.

## 2.2 Il primo programma, passo per passo

Apri l'editor, scrivi queste tre righe e salva il file con il nome `ciao.triad` nella cartella del repository:

```
# ciao.triad — il mio primo programma
nome = "Triad"
println("ciao, " * nome)
```

Da terminale, esegui il comando:

```
node seed/dev/triad_core.cjs ciao.triad
```

Output:

```
ciao, Triad
```

Se vedi `ciao, Triad`, hai scritto, salvato ed eseguito il tuo primo programma: congratulazioni, sei un programmatore. Se invece vedi un messaggio che comincia con `error:` o `parse error:`, non è successo niente di male: il capitolo 12 spiega come leggere quei messaggi, che sono il modo del computer di dire «non ho capito questa parte».

Analizziamo le tre righe, perché contengono già quasi tutti i concetti dei prossimi capitoli.

- La prima riga, `# ciao.triad — il mio primo programma`, è un **commento**: inizia con `#` e non è un'istruzione. L'interprete la ignora completamente. I commenti servono a spiegare il programma a chi lo legge (tra un mese, quel chi sarai tu).
- La seconda riga, `nome = "Triad"`, crea una **variabile**: un nome a cui è associato un valore, qui il testo `Triad`. Da questo punto in poi, ogni volta che il programma usa `nome`, intende quel valore.
- La terza riga, `println("ciao, " * nome)`, è una chiamata: chiede a `println` di mostrare il risultato di `"ciao, " * nome`. Il simbolo `*` tra due stringhe significa **concatenarle**, cioè incollarle una dopo l'altra (in Triad `+` serve solo per i numeri). Il risultato è `ciao, Triad`, che viene mostrato a schermo.

## 2.3 Il ciclo di lavoro

Scrivere programmi è un ciclo di quattro passi, sempre gli stessi: **scrivi** il programma, **eseguilo**, **guarda il risultato**, **correggi**. Non esiste programmatore che scriva un programma giusto al primo colpo: la differenza tra principiante ed esperto è che l'esperto si aspetta gli errori e

ha un metodo per trovarli. Alcune abitudini che fin da subito fanno risparmiare tempo: cambia una cosa alla volta e riesegui; scrivi programmi piccoli e fai crescere un pezzo alla volta; quando qualcosa non funziona, non indovinare — rileggi.

> ***Attenzione.*** *Il nome del file deve finire con* `.triad` *(o con l'alias storico* `.td`*), e va salvato come* ***testo semplice****. Se l'editor salva con nomi come* `ciao.triad.txt`*, l'interprete non troverà il programma. Controlla sempre il nome completo del file.*

## Esercizi

1. **Nel posto giusto.** Apri il terminale, entra nella cartella `triad2.0` e verifica con `ls` / `dir` di vederle cartelle `seed/`, `conformance/`, `docs/`.
2. **Il primo programma.** Scrivi `ciao.triad` come nel capitolo ed eseguilo con `node seed/dev/triad_core.cjs ciao.triad`: vedi `ciao, Triad` ?
3. **Un programma tuo.** Crea `mio.triad` che stampi `ciao, <tuo nome>` e fallo girare.
4. **Il nome del file.** Salva volutamente il programma come `ciao.triad.txt` ed eseguilo: che cosa succede? Correggi il nome e riesegui.
5. **Un errore di battitura.** Cambia `println` in `printn`, esegui e leggi il messaggio: a quale delle due famiglie di errori del capitolo 12 appartiene?
6. **Il commento.** Aggiungi tre commenti `# ...` al tuo programma: che cosa cambia nell'output? E nel testo del file?
7. **Concatenazione.** Modifica `ciao.triad` in modo che stampi `ciao, Triad!` usando `*` e la stringa `"!"`.

8. **Ciclo di lavoro.** Trasforma il saluto in due righe separate, **una modifica alla volta**, rieseguendo dopo ogni modifica: perché conviene lavorare così?
9. **Cartella sbagliata.** Esegui il programma da un'altra cartella usando il percorso completo del file: funziona? E se usi solo il nome del file?
10. **Secondo controllo.** Esegui `node seed/dev/triad_core.cjs conformance/core/cases/aritmetica/program.td` e confronta l'output con la colonna di numeri attesa: comincia con `14`, `20`, `1`, `1024`?

---

## 3. Valori: i dati con cui si calcola

Un programma manipola **valori**: numeri, testi, risposte vero/falso, elenchi di cose. Ogni valore ha un **tipo**, che dice che cosa quel valore è e che operazioni si possono farci. In Triad i tipi di base che incontri all'inizio sono questi:

| tipo | che cos'è | esempi |
|---|---|---|
| `Int` | un numero intero, con segno | `42`, `0`, `-7`, `1_000_000` |
| `String` | un testo, tra virgolette doppie | `"ciao"`, `"42"`, `""` |
| `Bool` | una verità: vero o falso | `true`, `false` |
| `Symbol` | un nome o un'etichetta, con i due punti davanti | `:ok`, `:alice` |
| `List` | un elenco ordinato di valori, tra parentesi quadre | `[1, 2, 3]`, `[]` |

| tipo | che cos'è | esempi |
|---|---|---|
| `Nothing` | «nessun valore» | `nothing` |

Osserva subito una distinzione importante: `42` è un numero, `"42"` è un testo. Sono due cose diverse, anche se si vedono uguali a schermo: `42 + 1` fa `43`, mentre `"42" + 1` è un errore, perché non si può aggiungere un numero a un testo. Le virgolette dicono all'interprete: «questo è testo, trattalo come tale».

I numeri interi di Triad-Core sono a 64 bit, cioè hanno un limite enorme (circa 9 miliardi di miliardi) ma un limite: se lo superi, il contatore «gira» e riparte dal numero negativo più piccolo. Nei primi programmi non ci penserai, ma è bene saperlo. In Triad-Core non esistono i numeri decimali (quelli con la virgola): fanno parte di Triad-Full, la versione completa del linguaggio descritta più avanti nel libro.

Per sapere di che tipo è un valore si usa `typeof`, che risponde con il nome del tipo:

```
println(42)
println("ciao")
println(true)
println(nothing)
println(:alice)
println([1, 2, 3])
println(typeof(42), " ", typeof("ciao"), " ",
typeof(true))
```

Output:

```
42
ciao
true
nothing
alice
[1, 2, 3]
Int String Bool
```

Due dettagli di questo esempio meritano attenzione, perché sono il primo assaggio del fatto che un linguaggio di programmazione è preciso fino in fondo. Primo: `println(:alice)` mostra `alice` senza i due punti, ma il valore resta un simbolo — i due puntini servono a scriverlo nel codice, non fanno parte del nome. Secondo: `println` accetta **più valori separati da virgola** e li stampa uno accanto all'altro senza spazi; `println()` senza argomenti serve solo per andare a capo.

## Esercizi

1. **Riconoscere i tipi.** Scrivi un programma che stampi `typeof` per `42`, `"42"`, `true`, `:ok`, `nothing` e `[1, 2]`.
2. **Numero o testo?** Esegui `println(42 + 1)` e poi `println("42" + 1)`: perché uno funziona e l'altro no? Leggi il messaggio d'errore.
3. **Stessa faccia, tipo diverso.** Stampa `42` e `"42"`: l'output è uguale? In che cosa i due valori sono diversi per il programma?
4. **Verità.** Stampa il risultato di `2 == 2` e di `2 != 2`: di che tipo sono quei valori?
5. **Etichette.** Stampa i simboli `:ok` e `:ciao`: compaiono con i due punti? Di che tipo sono secondo `typeof`?

6. **Liste.** Crea una lista con tre valori di tipi diversi e stampala: Triad lo permette?
7. **Nessun valore.** Che cosa stampa `println(nothing)`? E `println(typeof(nothing))`?
8. **Vuoti, ma valori.** Che tipo hanno `""` e `[]`? Sono «assenza di valore» o valori veri e propri? Verifica con `typeof`.
9. **Prevedi e verifica.** Senza eseguire, scrivi l'output di `println(1_000)`, `println(-7)`, `println(typeof(1_000))`; poi controlla.
10. **Classifica.** Assegna a ogni esempio il tipo giusto: `42`, `:alice`, `[]`, `false`, `"42"`, `nothing`, `[1, "due"]`, `true`.

---

## 4. Variabili: dare un nome a un valore

Una **variabile** è un nome a cui è associato un valore. Il nome serve a ritrovare il valore più avanti nel programma; l'associazione si crea con `=`, che si chiama **assegnamento**:

```
eta = 42
```

Da qui in poi `eta` significa `42`. L'assegnamento non è una formula matematica: `eta = 42` non «dice che eta è uguale a 42», **fa sì** che lo diventi. Per questo una riga come questa ha senso:

```
eta = 42
println(eta)
eta = eta + 1
println(eta)
eta += 5
```

```
println(eta)
nome = "Ada"
messaggio = "ciao " * nome
println(messaggio)
```

Output:

```
42
43
48
ciao Ada
```

Riga per riga: `eta = 42` crea il legame; la prima `println(eta)` mostra `42`; `eta = eta + 1` legge il valore corrente (`42`), calcola `43` e **sostituisce** il valore di `eta`; dopo `eta += 5` il valore è `48`. La forma `+=` è una scorciatoia per `eta = eta + 5`: esistono anche `-=` e `*=`.

Alcune regole e alcune buone abitudini:

- Un nome può contenere lettere, cifre e il carattere `_`, e non può cominciare con una cifra. Vanno bene `eta`, `eta_utente`, `x2`, `somma_totale`; non vanno bene `2eta`, `eta utente`.
- I nomi distinguono maiuscole e minuscole: `eta` e `Eta` sono due variabili diverse. Scegli un criterio e usalo sempre (in questo libro i nomi sono minuscoli, con `_` tra le parole).
- Il nome deve dire qualcosa: `n` va bene per un contatore in dieci righe, `numero_di_studenti` è meglio in un programma lungo. Il nome è la prima documentazione del programma.

- Usare una variabile **prima** di assegnarle un valore è un errore: in Triad non esistono variabili «vuote» che valgono zero o niente. Se il programma dice `error: undefined variable`, il nome è sbagliato o l'assegnamento manca.
- Una variabile può cambiare tipo durante il programma (`x = 1` poi `x = "ciao"` è permesso), ma farlo senza un motivo rende il programma difficile da leggere: di solito a una variabile conviene dare un tipo e tenerlo.

Un esercizio mentale utile, che i programmatori fanno di continuo: segui il programma riga per riga tenendo in mano una tabella dei valori correnti. Dopo ogni istruzione, «che valore ha `eta`? E `nome`?» Se sai rispondere sempre, sai già prevedere il comportamento del programma: è la competenza fondamentale del programmatore.

## Esercizi

1. **Assegnare e stampare.** Crea le variabili `nome` e `cognome`, costruisci il saluto con `*` e stampalo.
2. **Tabella dei valori.** Segui riga per riga `eta = 42`, `eta = eta + 1`, `eta += 5` scrivendo su un foglio il valore di `eta` dopo ogni istruzione; poi verifica con `println`.
3. **Nomi validi.** Quali di questi nomi vanno bene? `eta`, `2eta`, `eta_utente`, `eta utente`, `Eta`, `x2`, `nome-cognome`. Prova a usarli davvero.
4. **Variabile indefinita.** Stampa una variabile a cui non hai mai assegnato un valore: leggi il messaggio e spiega in italiano che cosa è successo.
5. **Maiuscole e minuscole.** `eta = 1` e `Eta = 2`: sono la stessa variabile? Verifica stampando entrambe.
6. **Cambiare tipo.** `x = 1` e poi `x = "ciao"`: è permesso? Stampa `typeof(x)` prima e dopo la seconda assegnazione.

7. **La scorciatoia `+=`.** Scrivi `n = 10` e `n += 3`; poi riscrivi la seconda riga senza `+=`. L'output è lo stesso?
8. **Il contadino e le mele.** Un cesto contiene `mele = 7`; ne vendi 3 e ne compri altre 10. Traccia il valore di `mele` a ogni passo e stampa il risultato finale.
9. **Scambio.** Dati `a = 1` e `b = 2`, scambia i loro valori usando una terza variabile, e stampa `a` e `b` alla fine.
10. **Nomi che parlano.** Prendi un programma di esempio con variabili `x`, `y`, `z` e rinominale con nomi che dicano che cosa contengono: il programma è più facile da leggere?

---

## 5. Espressioni: combinare i valori

Un'**espressione** è un calcolo che produce un valore. `42` è un'espressione (produce `42`); `2 + 3` è un'espressione (produce `5`); `"ciao " * nome` è un'espressione (produce un testo). Le espressioni si compongono: dentro `eta + 1` ci sono due espressioni più piccole, `eta` e `1`, e un **operatore**, il `+`, che le combina.

Gli operatori aritmetici di base sono:

| operatore | che cosa fa | esempio | risultato |
|---|---|---|---|
| `+` | addizione | `2 + 3` | `5` |
| `-` | sottrazione | `10 - 4` | `6` |
| `*` | moltiplicazione | `6 * 7` | `42` |
| `%` | resto della divisione intera | `7 % 3` | `1` |
| `^` | potenza | `2 ^ 10` | `1024` |

In Triad-Core non c'è l'operatore `/` della divisione: la divisione decimale appartiene a Triad-Full. Con `%` hai comunque il **resto** della divisione intera, che serve in moltissime situazioni (sapere se un numero è pari, «girare» un contatore su un intervallo, estrarre cifre). `7 % 3` vale `1` perché 7 diviso 3 fa 2 con resto 1; `10 % 5` vale `0` perché la divisione è esatta.

Quando in un'espressione ci sono più operatori, decide la **precedenza**: la moltiplicazione si esegue prima dell'addizione, quindi `2 + 3 * 4` vale `14`, non `20`. Quando hai dubbi, usa le parentesi: `(2 + 3) * 4` vale `20` e non si sbaglia a leggerlo. Le parentesi sono tue amiche: rendono il programma chiaro a chi lo legge, e chi lo legge sarà spesso te stesso.

Oltre ai numeri, ci sono gli operatori di **confronto**, che producono un `Bool` (`true` o `false`), e gli operatori **logici**, che combinano le verità:

| operatore | significato | esempio | risultato |
|---|---|---|---|
| `==` | uguale a | `2 == 2` | `true` |
| `!=` | diverso da | `2 != 2` | `false` |
| `<` `<=` `>` `>=` | minore, minore o uguale, maggiore, maggiore o uguale | `10 > 3` | `true` |
| `&&` | e (vero se entrambi sono veri) | `true && false` | `false` |
| `\|\|` | o (vero se almeno uno è vero) | `true \|\| false` | `true` |
| `!` | non (inverte la verità) | `!false` | `true` |

```
println(2 + 3 * 4)
println((2 + 3) * 4)
println(7 % 3)
println(2 ^ 10)
println(10 > 3, " ", 2 == 2, " ", 2 != 2)
println(true && false, " ", true || false, " ", !false)
eta = 17
println(eta >= 18 ? "maggiorenne" : "minorenne")
```

Output:

```
14
20
1
1024
true true false
false true true
minorenne
```

L'ultima riga introduce una forma comoda, il **condizionale ternario**: `condizione ? a : b` produce `a` se la condizione è vera, `b` se è falsa. Serve quando il programma deve scegliere un valore, non eseguire un blocco di istruzioni: la forma estesa della scelta è l'`if` del capitolo 7.

Un avvertimento su due simboli che confondono tutti i principianti: `=` **assegna** (fa diventare qualcosa), `==` **confronta** (chiede se due cose sono uguali). Scrivere `if x = 0` quando si intendeva `if x == 0` è l'errore più classico della storia della programmazione: in Triad produce un errore del linguaggio invece di un comportamento misterioso, ma l'abitudine di leggere `=` e `==` come due parole diverse va presa subito.

## Esercizi

1. **Precedenza.** Senza eseguire, calcola `2 + 3 * 4`, `2 * 3 + 4`, `(2 + 3) * 4`; poi verifica con un programma.
2. **Il resto.** Che cosa stampano `7 % 3`, `10 % 5`, `12 % 10`? Come si legge `n % 2` a parole?
3. **Potenza.** Calcola a mano `2 ^ 10` e `3 ^ 3`, poi controlla con il programma.
4. **Ternario.** Dato `eta = 20`, stampa `maggiorenne` o `minorenne` usando `condizione ? a : b`.
5. **`=` o `==`.** Scrivi apposta `if x = 0` (dopo aver assegnato `x = 0`): che cosa succede? Perché è un errore e non un comportamento misterioso?
6. **Confronti.** Stampa il valore di `10 > 3`, `2 == 2`, `2 != 2`, `!false`: di che tipo sono?
7. **Logica.** Valuta a mano poi verifica: `true && false`, `true || false`, `!(3 > 5)`, `2 > 1 && 5 > 3`.
8. **Parentesi.** Aggiungendo solo parentesi a `1 + 2 * 3 - 4`, ottieni almeno due risultati diversi da quello senza parentesi? Trovali e verificali.
9. **Intervallo.** Scrivi un'espressione Bool che sia `true` se e solo se `n` è compreso fra 10 e 20 estremi inclusi; provala con `n = 5`, `n = 10`, `n = 15`, `n = 25`.
10. **Tutti e tre.** Scrivi un'espressione che usi insieme `&&`, `||` e `!` e stampi `true`; spiega riga per riga come viene calcolata.

---

## 6. Stringhe: lavorare col testo

Una **stringa** è un testo trattato come dato: si scrive tra virgolette doppie, si può associare a una variabile, passare a una funzione, costruire pezzo per pezzo. Le operazioni di base sono la **concatenazione** con `*`, che unisce due stringhe, e qualche funzione per misurarle e convertirle:

| operazione | che cosa fa | esempio | risultato |
|---|---|---|---|
| `a * b` | incolla le due stringhe | `"ciao " * "Ada"` | `"ciao Ada"` |
| `length(s)` | quanti byte contiene `s` | `length("ciao")` | `4` |
| `string(x)` | trasforma un valore in testo | `string(40 + 2)` | `"42"` |
| `Int(s)` | trasforma un testo in numero intero | `Int("41") + 1` | `42` |

```
nome = "Triad"
saluto = "ciao " * nome
println(saluto)
println("riga1\nriga2")
println("la risposta è " * string(40 + 2))
println(length("ciao"))
println(Int("41") + 1)
```

Output:

```
ciao Triad
riga1
```

```
riga2
la risposta è 42
4
42
```

La seconda riga dell'output mostra gli **escape**: dentro una stringa, `\n` non sono due caratteri ma il comando «vai a capo», e `\t` è una tabulazione. Se vuoi scrivere una virgoletta dentro una stringa, usa `\"`. Il carattere `\` diventa `\\`. Questi sono tutti gli escape di Triad-Core; se ne vedi altri, non fidarti.

**Nota di implementazione.** Gli escape valgono come descritto, ma il seed in codice macchina attuale non li processa: legge i byte grezzi fino alla virgoletta di chiusura, quindi `println("riga1\nriga2")` su quel runtime stampa `riga1\nriga2` su una riga sola. Anche `Int("41")` non è ancora disponibile sul seed. Sull'interprete di riferimento l'output è quello mostrato (`TASK.md`, «Colmare i limiti del seed»).

Tre regole da sapere sulle stringhe:

- Le stringhe sono **immutabili**: non si può cambiare un carattere dentro una stringa esistente. Ogni operazione come la concatenazione ne produce una **nuova**. Non è un limite pratico, è una regola che rende i programmi più prevedibili.
- `length` conta i **byte**, non le lettere: per l'italiano con le accentate la differenza si vede (`length("è")` è `2`, non `1`). Nei primi programmi non importa; è bene solo saperlo.
- `Int("42")` funziona se il testo **è** un numero: `Int("ciao")` è un errore del programma, non un risultato «nullo». Chi converte testo in numero deve prima controllare che il testo sia un numero.

Il motivo per cui esistono `string` e `Int` è presto detto: i numeri si calcolano, i testi si stampano e si leggono. Per passare dall'uno all'altro mondo serve la conversione esplicita — un altro piccolo atto di precisione che il linguaggio ti chiede.

## Esercizi

1. **Concatenazione.** Costruisci e stampa `ciao, Ada!` usando `*` e almeno tre pezzi di testo.
2. **Lunghezza.** Prevedi e verifica `length("ciao")`, `length("è")`, `length("")`.
3. **Da numero a testo.** Stampa la riga `la risposta è 42` usando `string(40 + 2)`.
4. **Da testo a numero.** Esegui `println(Int("41") + 1)`; poi prova `Int("ciao")`: che cosa succede? Perché conviene controllare prima di convertire?
5. **Escape.** Stampa `prima` e `dopo` su due righe usando una sola `println` con `\n`.
6. **Virgolette dentro le virgolette.** Stampa la frase `disse: "arrivo"` (con le virgolette visibili nell'output).
7. **Immutabilità.** Dato `s = "ciao"`, costruisci `s2 = s * "!"` e stampa sia `s` sia `s2`: la stringa originale è cambiata?
8. **Una frase da pezzi.** Dati `nome = "Ada"` e `eta = 36` (intero), costruisci e stampa `Ada ha 36 anni`.
9. **Andata e ritorno.** Parti da `"123"`, convertilo in intero, aggiungi 1, riconvertilo in testo e stampa il risultato atteso: `124`.
10. **Che cosa funziona?** Esegui tutte e tre e spiega i risultati: `println("a" * "b")`, `println("a" + "b")`, `println(length("ab"))`.

## 7. Decisioni: `if`, `elseif`, `else`

Fino a qui ogni istruzione è stata eseguita una volta, in ordine. Il primo salto di qualità è poter dire: **se** è vero questo, fai una cosa, **altrimenti** ne fai un'altra. Si scrive con `if`:

```
punteggio = 15
if punteggio >= 18
    println("promosso")
else
    println("bocciato")
end
```

Output:

```
bocciato
```

La struttura è sempre la stessa: la parola `if`, una **condizione** (un'espressione che vale `true` o `false`), le istruzioni da eseguire se la condizione è vera, poi `else` con le istruzioni del caso contrario, e infine `end` che chiude la costruzione. L'indentazione (gli spazi a inizio riga) non è obbligatoria per il computer, ma lo è per chi legge: rende visibile la struttura.

Quando i casi sono più di due, si aggiungono rami con `elseif`:

```
x = -3
if x > 0
    println("positivo")
elseif x == 0
    println("zero")
else
```

```
    println("negativo")
end
```

Output:

```
negativo
```

L'`if` prova le condizioni una alla volta, dall'alto in basso, ed esegue **solo** il primo ramo la cui condizione è vera; se nessuna condizione è vera esegue `else`, se non c'è `else` non fa niente. Per questo l'ordine dei rami conta: `elseif x == 0` va dopo `x > 0` e prima dell'`else`, che è il ramo «per tutti gli altri casi».

Le regole da ricordare sono tre. La condizione deve essere un `Bool`: in Triad non esistono valori «falsi per convenzione» come lo zero o il testo vuoto in altri linguaggi, e `if 1` è un errore (`error: if cond`). Ogni `if` va chiuso con `end`, e dimenticarlo produce un `parse error`. Infine `else if` (con lo spazio) **non** è `elseif`: sono due parole che aprono un `if` annidato, che richiede un `end` in più; se ti ritrovi con `end` che non tornano, guarda lì.

Un `if` è anche un'espressione, e produce un valore — quello del ramo eseguito. Questo permette scelte compatte come `stato = eta >= 18 ? "maggiorenne" : "minorenne"`, oppure:

```
x = 3
etichetta = if x > 2
    :grande
else
    :piccolo
end
println(etichetta)
```

Output:

```
grande
```

## Esercizi

1. **Pari o dispari.** Dato `n`, stampa `pari` o `dispari` con un `if`.
2. **I voti.** Dato un voto da 0 a 10, stampa `insufficiente` (fino a 5), `sufficiente` (6), `buono` (fino a 8), `ottimo` (oltre), usando `elseif`.
3. **Il più grande di tre.** Dati tre numeri, stampa il più grande usando `if` (senza funzioni).
4. **`if 1`.** Perché `if 1` è un errore? Che cosa dice il messaggio? Che regola del linguaggio rispecchia?
5. **`else if` o `elseif`?** Scrivi apposta un `else if` (con lo spazio): quanti `end` servono per chiudere tutto? Dove si nasconde l'`if` annidato?
6. **L'ordine conta.** Scrivi un `if` con le condizioni `x > 0` e `x > 10`: in quale ordine vanno messe perché `x = 15` dia la risposta giusta? Prova entrambe le versioni.
7. **`if` come espressione.** Assegna a `etichetta` il valore di un `if` che produce `:grande` se `x > 2` e `:piccolo` altrimenti; stampa `etichetta`.
8. **Anno bisestile.** Dato `anno`, stampa `bisestile` o `non bisestile` secondo la regola del capitolo 13; provalo con 2000, 2005, 1900, 2024.
9. **Senza `else`.** Che cosa succede se nessuna condizione è vera e l'`if` non ha `else`? Verifica con un programma.

10. **Fuori intervallo.** Dato un voto, stampa il giudizio come all'esercizio 2, ma per voti negativi o maggiori di 10 stampa `voto non valido`.

---

## 8. Ripetizioni: `while`

Il computer è bravissimo a ripetere: è letteralmente ciò che sa fare meglio. La ripetizione si chiama **ciclo** (o *loop*), e il ciclo di Triad-Core è il `while`: «finché questa condizione è vera, ripeti queste istruzioni».

```
i = 1
while i <= 5
    println(i)
    i += 1
end
```

Output:

```
1
2
3
4
5
```

Il `while` funziona così: controlla la condizione; se è falsa, salta tutto il corpo del ciclo e prosegue dopo `end`; se è vera, esegue le istruzioni del corpo e ricontrolla la condizione. Le componenti del ciclo sono quindi sempre le stesse: una **variabile contatore** che cambia a ogni girata (`i`), una **condizione di uscita** che arriva a essere falsa (`i <= 5`), e l'**avanzamento** che porta lì (`i += 1`).

Se l'avanzamento manca o non viene eseguito, il ciclo non finisce mai: è il **ciclo infinito**, e il programma gira senza produrre nulla di utile finché non lo interrompi tu (su terminale, con `Ctrl+C`). Non è un dramma: capita a tutti. Se sospetti un ciclo infinito, guarda se la variabile del contatore cambia davvero a ogni girata e se la condizione può diventare falsa.

Due precisazioni utili. La condizione di un `while` deve essere un `Bool`: in Triad non esiste la «verità implicita» per cui `0`, una stringa vuota o una lista vuota valgono `false` (SPEC.md). E se un ciclo infinito sfugge, in Triad-Full c'è un freno automatico: `with budget(steps = N) … end` (cap. 13 del Manuale) interrompe il calcolo al posto tuo e restituisce un errore `Err(:budget)`; in Triad-Core il freno resta il `Ctrl+C`.

Con il ciclo si risolvono tutti i problemi «fai questa cosa N volte» e «passa in rassegna questi valori». Ecco due esempi classici — sommare e cercare:

```
somma = 0
n = 1
while n <= 100
    somma += n
    n += 1
end
println("somma: " * string(somma))

voti = [7, 4, 9, 6]
trovato = false
i = 1
while i <= length(voti)
    if voti[i] < 5
        trovato = true
```

```
    end
    i += 1
end
println(trovato)
```

Output:

```
somma: 5050
true
```

**Nota di implementazione.** La parte «sommare» gira su ogni runtime. La parte «cercare» usa `voti[i]`, l'accesso a un elemento di lista che imparerai al capitolo 9: il seed in codice macchina attuale non lo implementa ancora — `voti[i]` non seleziona l'elemento — e quel programma oggi stampa `false` invece di `true`, senza dare errori. Sull'interprete di riferimento l'output è quello mostrato. I limiti misurati del seed sono elencati in `STATUS.md`; il lavoro sull'indice di lista è tracciato in `TASK.md` (S0.2).

Due strumenti completano il quadro. `break` esce **immediatamente** dal ciclo, qualunque cosa dica la condizione; `continue` salta alla girata successiva, cioè al nuovo controllo della condizione:

```
i = 0
while true
    i += 1
    if i % 2 == 0
        continue
    end
    if i > 7
        break
    end
```

```
    println(i)
end
println("fatto")
```

Output:

```
1
3
5
7
fatto
```

Attenzione all'ordine dentro il corpo: se `continue` salta la riga che aggiorna il contatore, il ciclo non finirà mai. L'aggiornamento va fatto **prima** del `continue`, come nell'esempio.

Entrambi agiscono sul ciclo **più interno** che li contiene: dentro due cicli annidati un `break` ferma solo quello di dentro, e il ciclo esterno prosegue con le sue girate:

```
i = 1
while i <= 2
    j = 1
    while j <= 3
        if j == 2
            break
        end
        println(string(i) * "-" * string(j))
        j += 1
    end
```

```
    i += 1
end
```

Output:

```
1-1
2-1
```

Per uscire in un colpo da **entrambi** i cicli non esiste una scorciatoia: si usa una variabile di stato, che le condizioni di tutti e due controllano:

```
esci = false
a = 1
while a <= 3 && !esci
    b = 1
    while b <= 3 && !esci
        if a * b == 6
            esci = true
        else
            println(string(a) * "x" * string(b) * " = "
* string(a * b))
        end
        b += 1
    end
    a += 1
end
println("fine")
```

Output:

```
1x1 = 1
1x2 = 2
1x3 = 3
2x1 = 2
2x2 = 4
fine
```

Appena `a * b` arriva a 6, `esci` diventa `true` ed entrambe le condizioni lo vedono: il ciclo interno chiude la girata corrente e quello esterno non ne comincia nessun'altra.

**Nota di implementazione.** `break` funziona su tutti i runtime attuali; `continue` fa parte di Triad-Core (è nella grammatica di `SPEC-CORE.md`) ma il seed in codice macchina non lo implementa ancora: con `bin/triad` l'esempio qui sopra si interrompe alla prima `continue` con codice 1. Finché non sarà pronto (`TASK.md`, S0.4), la stessa identica logica si scrive capovolgendo la condizione — e gira oggi su entrambi i runtime:

```
i = 0
while true
    i += 1
    if i > 7
        break
    end
    if i % 2 == 1
        println(i)
    end
end
println("fatto")
```

Output:

```
1
3
5
7
fatto
```

---

## 8.1 Esercizi per sottogenere

Ogni problema «a ciclo», per quanto sembri diverso dagli altri, rientra quasi sempre in una di queste cinque forme. Riconoscere la forma ti dice subito quali pezzi ti servono:

| sottogenere | domanda tipo | pezzi che servono |
|---|---|---|
| **Contatore** | «quante volte?» | contatore, condizione, avanzamento |
| **Accumulatore** | «qual è il risultato di tutti insieme?» | variabile risultato, aggiornamento a ogni girata |
| **Ricerca e decisione** | «ce n'è uno che…?» | condizione di scoperta, `break` o variabile di stato |
| **Cicli annidati** | «quali combinazioni?» | contatore interno ri-azzerato a ogni girata |
| **Sentinella** | «fino a quando?» | `while true`, `break` alla condizione di uscita |

Tutte le soluzioni qui sotto girano identiche sul seed e sull'interprete di riferimento: prova prima a scriverle da solo, esegui e confronta l'output, e solo allora leggi la soluzione.

**Contatore**

1. **Tabellina del 7.** Stampa i multipli di 7 da 7 a 70, uno per riga.
2. **Conto alla rovescio.** Stampa i numeri da 3 a 1, uno per riga, e poi la riga `via!`.

**Accumulatore**

3. **Fattoriale.** Calcola `10!` (cioè `1 * 2 * … * 10`) e stampalo.
4. **Fila di numeri.** Costruisci la stringa `1-2-3-4-5` unendo con `-` i numeri da 1 a 5 e stampala. *(Parti da `s = ""` e a ogni giro aggiungi `string(i)` — e il trattino, tranne all'ultimo.)*

**Ricerca e decisione**

5. **Primo quadrato grande.** Qual è il primo numero il cui quadrato supera 5000? Stampalo.
6. **Verifica con uscita anticipata.** Controlla i numeri da 1 a 10: se uno ha il quadrato maggiore o uguale a 100 il controllo fallisce e il ciclo si ferma. Stampa `true` se tutti passano, `false` se fallisce; in questo secondo caso stampa anche il numero che ha fatto fallire il controllo.

**Cicli annidati**

7. **Tabella dei prodotti.** Stampa la tabella 3×3 dei prodotti: tre righe (`1 2 3`, `2 4 6`, `3 6 9`), con uno spazio fra i valori.
8. **Triangolo.** Dato `n = 3`, stampa `n` righe: la prima con una `+`, la seconda con due, la terza con tre.

**Sentinella**

9. **Potenze di due.** Partendo da 1 e raddoppiando, quante potenze di 2 minori di 1000 ci sono? Contale con un ciclo

`while true` che si ferma con `break`.

10. **Multipli di 7.** Quanti sono i multipli di 7 da 7 a 100 compreso? Contali con un ciclo `while true`.

## 8.2 Soluzioni

**1. Tabellina del 7**

```
n = 7
while n <= 70
    println(n)
    n += 7
end
```

Output:

```
7
14
21
28
35
42
49
56
63
70
```

**2. Conto alla rovescio**

```
i = 3
while i >= 1
    println(i)
```

```
    i = i - 1
end
println("via!")
```

Output:

```
3
2
1
via!
```

**3. Fattoriale** — il risultato si accumula in `f`, che parte da 1 (il neutro della moltiplicazione, come `0` per la somma):

```
f = 1
i = 1
while i <= 10
    f = f * i
    i += 1
end
println(f)
```

Output:

```
3628800
```

**4. Fila di numeri**

```
s = ""
i = 1
while i <= 5
```

```
    s = s * string(i)
    if i < 5
        s = s * "-"
    end
    i += 1
end
println(s)
```

Output:

```
1-2-3-4-5
```

**5. Primo quadrato grande** — qui il ciclo non «fa» nulla dentro: avanza finché la condizione è vera e alla fine basta leggere il contatore:

```
i = 1
while i * i <= 5000
    i += 1
end
println(i)
```

Output:

```
71
```

( `70 * 70` è `4900`, ancora sotto; `71 * 71` è `5041`, sopra.)

**6. Verifica con uscita anticipata**

```
n = 1
ok = true
while n <= 10
```

```
    if n * n >= 100
        ok = false
        break
    end
    n += 1
end
println(ok)
println("fallito a " * string(n))
```

Output:

```
false
fallito a 10
```

**7. Tabella dei prodotti** — il contatore interno `j` riparte da 1 a ogni girata di `i`:

```
i = 1
while i <= 3
    riga = ""
    j = 1
    while j <= 3
        riga = riga * string(i * j)
        if j < 3
            riga = riga * " "
        end
        j += 1
    end
    println(riga)
    i += 1
end
```

Output:

```
1 2 3
2 4 6
3 6 9
```

**8. Triangolo**

```
riga = 1
while riga <= 3
    s = ""
    col = 1
    while col <= riga
        s = s * "+"
        col += 1
    end
    println(s)
    riga += 1
end
```

Output:

```
+
++
+++
```

**9. Potenze di due** — la sentinella `while true` si ferma con `break` appena la potenza raggiunge la soglia:

```
p = 1
conta = 0
```

```
while true
    if p >= 1000
        break
    end
    conta += 1
    p = p * 2
end
println(conta)
```

Output:

```
10
```

(Sono `1, 2, 4, 8, 16, 32, 64, 128, 256, 512` : dieci valori.)

**10. Multipli di 7**

```
conta = 0
n = 7
while true
    if n > 100
        break
    end
    conta += 1
    n += 7
end
println(conta)
```

Output:

```
14
```

## 9. Liste: tenere insieme molti valori

Una variabile contiene un valore; una **lista** contiene un elenco di valori, nell'ordine in cui sono scritti. Si scrive tra parentesi quadre, con i valori separati da virgola:

```
frutta = ["mele", "pere", "banane"]
println(frutta)
println(frutta[1])
push!(frutta, "ciliegie")
println(frutta)
println(length(frutta))
```

Output:

```
[mele, pere, banane]
mele
[mele, pere, banane, ciliegie]
4
```

**Nota di implementazione.** Questo programma è verificato sull'interprete di riferimento. Il seed in codice macchina attuale non esegue ancora `frutta[1]`, non esegue `push!` scritto da solo (la forma `frutta = push!(frutta, "ciliegie")` invece funziona) e non stampa liste di testi: il programma qui sopra oggi si ferma dopo la sola `[`. Finché non saranno pronti (`TASK.md`, S0.2), questi esempi vanno letti come sintassi corretta del linguaggio.

Le operazioni di base sono queste:

- `[a, b, c]` crea una lista; `[]` è la lista vuota. Una lista può contenere valori di tipi diversi, e anche altre liste.

- `xs[i]` è l'elemento nella posizione `i`. **Le posizioni partono da 1**: `frutta[1]` è il primo elemento, l'ultimo è `frutta[length(frutta)]`. Una posizione fuori dall'intervallo è un errore (`error: index out of range`), non un valore «vuoto».
- `xs[i] = v` sostituisce l'elemento in posizione `i`.
- `length(xs)` conta gli elementi.
- `push!(xs, v)` aggiunge `v` alla fine della lista. Il `!` nel nome dice che la funzione **modifica** la lista: è una convenzione di nomenclatura che Triad riprende da altri linguaggi.

Le liste sono **mutabili**, e questo ha una conseguenza che sorprende i principianti: due nomi possono indicare la **stessa** lista.

```
xs = [1, 2, 3]
alias = xs
push!(alias, 4)
println(xs)
```

Output:

```
[1, 2, 3, 4]
```

Ho aggiunto a `alias` eppure è cambiata anche `xs`: non perché ci sia un errore, ma perché dopo `alias = xs` i due nomi guardano lo stesso oggetto, come due etichette sulla stessa scatola. Con i numeri non succede (`y = x` copia il valore), con le liste sì. Se ti serve una copia vera, crea una lista nuova e copia gli elementi uno a uno con un ciclo.

Il giro d'insieme di una lista — passare in rassegna tutti gli elementi — si scrive con un `while` e un contatore, ed è talmente comune che vale la pena impararlo a memoria:

```
numeri = [3, 5, 8, 1]
totale = 0
i = 1
while i <= length(numeri)
    totale += numeri[i]
    i += 1
end
println(totale)
```

Output:

```
17
```

## Esercizi

1. **Creare e stampare.** Crea una lista di quattro frutti, stampala intera, poi il primo e l'ultimo elemento.
2. **Posizioni.** Dato `xs = [10, 20, 30, 40]`, che cosa stampano `xs[1]`, `xs[4]`, `xs[5]`? Esegui e spiega l'ultimo risultato.
3. **Sostituire.** Dato `xs = [1, 2, 3]`, sostituisci l'elemento in posizione 2 con `20` e stampa la lista.
4. **`push!` e `length`.** Parti da una lista vuota e aggiungi tre numeri, stampando la lunghezza dopo ogni aggiunta.
5. **Giro completo.** Stampa tutti gli elementi di una lista con un `while` e un contatore.

6. **Somma.** Calcola e stampa la somma degli elementi di `[3, 5, 8, 1]`.
7. **Il più grande.** Trova il valore più grande di una lista di interi con un ciclo.
8. **Conta i pari.** Conta quanti elementi di una lista di interi sono pari.
9. **Alias o copia?** Dimostra che dopo `alias = xs` le due variabili indicano la stessa lista (come nell'esempio del capitolo); poi crea una **copia vera** con un ciclo e verifica che modificarla non cambi l'originale.
10. **Liste dentro le liste.** Crea `[[1, 2], [3, 4]]`, stampa `xs[1]` e spiega che cosa vedi: è un numero o un'altra lista?

---

## 10. Funzioni: dare un nome a un'operazione

Scrivere le stesse quattro righe dieci volte è noioso e pericoloso: se trovi un errore, devi correggerlo dieci volte. La soluzione di ogni linguaggio di programmazione è la **funzione**: un pezzo di programma a cui dai un nome, che puoi **chiamare** quante volte vuoi.

```
quadrato(x) = x * x

function saluta(nome)
    "ciao " * nome
end

println(quadrato(7))
println(saluta("Ada"))
```

Output:

```
49
ciao Ada
```

La prima funzione usa la **forma breve**: `quadrato(x) = x * x` significa «`quadrato` è la funzione che, dato `x`, produce `x * x`». La seconda usa la **forma blocco**: `function` apre, `end` chiude, e il valore prodotto dalla funzione è quello dell'**ultima espressione** del corpo (qui `"ciao " * nome`). Le due forme sono equivalenti: la breve quando il corpo è un'espressione sola, la blocco quando servono più istruzioni.

Quando scrivi `quadrato(7)`, si chiama la funzione: il valore `7` viene associato al parametro `x`, il corpo viene eseguito e produce `49`, che prende il posto della chiamata nell'espressione. I valori passati alla chiamata si chiamano **argomenti**; i nomi nella definizione si chiamano **parametri**. Sono due parole per la stessa cosa, viste dai due lati.

Una funzione può avere più parametri (separati da virgola), può servire a isolare un calcolo complesso, e — questa è la parte importante — può essere chiamata **da un'altra funzione**. Costruire programmi significa proprio questo: comporre pezzi piccoli e affidabili in pezzi più grandi.

```
function massimo(a, b)
    if a >= b
        a
    else
        b
    end
end

function area_rettangolo(larghezza, altezza)
    larghezza * altezza
end
```

```
println(massimo(3, 9))
println(area_rettangolo(5, 4))
```

Output:

```
9
20
```

Per uscire da una funzione prima della fine si usa `return`: `return espressione` termina la funzione subito, producendo quel valore. Nella maggior parte dei casi non serve, perché basta l'ultima espressione, ma è l'ideale quando ci sono più strade di uscita:

```
function descrivi(n)
    if n < 0
        return "negativo"
    end
    if n == 0
        return "zero"
    end
    "positivo"
end

println(descrivi(-5), " ", descrivi(0), " ",
descrivi(5))
```

Output:

```
negativo zero positivo
```

Le regole pratiche da ricordare: il numero di argomenti nella chiamata deve coincidere con il numero di parametri nella definizione (altrimenti `error: arity mismatch`); i parametri esistono solo dentro la funzione — una variabile nata dentro la funzione non esiste fuori; e la funzione produce un valore, che puoi usare, associare a una variabile o passare a un'altra funzione.

## Esercizi

1. **Forma breve.** Scrivi `doppio(x) = x * 2` e usala in un'espressione che stampi il doppio di `21`.
2. **Forma blocco.** Scrivi `massimo(a, b)` con `function` / `end` e un `if`, e provala con le coppie `(3, 9)`, `(9, 3)`, `(4, 4)`.
3. **`return` anticipato.** Scrivi `descrivi(n)` che restituisca `negativo`, `zero` o `positivo` usando due `return` e l'ultima espressione.
4. **Argomenti e parametri.** Nel programma `quadrato(7)`, quali sono la chiamata, l'argomento, il parametro? Scrivi un commento che li nomini tutti.
5. **Arity mismatch.** Chiama una funzione a un parametro con due argomenti: leggi l'errore e spiega in italiano che cosa significa.
6. **Comporre.** Scrivi `area_rettangolo(l, h)` e poi `doppia_area(l, h)` che riusi la prima funzione: stampa entrambi i risultati per `(3, 4)`.
7. **Variabili locali.** Crea dentro una funzione una variabile `temp` e provaci ad accedere fuori: che cosa succede?
8. **Non è una funzione.** Esegui `x = 5` e poi `x()`: quale errore ottieni? Che cosa voleva dire il linguaggio?

9. **Testo ripetuto.** Scrivi `ripeti(s, n)` che restituisca `s` concatenata `n` volte e provala con `("ab", 3)`: atteso `"ababab"`.
10. **Due saluti.** Scrivi `saluto(nome)` che restituisca `"ciao " * nome` e un programma che la chiami due volte con argomenti diversi, usando il risultato in due `println`.

---

## 11. Ricorsione: una funzione che chiama sé stessa

Una funzione può chiamare altre funzioni — compresa sé stessa. Si chiama **ricorsione**, e serve per i problemi che si riducono allo stesso problema più piccolo: contare alla rovescia, calcolare un fattoriale, attraversare una struttura ad albero. Vediamola in azione:

```
function conta_alla_rovescia(n)
    if n <= 0
        println("via!")
        return
    end
    println(n)
    conta_alla_rovescia(n - 1)
end

conta_alla_rovescia(3)
```

Output:

```
3
2
```

```
1
via!
```

Seguiamo cosa succede: `conta_alla_rovescia(3)` stampa `3` e chiama `conta_alla_rovescia(2)`; questa stampa `2` e chiama `conta_alla_rovescia(1)`; questa stampa `1` e chiama `conta_alla_rovescia(0)`; quest'ultima trova `n <= 0`, stampa `via!` e termina con `return`. Ogni chiamata aspetta che la chiamata successiva finisca, poi finisce anche lei. Le chiamate in attesa si accumulano (il computer le tiene in una struttura chiamata *stack*) e si risolvono nell'ordine inverso.

Ogni funzione ricorsiva ha bisogno di due pezzi: il **caso base**, che risolve subito il problema più piccolo e **interrompe la catena** delle chiamate (qui `n <= 0`), e il **passo ricorsivo**, che riduce il problema e richiama la funzione (qui `conta_alla_rovescia(n - 1)`). Se manca il caso base, o se il problema non si riduce mai, le chiamate si accumulano all'infinito e il programma termina con un errore: la ricorsione infinita è il corrispettivo ricorsivo del ciclo infinito.

L'esempio classico per capire il valore della ricorsione è il **fattoriale**: il fattoriale di `n` (o `n!`) è `n × (n-1) × … × 1`, e si può definire come «`n` per il fattoriale di `n-1`», con il fattoriale di `1` che vale `1`:

```
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
println(fattoriale(5))
```

Output:

```
120
```

La lettura è: `fattoriale(5)` vale `5 * fattoriale(4)`, che vale `5 * 4 * fattoriale(3)`, e così via fino a `fattoriale(1)`, che vale `1` e chiude la catena. Il risultato `5 * 4 * 3 * 2 * 1 = 120` risale indietro per le chiamate in attesa.

Non tutti i problemi chiedono la ricorsione: per «ripeti N volte» e «passa in rassegna la lista» il ciclo `while` è più diretto. La ricorsione diventa naturale quando il problema **è** un albero o una struttura fatta di pezzi simili, oppure quando la definizione del problema è già ricorsiva, come il fattoriale. In entrambi i casi la domanda da farsi è sempre la stessa: qual è il caso base, e come si riduce il problema?

## Esercizi

1. **Conto alla rovescia.** Scrivi `conta(n)` che stampi i numeri da `n` a 1 e poi `via!` in forma ricorsiva.
2. **Fattoriale.** Scrivi `fattoriale(n)` in forma breve con il ternario e verifica `fattoriale(5)` (`120`) e `fattoriale(1)` (`1`).
3. **Somma.** Scrivi `somma(n)` che restituisca `1 + 2 + … + n` in forma ricorsiva; confronta il risultato con la versione a ciclo del capitolo 8.
4. **Potenza.** Scrivi `potenza(x, n)` che calcoli `x^n` moltiplicando, in forma ricorsiva.
5. **Fibonacci.** Scrivi `fib(n)` ricorsivo (`fib(1) = 0`, `fib(2) = 1`) e stampa i primi dieci valori.
6. **Senza caso base.** Togli il caso base a `fattoriale` ed esegui: che cosa succede? Perché il programma si ferma invece di girare all'infinito?
7. **La catena.** Scrivi su un foglio tutte le chiamate che fa `fattoriale(5)` prima di produrre `120`, nell'ordine in cui cominciano e in cui finiscono.

8. **Troppe chiamate.** Esegui `conta_alla_rovescia(10000)`: che cosa succede quando le chiamate in attesa diventano troppe?
9. **Ciclo o ricorsione?** Riscrivi `fattoriale` con un `while` e confronta i due programmi: quale preferisci e perché?
10. **Doppia ricorsione.** Scrivi una funzione che, dato `n`, stampi i numeri da 1 a `n` e poi da `n` a 1, ricorsivamente (suggerimento: due funzioni che si chiamano a vicenda).

---

## 12. Errori e debug: cosa fare quando il programma non funziona

Un programma che non funziona non è un fallimento: è la normalità del mestiere. Anche gli programmatori esperti passano gran parte del tempo a correggere errori (in inglese, *bug*) e a cercarli (*debug*). La differenza è che loro sanno leggere ciò che il computer risponde, e hanno un metodo. Questo capitolo ti dà entrambe le cose.

### 12.1 Le due famiglie di errori

Quando esegui un programma sbagliato, il messaggio che vedi appartiene a una di due famiglie:

- **`parse error:`** — l'interprete non è riuscito a **leggere** il programma. Qualcosa non torna nella forma: una parentesi non chiusa, un `end` mancante, una parola scritta male. Il programma non è nemmeno cominciato: l'interprete si è fermato mentre lo analizzava.
- **`error:`** — il programma è stato letto, è cominciato, e si è fermato **durante l'esecuzione**: una variabile senza valore, un'operazione tra tipi che non si combinano, una posizione fuori lista. L'esecuzione si interrompe nel punto dell'errore.

In entrambi i casi il programma termina con **codice di uscita** `1` (il codice è il numero con cui un programma «risponde» al terminale: `0` significa tutto bene). I messaggi vanno sul canale di errore del terminale, ma ti arrivano comunque sugli occhi.

## 12.2 I messaggi più comuni, tradotti in italiano

| messaggio | che cosa significa | la causa tipica |
|---|---|---|
| `parse error: expected IDENT end` | manca un `end` oppure c'è una parola inattesa | `if` o `function` o `while` non chiusi |
| `parse error: expected RPAREN` | manca una parentesi tonda | `println("ciao"` senza chiudere |
| `error: undefined variable x` | `x` non è mai stata assegnata | refuso nel nome, o ordine sbagliato |
| `error: arity mismatch` | numero di argomenti sbagliato nella chiamata | `f(1, 2)` per una funzione con un parametro |
| `error: + type error` | `+` usato tra valori di tipo incompatibile | `"ciao" + 1` (serve `*` e `string`) |
| `error: index out of range` | posizione fuori dalla lista | `xs[10]` quando la lista ha 4 elementi |
| `error: if cond` | la condizione non è un `Bool` | `if 1` oppure `if x = 0` (assegna!) |
| `error: not a function` | si è «chiamato» qualcosa che non è una funzione | `x()` dove `x` è un numero |

Leggere il messaggio è già mezzo lavoro: dice **che cosa** non è andato e, spesso, **dove**. La prima domanda da farsi non è «perché non funziona?», ma «che cosa esattamente non ha funzionato, secondo il messaggio?».

## 12.3 Il metodo in quattro passi

1. **Leggi il messaggio tutto intero**, senza saltare. Traducilo: quale regola del linguaggio è stata violata?
2. **Trova il punto**. Se il messaggio non dice la riga, guarda l'ultima riga di codice che hai modificato: il più delle volte l'errore è lì.
3. **Riduci il problema**. Commenta (con `#`) metà del programma e riesegui: se l'errore resta, è nella metà attiva; se sparisce, era nell'altra. Ripeti e dimezzi: in pochi passi sei alla riga colpevole.
4. **Verifica le ipotesi, una alla volta**. Stampa le variabili con `println` proprio prima del punto sospetto: i valori reali sono più affidabili di quelli che credi ci siano.

Quest'ultimo trucco — stampare i valori intermedi — è il debugger più antico del mondo e funziona ancora benissimo. Un `println("qui, n = " * string(n))` inserito nel punto giusto dice se il programma arriva lì e con quali valori: in un solo esame sai se il problema è «non ci arriva» o «ci arriva con il valore sbagliato».

## 12.4 Gli errori dei principianti, per nome

- `=` **invece di** `==` in una condizione. Assegna invece di confrontare.
- `end` **mancante** (o in più). Ogni `if`, `while`, `function`, `let` aperti vanno chiusi.
- `else if` **invece di** `elseif`.
- `+` **tra stringhe**. In Triad la concatenazione è `*`.

- **Indice da 0**. Le liste di Triad partono da 1: il primo elemento è `xs[1]`.
- **Contatore del ciclo non aggiornato**, spesso per un `continue` scritto prima dell'aggiornamento: il ciclo non finisce più.
- **Variabile aggiornata nel punto sbagliato**: se la somma viene azzerata dentro il ciclo invece che prima, il risultato è sempre l'ultimo pezzo.
- **Refuso in un nome** ( `punteggio` vs `punteggio_totale` ): il linguaggio dice `undefined variable`, e ha ragione.

A tutto questo si aggiunge un errore «di mancanza»: credere che il computer faccia quello che avevi in mente invece di quello che hai scritto. Il programma fa **esattamente** ciò che dice il testo, mai ciò che volevi dire. Quando il risultato è misterioso, la domanda giusta è «che cosa sta eseguendo davvero il programma, riga per riga?», e la risposta si trova rileggendo.

## Esercizi

1. **Fine mancante.** Scrivi un `if` senza `end` ed esegui: a quale famiglia appartiene l'errore? Leggi il messaggio tutto intero.
2. **Due famiglie.** Classifica questi errori in `parse error` o `error`: parentesi tonda non chiusa, variabile senza assegnamento, `end` mancante, operazione `"ciao" + 1`.
3. **Traduci il messaggio.** Dato `error: + type error`, in italiano che cosa è successo? Qual è la riga sospetta in un programma che calcola il totale di due prezzi?
4. **Il metodo in quattro passi.** Un programma che dovrebbe sommare i numeri da 1 a 5 stampa sempre `0`: applica i quattro passi del capitolo e trova il bug.

5. **Debug con `println`.** Prendi un programma tuo con un errore e aggiungi `println` di controllo finché non individui la riga colpevole: quante ne sono bastate?
6. **`end` in più.** Aggiungi un `end` di troppo a un programma corretto: quale messaggio ottieni?
7. **Fuori lista.** Esegui `xs = [1, 2]` e `println(xs[3])`: che errore? Come si chiama la regola violata?
8. **Refuso nel nome.** Scrivi un programma che usi `punteggio` e poi `punteggio_totale` per errore: quale messaggio? Come si riconosce subito?
9. **Diagnosi.** Un programma stampa sempre l'ultimo valore invece della somma: quale errore classico del capitolo 8.1 potrebbe esserne la causa? Verifica la tua ipotesi.
10. **Allenamento al metodo.** Scrivi tre programmi sbagliati apposta (uno di sintassi, uno di tipo, uno di logica) e riscoprili usando solo il metodo in quattro passi.

---

## 13. Pensare da programmatore: scomporre un problema

Scrivere programmi non è digitare velocemente: è **risolvere problemi**. Il computer esegue, ma non aiuta: tocca a te capire che cosa fare, scomporlo in passi piccoli abbastanza da poter essere scritti come istruzioni, e controllare che ogni passo faccia davvero quello che pensavi. Questa è la parte che i programmatori chiamano *progettazione*, e si impara solo facendola. Un metodo semplice, in cinque passi, basta per i primi programmi e regge anche per quelli grandi.

## 13.1 Capire il problema, con esempi in mano

Prima di scrivere una riga, riformula il problema con le tue parole e scrivi **almeno due esempi di risultato atteso**, fatti a mano. Se non sai dire che cosa deve comparire a schermo per un caso semplice, non lo saprà nemmeno il programma. Gli esempi servono anche più avanti: sono il criterio con cui dire «funziona» oppure no — lo stesso meccanismo dei task e dei PASS/FAIL del capitolo 16.

Prendiamo un problema vero, piccolo ma non banale: *stampare tutti gli anni bisestili tra il 2000 e il 2040*. Esempi fatti a mano: il 2000 è bisestile (è divisibile per 400), il 2004 è bisestile, il 2005 no. Una regola nota dice: un anno è bisestile se è divisibile per 4, e in più non è un secolo oppure è divisibile per 400.

## 13.2 Scomporre in passi, in italiano

Scrivi i passi come se dovessi spiegarli a una persona pazientissima che non sa nulla del problema. In italiano, non in codice: questa è la bozza del programma.

1. parti dall'anno 2000;
2. finché l'anno è minore o uguale a 2040:
3. — se l'anno è bisestile, stampalo;
4. — passa all'anno successivo;
5. quando superi il 2040, fermati.

Sembra banale scritto così, ed è esattamente il punto: la programmazione consiste nel rendere espliciti passi che nella nostra testa erano impliciti. Nota che ogni riga della bozza corrisponde a un costrutto che conosci già: un assegnamento, un `while`, un `if`, una `println`.

## 13.3 Prima la versione semplice, poi aggiungi

L'errore più comune del principiante è voler scrivere il programma completo subito. Invece si procede per versioni: prima una versione che funziona **in un caso più semplice**, poi la si guarnisce. La nostra bozza ha due pezzi indipendenti: il giro sugli anni e il riconoscimento dell'anno bisestile. Prima il giro, che sappiamo già scrivere:

```
anno = 2000
while anno <= 2040
    println(anno)
    anno += 1
end
```

Output (primi tre e ultima riga):

```
2000
2001
2002
...
2040
```

Funziona: il giro c'è. Ora resta da filtrare, e il filtro è la regola del caso a mano: divisibile per 4 ( `anno % 4 == 0` ) **e** ( `&&` ) non un secolo ( `anno % 100 != 0` ) **oppure** ( `||` ) divisibile per 400. Attenzione alle parentesi attorno al secondo pezzo: senza, la regola si leggerebbe in un altro modo:

```
anno = 2000
while anno <= 2040
    if anno % 4 == 0 && (anno % 100 != 0 || anno % 400
== 0)
        println(anno)
```

```
    end
    anno += 1
end
```

Output:

```
2000
2004
2008
2012
2016
2020
2024
2028
2032
2036
2040
```

Confronta l'output con gli esempi del passo 1: il 2000 c'è, il 2004 c'è, il 2005 non c'è. Il caso limite più interessante è proprio il 2000: un anno secolo **è** bisestile solo se divisibile per 400, e il programma lo stampa. Se il compito avesse compreso anche il 2100, il tuo esempio a mano avrebbe scoperto il caso: per questo conviene inventarsene sempre qualcuno di scomodo.

## 13.4 Scrivere poco, verificare spesso

Ogni versione va **eseguita** prima di passare alla successiva. Se qualcosa non torna, non continuare: un errore scoperto oggi costa un decimo di quello scoperto tra dieci righe. E quando il programma è complesso, i tuoi

migliori alleati sono le `println` di controllo — stampare un valore intermedio «per vedere com'è» non è roba da principianti: è quello che fa anche chi programma da trent'anni.

## 13.5 Quando sei bloccato

Succede a tutti, letteralmente. Una sequenza che funziona: **rileggi il testo del problema** (molte volte il programma fa la cosa sbagliata per bene); **rifallo a mano** con un caso piccolo scrivendo ogni passaggio; **simplifica** (commenta metà programma, vedi il capitolo 12); **rileggi il messaggio d'errore** per intero; e se proprio non ne vuoi uscire, **stacca**. Il problema non scappa, e spesso la soluzione arriva mentre non stai guardando.

Ultima cosa, la più importante: progettare bene si impara solo progettando. Prendi gli esercizi del capitolo 14, applica questi cinque passi a ciascuno — capire, esempi, scomporre, versione semplice, verificare — e dopo una decina di problemi il metodo non sarà più una lista nel libro: sarà il tuo modo di lavorare.

## Esercizi

1. **Esempi in mano.** Per il problema «stampare i numeri da 1 a 30 che sono multipli di 3 o di 5», scrivi tre esempi di output atteso **prima** di scrivere codice.
2. **Scomporre.** Scrivi la bozza in italiano (solo passi, nessun codice) per «contare quanti numeri di una lista sono maggiori di 10».
3. **Versione semplice.** Risolvi prima «stampare tutti i numeri da 1 a 20» e solo poi aggiungi il filtro dei multipli: esegui entrambe le versioni.
4. **Casi scomodi.** Per il riconoscimento dell'anno bisestile, elenca almeno quattro casi «scomodi» da provare e spiega perché proprio quelli (2000, 1900, 2040, 2023…).

5. **Verificare spesso.** Scrivi il programma dei multipli in tre versioni successive, eseguendo **ognuna** prima di passare alla successiva.
6. **Rifallo a mano.** Un programma dice che il 1900 è bisestile: rifai il calcolo a mano e individua l'errore nella regola scritta.
7. **Due pezzi indipendenti.** Scomponi «stampare la tabellina del 7 in forma `7 x 1 = 7`» in due pezzi e risolvili separatamente prima di unirli.
8. **Sei bloccato.** Applica la sequenza del capitolo 13.5 a un problema su cui sei davvero bloccato: quale passo ti ha sbloccato?
9. **Riusare il metodo.** Applica i cinque passi a «stampare tutti gli anni bisestili tra 1900 e 2000»: scrivi esempi, bozza, versione semplice, versione completa.
10. **Un problema tuo.** Inventati un piccolo problema che richieda un `if` e un `while`, risolvilo col metodo dei cinque passi e scrivi in fondo «come ho verificato che funziona».

---

## 14. Esercizi

Gli esercizi sono in ordine di difficoltà. Per ciascuno scrivi il programma, eseguilo, confronta l'output: solo dopo, se serve, guarda la soluzione in fondo. Un esercizio «fatto» è un esercizio **eseguito**, non letto.

1. **Saluto personalizzato.** Scrivi un programma che associ a una variabile `nome` il tuo nome e stampi `ciao, <nome>!`.
2. **Doppio e quadrato.** Dato `n = 7`, stampa il suo doppio e il suo quadrato, ciascuno su una riga.
3. **Pari o dispari.** Data una variabile `n`, stampa `pari` o `dispari` a seconda del valore. *(Suggerimento: guarda il*

*resto della divisione per 2.)*

4. **Tavola del 3.** Stampa i primi dieci multipli di 3, uno per riga, usando un ciclo `while`.
5. **Conto alla rovescia.** Chiedi al programma di stampare i numeri da 10 a 1, uno per riga.
6. **Il più grande.** Data una lista di interi `[4, 19, 2, 8]`, stampa il valore più grande.
7. **Conta i negativi.** Data una lista di interi, stampa quanti sono minori di zero.
8. **Somma di liste.** Scrivi una funzione `somma_lista(xs)` che restituisca la somma degli elementi di una lista, e usala con due liste diverse.
9. **Password semplice.** Scrivi una funzione `controlla(password)` che restituisca `:ok` se la stringa è lunga almeno 8 byte, `:corta` altrimenti. Provala con tre password diverse.
10. **Fibonacci.** Scrivi una funzione `fib(n)` che produca l'ennesimo numero di Fibonacci (0, 1, 1, 2, 3, 5, 8, …; ogni numero è la somma dei due precedenti). Prima con un ciclo, poi — se ti va — in forma ricorsiva.
11. **Stella.** Scrivi un programma che, dato un intero `n`, stampi `n` righe: la prima con una `*`, la seconda con due, e così via.
12. **Traduttore di voti.** Scrivi una funzione che, dato un voto da 0 a 10, restituisca il giudizio: `:insufficiente` fino a 5, `:sufficiente` fino a 6, `:buono` fino a 8, `:ottimo` oltre. Controlla anche il caso del voto fuori intervallo.

## Soluzioni

**1. Saluto personalizzato**

```
nome = "Ada"
println("ciao, " * nome * "!")
```

Output:

```
ciao, Ada!
```

**2. Doppio e quadrato**

```
n = 7
println(n * 2)
println(n * n)
```

Output:

```
14
49
```

**3. Pari o dispari** — il resto della divisione per 2 è `0` per i pari:

```
n = 7
if n % 2 == 0
    println("pari")
else
    println("dispari")
end
```

Output:

```
dispari
```

**4. Tavola del 3** — il contatore `i` gira da 1 a 10 e a ogni giro stampa `3 * i`:

```
i = 1
while i <= 10
    println(3 * i)
    i += 1
end
```

Output:

```
3
6
9
12
15
18
21
24
27
30
```

**5. Conto alla rovescia** — il contatore scende, e la condizione si legge al contrario:

```
n = 10
while n >= 1
    println(n)
    n -= 1
end
```

Output:

```
10
9
8
7
6
5
4
3
2
1
```

**6. Il più grande** — si parte dal primo elemento come massimo provvisorio e si confronta con gli altri. La funzione `massimo` del capitolo 10 torna utile:

```
xs = [4, 19, 2, 8]
migliore = xs[1]
i = 2
while i <= length(xs)
    if xs[i] > migliore
        migliore = xs[i]
    end
    i += 1
end
println(migliore)
```

Output:

```
19
```

**7. Conta i negativi** — schema «conta se soddisfa»:

```
xs = [3, -1, -7, 0, 2]
quanti = 0
i = 1
while i <= length(xs)
    if xs[i] < 0
        quanti += 1
    end
    i += 1
end
println(quanti)
```

Output:

```
2
```

**8. Somma di liste** — la stessa funzione, chiamata su liste diverse:

```
function somma_lista(xs)
    totale = 0
    i = 1
    while i <= length(xs)
        totale += xs[i]
        i += 1
    end
    totale
end

println(somma_lista([1, 2, 3]))
println(somma_lista([10, 20]))
```

Output:

```
6
30
```

**9. Password semplice** — `length` conta i byte; si restituiscono simboli come etichette:

```
function controlla(password)
    if length(password) >= 8
        :ok
    else
        :corta
    end
end

println(controlla("segretissimo"), " ",
controlla("abc"))
```

Output:

```
ok corta
```

**10. Fibonacci** — versione con ciclo, che aggiorna due valori a ogni giro; poi la versione ricorsiva, che è quasi la definizione stessa del problema:

```
function fib(n)
    a = 0
    b = 1
    i = 1
    while i <= n
```

```
        t = a + b
        a = b
        b = t
        i += 1
    end
    a
end

fib_ric(n) = n < 2 ? n : fib_ric(n - 1) + fib_ric(n -
2)

println(fib(10), " ", fib_ric(10))
```

Output:

```
55 55
```

**11. Stella** — doppio ciclo: quello esterno sceglie la riga, quello interno stampa le `*`:

```
n = 4
riga = 1
while riga <= n
    riga_di_stelle = ""
    j = 1
    while j <= riga
        riga_di_stelle = riga_di_stelle * "*"
        j += 1
    end
    println(riga_di_stelle)
```

```
    riga += 1
end
```

Output:

```
*
**
***
****
```

**12. Traduttore di voti** — una catena di `elseif`, con un controllo finale sui valori fuori intervallo. Attenzione all'ordine: il controllo sui fuori intervallo va **prima** degli altri:

```
function giudizio(voto)
    if voto < 0 || voto > 10
        :errore
    elseif voto <= 5
        :insufficiente
    elseif voto == 6
        :sufficiente
    elseif voto <= 8
        :buono
    else
        :ottimo
    end
end

println(giudizio(3), " ", giudizio(6), " ",
giudizio(7), " ", giudizio(10), " ", giudizio(11))
```

Output:

```
insufficiente sufficiente buono ottimo errore
```

---

## 15. Glossario dei termini informatici

Le parole che seguono sono il vocabolario minimo del mestiere: il Manuale e il Tutorial le usano dandole per note. Qui ciascuna ha la sua spiegazione in una riga.

| termine | che cosa significa |
|---|---|
| argomento | il valore passato a una funzione al momento della chiamata |
| assegnamento | l'operazione `=`: associare un valore a un nome di variabile |
| bug | un errore in un programma |
| caso base | la condizione che interrompe una ricorsione, risolvendo il caso più piccolo |
| chiamata | l'uso di una funzione, con i suoi argomenti: `quadrato(7)` |
| ciclo (loop) | un blocco di istruzioni eseguito più volte |
| codice / sorgente | il testo di un programma |
| compilatore | programma che traduce un intero programma in una forma eseguibile dal computer |
| condizione | un'espressione che vale `true` o `false` |
| contatore | variabile che tiene il conto delle girate di un ciclo |
| debug | l'attività di trovare e correggere gli errori di un programma |

| termine | che cosa significa |
|---|---|
| espressione | un calcolo che produce un valore |
| file | un documento salvato su disco, identificato da un nome |
| funzione | un pezzo di programma con un nome, che riceve argomenti e produce un valore |
| interprete | programma che esegue un altro programma istruzione per istruzione |
| iterazione | una singola girata di un ciclo |
| lista | un elenco ordinato di valori, raggiungibili per posizione |
| parametro | il nome con cui una funzione riceve un argomento |
| percorso | l'indirizzo di una cartella o di un file nel computer |
| ricorsione | una funzione che chiama sé stessa su un problema più piccolo |
| runtime | il programma che esegue i programmi Triad (interprete o compilatore) |
| stringa | un testo trattato come dato |
| terminale | la finestra in cui si danno comandi al computer scrivendoli |
| tipo | la categoria di un valore: numero intero, testo, verità, lista… |
| variabile | un nome associato a un valore, che può cambiare nel tempo |
| valore | un dato con cui il programma calcola: un numero, un testo, una lista… |

## Esercizi

1. **Argomento o parametro?** In `saluta("Ada")` e in `function saluta(nome)`, dove sono l'argomento e il parametro? Spiega la differenza con le tue parole.

2. **Due mestieri.** Differenza tra compilatore e interprete: quale dei due usa Triad oggi e come si vede nel tuo ciclo di lavoro?
3. **I pezzi di un ciclo.** Prendi il programma del capitolo 8 che somma i numeri da 1 a 100 e nomina: contatore, condizione, avanzamento, iterazione, caso (o girata).
4. **Vero o falso.** Valuta: «le liste sono immutabili», «il caso base interrompe una ricorsione», «una variabile può cambiare tipo», `"42"` e `42` sono lo stesso valore», « `println` è una funzione».
5. **La tua definizione.** Scrivi con le tue parole, una frase ciascuna: *espressione*, *tipo*, *bug*, *runtime*. Confronta con il glossario.
6. **Associa.** Collega ogni termine al suo esempio: *funzione*, *chiamata*, *parametro*, *argomento*, *ritorno* — esempi: `quadrato(x) = x * x`, `quadrato(7)`, `x`, `7`, `49`.
7. **Il glossario di un programma.** Prendi un tuo programma e annota accanto a ogni riga almeno un termine del glossario (assegnamento, espressione, condizione…).
8. **Quale parola?** «Un elenco ordinato di valori raggiungibili per posizione», «un nome associato a un valore che può cambiare», «una finestra in cui si danno comandi»: quali termini del glossario descrivono queste frasi?
9. **Parole che confondono.** Scrivi una frase che distingua *parametro* da *argomento* e una che distingua *valore* da *variabile*.
10. **Ripasso senza libro.** Elenca a memoria i sei tipi di base di Triad con un esempio per ciascuno, poi controlla con la tabella del capitolo 3.

# 16. Una domanda che capita spesso: cosa sono «success» e «fail»?

Chi guarda il progetto da fuori — sito, repository, benchmark — trova descrizioni come questa: Triad si impara risolvendo piccoli **task** con operazioni aritmetiche e su stringhe, e le soluzioni sono sottoposte a **tentativi** che finiscono in **success** (successo) o **fail** (fallimento). La domanda che ne nasce è giusta, e merita una risposta precisa: *tentativi di che cosa? e successo o fallimento rispetto a quale riferimento?*

## 15.1 Il riferimento: la verifica automatica di un task

Un **task** è un piccolo esercizio di programmazione: un problema da risolvere scritto in linguaggio naturale, con alcuni esempi di ciò che il programma deve produrre. Nel repository, la cartella `ai/tasks/` ne contiene alcuni che riguardano proprio le operazioni di base: una somma (`01_sum`), un fattoriale (`02_factorial`), la successione di Fibonacci (`03_fibonacci`), operazioni su stringhe (`04_strings`), una funzione che restituisce un'altra funzione (`05_closures`). Il task consegna il problema; la soluzione è un programma Triad.

La **verifica** di una soluzione è interamente meccanica, e qui sta il punto: il programma viene **eseguito** con l'interprete e il suo output viene **confrontato** con quello che il task dichiara atteso. Se coincidono, l'esito è **PASS** (successo); se non coincidono — o se il programma va in errore e non produce nulla — l'esito è **FAIL** (fallimento). «Successo» e «fallimento» non sono dunque proprietà misteriose del linguaggio: sono il **verdetto di un confronto** tra ciò che il programma fa e ciò che il task si aspetta. Il riferimento, in altre parole, è **l'output atteso scritto nel task** — nient'altro.

Si può vedere tutta la faccenda in piccolo, con un programma che fa entrambe le parti — calcola la risposta e la confronta con l'atteso:

```
atteso = "42"

risposta() = 6 * 7

if string(risposta()) == atteso
    println("PASS")
else
    println("FAIL")
end
```

Output:

```
PASS
```

Se `atteso` fosse `"43"`, lo stesso programma stamperebbe `FAIL`: non perché il calcolo sia sbagliato, ma perché il **confronto** non è andato a buon fine. Il verdetto dice «questa soluzione soddisfa il task», non «questo programma è giusto in assoluto» — ed è per questo che ogni task specifica con cura i propri esempi attesi: sono il criterio con cui si misura.

## 15.2 I tentativi: pass@1 e pass@k

Restano i «tentativi». Quando si misura la capacità di un modello di intelligenza artificiale di scrivere Triad, non gli si chiede una soluzione sola: gli si chiedono **più soluzioni dello stesso task**, una per tentativo, e ciascuna viene verificata da sola, con lo stesso confronto di prima. Un tentativo è quindi **una generazione della soluzione**, e il suo esito è PASS o FAIL:

```
atteso = "42"
tentativi = ["41", "43", "42"]
i = 1
```

```
while i <= length(tentativi)
    esito = tentativi[i] == atteso ? "PASS" : "FAIL"
    println("tentativo " * string(i) * ": " *
tentativi[i] * " -> " * esito)
    i += 1
end
```

Output:

```
tentativo 1: 41 -> FAIL
tentativo 2: 43 -> FAIL
tentativo 3: 42 -> PASS
```

Le metriche del benchmark (`ai/README.md`) si leggono esattamente così. **pass@1** significa: «il primo tentativo risolve il task» — la misura più severa, quella che conta solo la soluzione che esce giusta subito. **pass@k** significa: «dati k tentativi per task, ne basta uno che riesca» — *il primo che riesce conta*. Sono due modi di rispondere alla domanda «quanto è affidabile chi scrive questo codice?»: pass@1 misura la precisione al primo colpo, pass@k misura se, dato qualche tentativo, il task alla fine si risolve. Un modello con pass@k alto e pass@1 basso «ci arriva», ma ci riprova.

## 15.3 Le altre due facce di «successo e fallimento» in Triad

Le stesse due parole ricorrono in altri due punti del progetto, ed è utile saperle riconoscere per non confonderle con quelle del benchmark.

Il primo l'hai già incontrato nel capitolo 12: quando un programma termina, lo fa con un **codice di uscita** — `0` significa successo, `1` errore, `2` uso non valido del comando. È il verdetto sull'intera esecuzione, non

su una singola risposta: un programma che stampa l'output giusto ma va in errore alla fine esce comunque con `1`.

Il secondo appartiene alla **programmazione logica** di Triad-Full, la parte del linguaggio in cui si descrivono fatti e regole e si fanno domande (`solve`) al sistema. Lì ogni domanda — ciò che il linguaggio chiama *goal* — può **riuscire o fallire**: riesce se il sistema trova un modo di renderla vera collegando le variabili, fallisce se nessun modo esiste. E poiché una domanda può ammettere più risposte, il motore fa letteralmente dei **tentativi**: prova una clausola, e se quella strada fallisce torna indietro e ne prova un'altra (si chiama *backtracking*: riprendere da un bivio precedente e tentare la strada successiva). La traccia di una ricerca (`solve … trace`) registra proprio questi tentativi con quattro etichette: `:call` (il tentativo comincia), `:redo` (un nuovo tentativo sulla stessa domanda), `:exit` (il tentativo riesce), `:fail` (il tentativo fallisce). Se incontri `success` e `fail` parlando di `fact`, `rule` o `solve`, il riferimento è a questo: l'esito dei tentativi del motore logico, non la verifica di un task.

## 15.4 La domanda dietro la domanda

Tutte queste facce hanno lo stesso scheletro: il computer **confronta qualcosa con un criterio preciso** — un output atteso, un codice di uscita, una condizione da dimostrare — e ne trae un verdetto binario. Non c'è un «fallimento» da temere: un FAIL è informazione, il modo in cui la verifica dice dove guardare. Anzi, nel lavoro di tutti i giorni — e questo libro lo darà per fatto nei capitoli successivi — il FAIL arriva prima del PASS quasi sempre, e leggerlo bene è metà del mestiere.

## Esercizi

1. **Mini-verificatore.** Scrivi il programma PASS/FAIL dell'esempio, poi cambia `atteso` in `"43"` e verifica di ottenere `FAIL`.

2. **Cinque tentativi.** Dati i tentativi `["41", "43", "42", "42", "40"]`, stampa una riga per tentativo con il suo esito, come nell'esempio del capitolo.
3. **pass@1 e pass@k.** Con i tentativi dell'esercizio 2: il pass@1 è PASS o FAIL? E il pass@3? Spiega la differenza fra le due metriche.
4. **Codice di uscita.** Esegui un programma corretto e uno che va in errore: come controlli il codice di uscita dal terminale? (`echo $?` su Linux/macOS, `echo %ERRORLEVEL%` su Windows.)
5. **Due significati di «fail».** Spiega la differenza tra il `FAIL` della verifica di un task e il fallimento di un goal nella programmazione logica del capitolo 16.3.
6. **Scrivi un task.** Progetta un task come quelli di `ai/tasks/`: il problema in linguaggio naturale e due esempi di output atteso.
7. **Verdetto con `if`.** Riscrivi il mini-verificatore usando un `if` completo invece del ternario: l'output è lo stesso?
8. **Conta i PASS.** Dati cinque tentativi con output diversi, stampa quanti sono PASS e quanti FAIL.
9. **Perché il FAIL serve.** Un compagno dice che un FAIL significa «ho sbagliato tutto»: scrivi due righe per spiegargli che il FAIL è invece informazione su dove guardare.
10. **Verifica automatica.** Scrivi un programma che controlli che `fattoriale(5)` sia `120` e stampi `PASS` o `FAIL`; poi rendi «sbagliata» la funzione e verifica che il verdetto cambi.

---

## 17. Domande da neofita

Le domande che seguono sono quelle che ogni persona pone nei primi mesi di programmazione — nessuna è sciocca, e ognuna ha una risposta secca. Se ti riconosci in una, sei nel posto giusto.

**Devo imparare tutto a memoria?** No. Nessun programmatore conosce a memoria un intero linguaggio: si ricordano i costrutti che si usano ogni giorno e si consulta il resto. Questo libro ha apposta una scheda rapida (Appendice B) e una grammatica: usarli non è «barare», è il modo normale di lavorare. La memoria che conta è un'altra: riconoscere i problemi e i modelli con cui si risolvono, e quella si costruisce solo scrivendo programmi.

**Perché il programma fa esattamente quello che scrivo e non quello che voglio?** Perché il computer non legge nella mente, legge il testo. Se hai scritto `+` dove intendevi `*`, o `=` dove intendevi `==`, il programma esegue fedelmente il tuo testo — e il risultato sembra "sbagliato" solo rispetto a ciò che avevi in mente. È la prima lezione del mestiere, e non è una tua mancanza: è la natura stessa del programmare.

**Quanto tempo ci vuole per imparare?** Per i fondamentali di questo libro bastano alcune settimane di esercizi regolari; per sentirsi a proprio agio servono mesi di pratica, come per qualsiasi mestiere. La variabile che conta non è il talento, è la costanza: un'ora al giorno batte dieci ore il sabato.

**Va bene sbagliare così tanto?** Gli errori non sono un incidente del lavoro: **sono** il lavoro. Anche un programmatore esperto passa gran parte del tempo a leggere messaggi d'errore e correggere. L'unica differenza è che a lui non spaventano più. Il capitolo 12 ti ha dato il metodo; il resto viene con la pratica.

**Dove scrivo il codice? Serve un programma speciale?** In qualsiasi **editor di testo**: il Blocco note va bene per cominciare, Visual Studio Code è meglio (colora il codice e ti avvisa degli errori). Non usare il programma di videoscrittura (Word e simili): salva formattazione invisibile che rovina il sorgente. Vedi Appendice A.

**Ho scritto il programma ma non parte: il computer dice che il comando non esiste.** Quasi sempre il terminale è aperto nella cartella sbagliata, il nome del file ha un refuso, oppure Node.js non è installato o non è nel `PATH`. La tabella diagnostica dell'Appendice A elenca sintomo per sintomo i casi più comuni.

**Come faccio a sapere cosa si può scrivere in Triad?** Tre riferimenti: la **scheda rapida** in Appendice B per ciò che hai imparato finora; la **grammatica** e le schede del Manuale per l'elenco completo; e il runtime stesso, che ti dice subito se una forma non è ammessa. Il confine tra Triad-Core (eseguibile oggi) e Triad-Full (definito dallo standard) è segnato nel libro dai marcatori **[Core]** e **[Full]**.

**Posso fare programmi utili adesso?** Sì, entro i limiti di Triad-Core, che è piccolo apposta: calcoli, conversioni, analisi di liste e di testi, piccoli algoritmi, giochi da terminale con argomenti passati da riga di comando (`ARGS`, vedi il Tutorial). I programmi "da vero mondo" — file, finestre, rete — richiedono librerie e parti del linguaggio che arriveranno con il completamento di Triad-Full e del bootstrap: nel frattempo, ciò che stai imparando è la parte che non cambia mai.

**Il mio programma gira all'infinito e non si ferma.** Hai un ciclo senza uscita: premi `Ctrl+C` sul terminale per interromperlo e rileggi il capitolo 8. Nove volte su dieci manca l'aggiornamento del contatore, o la condizione del `while` non può mai diventare falsa.

**Cosa sono i numeri e le scritte in inglese negli errori?** Sono i messaggi del runtime: il linguaggio li scrive in inglese perché è la convenzione dei linguaggi di programmazione. Il capitolo 12 traduce i più comuni; dopo un po' li riconoscerai a colpo d'occhio.

**Perché `42 + 1` funziona e `"42" + 1` no?** Perché il primo è un numero, il secondo un testo: sono tipi diversi e non si combinano con `+`. La spiegazione è nel capitolo 3, la conversione esplicita nel capitolo 6. È il tipo di errore che impara a evitare dopo una settimana — e a prevedere da subito.

**Devo sapere l'inglese?** Non per questo libro, che è in italiano. Aiuta, perché la documentazione tecnica e i messaggi dei linguaggi sono quasi sempre in inglese: ma si impara anche strada facendo, e i termini tecnici essenziali sono nel glossario del capitolo 15.

## Esercizi

1. **Ciclo infinito, apposta.** Scrivi un ciclo senza uscita, interrompilo con `Ctrl+C`, poi correggilo e riesegui: quale pezzo mancava?
2. **`+` tra testi.** Dimostra con un programma perché `"42" + 1` è un errore e scrivi la forma corretta che ottiene `43`.
3. **Con la scheda in mano.** Usando solo l'Appendice B, scrivi un programma che usi un ciclo, una lista e una funzione: quanti sguardi alla scheda ti sono serviti?
4. **`=` per `==`.** Scrivi apposta un programma con questo errore, eseguilo e riconosci il problema dal comportamento e dal messaggio.
5. **Accentate.** Salva un programma che stampi `perché` e `già`: se le accentate escono strane, il file non è in UTF-8 — risalva e controlla.

6. **Lambda dalla scheda.** Consulta B.8 e scrivi una lambda che raddoppi il suo argomento; assegnala a un nome e chiamala.
7. **Memoria.** Senza guardare, scrivi lo scheletro di un `if` e di un `while`; poi confronta con B.5 e B.6.
8. **Un programma utile.** Scrivi un piccolo programma che converta dei minuti in secondi e stampi la frase completa: è già un programma «da vero mondo» entro i limiti di Core.
9. **Errori in inglese.** Procurati tre messaggi d'errore diversi e traducili con la tabella del capitolo 12: quale regola del linguaggio viola ciascuno?
10. **Il tuo elenco.** Prendi spunto dalla domanda «quanto tempo ci vuole?» e scrivi l'elenco di cinque piccoli problemi che risolverai nella prossima settimana.

---

## 18. Come proseguire

Hai appena costruito il vocabolario di base: valori, variabili, espressioni, decisioni, ripetizioni, liste, funzioni, ricorsione, errori. Con questi concetti — e non servono altri per iniziare — il resto del libro diventa leggibile. Ecco come è organizzato e come ti conviene proseguire.

- Il **Manuale di Triad 2.0** spiega il linguaggio capitolo per capitolo: lessico, valori, controllo di flusso, funzioni, logica, errori, contratti, con esercizi a fine capitolo e riferimenti finali (grammatica, schede rapide, tabella degli errori, glossario, appendici). È il testo in cui approfondire ogni concetto che qui abbiamo solo sfiorato.
- Il **Tutorial di Triad 2.0** è il percorso pratico in cinque parti: primi passi (installare ed eseguire), Triad-Core passo per passo, Triad-Full (la parte completa dello standard), il codice

del progetto e un riferimento rapido. Le parti I e II sono il seguito naturale di queste *Fondamenta*.

Due cose è bene sapere prima di proseguire. La prima: Triad ha **due profili di conformità**. **Triad-Core** è il sottoinsieme del linguaggio che oggi è eseguibile — quello che abbiamo usato in questa parte. **Triad-Full** è lo standard completo, con tipi dichiarati, programmazione logica, macro, contratti e molto altro: definito e documentato, ma non ancora eseguibile dall'runtime corrente. Nel libro, i marcatori **[Core]** e **[Full]** dicono a quale profilo appartiene ogni costrutto, e i blocchi di codice sono etichettati `triad` (eseguibile oggi) o `triad-full` (definito dallo standard). Ogni programma Core è anche un programma Full valido.

La seconda: per eseguire gli esempi Core basta l'interprete di riferimento, il comando `node seed/dev/triad_core.cjs file.triad` che hai già usato. Niente altro da installare, niente altro da imparare per ora: la macchina è già pronta.

Il percorso che consigliamo a chi parte da zero è questo: **Fondamenta** (questa parte) → **Tutorial, parti I e II** (scrivere programmi Core veri) → **Manuale** (approfondire, con gli esercizi) → **Tutorial, parte III** e capitoli del Manuale su Triad-Full (lo standard completo) → **Tutorial, parte IV** (il codice del progetto, se vuoi vedere come il linguaggio è costruito).

Ultima raccomandazione, la più importante di tutto il libro: **scrivi codice**. La programmazione si impara programmando, come il pianoforte si impara suonando: leggere non basta. Prendi gli esempi, modificali, rompili e aggiustali; inventati i tuoi problemi piccoli e risolvili. Ogni programma che esegui con successo ti lascia qualcosa che nessuna pagina può darti.

Due appendici chiudono questa parte e restano tue compagne per il cammino: l'**Appendice A**, l'installazione guidata passo passo con la diagnosi dei guasti comuni, e l'**Appendice B**, la scheda rapida della

sintassi di tutto ciò che hai imparato, da tenere a portata di mano mentre scrivi i tuoi primi programmi.

Buon lavoro, e buon codice.

## Esercizi

1. **Mappa del volume.** In quale parte del libro trovi la spiegazione di `solve`? E degli errori come dati? E della grammatica?
2. **Core o Full?** Classifica questi costrutti: `while`, `fact`, `struct`, `lambda`, `for`, `macro`, `push!`, `solve`, `try`.
3. **Il primo esempio del Tutorial.** Scegli un programma della parte II del Tutorial, eseguilo e confronta l'output con quello del libro.
4. **Perché non gira?** Un esempio con `fact` non viene eseguito dal runtime attuale: spiega perché, usando la distinzione Core/Full.
5. **I marcatori.** Che cosa significano **[Core]** e **[Full]**? Trova tre esempi di ciascuno nel libro.
6. **I comandi.** Scrivi a memoria il comando per eseguire `ciao.triad` e quello per eseguire un caso di conformance; poi controlla.
7. **Il tuo percorso.** Scrivi l'ordine dei capitoli che studierai dopo le Fondamenta e perché.
8. **Verso il Manuale.** Scegli un capitolo del Manuale, leggilo e risolvi almeno tre dei suoi esercizi.
9. **Verso il Tutorial.** Esegui il programma del capitolo 3 del Tutorial passandogli due argomenti da riga di comando e osserva `ARGS`.

10. **Il programma finale.** Scrivi un programma che usi almeno quattro costrutti visti nelle Fondamenta (variabili, stringhe, `if`, `while`, liste o funzioni) e commentalo riga per riga.

---

# Appendice A — Installazione guidata passo passo

Questa appendice è per chi non ha mai installato uno strumento di programmazione: ogni passaggio ha un **controllo** da fare prima di proseguire, così se qualcosa non torna lo sai subito e non dopo venti passaggi. Al termine avrai sul computer tutto ciò che serve per eseguire ogni esempio di questo libro. Tempo stimato: un quarto d'ora.

## A.1 Che cosa ti serve

- un computer Windows, macOS o Linux, con una connessione internet;
- **Node.js**, il programma che esegue i programmi Triad (l'interprete);
- un **editor di testo** per scrivere i sorgenti (il Blocco note va bene per cominciare);
- il **repository del progetto**, cioè la cartella con gli esempi e l'interprete.

## A.2 Passo 1 — Installare Node.js

1. Apri il browser e vai su https://nodejs.org.
2. Scarica la versione **LTS** (il sito la indica come «consigliata»): il sito riconosce da solo il tuo sistema operativo.
3. Avvia l'installatore e prosegui con le impostazioni predefinite («Avanti», «Avanti», «Installa»). Su Linux segui le istruzioni del sito per la tua distribuzione.

4. **Chiudi e riapri il terminale**: un terminale aperto prima dell'installazione non vede i programmi nuovi.
5. Controlla, digitando nel terminale:

```
node --version
```

Devi vedere una riga come `v20.11.0`: il primo numero deve essere **18 o superiore**. Se il terminale risponde che il comando non esiste, vai alla tabella A.6.

## A.3 Passo 2 — Ottenere il repository del progetto

Ci sono due strade equivalenti; scegli quella che ti è più comoda.

- **Senza Git** (adatta a chi comincia): apri la pagina https://github.com/GPuntobit/triad2.0, premi il pulsante verde **Code** e scegli **Download ZIP**. Estrai l'archivio in una cartella di tua scelta, per esempio nei Documenti: ne nascerà una cartella chiamata `triad2.0`.
- **Con Git**, se lo conosci già:

```
git clone https://github.com/GPuntobit/triad2.0.git
```

Ricorda dove hai messo la cartella `triad2.0`: ti servirà al passo successivo.

## A.4 Passo 3 — Il terminale nella cartella giusta

Il terminale lavora sempre **in una cartella**: prima di eseguire un programma devi portarlo dove si trova il programma. Il comando è `cd` seguito dal percorso della cartella:

```
cd Documenti/triad2.0
```

Su Windows il percorso usa la barra rovesciata, per esempio:

```
cd C:\Users\tuo-nome\Documents\triad2.0
```

Controllo: digita `ls` (Windows: `dir`). Devi vedere elencate cartelle come `seed/`, `conformance/`, `docs/`, `cli/`. Se le vedi, sei nel posto giusto.

## A.5 Passo 4 — Il programma di prova

Scrivi il programma del capitolo 2 in un file chiamato `ciao.triad`, salvalo dentro la cartella `triad2.0`, e dal terminale esegui:

```
node seed/dev/triad_core.cjs ciao.triad
```

Atteso:

```
ciao, Triad
```

Se vedi `ciao, Triad`, l'installazione è completa e questo libro è tutto alla tua portata. Vuoi un secondo controllo, con un programma già pronto nel repository? Esegui:

```
node seed/dev/triad_core.cjs
conformance/core/cases/aritmetica/program.td
```

Atteso: una colonna di numeri che comincia con `14`, `20`, `1`, `1024`.

## A.6 Se qualcosa non funziona

Prima di pensare a un problema grave, guarda la tabella: i guasti di installazione sono pochi e sempre gli stessi.

| sintomo | causa probabile | rimedio |
|---|---|---|
| « `node` non è riconosciuto come comando» (Windows) o `command not found: node` | Node.js non installato, oppure il terminale era aperto prima dell'installazione | reinstalla Node.js da nodejs.org, poi **chiudi e riapri** il terminale |
| `Cannot find module ... triad_core.cjs` | il terminale è aperto nella cartella sbagliata | `cd` nella cartella `triad2.0` e ricontrolla con `ls` / `dir` (passo A.4) |
| `error: file non leggibile` o `ENOENT` con il nome del programma | il file non è dove pensi, o il nome è diverso | controlla nome ed estensione: deve essere `ciao.triad`, non `ciao.triad.txt` |
| il file si apre in Word, o con icone strane | salvato come documento formattato | salva di nuovo come **testo semplice**: «Salva con nome» → tipo «Tutti i file» → nome `ciao.triad` → codifica UTF-8 |
| il programma non si ferma più | ciclo senza uscita | premi `Ctrl+C` sul terminale; poi capitolo 8 e capitolo 17 |
| nell'output le accentate appaiono strane | il file non è salvato in UTF-8 | salva in codifica UTF-8 (l'editor la chiama «UTF-8» o «Unicode») |
| il programma risponde `error: …` o `parse error: …` | non è un guasto di installazione: è il programma che segnala un problema | capitolo 12 |

### A.7 Windows, macOS e Linux: le differenze

Sono poche e nessuna cambia il programma: i percorsi usano `\` su Windows e `/` altrove; per elencare i file si digita `dir` su Windows e `ls` su macOS e Linux; il resto — editor, comandi `node`, testo dei programmi — è identico. Dove servono entrambe le forme, questo libro mostra quella Linux/macOS: su Windows cambia solo la barra.

---

## Appendice B — Scheda rapida della sintassi (Triad-Core)

Questa è la pagina da tenere aperta mentre scrivi: tutto ciò che le Fondamenta insegnano, in ordine di uso, con la forma esatta da digitare. Solo **Triad-Core**, cioè ciò che oggi è eseguibile; il resto del linguaggio (Triad-Full) è nel Manuale e nella parte III del Tutorial.

### B.1 Il minimo indispensabile

| cosa | forma | note |
|---|---|---|
| commento di riga | `# fino a fine riga` | ignorato dall'interprete |
| commento lungo | `#= … =#` | non si annida |
| fine istruzione | a capo oppure `;` | uno spazio non basta |
| nome del file | `programma.triad` | alias storico: `.td` |
| eseguire | `node seed/dev/triad_core.cjs programma.triad` | da terminale, nella cartella giusta |

## B.2 Valori e tipi

| tipo | esempi | `typeof(v)` |
|---|---|---|
| `Int` | `42`, `0`, `-7`, `1_000` | `:Int` |
| `String` | `"ciao"`, `"42"`, `""` | `:String` |
| `Bool` | `true`, `false` | `:Bool` |
| `Symbol` | `:ok`, `:alice` | `:Symbol` |
| `List` | `[1, 2, 3]`, `[]` | `:List` |
| `Nothing` | `nothing` | `:Nothing` |

## B.3 Operatori

| gruppo | operatori | esempi |
|---|---|---|
| aritmetica | `+ - * % ^` | `7 % 3` → `1`; `2 ^ 10` → `1024` |
| confronto | `== != < <= > >=` | `2 == 2` → `true` |
| logica | `&& \|\| !` | `true && false` → `false` |
| assegnamento | `= += -= *=` | `n += 1` |
| scelta compatta | `cond ? a : b` | `x > 2 ? :grande : :piccolo` |

Attenzione: `=` **assegna**, `==` **confronta**. In Core non esistono `/` né i numeri decimali: per dividere si usa il resto (`%`) o si aspetta Triad-Full. Quando hai dubbi sul raggruppamento, usa le parentesi.

## B.4 Stampa e stringhe

| cosa | forma | esempio |
|---|---|---|
| stampare e andare a capo | `println(a, b)` | `println("ciao, ", "mondo")` |
| stampare senza a capo | `print(a)` | poi `println()` per chiudere la riga |
| unire stringhe | `a * b` | `"ciao " * "Ada"` |
| escape | `\n \t \" \\` | `"riga1\nriga2"` |
| lunghezza in byte | `length(s)` | `length("ciao")` → `4` |
| da valore a testo | `string(x)` | `string(40 + 2)` → `"42"` |
| da testo a intero | `Int(s)` | `Int("41") + 1` → `42` |

## B.5 Decisioni

```
if condizione
    …
elseif altra_condizione
    …
else
    …
end
```

La condizione deve essere un `Bool`; ogni `if` si chiude con `end`; il ramo corretto è `elseif`, non `else if`.

## B.6 Cicli

```
i = 1
while i <= n
    …            # corpo
    i += 1       # avanzamento: non dimenticarlo!
end
```

`break` esce dal ciclo, `continue` passa alla girata successiva (dopo aver aggiornato il contatore!).

## B.7 Liste

| cosa | forma | nota |
|---|---|---|
| creare | `xs = [1, 2, 3]` | anche miste e annidate |
| leggere | `xs[1]` | gli indici partono **da 1** |
| scrivere | `xs[i] = v` | anche `xs[i] += v` |
| contare | `length(xs)` | numero di elementi |
| aggiungere in coda | `push!(xs, v)` | modifica la lista ( `!` ) |
| giro completo | `while i <= length(xs)` … | contatore da 1 |

## B.8 Funzioni

| forma | esempio | quando |
|---|---|---|
| breve | `quadrato(x) = x * x` | corpo di una sola espressione |
| blocco | `function saluta(nome)` … `end` | più istruzioni; il valore è l'ultima espressione |
| anonima | `function (a, b)` … `end` | da passare o assegnare a un nome |
| lambda | `x -> x * 2` | forma compatta dell'anonima |
| uscita anticipata | `return espr` | termina la funzione subito |

Regole: gli argomenti devono essere **esattamente** quanto i parametri; le variabili nate nella funzione esistono solo lì dentro; una funzione si può chiamare solo dopo la sua riga di definizione.

## B.9 I messaggi d'errore, in pillole

| messaggio | significato |
|---|---|
| `parse error: …` | il programma non si legge: sintassi (spesso un `end` mancante) |
| `error: undefined variable x` | `x` non è mai stata assegnata (refuso?) |
| `error: arity mismatch` | numero di argomenti sbagliato nella chiamata |
| `error: + type error` | operazione tra tipi incompatibili ( `"ciao" + 1` ) |
| `error: index out of range` | posizione fuori dalla lista |

| messaggio | significato |
|---|---|
| `error: if cond` | la condizione non è un `Bool` |
| `error: not a function` | hai «chiamato» qualcosa che non è una funzione |

## B.10 Che cosa NON c'è in Triad-Core

Numeri decimali (`1.5`), la divisione `/`, il ciclo `for` e i range, l'interpolazione `"$x"`, funzioni di lista come `map` / `filter` / `sort`, la lettura da tastiera. Sono parti di **Triad-Full** (studiato nel Manuale e nella parte III del Tutorial) o di librerie che arriveranno: se li cerchi e non li trovi, non è un tuo errore.

# Manuale di Triad 2.0

> *Il manuale completo del linguaggio: lessico, valori, controllo di flusso, funzioni, dispatch, logica, codice come dato, errori, contratti, budget, tracce, effetti, esercizi e riferimento.*

Triad è un linguaggio di programmazione autonomo: sintassi a blocchi e tipi dichiarati, multiple dispatch, programmazione logica integrata e codice come dato, in una sola sintassi. La sua definizione è una specifica normativa indipendente da qualsiasi implementazione (`SPEC.md`, standard `0.2.0`), accompagnata da una specifica eseguibile (`SPEC-CORE.md`) e da una suite di conformità con golden verificabili (`conformance/`).

Questo manuale insegna il linguaggio per come è definito dallo standard. Non è un riassunto della specifica: è un percorso di studio, con spiegazioni, esempi ed esercizi. Puoi leggerlo dall'inizio alla fine senza altro materiale: ogni concetto che serve per capire un capitolo è spiegato prima o in quel capitolo, e i rimandi esterni sono approfondimenti facoltativi. Non è una raccolta di pagine da consultare a caso: è un libro, e l'ordine dei capitoli costruisce il modello mentale del linguaggio un pezzo alla volta.

---

# Come leggere questo manuale

## A chi è indirizzato

Sai programmare, in qualunque linguaggio: sai cosa sono una variabile, una funzione, un ciclo, una ricorsione. Questo manuale non spiega la programmazione in generale, spiega **Triad**. Se non hai mai programmato, parti dalle *Fondamenta: programmare da zero con Triad*, la parte del volume che insegna la programmazione da capo e costruisce i concetti che qui si danno per noti.

Il manuale non presume che tu abbia letto nient'altro: ogni concetto è spiegato dove serve. I documenti normativi del progetto — `SPEC.md`, `SPEC-CORE.md` — e la suite di conformità in `conformance/` sono i testi e le prove a cui puoi risalire per approfondire o per controllare un caso limite, ma non sono prerequisiti: sono il riferimento di chi vuole verificare, non il manuale di chi vuole imparare. Per questo il testo distingue sempre ciò che è regola del linguaggio da ciò che è comportamento di una particolare implementazione.

## Due profili, una sola lingua

Lo standard definisce due profili di conformità, ed è utile saperli distinguere prima di cominciare:

| profilo | che cosa contiene | dove è definito |
|---|---|---|
| **Triad-Core** | valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O | `SPEC-CORE.md` |
| **Triad-Full** | tutto: tipi dichiarati, dispatch, logica, macro, contratti, budget, tracce, effetti | `SPEC.md` |

Triad-Core è un **sottoinsieme** di Triad-Full: ogni programma Core è anche un programma Full valido e deve comportarsi allo stesso modo. Il manuale presenta i due profili **sullo stesso piano**, per costruzione logica: ogni capitolo introduce un costrutto e dice in quale profilo esiste. Non perché Full sia "più avanti", ma perché sono due facce della stessa specifica.

Il profilo Core non è una versione semplificata per comodità didattica: è il sottoinsieme scelto perché un interprete di bootstrap, scritto in poche migliaia di righe, possa eseguirlo, e perché lo stesso linguaggio possa descrivere se stesso. Full aggiunge i costrutti che richiedono un motore più ricco — dispatch multiplo, unificazione, espansione di macro — ma non cambia il significato di quelli che Core ha già. Imparare Core, quindi, non è tempo speso per un linguaggio diverso.

Dove la differenza conta, trovi due marcatori:

- **[Core]** — il costrutto esiste nel sottoinsieme di bootstrap;
- **[Full]** — il costrutto esiste solo nello standard completo.

Un marcatore di profilo descrive ciò che lo standard definisce, non ciò che ogni programma installato sa già eseguire. Per questo il manuale distingue tre livelli: la regola normativa (che cosa deve significare il costrutto), la disponibilità nel profilo Core o Full e lo stato delle implementazioni presenti nel repository. Un esempio Full può essere corretto secondo lo standard e, allo stesso tempo, non essere ancora eseguibile dal runtime corrente. Quando un esempio riporta **Output (golden Full)**, il risultato è un valore atteso conservato dalla suite, non una verifica appena eseguita da un interprete Full completo.

Per studiare un costrutto senza confondere questi livelli, chiediti nell'ordine: che forma accetta la grammatica? quali valori legge e produce? quali effetti o errori può generare? in quale profilo è definito? quale implementazione e quale caso di conformance lo verificano?

Questa sequenza permette di passare dal codice alla semantica e poi alle prove, invece di trattare l'output di un singolo runtime come definizione del linguaggio.

## Convenzioni di notazione

| notazione | significa |
|---|---|
| `monospace` | codice Triad o un valore letterale |
| **Output:** | l'output esatto prodotto dal programma |
| **[Core]** / **[Full]** | il profilo in cui il costrutto esiste |
| `SPEC §n` | la sezione normativa corrispondente |

I blocchi di codice sono marcati con il loro livello di eseguibilità:

| fence | significa |
|---|---|
| ```` ```triad ```` | programma **eseguibile oggi** con l'interprete di riferimento |
| ```` ```triad-full ```` | programma **specifica dello standard**: definito e normativo, ma non ancora eseguibile dal runtime corrente |

## Come eseguire gli esempi

Gli esempi marcati `triad` si eseguono con l'interprete di riferimento JavaScript:

```
node seed/dev/triad_core.cjs programma.triad
```

Il programma riceve gli argomenti in `ARGS` (lista di stringhe, senza il nome del programma) e lo shebang `#!` in prima riga viene ignorato dal parser.

```
#!/usr/bin/env triad
# saluto.triad
println("ciao, " * "Triad")
println(6 * 7)
```

```
node seed/dev/triad_core.cjs saluto.triad uno due
```

**Output:**

```
ciao, Triad
42
```

L'esempio mostra tre cose che valgono in tutto il libro. Lo shebang è solo una comodità dei sistemi operativi: il parser lo salta, quindi il programma comincia davvero dalla seconda riga. `println` non inserisce separatori tra gli argomenti, quindi la virgola e lo spazio che vedi nell'output sono caratteri scritti nel programma, non una formattazione automatica. Infine `6 * 7` viene calcolato prima della stampa: gli argomenti di una chiamata sono espressioni, non testo. Gli argomenti passati da riga di comando (`uno`, `due`) non diventano variabili con quei nomi: finiscono tutti nella lista `ARGS`, come spiegato nel capitolo 20.

# Parte I — Il linguaggio

La prima parte costruisce il linguaggio strato per strato: prima i mattoni (lessico e valori), poi le espressioni e il controllo di flusso, quindi le funzioni e i modelli di calcolo che Triad aggiunge — dispatch, chiusure, logica, codice come dato — e infine gli strumenti con cui un programma

comunica con l'esterno (errori, contratti, budget, tracce, effetti). Ogni capitolo si appoggia ai precedenti; gli esempi sono brevi apposta, perché il punto non è ammirare un programma ma capire una regola.

# 1. Lessico

## 1.1 Identificatori

Un identificatore comincia con una lettera o `_`, seguito da lettere, cifre o `_`. Un `!` finale è ammesso se non è seguito da `=`. **[Core]**

```
x = 1
somma_2 = 2
_ = 3
nome_valido = 4
```

`push!`, `sort!` e simili sono nomi legittimi: la convenzione del `!` finale è per le funzioni che mutano il loro argomento (cap. 9).

Il `!` non cambia il significato del nome e non è un operatore: è un segnale per chi legge. Il parser lo accetta solo come ultimo carattere, e solo se il carattere seguente non è `=`; altrimenti comincerebbe un confronto. Questa è la prima delle convenzioni che Triad eredita da Julia: il nome dichiara un contratto, ma è il programmatore a doverlo rispettare.

In Triad-Full gli identificatori ammettono anche lettere non ASCII. **[Full]**

```
età = 42
```

## 1.2 Numeri

Gli interi si scrivono con cifre decimali; `_` funziona come separatore. Il segno è un operatore unario, non parte del letterale. **[Core]**

La distinzione è più importante di quanto sembri: `-5` è l'applicazione dell'operatore `-` al letterale `5`, non un numero negativo scritto in un modo speciale. Per questo non esiste una "sintassi negativa" da ricordare: le parentesi e i livelli di precedenza del capitolo 4 spiegano già che cosa succede in un'espressione come `-x * y`.

```
println(42)
println(1_000_000)
println(-5)
```

**Output:**

```
42
1000000
-5
```

> ***Nota di implementazione.*** *Il seed in codice macchina legge solo cifre: `_` non è per lui un separatore (`1_000` termina con codice 1) e i letterali maggiori di 2^61-1 vengono corrotti durante il parsing (misurato: `println(9223372036854775807)` stampa `-1`), perché il seed codifica gli interi con due bit di tag. L'interprete di riferimento accetta `_` e l'intero intervallo a 64 bit con wrap. Dati misurati: `docs/libro/_dati_dal_codice.md`, `TASK.md`.*

`Float` esiste solo in Triad-Full. **[Full]**

```
println(3.14)
println(1e-9)
```

### 1.3 Stringhe

Una stringa è una sequenza di byte UTF-8 **immutabile**, tra apici doppi. Gli escape sono `\n`, `\t`, `\"`, `\\`. **[Core]**

```
println("ciao\n")
println("tab:\t|")
println("virgolette: \"")
println("backslash: \\")
```

**Output:**

```
ciao

tab:	|
virgolette: "
backslash: \
```

In Core **non esiste l'interpolazione**: `"x = $x"` stampa letteralmente `x = $x`. In Full esiste, e si scrive `$x` per un'espressione semplice e `$(espr)` per una complessa. **[Full]**

```
x = 42
println("x = $x")
println("x + 1 = $(x + 1)")
```

## 1.4 Simboli

Un simbolo è `:` seguito da un nome o da un operatore: `:alice`, `:+`. Due simboli sono uguali se hanno lo stesso nome. In Triad-Full il simbolo è anche l'atomo della logica (cap. 16). **[Core]**

Un simbolo serve a dare un nome stabile a una cosa — una categoria, una chiave, un atomo — senza la flessibilità e il costo di una stringa. La differenza rispetto a una stringa non è solo di comodità: `:alice` è un'identità, `"alice"` è una sequenza di byte; due simboli con lo stesso nome sono lo stesso valore ovunque compaiano, mentre due stringhe vanno confrontate carattere per carattere. È questa proprietà che rende i simboli adatti alle chiavi dei record (cap. 2), ai tag delle liste e agli atomi dei fatti (cap. 16).

```
println(:alice)
println(:+)
println(:ok == Symbol("ok"))
```

**Output:**

```
alice
+
true
```

## 1.5 Commenti

`#` commenta fino a fine riga. `#= … =#` commenta su più righe; lo standard descrive questa forma ma **non** stabilisce che i commenti a blocco si annidino, e l'interprete di riferimento chiude al primo `=#`: non contare sull'annidamento. **[Core]**

```
#= commento
   su più righe =#
x = 1_000   # commento di riga
println(x)
```

**Output:**

```
1000
```

### 1.6 Separatori e fine-riga

Un'istruzione termina con un a capo o con `;`. Gli a capo **chiudono** un'istruzione, tranne dentro `( )`, `[ ]` e `:( )`, e tranne subito dopo un operatore binario o una virgola di `rule`.

```
println(1); println(2)
```

Una conseguenza pratica: **in Triad-Core ogni a capo chiude l'istruzione**. La grammatica di Core non ha una regola di continuazione di riga: un'espressione spezzata su più righe non è un programma valido. Scrivi **una istruzione per riga** e usa `;` per separare due istruzioni.

```
totale = 1 + 2 + 3
println(totale);
println("seconda istruzione")
```

**Output:**

```
6
seconda istruzione
```

La regola ha una conseguenza sulla formattazione: quando vuoi spezzare un'espressione lunga, le parentesi non bastano in Core. L'interprete di riferimento chiude l'istruzione al primo a capo anche dentro una chiamata, quindi `println(1 +` e `2)` su righe diverse non sono un programma valido oggi. Per il codice destinato al runtime attuale vale una regola sola: una istruzione per riga, e `;` solo quando due istruzioni devono stare sulla stessa riga.

> ***In Triad-Full*** *la regola è più permissiva:* `SPEC §1` *stabilisce che un a capo* ***non*** *chiude l'istruzione dentro* `( )`*,* `[ ]` *e* `:( )`*, né subito dopo un operatore binario. L'interprete di riferimento attuale non implementa ancora questa eccezione, quindi anche in Full la forma sicura resta una istruzione per riga.*

## 1.7 La regola dei due punti e del punto interrogativo

`:` ha tre significati e `?` ne ha due. La specifica li chiama esplicitamente «il problema classico di `:`» — è il primo punto in cui Triad si discosta dalla sintassi che conosci.

| scrittura | significato |
|---|---|
| `1:n`, `xs[2:end]` | `:` attaccato a un valore e senza spazi → **range** |
| `f(:a)`, `x = :a` | `:` non preceduto da un valore e seguito da un nome → **simbolo** |
| `:( … )` | → **codice quotato** |
| `c ? a : b` | `:` circondato da spazi → **separatore del ternario** |

| scrittura | significato |
|---|---|
| `?x` | `?` attaccato → **variabile logica** |

| scrittura | significato |
|---|---|
| `?` | `?` con spazio dopo → **ternario** |

Da qui la regola pratica: quando `:` è ambiguo, **metti o togli gli spazi**. `xs[2:end]` è un range; `c ? a : b` è un ternario perché i due spazi lo circondano.

Le due tabelle sembrano un elenco di casi speciali, ma discendono da un'unica idea: un carattere aderente al token precedente appartiene a quel token, uno separato appartiene alla sintassi. Per questo in Triad la spaziatura non è questione di stile: è informazione che il parser legge. Se scrivi `xs[2: end]` oppure `a ? b:c`, cambiando gli spazi non cambi l'aspetto del programma: cambi che cosa il parser riconosce, e il più delle volte ottieni un errore di sintassi. Il consiglio è di scrivere sempre gli spazi che rendono esplicito il significato.

## 1.8 Parole chiave

Lo standard definisce **29 parole chiave**:

```
function struct macro end if elseif else for in while
let begin return
break continue fact rule solve quote try catch with
requires ensures
true false nothing isa is
```

E **4 parole contestuali**, valide solo dopo `solve`:

```
limit depth steps trace
```

`true`, `false` e `nothing` sono valori, non parole strutturali.

Le parole chiave sono riservate perché il parser deve riconoscerle prima di sapere che cosa stanno facendo: se `if` potesse essere anche un nome di variabile, la lettura di una riga dipenderebbe dal contesto in modo imprevedibile. `true`, `false` e `nothing` fanno eccezione solo in apparenza: sono valori che il linguaggio predefinisce, non forme sintattiche. Le quattro parole contestuali mostrano lo stesso principio al contrario: `limit`, `depth`, `steps` e `trace` sono nomi normalissimi finché non seguono `solve`, e solo lì hanno un significato speciale. Questa distinzione riduce il numero di parole riservate e ti permette di usare nomi naturali nel resto del programma.

## 1.9 Identificatori avanzati e convenzioni

In Triad-Core gli identificatori sono limitati all'ASCII: lettere, cifre e `_`. In Triad-Full si ammettono anche lettere non ASCII (accentate, greche, cirilliche): `età`, `π`, `Δx` sono nomi validi. **[Full]**

```
età = 42
π = 3.14159
Δx = 0.001
println(età + π)
```

La convenzione del `!` finale (`push!`, `sort!`) indica funzioni che **mutano** il loro argomento. È l'unico carattere speciale ammesso dentro un identificatore: la grammatica accetta lettere, cifre, `_` e il `!` finale, e nulla di più. In particolare il `?` **non** fa parte del nome, perché in Triad introduce una variabile logica o un ternario: `valido?` non è un identificatore valido. Per un predicato usa un nome normale e lascia che sia il tipo `Bool` del risultato a dire che cos'è — `vuoto`, `is_valido`. **[Core]**

```
function push!(xs, v)
    # muta xs e lo restituisce
    xs
end

function vuoto(xs)
    length(xs) == 0
end
```

Il corpo di `push!` qui è volutamente vuoto: l'esempio parla dei nomi, non dell'implementazione. La mutazione vera delle liste, con `push!` e con l'assegnamento a un elemento, è nel capitolo 9.

Gli identificatori che coincidono con una parola chiave non sono ammessi: `function`, `end`, `if` non possono essere usati come nomi di variabile o funzione. **[Core]**

## 1.10 Escape e stringhe avanzate

Lo standard fissa quattro escape nelle stringhe: `\n`, `\t`, `\"`, `\\`; in Triad-Full si aggiunge `\$`, perché il `$` introduce l'interpolazione e va neutralizzato quando deve restare un carattere normale. Le forme estese `\r`, `\0`, `\u{…}` e `\x{…}` che compaiono nel frammento seguente non sono elencate in `SPEC §1`: trattale come estensioni di una singola implementazione, non come garanzia del linguaggio. Un programma portabile usa le escape documentate. **[Full]**

> ***Nota di implementazione.*** *Sul seed in codice macchina le stringhe sono byte grezzi copiati fino alla virgolette di chiusura: le escape non vengono processate. Misurato:* `println("a\nb")` *stampa* `a\nb` *letterale su una riga e* `length("a\nb")` *vale* `4`*;*

> *sull'interprete di riferimento stampa due righe e `length` vale `3`. Anche un newline reale dentro una stringa è accettato da entrambi i runtime attuali.*

```
println("a\r\nb")          # ritorno a capo CRLF
println("unicode: \u{1F600}")  # emoji
println("hex: \x{41}")     # 'A'
```

Una stringa è una sequenza di byte UTF-8 e le stringhe sono **immutabili**: ogni operazione di concatenazione crea una nuova stringa, come mostra l'esempio qui sotto. La lunghezza si misura in byte e non in caratteri, un dettaglio che conta appena esci dall'alfabeto latino (cap. 10). **[Core]**

```
a = "ciao"
b = a * " mondo"
println(a)        # "ciao" — inalterato
println(b)        # "ciao mondo"
```

**Output:**

```
ciao
ciao mondo
```

## 1.11 Esercizi sul lessico

1. Scrivi un programma che stampi il tuo nome e la tua età su due righe, usando `\n` e concatenazione di stringhe.
2. Definisci un identificatore con `_` iniziale e uno con `!` finale; stampali.

3. Prova a scrivere `x = 1_000_000` e verifica che l'output sia `1000000`.
4. (Full) Scrivi una stringa con interpolazione `$x` e `$(x + 1)`.
5. (Full) Definisci il simbolo `:ok` e confrontalo con `Symbol("ok")`.
6. Scrivi un programma con un commento `#` e un commento lungo `#= ... =#`: che cosa cambia nell'output? E nel sorgente?
7. Scrivi tre istruzioni su una sola riga separate da `;`: il programma funziona ancora?
8. Quale di questi è un identificatore valido? `_x`, `x!`, `x-y`, `2x`, `x2`, `!x`, `x y`. Prova a usarli davvero, non solo a guardarli.
9. Scrivi una stringa che contenga tutti e quattro gli escape Core (`\n`, `\t`, `\"`, `\\`) e stampala: l'output corrisponde a ciò che avevi in mente?
10. Quali di queste righe vengono accettate? `x = 1_000_000`, `y = 1__0`, `z = 1.5` (l'ultimo caso è Full: `Float`).
11. Prova a innestare due commenti lunghi (`#= ... #= ... =# ... =#`): quale `=#` chiude il commento?
12. Scrivi una chiamata a `println` spezzata su due righe: che cosa succede e perché?

---

# 2. Valori e tipi

## 2.1 I tipi

Un valore Triad appartiene sempre a un tipo, e il tipo si può sempre osservare: la tabella elenca i tipi dello standard, con il nome che `typeof` restituisce e il profilo in cui esistono.

| tipo | valori | `typeof(v)` |
|---|---|---|
| `Int` | interi con segno a 64 bit, aritmetica con wrap | `Int` |
| `Float` | numeri in virgola mobile **[Full]** | `Float` |
| `Bool` | `true`, `false` | `Bool` |
| `String` | sequenza di byte UTF-8 immutabile | `String` |
| `Symbol` | `:nome` | `Symbol` |
| `List` | lista eterogenea mutabile, indici da 1 | `List` |
| `Function` | funzione nominata, lambda, builtin | `Function` |
| `Nothing` | `nothing` | `Nothing` |
| `Expr` | codice quotato **[Full]** | `Expr` |
| `Term`, `LVar` | termini e variabili logiche **[Full]** | — |
| `Solution` | `{who = :dave}` **[Full]** | — |
| `Err` | errore come dato **[Full]** | — |
| `Type` | i tipi stessi sono valori **[Full]** | — |
| struct utente | `Point(3.0, 4.0)` **[Full]** | `Point` |

Il set dei tipi Core è **chiuso e piccolo**: `Int`, `Bool`, `String`, `Symbol`, `List`, `Function`, `Nothing`. In Core non esistono `Float`, `Term`, `Solution`, `Err` come valori del linguaggio: un errore è una lista taggata `[:Err, kind, msg]` (cap. 11) e una struct è `[:Point, x, y]`. Il set chiuso non è una limitazione arbitraria: un interprete di

bootstrap deve poter rappresentare ogni valore con poche strutture, e i concetti Full che mancano — errore, struct, soluzione — si ricostruiscono per convenzione sopra liste e simboli.

`typeof` restituisce il **nome del tipo come simbolo**. Il dettaglio del profilo conta: in Core il risultato è un `Symbol`, quindi `typeof(1) == :Int` è il confronto naturale; in Triad-Full i tipi sono valori di tipo `Type`, per cui `typeof(3) == Int` confronta direttamente con il tipo. La stampa è la stessa ( `Int` ), ma l'operazione che scrivi è diversa — ed è per questo che la tabella qui sopra ha una colonna `typeof(v)` con il nome e non con il tipo.

È utile separare tre idee che nella sintassi possono sembrare la stessa cosa. Il **valore** è il dato concreto, per esempio `5` o `"ciao"`; il **tipo dinamico** descrive la categoria del valore che il programma sta manipolando; un **vincolo dichiarato** restringe invece quali valori può assumere un nome o un parametro. In Core il tipo si osserva con `typeof`, ma non si dichiara una variabile come `x::Int`. In Full i tipi entrano anche nelle firme, nelle struct e nel dispatch. Una conversione come `Float(3)` costruisce un valore compatibile; un'asserzione come `x::Int` controlla un valore già ottenuto. Sono operazioni diverse: convertire può cambiare il valore, asserire non è una richiesta di conversione.

Un'altra distinzione importante è tra tipo **nominale** e forma strutturale. In Full una `struct Point` dichiara un'identità di tipo, quindi un valore `Point` non diventa un altro tipo solo perché possiede campi simili. In Core, dove non ci sono struct, la stessa informazione è convenzionale: una lista come `[:Point, 3, 4]` porta il tag `:Point` e i campi in posizioni concordate dal programma. Il tag rende possibile riconoscere il record, ma non aggiunge da solo controlli di campo, immutabilità o dispatch.

```
xs = [1, :a, "s"]
println(typeof(1), " ", typeof("s"), " ", typeof(:a), "
", typeof(xs))
println(typeof(nothing), " ", typeof(true), " ",
typeof(x -> x))
```

**Output:**

```
Int String Symbol List
Nothing Bool Function
```

## 2.2 Gerarchia dei tipi

`Int <: Number`, `Float <: Number`, e ogni tipo è sottotipo di `Any`. `Any` è il supertipo usato nelle firme quando un parametro accetta qualunque cosa. **[Full]**

La gerarchia esiste per il dispatch (cap. 7): un metodo dichiarato su `Number` accetta anche gli `Int`, perché `Int` è un sottotipo di `Number`. In Core la gerarchia non si può osservare, perché non c'è dispatch multiplo, ma resta il motivo per cui i tipi hanno un nome e non sono solo etichette di comodo: la relazione fra i nomi è ciò che permette di scrivere un metodo una volta e di applicarlo a una famiglia di valori.

```
function accetta_qualsiasi(x::Any)
    "qualunque cosa: " * string(x)
end
```

## 2.3 Struct

Una `struct` è un record immutabile con campi tipizzati e costruttore posizionale. Il costruttore controlla i campi; un `Int` passato a un campo `Float` viene convertito, un valore non convertibile produce `Err(:type)`. **[Full]**

```
struct Point
    x::Float
    y::Float
end

p = Point(3, 4)
println(p.x, " ", p.y)
println(typeof(p) == Point)
println(p isa Point)
```

**Output (golden Full):**

```
3.0 4.0
true
true
```

In Core lo stesso concetto si rappresenta con una **lista taggata**: `[:Point, x, y]`, con il simbolo del tipo in prima posizione. È così che gli interpreti di `selfhost/` rappresentano le struct Full. La differenza tra il record Full e quello Core non è solo di scrittura: la struct ha un'identità di tipo e campi immutabili, il record Core è una lista, quindi può essere mutata e nessuno impedisce di costruirne una con un tag sbagliato. Il tag rende il dato riconoscibile, non lo rende sicuro: i controlli che una struct fa da sola, in Core li deve fare il programma.

```
punto(x, y) = [:Point, x, y]
px(p) = p[2]
py(p) = p[3]
println(punto(1, 2), " ", px(punto(1, 2)))
```

**Output:**

```
[:Point, 1, 2] 1
```

## 2.4 Conversioni

`Int(x)`, `Float(x)`, `String(x)`, `Symbol(x)`. Il nome del tipo usato come funzione converte; il nome di una struct usato come funzione **costruisce**. **[Full]**

La conversione è un'operazione esplicita: Triad non trasforma i tipi da solo, tranne nei punti in cui la specifica lo prescrive — un `Int` passato a un campo `Float` di una struct, o il valore di ritorno controllato con `::R`. Altrove la conversione la chiedi tu, e questo rende leggibile il punto in cui il tipo cambia. `string(x)`, che vedi nella tabella del capitolo 10, è un caso a parte: è definita per ogni valore e serve a costruire testo, per questo nei programmi Core è il modo normale di convertire un numero in stringa.

```
println(Int("42") + 1)
println(Float(3))
println(Symbol("ciao"))
println(String(42))
```

`Int(s)` e `Symbol(s)` esistono anche in Core, ma solo per stringhe e simboli. **[Core]**

```
println(Int("41") + 1)
println(Symbol("bob"))
```

**Output:**

```
42
bob
```

## 2.5 Asserzione di tipo

`e::T` restituisce `e` se è di tipo `T`, altrimenti produce `Err(:type)`. **[Full]**

```
x = 3
println(x::Int)
println(iserr(x::String))
```

## 2.6 Il vincolo del dichiarato

`x::T = e` — e un parametro `x::T` — **fissa** il tipo di quel nome nell'ambiente in cui nasce. Gli assegnamenti successivi sono controllati e convertiti contro `T`. Una nuova dichiarazione `x::U = …` sostituisce il vincolo; un nome non dichiarato con tipo resta libero; una rilegatura in un ambiente interno (`let x = 2.5`) non eredita il vincolo esterno. **[Full]**

```
x::Int = 5
x = 6            # bene
x = 5.5          # Err(:type)

y::Float = 1.0
y = 2            # 2.0: conversione Int -> Float
```

```
s = "a"
s = 3            # bene: s non è dichiarato con un tipo
```

Il vincolo vale per l'ambiente in cui il nome nasce, non per sempre: è un'annotazione locale, non una dichiarazione globale. Questo permette di introdurre i tipi in un programma Core già scritto senza riscriverne la struttura, e spiega perché `let x = 2.5` dentro una funzione non è un errore anche quando fuori `x` era vincolato a `Int`. Il controllo avviene a ogni assegnamento, quindi il costo è un controllo in più per ogni scrittura; in cambio, un valore del tipo sbagliato non entra mai nel nome, e l'errore arriva dove il tipo è stato promesso, non molto più tardi.

## 2.7 Le condizioni accettano solo `Bool`

Non esiste il concetto di «verità implicita»: un valore non diventa vero o falso per convenzione. `if 1` è un errore, non `true`. Questo vale per `if`, `while`, `&&`, `||`, `!`, il ternario e i filtri delle comprehension. **[Core]**

```
println(1 < 2 && 2 < 3)
x = 3
println(x > 2 ? "grande" : "piccolo")
```

**Output:**

```
true
grande
```

La scelta è deliberata. In un linguaggio con tipi dinamici e con gli errori come valori, l'unica cosa che una condizione deve fare è produrre `true` o `false`; ammettere valori "veri" per convenzione (`0`, `""`, `[]`)

renderebbe ambiguo il significato di un programma e sposterebbe l'errore lontano dalla sua causa. Con la regola stretta, `if xs` è un errore immediato e dice che lì serve un confronto — per esempio `if length(xs) > 0`. Lo stesso vale per `&&` e `||`: entrambi gli operandi che vengono valutati devono essere `Bool`, anche se il cortocircuito ne risparmia uno.

> ***Nota di implementazione.*** *La regola oggi è violata in silenzio dal seed in codice macchina: con una condizione non* `Bool`*,* `if` *prende il ramo* `else` *e* `while` *salta il corpo, senza errori (misurato). L'interprete di riferimento termina invece con* `error: if cond` */* `while cond`*. Non usare il comportamento del seed come semantica: è una lacuna da colmare (* `TASK.md` *).*

## 2.8 Come stampa un valore

`print` e `println` concatenano gli argomenti **senza separatori**; `println` aggiunge un a capo.

| valore | come viene stampato |
|---|---|
| `Int` | decimale |
| `Bool` | `true` / `false` |
| `String` | così com'è |
| `Nothing` | `nothing` |
| `Symbol` | **senza** `:` se passato direttamente, **con** `:` dentro una lista |
| `List` | `[a, b]`; le stringhe dentro non hanno virgolette |
| `Function` | `#lambda(x)` |

```
xs = [1, :a, "s", true, nothing, [2, 3]]
println(xs)
println(:alice)
print("senza ", "a capo")
println()
```

**Output:**

```
[1, :a, s, true, nothing, [2, 3]]
alice
senza a capo
```

> ***Nota di implementazione.*** *Sul seed in codice macchina è disponibile solo* `println` *con* ***un*** *argomento:* `print` *e le chiamate a più argomenti (*`println(1, 2, "tre")`*) terminano con codice 1. Una funzione stampata dà* `fun` *invece di* `#lambda(x)`*, e una lista con elementi diversi dagli interi non è stampabile (il seed si ferma dopo* `[`*).*

La differenza tra un simbolo stampato da solo e dentro una lista non è un capriccio: fuori da una lista il simbolo è l'unico contenuto dell'output, dentro una lista serve un modo per distinguerlo da una stringa con lo stesso testo. Per questo `println(:a)` stampa `a` mentre le liste usano `:a`, e per questo la tabella va letta come «come appare un valore quando è stampato», non come «come è fatto il valore». Le stringhe dentro una lista non hanno virgolette per la stessa ragione: la stampa è pensata per essere leggibile, non per essere riletta dal parser.

L'uguaglianza confronta numeri, stringhe e simboli **per valore**, le liste **elemento per elemento** (anche annidate), le funzioni **per identità**. **[Core]**

```
println([1, [2, [3]]] == [1, [2, [3]]])
println("ab" == "a" * "b", " ", :a == :a, " ", 1 == 2)
```

**Output:**

```
true
true true false
```

## Esercizi

1. **Tutti i tipi.** Scrivi un programma che stampi `typeof` di un valore per ciascuno dei tipi di base e confronta i risultati con la tabella 2.1.
2. **Gerarchia. [Full]** Con `isa` verifica che `1` è `Int`, `Number` e `Any`, ma non `String`.
3. **Struct. [Full]** Definisci `struct Punto x::Int y::Int end`, costruisci un valore e stampalo; poi accedi a un campo.
4. **Conversioni. [Full]** Converti `3` in `Float`, `"42"` in `Int`, `42` in `String`, `"ciao"` in `Symbol` e stampa tutto.
5. **Conversioni Core.** Quali conversioni funzionano anche in Core? Prova `Int("41") + 1` e `Symbol("bob")` e confronta con il caso `Float(3)`.
6. **Asserzione di tipo. [Full]** Dato `x = 3`, che cosa producono `x::Int` e `x::String`? Come verifichi il secondo caso senza far terminare il programma?
7. **Vincolo del dichiarato. [Full]** `x::Int = 5`, poi `x = 6`, poi `x = 5.5`: quale riga fallisce, e perché l'errore arriva proprio lì?
8. **Vincolo e `let`. [Full]** Dato `x::Int = 5`, che cosa succede a `let x = 2.5` dentro una funzione? Che regola del

capitolo 2.6 si applica?

9. **Solo `Bool`.** Quali di queste condizioni sono ammesse? `if 1`, `if x`, `if x == 0`, `if length(xs) > 0`. Prova a eseguirle tutte.
10. **Come stampa.** Prevedi e verifica l'output di `println([1, :a, "s", true, nothing, [2, 3]])` e di `println(:alice)`: perché il simbolo appare in due forme diverse?
11. **Uguaglianza.** Che cosa valgono `[1, [2]] == [1, [2]]`, `"ab" == "a" * "b"` e `f == f` dove `f(x) = x`? Spiega la regola applicata in ciascun caso.
12. **Funzione stampata.** Assegna una lambda a un nome e stampala: che cosa vedi? Confronta con la tabella 2.8 e con la nota sul seed.

---

## 3. Variabili e assegnamento

`nome = espr` lega o aggiorna un nome. L'assegnamento **è un'espressione** e vale il valore assegnato. `+=`, `-=`, `*=` aggiornano una variabile esistente e valutano il lato sinistro una volta sola. Usare un nome mai assegnato è un errore di esecuzione. **[Core]**

Il segno `=` non si legge «scrivi in memoria»: lega un nome a un valore. Quando il nome non esiste lo crea, quando esiste ne sostituisce il legame. Triad non distingue i due casi con parole chiave diverse e non ha una parola per dichiarare una variabile: `let` introduce uno scope, non dichiara un tipo. Questa semplicità ha un prezzo che conviene conoscere subito: un nome scritto male non viene segnalato mentre scrivi, ma solo quando quella riga viene eseguita.

```
x = 42
y = x + 8
x = x + 1
z = (x + y) * 2
println(x, " ", y, " ", z)
n = 10
n += 5
n -= 3
n *= 2
println(n)
```

**Output:**

```
43 50 186
24
```

Non esistono dichiarazioni: le variabili sono dinamiche. Le regole di **visibilità** si vedono nel capitolo 8.

Perché mai un assegnamento dovrebbe avere un valore? Perché in Triad tutto ciò che può comparire in un programma è un'espressione, e l'assegnamento non fa eccezione: così `x = 42` può stare dentro un `if`, un `let` o un argomento di funzione senza una sintassi speciale. Il valore è quello assegnato, non un `Bool` di successo come in altri linguaggi; per questo non scriverai `if x = 5` aspettandoti un controllo, perché `5` non è un `Bool` e la condizione fallisce con un errore di tipo.

Anche `+=`, `-=` e `*=` sono espressioni e valgono il valore aggiornato. Non sono forme dichiarative e non introducono un nome nuovo: aggiornano un legame che deve già esistere. In cambio valutano il lato

sinistro una volta sola, un dettaglio che con un semplice nome non si nota ma che conta quando il bersaglio è un elemento di lista (`xs[i] += 1`, cap. 9): così l'indice non viene calcolato due volte.

Leggi l'esempio distinguendo il **nome** dal valore: dopo `x = 42`, `x` è un legame dell'ambiente corrente; `y = x + 8` legge quel legame e memorizza il risultato `50` in un legame diverso; `x = x + 1` calcola prima `43` e poi rimpiazza il valore associato a `x`. Di conseguenza `y` resta `50`: non è un alias automatico dell'espressione che lo ha prodotto. L'assegnamento ritorna il nuovo valore, perciò può comparire dentro un'espressione, ma questo non significa che ogni assegnamento sia globale: funzione, lambda e `let` definiscono regole di scope distinte.

Infine, riassegnare un nome non equivale a mutare un oggetto. Con `x = 43` cambia il legame di `x`; con `xs[1] = 43` cambia un elemento della lista a cui `xs` fa riferimento. Se un secondo nome indica la stessa lista, vedrà la mutazione; una riassegnazione locale di `xs`, invece, non cambia automaticamente il legame dell'altro nome. Questa differenza diventa essenziale quando una funzione condivide una lista con il chiamante.

## Esercizi

1. **L'assegnamento vale un valore.** Che cosa stampa `println(x = 42)`? Perché in Triad ha senso che un assegnamento sia un'espressione?
2. **Senza scorciatoie.** Riscrivi `n += 5`, `n -= 3`, `n *= 2` nella forma esplicita: l'output finale cambia?
3. **Valore aggiornato.** Dopo `n = 10`, che cosa stampa `println(n += 1)`? Di che tipo è quel valore?
4. **Una volta sola.** Spiega perché in `xs[i] += 1` il lato sinistro viene valutato una sola volta, e che cosa succederebbe se venisse valutato due volte.

5. **Nome inesistente.** `println(contatore)` senza assegnamento: quale errore? Perché nessun controllo segnala il refuso mentre scrivi?
6. **Riassegnare o mutare.** Dati `xs = [1, 2]` e `alias = xs`: che cosa vede `alias` dopo `xs[1] = 9`? E dopo `xs = [5]`?
7. `if x = 5`**.** Perché è un errore anche se l'assegnamento produce un valore? Quale regola del capitolo 2.7 si applica?
8. **Tabella dei valori.** Traccia `x = 42`, `y = x + 8`, `x = x + 1`, `z = (x + y) * 2` e conferma l'output `43 50 186`.
9. **Due legami indipendenti.** Dimostra con un programma che cambiare `x` dopo `y = x + 8` non cambia `y`.
10. **Vincolato e libero. [Full]** Confronta `x::Int = 5` e `s = "a"`: come si comportano gli assegnamenti successivi con tipi diversi? Scrivi un esempio per ciascuno.

---

## 4. Operatori e precedenza

Il parser è un **Pratt** con dodici livelli. «Pratt» significa che ogni operatore ha un numero di precedenza e il parser, leggendo da sinistra, sa quando fermarsi e restituire il controllo al livello più basso: è un modo compatto di descrivere l'albero senza scrivere una regola per ogni combinazione di operatori. Dalla precedenza più bassa alla più alta:

| livello | operatori | associatività |
|---|---|---|
| 1 | `=` `+=` `-=` `*=` | destra |
| 2 | `x -> e`, `(a, b) -> e`, `c ? a : b` | destra |

| livello | operatori | associatività |
|---|---|---|
| 3 | `\|\|` | sinistra, cortocircuito |
| 4 | `&&` | sinistra, cortocircuito |
| 5 | `==` `!=` `<` `<=` `>` `>=` `in` `isa` `is` | sinistra |
| 6 | `a:b` | range, senza spazi |
| 7 | `+` `-` | sinistra |
| 8 | `*` `/` `%` | sinistra |
| 9 | `-x` `!x` `+x` `$x` | prefissi |
| 10 | `^` | **destra** |
| 11 | `e::T` | asserzione di tipo |
| 12 | `f(args)` `xs[i]` `a.campo` | postfissi, senza spazio prima di `(` e `[` |

I livelli 6, 8 (la divisione), 9 (`$x`), 11 (`::`) e alcuni operatori di confronto esistono solo in Triad-Full. In Triad-Core i livelli effettivamente usati sono **dieci**, perché mancano il range, la divisione, `$x`, `::` e gli operatori `in`, `isa`, `is`; tolti quelli, la tabella si compatta ma il rapporto fra gli operatori che restano è lo stesso. Il numero dodici, che il glossario riporta, è quindi il conteggio del profilo Full: non è in contraddizione con i dieci livelli di Core, è il conto di un profilo più grande. **[Core]**

La precedenza decide **come il parser raggruppa** gli operatori, non quale calcolo avviene per primo in ogni possibile programma. Per esempio, `2 + 3 * 4` viene letto come `2 + (3 * 4)`: la moltiplicazione appartiene a un livello più forte della somma. Le

parentesi non cambiano la regola del linguaggio; rendono esplicita una scelta del programmatore e aiutano chi legge. L'associatività risolve invece la parità di precedenza: `2 ^ 3 ^ 2` è `2 ^ (3 ^ 2)`, mentre le sottrazioni a catena si raggruppano a sinistra. Quando la precedenza è poco familiare, scrivere le parentesi è una forma di documentazione eseguibile.

Il raggruppamento non va confuso con la valutazione. `&&` e `||` sono operatori a cortocircuito: il secondo operando viene valutato solo se il primo non determina già il risultato. Ciò consente di mettere a destra un controllo che dipende da un valore valido a sinistra, per esempio prima verificare che un indice sia entro i limiti e poi leggere la lista. In Core ogni condizione deve comunque produrre un `Bool`; un intero o una lista non vengono convertiti implicitamente in vero/falso.

## 4.1 Aritmetica intera

L'aritmetica di Core è interamente intera e queste quattro regole vanno lette come un modello unico, non come curiosità isolate: il tipo `Int` è un blocco di 64 bit, le operazioni lo trattano come tale, e la divisione è lasciata a Full perché richiede un altro tipo di numero.

- `+ - * ^` su `Int` con **wrap a 64 bit** (complemento a due);
- `%` è il **resto con il segno del dividendo**, come `rem` di Julia: `-7 % 3 == -1`;
- `^` con esponente negativo è un errore; `%` per zero è un errore;
- `/` **non esiste in Core**: la divisione è di Triad-Full.

```
println(2 + 3 * 4, " ", (2 + 3) * 4, " ", 2 ^ 3 ^ 2)
println(7 % 3, " ", -7 % 3, " ", 10 % 4)
println(9223372036854775807 + 1)
```

**Output:**

```
14 20 512
1 -1 2
-9223372036854775808
```

Nota `2 ^ 3 ^ 2 == 512`: l'esponente è associativo a destra, come in matematica.

Il wrap non è un difetto da aggirare: `Int` è un intero a 64 bit e l'aritmetica è quella del complemento a due, quindi superare il massimo riporta al minimo. Se il tuo calcolo può crescere oltre i 64 bit, in Full puoi passare a `Float` (con i suoi limiti di precisione); in Core devi progettare il calcolo per restare nei limiti, per esempio controllando i valori prima di moltiplicare.

> ***Nota di implementazione.*** *L'esempio qui sopra è verificato sull'interprete di riferimento. Sul seed in codice macchina il letterale `9223372036854775807` supera i 2^61-1 utilizzabili e l'espressione stampa `0`; inoltre `%` per zero non è un errore gestito ma termina il processo con codice **127** (divisione hardware per zero) e nessun messaggio, mentre l'interprete di riferimento scrive `error: div by zero` ed esce con 1.*

`%` segue il segno del dividendo perché è il resto della divisione intera troncata verso zero, non il modulo matematico, che sarebbe sempre non negativo: `-7 % 3` è `-1` e non `2`. La distinzione conta negli algoritmi che usano il resto per alternare indici o per normalizzare valori negativi.

Infine `/`: in Core non esiste perché la divisione esatta di due interi non ha un tipo in cui vivere senza i numeri in virgola mobile. In Full `/` produce un `Float` e `div` fa la divisione intera; se lavori in Core, la

divisione intera la scrivi a mano, come nell'esempio del capitolo 11.4.

## 4.2 Logica e confronti

`&&` e `||` cortocircuitano. I confronti **non** si concatenano: `1 < 2 < 3` è un errore (si confronta `Bool < Int`); scrivi `1 < 2 && 2 < 3`. **[Core]**

```
println(!false, " ", true && false, " ", false || true)
println(1 < 2 && 2 < 3)
```

**Output:**

```
true false true
true
```

## Esercizi

1. **Precedenza.** Calcola a mano `2 + 3 * 4`, `2 ^ 3 ^ 2`, `!true && false`, `1 + 2 == 3` e verifica con un programma.
2. **`^` è destra.** Perché `2 ^ 3 ^ 2` vale `512` e non `64`? Scrivi con le parentesi il raggruppamento effettivo.
3. **Cortocircuito. [Full]** Scrivi una condizione che usi `&&` per evitare la divisione per zero in `a / b > 1`, e provala con `b = 0`.
4. **Ternario e lambda.** A che livello di precedenza stanno `?:` e `->`? Scrivi un'espressione che usi entrambi e spiegane il raggruppamento.
5. **Resto dei negativi.** Che cosa valgono `-7 % 3` e `7 % -3`? Verifica e confronta con la tua idea di «resto».
6. **Catene di confronto.** Che cosa succede con `1 < 2 < 3`? Perché il risultato (o l'errore) è quello che vedi?

7. **`in`, `isa`, `is`. [Full]** Prova `1 in [1, 2]`, `isa(1, Int)` e `is(1, 1)`: che ruolo ha ciascuno dei tre operatori?
8. **Range senza spazi. [Full]** Scrivi `1:5` e poi `1 : 5`: perché il secondo non è un range?
9. **Segno e potenza.** Prevedi i valori di `-2 ^ 2` e `(-2) ^ 2`, poi verifica: a quale regola di precedenza si riconduce la differenza?
10. **Parentesi esplicite.** Riscrivi `1 + 2 * 3 == 7 || false` introducendo tutte le parentesi che rispecchiano la tabella dei livelli: il risultato cambia?

---

## 5. Controllo di flusso

In Triad **tutto è un'espressione**: `if`, `while`, `let`, `begin` hanno un valore, quello dell'ultima istruzione eseguita (`nothing` se non ce n'è).

Le eccezioni sono `return`, `break` e `continue`: non producono un valore, interrompono il flusso. L'assegnamento invece è un'espressione (cap. 3), e questa uniformità è ciò che rende possibile scrivere `v = if condizione … end` senza una sintassi speciale: un blocco è un'espressione come lo è una somma.

Questa regola unifica due usi che in altri linguaggi spesso hanno sintassi separate: scegliere un ramo e calcolare un risultato. In `v = if condizione ... end`, l'espressione completa produce il valore dell'ultima espressione del ramo scelto; il codice dei rami non scelti non viene eseguito. Le istruzioni che producono output o mutano una lista possono stare nello stesso blocco di un'espressione che restituisce un valore: il risultato del blocco è l'ultima espressione, non una combinazione automatica di tutti i risultati intermedi.

Le condizioni sono intenzionalmente restrittive: `if`, `while`, ternario, `&&` e `||` chiedono un valore `Bool`. Non si usa una regola di «verità implicita» che interpreti `0`, stringhe vuote o liste vuote come falsi. Questo rende visibili gli errori di tipo vicino alla decisione che li ha provocati, invece di trasformarli in rami inattesi.

### 5.1 `if` / `elseif` / `else` [Core]

```
function classifica(x)
    if x < 0
        "negativo"
    elseif x == 0
        "zero"
    else
        "positivo"
    end
end
println(classifica(-3), " ", classifica(0), " ",
classifica(7))
v = if 2 > 1 :si else :no end
println(v)
```

**Output:**

```
negativo zero positivo
si
```

Nell'esempio `classifica` restituisce l'ultima espressione del ramo scelto, quindi non serve nessun `return`. `v = if 2 > 1 :si else :no end` mostra la stessa cosa senza funzione: il blocco è un valore e `v`

riceve `:si`. Il ramo che non vince non viene eseguito affatto, non viene valutato e poi scartato: se contenesse una stampa o una mutazione, non la vedresti.

Il ternario `c ? a : b` è la forma breve di `if` e ha la stessa precedenza del `->`. **[Core]**

### 5.2 `while`, `break`, `continue` [Core]

`break` e `continue` agiscono sul ciclo **più interno**.

```
i = 0
somma_dispari = 0
while i < 10
    i += 1
    if i % 2 == 0
        continue
    end
    somma_dispari += i
end
println(somma_dispari)
while true
    break
end
println(:fine)
```

**Output:**

```
25
fine
```

Il primo ciclo somma solo i dispari: `continue` salta il resto del corpo, quindi l'aggiunta non avviene per i pari, e il contatore `i` è già stato incrementato prima del salto. Se l'incremento fosse stato in fondo al corpo, `continue` lo avrebbe saltato e il ciclo sarebbe stato infinito: è la trappola classica di `continue` nei cicli con contatore manuale, e il motivo per cui conviene incrementare subito, prima di ogni possibile uscita anticipata.

`break` e `continue` agiscono sul ciclo **più interno**; non esiste un'etichetta per uscire da due cicli in un colpo, quindi in quel caso si usa una variabile di stato.

In Core **non esiste** `for`: si itera con un contatore e `while`. In Full `for` esiste (cap. 5.6).

## 5.3 `let` e `begin` [Core]

`let` introduce un nuovo scope con legami locali. `begin … end` raggruppa istruzioni in una sola espressione, ma non è «senza conseguenze» sullo scope: l'interprete di riferimento aggiorna un nome già legato fuori dal blocco, mentre un nome **nuovo** assegnato dentro `begin` non risulta visibile fuori. La specifica non elenca `begin` fra i costrutti che introducono ambienti e nessun caso di conformità fissa il punto: per i nomi nuovi è prudente trattare il blocco come un confine.

```
x = 1
let x = 10
    let x = 20
        println(x)
    end
    println(x)
end
println(x)
```

```
r = begin
    a = 2
    a * 21
end
println(r)
```

**Output:**

```
20
10
1
42
```

L'esempio mostra tre legami distinti per `x`: quello esterno, quello introdotto dal primo `let` e quello introdotto dal secondo. Ogni `let` crea un ambiente nuovo, e un nome viene cercato prima nell'ambiente più interno; per questo le tre stampe danno `20`, `10`, `1` e nessuna delle tre rilegature tocca le altre. `begin`, invece, è solo un raggruppamento: l'esempio assegna `a = 2` e usa `a * 21` nello stesso blocco, quindi non dipende da che cosa resta visibile dopo. La regola da tenere presente è che il blocco non serve a far uscire nomi nuovi: se un valore deve essere usato fuori, assegnalo a un nome che esiste già prima del `begin` o restituiscilo come valore dell'espressione. In pratica `let` serve quando vuoi isolare un nome o riusare un nome già preso senza alterarne il legame esterno; `begin` serve a raggruppare istruzioni in un'espressione unica.

## 5.4 `for` e i range [Full]

`for` itera su una lista, un range o un risultato di `solve`. `a:b` produce il range da `a` a `b` inclusi.

```
s = 0
for i in 1:10
    s += i
end
println(s)

xs = [1, 2, 3]
println(xs[end])
println(xs[2:end])
```

**Output (golden Full):**

```
55
3
[2, 3]
```

`end` è un nome speciale che vale la posizione dell'ultimo elemento e si può usare dentro le parentesi quadre di un indice: `xs[end]` è l'ultimo elemento, `xs[2:end]` è la coda dal secondo elemento in poi. Il range `a:b` è inclusivo su entrambi gli estremi, quindi `1:10` contiene dieci valori; è una forma di Triad-Full, e in Core si ottiene con un ciclo. La destrutturazione nel `for` è trattata nel capitolo 24.

## 5.5 Le comprehension [Full]

La comprehension `[e for x in xs]` costruisce una lista; con `if` aggiunge un filtro.

```
println([x^2 for x in 1:4])
println([x^2 for x in 1:4 if x > 2])
```

**Output (golden Full):**

```
[1, 4, 9, 16]
[9, 16]
```

La comprehension è zucchero per un ciclo che accumula in una lista: per ogni `x` di `1:4` valuta `x^2` e aggiunge il risultato, senza un contatore esplicito. Il filtro `if x > 2` agisce sull'elemento, non sull'indice: `9` e `16` restano perché `3` e `4` superano la soglia. In Core `[e for x in xs]` non esiste e lo stesso lavoro lo fanno le funzioni scritte a mano del capitolo 22.1; in Full spesso la comprehension è la forma più leggibile, purché l'espressione resti breve.

### 5.6 `try` / `catch` [Full]

In Full l'errore è un **valore**, quindi `try` ha due forme: con `catch` il ramo gestito produce il valore dell'espressione; senza `catch`, l'errore **diventa** il valore dell'espressione.

```
e = try div(1, 0) end
println(iserr(e), " ", e.kind)

r = try div(1, 0) catch err; 0 end
println(r)
```

**Output (golden Full):**

```
true domain
0
```

Le due forme non sono intercambiabili. `try e end` non gestisce l'errore: lo rende il valore dell'espressione, così puoi controllarlo con `iserr` e decidere dopo. `try e catch err; h end` invece recupera: lega `err` all'errore e valuta `h`, il cui valore diventa il risultato. Nel

primo caso `e` è un `Err`, nel secondo `r` è `0`. Scegliere fra i due è una decisione di progetto: la prima forma tiene l'errore nel flusso normale, la seconda lo confina in un punto preciso. È il motivo per cui il capitolo 11 insiste sul fatto che «errore come dato» non significa «il programma continua da solo»: senza un `try` che lo osservi, l'errore arriva al livello superiore e il programma termina.

## Esercizi

1. **`if` come espressione.** Usa un `if` come valore in un assegnamento e come argomento di `println`.
2. **Cerca con `break`.** Scrivi un ciclo che trovi il primo intero il cui quadrato superi 1000 e lo stampi.
3. **`continue`.** Scrivi un ciclo che stampi i dispari da 1 a 9 usando `continue`; poi riscrivilo senza `continue` (la forma equivalente del volume) e confronta gli output.
4. **Uscire da due cicli.** Con due `while` annidati e una variabile di stato, interrompi entrambi appena `a * b` supera 20.
5. **`let`.** Che cosa succede a una variabile creata dentro un `let` dopo il blocco? Verifica con un accesso dall'esterno.
6. **`begin`.** A cosa serve `begin … end`? Usalo dove serve un'espressione ma devono eseguirsi più istruzioni.
7. **`for` e range. [Full]** Riscrivi un `while` che somma 1..10 con un `for i in 1:10`.
8. **Comprehension. [Full]** Costruisci con una comprehension la lista dei quadrati da 1 a 5; poi aggiungi un filtro che tenga solo i dispari.
9. **`try` / `catch`. [Full]** Cattura `Int("ciao")` con `try/catch` e stampa la categoria dell'errore catturato.
10. **`while` come espressione.** Che cosa produce `x = while … end`? Provalo sull'interprete di riferimento e confronta con la

nota sul seed del capitolo 29.

---

# 6. Funzioni

## 6.1 Tre forme di definizione [Core]

Una funzione nominata si definisce in due modi — forma breve e forma a blocco — e la terza forma è una funzione anonima legata a un nome. La differenza è solo sintattica: la funzione che ottieni è la stessa. La forma breve si usa quando il corpo è una sola espressione; la forma `function … end` quando il corpo ha più istruzioni o un `return` che esce prima della fine; la lambda assegnata a un nome è utile quando il valore funzione nasce dentro un'espressione.

```
# forma breve: il corpo è un'espressione
quadrato(x) = x * x

# forma blocco: il valore è quello dell'ultima
espressione
function calcola(n)
    m = n + 1
    m * 2
end

# funzione anonima assegnata a un nome
moltiplica = function (a, b)
    a * b
end

println(quadrato(4), " ", calcola(3), " ",
moltiplica(6, 7))
```

**Output:**

```
16 8 42
```

`quadrato` e `calcola` sono funzioni nominate, `moltiplica` è un valore di tipo funzione legato a un nome: dal punto di vista della chiamata non c'è differenza. Scegli la forma che rende più leggibile il corpo, non quella che sembra più moderna; la forma breve con una sola espressione rende evidente che non ci sono effetti nascosti tra le righe.

## 6.2 `return`, ricorsione, arità

`return e` esce subito dalla funzione; `return` da solo restituisce `nothing`. Una funzione nominata è visibile nel proprio corpo, quindi la ricorsione diretta funziona. Il numero di argomenti deve coincidere con i parametri. **[Core]**

```
function taglia(n)
    if n > 100
        return :grande
    end
    :piccola
end
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)
mcd(a, b) = b == 0 ? a : mcd(b, a % b)
println(taglia(500), " ", taglia(5))
println(fattoriale(10), " ", fib(15), " ", mcd(48, 18))
```

**Output:**

```
grande piccola
3628800 610 6
```

Una chiamata può essere letta come una sequenza precisa: si valutano gli argomenti, si crea il contesto della funzione, si associano i valori ai parametri e si esegue il corpo. Il corpo termina in uno di due modi: raggiunge la fine e restituisce l'ultima espressione, oppure esegue `return`, che interrompe immediatamente il corpo e restituisce il valore indicato. `return` senza espressione restituisce `nothing`. Una ricorsione è una nuova chiamata con un proprio insieme di parametri locali; il caso base impedisce di continuare a creare chiamate senza fine.

L'arità è parte della firma anche in Core: `somma(1, 2)` non è intercambiabile con `somma(1)`. Triad-Full aggiunge i tipi alle firme e consente più metodi con lo stesso nome; questo non cambia la nozione di chiamata, ma aggiunge una fase di selezione del metodo prima di eseguirne il corpo.

Un'ultima nota pratica: la ricorsione profonda consuma spazio di chiamata, quindi per profondità grandi conviene un `while` con uno stato esplicito (cap. 22.6). Il caso base non è una formalità: senza di esso la chiamata ricorsiva non si ferma e il programma non arriva a un risultato.

## 6.3 Funzioni di ordine superiore

Le funzioni sono valori di prima classe: si passano, si restituiscono, si compongono. In Core `map`, `filter` e `reduce` **non esistono**: si scrivono a mano (cap. 22.1). In Full sono builtin. **[Core]**

```
function applica_tutti(f, xs)
    out = []
    i = 1
    while i <= length(xs)
```

```
        push!(out, f(xs[i]))
        i += 1
    end
    out
end
println(applica_tutti(x -> x * x, [1, 2, 3]))
```

**Output:**

```
[1, 4, 9]
```

In Full, `reduce(+, xs)` e `map(sq, xs)` funzionano perché anche gli **operatori sono funzioni**.

Il punto di `applica_tutti` non è la funzione specifica: è che `f` è un parametro come gli altri, e `x -> x * x` è un valore costruito al momento della chiamata. Questa è la differenza fra scrivere dieci cicli quasi uguali e scrivere un ciclo una volta sola; in Core `map` e `filter` non esistono perché il linguaggio di bootstrap deve restare piccolo, non perché l'idea sia estranea a Triad. Le funzioni del capitolo 22.1 sono la risposta pratica: scrivile una volta e riusale.

## 6.4 L'annotazione di effetto `!io`

Una funzione che stampa deve dichiarare `!io` nella firma. Core **accetta l'annotazione e la ignora**: scriverla rende il codice già pronto per Full. **[Core]**

```
function mostra() !io
    println("hello from block")
    20 + 22
```

```
end
println(mostra())
```

**Output:**

```
hello from block
42
```

L'annotazione non è decorativa. In Triad-Full una funzione dichiara le capacità che usa, e il runtime verifica che il chiamante le permetta (cap. 15); in Core il controllo non esiste, ma scrivere `!io` adesso significa non dover riscrivere la firma quando il programma passerà a Full. La parola `io` è un permesso, non un'etichetta descrittiva: dice che la funzione può produrre output, e chi la chiama deve essere autorizzato a farlo.

## Esercizi

1. **Tre forme.** Scrivi la stessa funzione (`triplo`) nelle tre forme: breve, blocco e anonima assegnata a un nome.
2. **Che cosa restituisce?** Che valore produce una funzione il cui corpo termina con `println(...)`? Verifica usando il risultato in un'espressione.
3. **`return` e ultima espressione.** Scrivi `segno(n)` in due versioni: con `return` anticipati e solo con l'ultima espressione.
4. **Arity.** Quali errori ottieni chiamando `f(1)` e `f(1, 2, 3)` per `function f(a, b)`?
5. **Ricorsione.** Scrivi `fib(n)` e `fattoriale(n)` ricorsivi e riscrivi `fattoriale` a ciclo: confronta i due programmi.
6. **Ordine superiore.** Scrivi `applica(f, x)` e usala con una lambda e con una funzione con nome.

7. **Funzione come valore.** Assegna una funzione a una variabile, passala come argomento e restituiscila da un'altra funzione.
8. **Effetto `!io`. [Full]** Perché una funzione senza `!io` che chiama `println` produce `Err(:effect)`? Scrivi l'esempio e correggi la firma.
9. **Effetti delle lambda. [Full]** Una lambda creata dentro una funzione `!io` può stampare? Verifica con un esempio.
10. **Ordine di definizione.** Che cosa succede se chiami una funzione prima della sua riga di definizione? Perché?
11. **Ricorsione sinistra. [Full]** `f(n) = f(n + 1)` senza limiti: che cosa succede? Avvolgi la chiamata in `with budget` e confronta i due comportamenti.

---

## 7. Tipi e multiple dispatch

Una funzione generica è un insieme di metodi. Ridefinire la stessa firma sostituisce il metodo precedente. Tra i metodi applicabili (stessa arità, ogni argomento sottotipo del parametro) vince il **più specifico**. **[Full]**

```
area(n::Number) = :number
area(n::Int) = :int

println(area(1))
println(area(1.5))
```

**Output (golden Full):**

```
int
number
```

L'esempio va letto come una ricerca: `area(1)` ha un `Int`, che è sottotipo sia di `Int` sia di `Number`; entrambi i metodi sono applicabili, ma `Int` è più specifico e vince. `area(1.5)` ha un `Float`, che è sottotipo di `Number` ma non di `Int`, quindi resta un solo metodo applicabile. Se avessi definito solo `area(n::Number)`, entrambe le chiamate avrebbero usato quello: il dispatch non sceglie il metodo «migliore», sceglie il più specifico fra quelli che accettano gli argomenti.

Un metodo A è più specifico di B se ogni suo parametro è sottotipo del corrispondente parametro di B e le firme differiscono.

- Se nessun metodo è applicabile: `Err(:method)`.
- Se più metodi sono massimali: `Err(:ambiguous)`.
- Se il nome è anche un builtin (per esempio `+` sui numeri), si usa il builtin.

Il dispatch si può modellare come una ricerca in due passaggi. Prima si eliminano i metodi con arità errata o con almeno un argomento che non soddisfa il tipo dichiarato; poi si confrontano quelli rimasti secondo la relazione di sottotipo, parametro per parametro. Un metodo è più specifico solo se non è più generale in nessuna posizione e lo è in almeno una. Non basta quindi scegliere il metodo aggiunto più di recente. Se restano due firme incomparabili, la chiamata è ambigua e produce `Err(:ambiguous)` invece di dipendere dall'ordine dei file o dall'ordine di registrazione. Questo comportamento consente di aggiungere metodi specializzati senza riscrivere la funzione generica, mantenendo però espliciti i casi che non hanno una risposta univoca.

Gli operatori si estendono come funzioni, ridefinendoli:

```
struct Point
    x::Float
    y::Float
```

```
end

function +(a::Point, b::Point)
    Point(a.x + b.x, a.y + b.y)
end

p = Point(1, 2) + Point(3, 4)
println(p.x, " ", p.y)
```

**Output (golden Full):**

```
4.0 6.0
```

## 7.1 Il tipo di ritorno

`::R` è controllato all'uscita, con la conversione `Int -> Float`. Se il valore non è conforme: `Err(:type)`. **[Full]**

```
function metà(n::Number)::Float
    n / 2
end
println(metà(3))
```

**Output (golden Full):**

```
1.5
```

Il controllo avviene all'uscita perché solo allora il valore esiste; la conversione `Int -> Float` permette di scrivere `n / 2` senza preoccuparti del tipo letterale del risultato. Se il valore non è convertibile,

ottieni un `Err(:type)` e non un'eccezione: anche una promessa di firma violata entra nel flusso degli errori come dato.

### 7.2 Funzioni locali

Una `function` definita dentro un'altra funzione è una **chiusura locale**, senza dispatch. **[Full]**

```
function esterno(x)
    function interno(y)
        y * 2
    end
    interno(x) + 1
end
println(esterno(5))
```

## Esercizi

1. **Tipo di ritorno. [Full]** In `function f(x)::Int`: che cosa succede se il corpo restituisce una stringa? E se restituisce un `Float`?
2. **Firme tipate. [Full]** Definisci `area(x::Int)` e `area(x::Float)`: quale metodo viene scelto per `area(1)` e per `area(1.5)`?
3. **Più specifico vince. [Full]** Aggiungi `area(x::Number)`: per `1.5` quale dei tre metodi scatta, e perché?
4. **Nessun metodo. [Full]** `area("x")`: quale errore? Che cosa contiene `data.types`?
5. **Ambiguità. [Full]** Costruisci due firme incomparabili (`f(a::Int, b::Number)` e `f(a::Number, b::Int)`) e osserva `Err(:ambiguous)` con `f(1, 1)`.

6. **Funzioni locali.** Definisci una funzione dentro un'altra: dove è visibile? Prova a chiamarla dall'esterno.
7. **Fallback su `Any`. [Full]** Un metodo su `Any`: quando viene scelto? Scrivi un esempio con due metodi più specifici e uno su `Any`.
8. **Ridefinire una firma. [Full]** Dichiarare due volte la stessa firma: sostituisce o aggiunge un metodo? Verifica.
9. **Metodi su un operatore. [Full]** Definisci `+` per una struct `Point` e prova `p1 + p2`: che cosa succede al builtin `+` per gli altri tipi?
10. **Conversione del ritorno. [Full]** Scrivi una funzione `::Float` che restituisce un `Int`: il risultato arriva convertito? Verifica stampandolo.

---

# 8. Lambda, chiusure e scope

## 8.1 Lambda [Core]

Una lambda è una funzione senza nome: `x -> x * 2` è un valore, non una definizione. La forma `(a, b) -> …` dichiara più parametri, e il corpo è un'espressione oppure un blocco. Le lambda si passano come argomenti e si restituiscono come risultati, ed è così che una funzione di ordine superiore (cap. 6.3) descrive un'operazione senza eseguirla subito.

```
doppio = x -> x * 2
somma = (a, b) -> a + b
applica = f -> f(3)
println(doppio(5), " ", somma(1, 2), " ", applica(x ->
x * x))
```

**Output:**

```
10 3 9
```

Una lambda con corpo a più istruzioni si scrive con `begin … end` o con `function (…) … end`. Le lambda senza parametri (`() -> …`) **non** sono supportate: usa `function () … end`.

## 8.2 Chiusure

Una lambda cattura l'ambiente in cui è creata. Secondo `SPEC-CORE §5` le variabili catturate sono **condivise**: le mutazioni sono visibili. **[Core]**

```
function add(n)
    x -> x + n
end
f = add(10)
g = add(100)
println(f(5), " ", g(5))

function contatore()
    v = 0
    x -> begin
        v += x
        v
    end
end
acc = contatore()
acc(5)
println(acc(10))
```

**Output:**

```
15 105
15
```

> ***Attenzione.*** *Il seed Windows cattura* ***per valore*** *e copre solo l'ultimo legame del frame: un accumulatore come* `contatore` *può non funzionare sul seed. L'interprete JS e la specifica usano il modello condiviso.*

Una chiusura non conserva una fotografia del solo testo della lambda: mantiene accesso ai legami che il suo corpo può leggere. Nel primo esempio la funzione restituita da `add(10)` mantiene il valore `n = 10` anche dopo che la chiamata a `add` è terminata; `add(100)` crea una chiusura distinta. Nel contatore, invece, la variabile `v` è un legame condiviso: ogni chiamata aggiorna lo stesso stato e la successiva lo osserva. È il motivo per cui una chiusura può rappresentare un piccolo oggetto con stato privato. È anche il motivo per cui il comportamento specifico del seed Windows va verificato separatamente dall'interprete di riferimento.

La differenza fra cattura condivisa e cattura per valore non è teorica: con il modello condiviso due chiusure create dallo stesso frame vedono lo stesso stato, con il modello per valore ognuna ha la sua copia. Quando progetti un oggetto con stato, sappi quale modello stai usando, perché è quello che decide se due contatori sono indipendenti o no.

## 8.3 Le regole di scope

Le regole seguono un principio unico: lo scope è **lessicale**, cioè conta dove il codice è scritto, non da dove viene chiamato. Da quel principio discendono le cinque regole che seguono; leggerle in ordine aiuta a prevedere che cosa modifica davvero un assegnamento.

1. `function`, lambda, `let`, `while`, `for` e le comprehension introducono **ambienti**; un nome è visibile nell'ambiente in cui è legato e in quelli interni (scope lessicale), mai in quelli esterni.
2. **Dentro una funzione nominata** l'assegnamento lega nel frame locale: una funzione **non modifica i globali**.
3. **Dentro una lambda** l'assegnamento a un nome già legato fuori aggiorna quel legame.
4. **Al top level** i cicli possono aggiornare i globali.
5. `let x = …` crea un nuovo `x`: assegnare `x` dentro il `let` non tocca l'`x` esterno.

```
x = 1
function f()
    x = 2
    x
end
println(f(), " ", x)
c = 0
function incrementa()
    c += 1
end
incrementa()
println(c)
```

**Output:**

```
2 1
0
```

Per far uscire un risultato da una funzione, restituiscilo — oppure usa una lista, che è mutabile.

L'esempio mostra il caso più importante: `f` restituisce `2` e il globale `x` resta `1`, perché l'assegnamento dentro una funzione nominata crea un legame locale. `incrementa` incrementa un legame locale `c` che nasce e muore nella chiamata, e il globale `c` resta `0`. Non è una svista del linguaggio: è ciò che rende il comportamento di una funzione indipendente dallo stato globale. Quando hai bisogno di stato condiviso, le strade sono due — una lista mutabile passata come argomento (cap. 9) o una chiusura (cap. 8.2) — e in entrambi i casi la condivisione è visibile nella firma o nella definizione.

## Esercizi

1. **Lambda e chiamata.** Scrivi `x -> x * 2`, assegnala a un nome e chiamala su un valore.
2. **Due parametri.** Riscrivi `function (a, b) a + b end` in forma lambda: quale limite ha questa forma sul seed Windows (cap. 29)?
3. **Un contatore.** Scrivi `contatore()` che restituisce una funzione che a ogni chiamata restituisce il valore successivo: quale tecnica di stato usi, e come si comporta sui due runtime?
4. **Visibilità.** Una funzione vede le variabili del programma che la contiene? E il programma vede quelle nate dentro la funzione? Verifica entrambe le direzioni.
5. **Shadowing con `let`.** Dato `x = 1` esterno, che cosa stampa `let x = 2` dentro il blocco e che cosa resta fuori?
6. **Parametri e argomenti.** Modificare un parametro dentro la funzione cambia l'argomento del chiamante? Prova con un `Int` e con una lista.

7. **Lambda come argomento.** Passa `f = x -> x + 1` a una funzione di ordine superiore e usala nel corpo.
8. **Chiusura su una lista.** Una chiusura che cattura una lista vede le modifiche fatte con `push!`? Verifica.
9. **Conteggio delle chiamate.** Scrivi una funzione che restituisca una lambda e osserva quante volte la cattura «per valore» del seed cambia il risultato rispetto alla cattura condivisa.
10. **Anonima o lambda?** Riscrivi ciascuna delle due forme nell'altra: `function (x) x * x end` e `(a, b) -> a - b`.

---

## 9. Liste

Una lista è eterogenea, annidabile e **mutabile**. Gli indici partono da **1**. **[Core]**

Una lista si scrive fra parentesi quadre, `[1, 2, 3]`; l'ordine degli elementi è significativo e i valori possono essere di tipi diversi. La mutabilità è la proprietà che distingue la lista da un record: la lista cresce e cambia, e più nomi possono osservare la stessa lista. È la struttura d'appoggio di quasi tutto il codice Core, dai record alle code, dagli ambienti ai parser.

```
xs = [1, 2, 3]
println(xs[1], " ", length(xs))
push!(xs, 4)
xs[1] += 100
println(xs)
alias = xs
push!(alias, 5)
println(xs)
```

**Output:**

```
1 3
[101, 2, 3, 4]
[101, 2, 3, 4, 5]
```

`push!` **muta** la lista e la restituisce. Le liste sono riferimenti: due nomi possono indicare la stessa lista, come si vede sopra.

Il fatto che `push!` restituisca la lista, e non `nothing`, è una scelta comoda: permette di concatenare l'aggiunta con un'altra operazione, ma rende anche possibile scrivere `xs = push!(xs, 4)`, che è ridondante ma innocuo. L'assegnamento a un elemento (`xs[1] += 100`) funziona anche con gli operatori composti: il lato sinistro viene valutato una volta sola, quindi l'indice non viene calcolato due volte.

Una lista è un oggetto mutabile; una variabile contiene un riferimento a quell'oggetto. Se `ys = xs`, i due nomi permettono di raggiungere la stessa lista: `push!(ys, valore)` e `xs[i] = valore` sono osservabili anche leggendo `xs`. Se invece scrivi `ys = [1, 2]`, hai sostituito soltanto il legame `ys`; `xs` continua a riferirsi alla lista precedente. Questa distinzione spiega perché assegnare una lista non ne crea automaticamente una copia e perché gli argomenti lista possono essere usati per condividere stato tra funzioni.

La convenzione 1-based è coerente con esempi e specifica: per visitare `n` elementi, il contatore inizia da 1 e la condizione tipica è `i <= length(xs)`. In un ciclo, aggiornare il contatore dopo aver letto l'elemento evita di saltare il primo elemento; aumentarlo in ogni iterazione assicura che la condizione prima o poi diventi falsa. Sul seed Windows la funzione ausiliaria sperimentale `get(xs, i)` è diversa e 0-based: non va usata come se fosse la semantica standard delle liste.

> ***Nota di implementazione.*** *Sul seed in codice macchina* `xs[i]` *non è ancora eseguibile e* `push!` *funziona solo* ***come espressione*** *(*`xs = push!(xs, 4)`*): la forma statement* `push!(xs, 4)` *termina con codice 1. Anche* `get(xs, i)` *esiste nel seed ma non nell'interprete di riferimento: oggi i due runtime non condividono nessuna forma di indice. Dati misurati:* `docs/libro/_dati_dal_codice.md` *(§7); lavoro tracciato in* `TASK.md` *(S0.2).*

La scelta di partire da 1 non è un dettaglio estetico: rende coerente l'accesso con le posizioni umane e con i range, e permette di usare `length(xs)` come ultimo indice valido. Il prezzo è che un indice calcolato pensando allo 0 va corretto; l'errore tipico è `xs[0]`, che non restituisce un valore ma produce un errore. Un indice fuori intervallo non è `nothing`: in Core il programma termina, in Full arriva un `Err(:bounds)` da gestire. Per questo, quando l'indice è calcolato, è buona norma controllarlo prima con `1 <= i && i <= length(xs)`.

La somma degli elementi con un ciclo esplicito:

```
xs = [101, 2, 3, 4, 5]
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
```

**Output:**

```
115
```

Il ciclo mostra la forma canonica di attraversamento in Core: contatore da 1, condizione `i <= length(xs)`, lettura di `xs[i]`. `length` viene ricalcolata a ogni iterazione, quindi se il corpo del ciclo cambia la lista il confine si sposta; mutare la lista mentre la si attraversa è possibile, ma va fatto con consapevolezza. Aggiungere elementi allunga la lista e il ciclo li visiterà; in Full, dove esistono `pop!` e `append!`, togliere elementi mentre l'indice cresce fa saltare quelli successivi.

In Full esistono anche `reverse`, `sort`, `sum`, `first`, `last`, `rest`, `pop!`, `append!`, `vcat`, `collect` e l'operatore `x in xs`. Nessuno di questi nomi esiste in Core: se ti servono, li scrivi a mano con `while` e `push!`, come nel capitolo 22.1. Non è una perdita: gli algoritmi scritti a mano mostrano che cosa fanno e quanto costano, e in un bootstrap questa trasparenza vale più della brevità.

## Esercizi

1. **Costruzione.** Crea liste miste e annidate e stampale: confronta l'output con le regole di formattazione del capitolo 2.8.
2. **Indici ed `end`. [Full]** Prova `xs[1]`, `xs[length(xs)]` e `xs[end]`: quale forma è più leggibile? Verifica anche in scrittura (`xs[end] = v`).
3. **Modificare la lista. [Full]** Usa `push!`, `pop!` e `append!` e stampa la lista dopo ogni operazione.
4. **`sum` scritto a mano.** Calcola la somma di una lista con un ciclo Core, poi confronta con il builtin `sum` **[Full]**.
5. **`reverse` scritto a mano.** Implementa l'inversione di una lista senza `reverse`, poi confronta con il builtin.

6. **Ordine superiore su liste. [Full]** Con `map`, `filter` e `count` seleziona i numeri maggiori di 10 e conta gli elementi pari.
7. **Alias e mutabilità.** Dati `xs` e `alias = xs`, dimostra che `xs[1] = 9` è visibile ad `alias`, mentre `xs = [5]` no.
8. **Contatore di frequenze.** Data una lista di interi 1..3, usa `xs[i] += 1` per contare le occorrenze di ciascun valore.
9. **Fuori intervallo.** Quali errori si ottengono con `xs[0]`, `xs[length(xs) + 1]` su una lista di 3 elementi? Confronta con la categoria `:bounds`.
10. **Liste e funzioni.** Scrivi `massimo(xs)` e `ultima_posizione(xs, v)` e provalle su casi limite (lista vuota, elemento assente): che cosa devono restituire?

---

## 10. Stringhe, simboli e conversioni

Le stringhe in Triad sono valori immutabili e le operazioni sono poche, volutamente esplicite: si concatena con `*`, si misura la lunghezza in byte, si converte con funzioni dal nome chiaro. La tabella raccoglie le operazioni disponibili in Core; dove Full si comporta diversamente lo trovi indicato sotto, con il perché.

| operazione | significato |
|---|---|
| `a * b` | concatenazione di stringhe |
| `length(s)` | numero di **byte** UTF-8 (`length("é") == 2`) |
| `string(x)` | rappresentazione testuale di qualsiasi valore (`string(:a) == "a"`, `string([1, 2]) == "[1, 2]"`) |

| operazione | significato |
|---|---|
| `Int(s)` | da stringa a intero ( `Int("42") == 42` ) |
| `Symbol(s)` | da stringa a simbolo ( `Symbol("ciao") == :ciao` ) |
| `typeof(x)` | in Core, il nome del tipo come simbolo (cap. 2.1) |

```
nome = "Triad"
saluto = "ciao " * nome
println(saluto, " (", length(saluto), " byte)")
println("risposta: " * string(40 + 2))
println(Int("41") + 1, " ", Symbol("bob"), " ",
string(true))
```

**Output:**

```
ciao Triad (10 byte)
risposta: 42
42 bob true
```

La prima cosa da notare è che `length(saluto)` vale `10`: "ciao Triad" ha dieci caratteri e dieci byte, perché l'alfabeto latino occupa un byte per carattere. Con `"é"` il conto cambia ( `length("é") == 2` ), e con un'emoji cambia ancora: la lunghezza è una proprietà della codifica UTF-8, non del testo come lo vedi. È il motivo per cui in Triad non si indicizza una stringa come una lista: `s[1]` restituirebbe un byte, non un carattere, e sarebbe una trappola silenziosa. Per lavorare sui singoli byte servono i primitivi di bootstrap (cap. 21).

La concatenazione con `*` non modifica `saluto`: crea una nuova stringa, e la vecchia resta disponibile a chiunque la stia usando. Questa immutabilità rende le stringhe sicure da passare alle funzioni, ma significa anche che ogni `*` alloca una stringa nuova: in un ciclo lungo il costo cresce con la stringa, non con il numero di giri.

`string(x)` e `Symbol(s)` vanno tenute distinte: `string` trasforma un valore in testo, `Symbol` trasforma testo in un'identità. `string(42)` produce la stringa `"42"`; `Symbol("42")` produce un simbolo, che è un valore completamente diverso. `Int("42")` fa il cammino inverso: è una conversione, non una valutazione, quindi il testo deve essere un intero scritto in decimale. Che cosa succeda con un testo non numerico non è fissato dallo standard: l'interprete di riferimento restituisce `0`, ma è un comportamento da verificare prima di contarci.

> ***Nota di implementazione.*** *Sul seed in codice macchina* `Int`*,* `Symbol` *e* `typeof` *non esistono (il programma termina con codice 1) e* `string(true)` *produce* `1` *invece di* `true` *(misurato): il seed tratta i* `Bool` *come interi in questa conversione. L'esempio qui sopra è verificato sull'interprete di riferimento.*

In Full le stringhe hanno in più l'interpolazione (cap. 1.3) e `repr`, che produce una rappresentazione pensata per essere riletta; in Core `string` è l'unica porta dal valore al testo, e non esiste l'interpolazione.

## Esercizi

1. **Concatenazione.** Costruisci una frase da almeno quattro pezzi usando `*` e `string()`.
2. **Byte, non lettere.** Spiega e verifica `length("è") == 2`: che implicazioni ha per un programma che «taglia» testo?

3. **Escape conosciuti e sconosciuti.** Prova `"a\nb"`, `"a\tb"`, `"a\"b"`, `"a\\b"` e poi `"a\q"`: quale riga produce `Err(:syntax)` e con quali informazioni?
4. **Interpolazione. [Full]** Riscrivi con l'interpolazione `"$x"` e `"$(x + 1)"` un saluto che in Core richiedeva concatenazione.
5. **`string` e `repr`. [Full]** Confronta `string(:a)` e `repr(:a)`, e `string("s")` con `repr("s")`: quando serve l'uno e quando l'altro?
6. **Da testo a numero.** Converti `"41"` con `Int`, aggiungi 1 e riconverti in testo; poi prova `Int("ciao")` e spiega l'errore.
7. **Simboli.** Confronta `:alice` con `Symbol("alice")` e spiega l'output di `println(:alice)` rispetto a `println([:alice])`.
8. **Funzione stampata. [Full]** Stampa una funzione (`repr` e `string`): che forma ha la sua rappresentazione?
9. **`ripeti` con un ciclo.** Scrivi `ripeti(s, n)` che concateni `s` `n` volte senza ricorsione.
10. **Conversioni a catena.** Partendo da `"123"`, arriva a `"124"` usando solo `Int`, `string` e aritmetica, e spiega ogni passaggio.

---

## 11. Errori come dati

Questa è una delle idee portanti di Triad: **l'errore è un valore**, non un'eccezione che interrompe il programma.

### 11.1 La forma `Err`

Un errore è un valore `Err` con tre campi:

- `kind::Symbol` — la categoria, su cui si può fare dispatch;
- `msg::String` — il messaggio;
- `data` — un valore qualsiasi, spesso un record `Info(...)` con i dettagli.

**[Full]**

I tre campi separano tre domande diverse: *che cosa* è successo (`kind`), *come raccontarlo* (`msg`) e *quali informazioni servono per reagire* (`data`). Tenerle separate è ciò che permette di scrivere codice che decide in base alla categoria senza analizzare il testo del messaggio, che è pensato per una persona e non per un programma.

```
e = err(:mio, "ops", 42)
println(e.kind, " ", e.msg)
println(iserr(e))
```

## 11.2 Produrre e propagare

La tabella riassume i costrutti che creano, propagano o osservano un errore: leggerla dall'alto in basso è anche l'ordine in cui un errore attraversa un programma, dal punto in cui nasce al punto in cui qualcuno decide che cosa farne.

| costrutto | effetto |
|---|---|
| `err(:kind, "msg"[, data])` | costruisce un `Err` **senza** sollevarlo |
| `throw(e)`, `error("msg")` | solleva (`error` produce `Err(:error, …)`) |
| `try b catch e h end` | se `b` solleva, `e` è legato all'`Err` e si valuta `h` |

| costrutto | effetto |
|---|---|
| `try b end` | **l'errore diventa il valore** dell'espressione |
| `iserr(x)` | controlla se `x` è un `Err` |

Tutti i fallimenti del linguaggio — divisione intera per zero, `sqrt(-1.0)`, indici fuori range, nessun metodo applicabile — arrivano come `Err` e **mai** come fallimenti fatali. **[Full]**

```
e = try div(1, 0) end
println(e.kind)
println(iserr(e))

r = try div(10, 2) catch err; 0 end
println(r)
```

**Output (golden Full):**

```
domain
true
5
```

È utile distinguere tre momenti: **produrre**, **propagare** e **gestire** un errore. `err(...)` costruisce un dato e non interrompe nulla; un'operazione che fallisce produce un `Err` secondo la specifica; `try` stabilisce il punto in cui quel valore viene osservato o recuperato. In Full, `try espressione end` restituisce l'errore come valore, mentre `try ... catch nome ... end` permette al ramo di recupero di esaminare la categoria, il messaggio o i dettagli. Se l'errore non viene gestito, arriva al livello superiore e diventa un codice di uscita non riuscito: "errore come dato" non significa che il programma continui automaticamente dopo qualunque operazione.

Il tipo del dato permette di scrivere politiche diverse per categorie diverse. Per esempio, un errore `:bounds` può essere presentato come input non valido, mentre `:method` può indicare che manca un'implementazione. Separare la categoria stabile (`kind`) dal testo leggibile (`msg`) evita di dover analizzare stringhe di diagnostica per decidere come reagire.

## 11.3 Le categorie di errore

La categoria è la parte stabile dell'errore: è un simbolo, il programma può confrontarlo e fare dispatch su di esso. La tabella elenca le categorie che lo standard definisce; i messaggi che vedi stampati sono invece dell'implementazione e possono cambiare.

| kind | quando |
|---|---|
| `:syntax` | errore di parse; `data` contiene `line`, `col` e `incomplete` |
| `:undef` | nome, funzione, macro o predicato non definito |
| `:type` | tipo sbagliato (campi, ritorno, `::`, condizioni non `Bool`) |
| `:method` | nessun metodo applicabile; `data.types` contiene i tipi degli argomenti |
| `:ambiguous` | più metodi ugualmente specifici |
| `:field`, `:bounds`, `:domain` | campo inesistente, indice fuori range, errore matematico |
| `:instantiation` | variabile logica non legata dove serve un valore |
| `:contract` | `requires` / `ensures` violata (cap. 12) |
| `:budget` | passi esauriti (cap. 13) |
| `:effect` | effetto non dichiarato (cap. 15) |

| kind | quando |
|---|---|
| `:host` | qualsiasi altro errore dell'interprete |

Un errore non gestito al top level interrompe il programma: il messaggio riporta la riga dell'istruzione (`riga 12: …`) e il codice di uscita è `1`.

## 11.4 In Core: errori come liste taggate

Core non ha `try` / `catch` né il tipo `Err`: un errore **termina il processo** con exit code `1` e un messaggio diagnostico su stderr. Ma il pattern "errore come dato" si può già scrivere, ed è esattamente così che S1 implementerà Full. La convenzione è `[:Err, kind, msg]`. **[Core]**

```
err(kind, msg) = [:Err, kind, msg]
is_err(v) = typeof(v) == :List && length(v) == 3 &&
v[1] == :Err

function div_intera(a, b)
    if b == 0
        return err(:domain, "divisione per zero")
    end
    q = 0
    while a >= b
        a -= b
        q += 1
    end
    q
end

r = div_intera(7, 0)
if is_err(r)
    println("errore ", r[2], ": ", r[3])
```

```
end
println(div_intera(17, 5))
```

**Output:**

```
errore domain: divisione per zero
3
```

Nota che `&&` cortocircuita: `v[1]` viene valutato solo se `v` è una lista di 3 elementi.

Il perché di una lista taggata è semplice: in Core non esiste un tipo `Err`, ma esistono le liste e i simboli, e una lista con il tag in prima posizione può rappresentare qualunque record. `is_err` è quindi una convenzione, non una garanzia di tipo: chiunque può costruire una lista `[:Err, ...]` con campi sbagliati, e chi la riceve deve controllare ciò che gli serve. È lo stesso patto che gli interpreti in `selfhost/` usano per le struct (cap. 2.3). La parte importante non è il contenitore, è l'idea: un fallimento è un valore che viaggia come gli altri, e chi lo riceve può scegliere se gestirlo, trasformarlo o restituirlo.

## Esercizi

1. **Costruire un errore. [Full]** Crea un `Err` con `err(...)` / `Err(...)`, ispeziona `.kind` e stampalo.
2. **Catturare. [Full]** Con `try … end` cattura un errore di tipo e stampa la sua categoria.
3. **Le categorie.** Abbina ogni categoria a un programma che la produce: `:syntax`, `:undef`, `:type`, `:method`, `:bounds`, `:contract`, `:budget`, `:effect`.
4. **Propagazione. [Full]** Un errore non gestito a che cosa porta? Verifica codice di uscita, messaggio e riga segnalata.

5. **Errori in Core.** Scrivi `err(kind, msg) = [:Err, kind, msg]` e `iserr(x)` come nel capitolo 11.4 e usali in un piccolo programma.
6. **Dispatch sulla categoria. [Full]** Scrivi una funzione che, dato un `Err`, restituisca un messaggio diverso per `:type` e per `:bounds`.
7. **`:method` o `:ambiguous`?** Scrivi un esempio per ciascuna delle due categorie e spiega la differenza.
8. **I dati dell'errore. [Full]** Che cosa contengono `data` per un `:syntax` (`line`, `col`, `incomplete`) e per un `:method` (`data.types`)? Costruisci un esempio per ciascuno.
9. **Fallire come valore.** Riscrivi una funzione che termina il programma con `exit`/errore in una che restituisce un `Err` (o una lista taggata) e gestiscilo al chiamante.
10. **Messaggi o categorie?** Perché il programma deve confrontare `kind` e non il testo del messaggio? Scrivi un esempio in cui il testo cambia ma la categoria no.

---

## 12. Contratti

Un contratto è una precondizione (`requires`) o una postcondizione (`ensures`) dichiarata nel corpo di una funzione. **[Full]**

Un contratto non è un commento: è una condizione eseguita, e una sua violazione è un errore come dato. Serve a rendere esplicito che cosa una funzione pretende da chi la chiama e che cosa promette a chi la riceve, senza affidare il patto alla buona volontà di chi legge. Non dimostra che la proprietà valga per ogni input possibile — la controlla per le esecuzioni che avvengono — ma trasforma una violazione silenziosa in un `Err` che riporta quale clausola è fallita.

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end
```

- `requires c` viene controllata **nel punto in cui compare**: per convenzione all'inizio del corpo.
- `ensures c` viene registrata e controllata all'uscita della funzione, con `result` legato al valore restituito, **dopo** il controllo del tipo di ritorno.
- Una violazione produce `Err(:contract, "requires violata in safe_div: b != 0", Info(fn, clause, cond[, result]))`. `cond` contiene il **codice** della condizione: è un dato, si può ispezionare.
- Le condizioni sono **pure**: un effetto dentro un contratto dà `Err(:effect)`.
- Al top level `requires c` funziona come un'asserzione.
- `ensures` ha senso solo dentro una `function`: usa `result`, che esiste soltanto quando la funzione sta per restituire un valore. Fuori da una funzione non c'è un risultato a cui riferirsi.

```
println(safe_div(6, 3))

e = try safe_div(1, 0) end
println(e.kind)
println(e.data.clause)
println(e.data.fn)
```

**Output (golden Full):**

```
2.0
contract
requires
safe_div
```

Nell'esempio `safe_div(6, 3)` produce `2.0`: il tipo di ritorno `Float` converte il risultato e lo espone come promessa mantenuta. `safe_div(1, 0)` non arriva mai a dividere: la precondizione viene controllata all'inizio, l'errore è già un valore e il programma continua. Il messaggio dell'errore porta con sé `clause = requires` e `fn = safe_div`, quindi chi chiama può distinguere una violazione d'ingresso da una promessa d'uscita non mantenuta senza leggere il testo.

La sequenza di controllo è significativa: il corpo valuta le clausole nell'ordine in cui compaiono; le precondizioni vengono verificate nel punto in cui sono dichiarate, mentre le postcondizioni vengono conservate fino al completamento della funzione. Il valore finale viene prima sottoposto al vincolo di ritorno e poi esposto come `result` alle clausole `ensures`. Un errore di contratto include quale clausola è fallita e il codice che la rappresenta, così chi chiama può distinguere una violazione d'ingresso da una promessa d'uscita non mantenuta. I contratti controllano un'invariante osservabile, ma non provano matematicamente che valga per ogni input possibile: vengono valutati per le esecuzioni che avvengono.

## Esercizi

1. **`requires`. [Full]** Scrivi `radice(x::Number)::Float` con una precondizione `x >= 0` e provala con input validi e non validi.

2. **`ensures` e `result`. [Full]** Aggiungi a una funzione una postcondizione sul valore restituito usando `result`.
3. **Leggere la violazione. [Full]** Dopo una `requires` violata, stampa `e.kind`, `e.data.clause` e `e.data.fn`: che cosa ti dicono?
4. **L'ordine dei controlli.** Perché il tipo di ritorno viene controllato prima delle `ensures`? Scrivi una funzione in cui l'ordine si nota.
5. **Condizioni pure. [Full]** Metti un `println` dentro un contratto: che errore ottieni e perché?
6. **`requires` al top level.** Che cosa fa `requires c` fuori da una funzione? Usalo come asserzione in un piccolo programma.
7. **`ensures` fuori da una funzione.** Perché non ha senso? Che cosa manca?
8. **`safe_div`.** Riscrivi l'esempio del capitolo e prova `safe_div(6, 3)` e `safe_div(1, 0)`: il secondo interrompe il programma?
9. **Contratti che controllano, non provano.** Scrivi un contratto che passi per alcuni input e fallisca per altri: che cosa ti dice sulla differenza tra controllo e dimostrazione?
10. **Contratto o test?** Scrivi la stessa proprietà prima come test e poi come contratto: quando conviene l'uno e quando l'altro?

---

## 13. Budget

Un **budget** limita quanti passi il programma può consumare. Un passo è una chiamata, un'iterazione di ciclo o un passo di risoluzione. Quando il budget si esaurisce, **il valore del `with` è `Err(:budget)`**: il programma non si blocca e non viene interrotto. **[Full]**

```
r = with budget(steps = 10_000)
    while true end
end
println(r.kind)
```

**Output (golden Full):**

```
budget
```

I budget si annidano e vale il più stretto. Un `with` converte in valore **solo** l'esaurimento del proprio budget, mentre quello di un budget esterno continua a propagarsi. `solve … steps N` è il budget specifico di una query.

Il budget misura il **lavoro**, non il tempo: due esecuzioni dello stesso programma con lo stesso budget consumano gli stessi passi, quindi il risultato è riproducibile. Un timeout basato sull'orologio non lo sarebbe e renderebbe i test instabili. Per questo il limite è parte del linguaggio e non dell'ambiente: un programma può decidere che cosa fare quando lo esaurisce, e quella decisione è anch'essa codice.

Il motore logico usa uno stack esplicito: la ricorsione infinita, anche sinistra, finisce in `Err(:budget)` e **non** in uno stack overflow.

Il limite è parte del risultato del calcolo perché il programma può scegliere come trattarlo: mostrare una diagnosi, ridurre il problema o restituire un risultato parziale solo se la propria logica lo consente. Non confondere `limit n`, che limita il numero di soluzioni restituite, `depth d`, che taglia i rami oltre una profondità, e `steps s`, che limita il lavoro complessivo. Una query che ha esplorato lo spazio previsto senza trovare soluzioni restituisce una lista vuota; una query fermata dall'esaurimento dei passi segnala invece `Err(:budget)`.

## Esercizi

1. **Il budget scatta. [Full]** Avvolgi un `while true` in `with budget(steps = 10_000)` e verifica che il valore del blocco sia `Err(:budget)`.
2. **Che cos'è un passo?** Individua nel capitolo tre costrutti che consumano passi e spiega perché il budget misura lavoro e non tempo.
3. **Budget annidati. [Full]** Con un budget interno di 100 passi dentro uno esterno di 1000: quale dei due scatta per primo e che cosa produce il blocco interno?
4. **Riproducibilità.** Perché due esecuzioni con lo stesso budget danno gli stessi passi? Che problema risolverebbe (e quale no) un timeout sull'orologio?
5. **Ricorsione sinistra. [Full]** `f(n) = f(n + 1)` dentro un budget: quale errore invece di uno stack overflow? Verifica.
6. **`limit`, `depth`, `steps`. [Full]** Distingui le tre opzioni di `solve` con un esempio per ciascuna e indica quale delle tre produce un errore.
7. **Diagnosi. [Full]** Scrivi un programma che, ricevuto `Err(:budget)`, stampi una diagnosi invece di propagare il panico.
8. **Quanti passi?** Stima e poi verifica quanti passi consuma un ciclo che gira 10 volte dentro un budget: quale relazione osservi tra iterazioni e passi?
9. **Vuoto o esaurito? [Full]** Spiega la differenza tra una query che restituisce una lista vuota e una terminata da `Err(:budget)`: scrivi due query che mostrino entrambi i casi.
10. **Perché nel linguaggio.** Scrivi due righe sul perché il budget è parte del linguaggio e non dell'ambiente di esecuzione.

## 14. Tracce

La traccia di un calcolo è un **valore**, non output stampato. **[Full]**

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)

t = with trace
    fib(3)
end
println(t.result)
println(length(t.trace))
```

**Output (golden Full):**

```
2
10
```

`fib(3)` effettua 5 chiamate (`fib(3)`, `fib(2)`, `fib(1)`, `fib(0)`, `fib(1)`): la traccia registra **due eventi per ogni chiamata**, `:call` e `:exit`, quindi 10 eventi. Per `fib(5)` sono 15 chiamate e 30 eventi — è il valore atteso dal caso di conformance `garanzie_core`.

`with trace … end` restituisce `Traced(result, trace)` con gli eventi `:call` e `:exit` di ogni funzione generica chiamata nel corpo. `solve … trace` restituisce `Traced(result, trace)` dove `result` è la lista delle soluzioni e gli eventi sono le porte del modello a scatole di Byrd (`call`, `exit`, `redo`, `fail`) più `depth`, quando un ramo viene tagliato da `depth`.

Gli eventi sono record: si filtrano e si contano come ogni altro valore.

Raccogliere una traccia non è gratis: ogni evento è un valore allocato, e su calcoli grandi la traccia può pesare più del risultato. Chiedila quando ti serve, non come impostazione predefinita.

Una traccia aggiunge una dimensione temporale ai dati di debug: due esecuzioni possono avere lo stesso risultato ma seguire percorsi diversi, per esempio se una query ha dovuto provare e scartare alternative. Invece di scrivere log dentro ogni funzione, il chiamante racchiude il calcolo in `with trace` oppure chiede `trace` a `solve`; il risultato e gli eventi vengono restituiti insieme. Questo mantiene separati il risultato applicativo e la diagnostica, e consente di analizzare la traccia con gli stessi strumenti usati per liste e record.

## Glossario degli eventi di traccia (porte di Byrd)

Quando si usa `solve … trace`, ogni passo della ricerca produce un evento che registra **a che riferimento** (quale goal, quale clausola, quali variabili logiche) si sta lavorando. Le quattro porte del modello a scatole di Byrd sono:

| porta | quando si verifica | significato |
|---|---|---|
| `:call` | si entra in un goal | inizio della valutazione della clausola candidata |
| `:exit` | unificazione **riuscita** | il goal è dimostrato: è il "success" della ricerca |
| `:redo` | backtracking | si torna indietro per cercare un'altra soluzione |
| `:fail` | nessuna clausola applicabile | il goal **fallisce**: nessuna soluzione per quel ramo |

Note importanti:

- **"success" e "fail" non sono esiti generici**: sono le porte `:exit` e `:fail` della traccia, riferite al goal/clausola in corso. Il fallimento di un goal non è un'eccezione: si manifesta come **lista di soluzioni vuota** nel `result` di `Traced(result, trace)`.
- `depth` appare come evento aggiuntivo quando un ramo viene tagliato dal limite di profondità (`solve … depth N`), segnalando che la ricerca è stata interrotta prima di un `:exit` / `:fail`.
- Gli eventi sono **record ordinari**: si filtrano (`[e for e in t.trace if e.port == :fail]`), si contano (`length`), si ispezionano (`e.args`, `e.depth`) come ogni altro valore Triad.
- Il riferimento di ogni evento è il **goal corrente e le variabili logiche** (`?x`, `?y`, …) coinvolte in quel passo: leggendo la traccia si ricostruisce *quale* alternativa è stata provata e *perché* è fallita.

```
# Conta i fallimenti di una ricerca
t = solve path(:a, :c) trace
fails = [e for e in t.trace if e.port == :fail]
println(length(fails))
```

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)
t = with trace
    fib(3)
end
println([e.args for e in t.trace if e.port == :call])
```

**Output (golden Full):**

```
[[3], [2], [1], [0], [1]]
```

## Esercizi

1. **La prima traccia. [Full]** Avvolgi un piccolo `solve` in `with trace` e leggi la sequenza degli eventi prodotti.
2. **Le porte di Byrd.** Che cosa significano `:call`, `:redo`, `:exit`, `:fail`? Collega ciascuna a un momento preciso della ricerca.
3. **Contare i `:redo`. [Full]** Traccia un goal con due soluzioni e conta quanti `:redo` compaiono: a cosa corrispondono nel backtracking?
4. **Contare i fallimenti.** Riscrivi l'esempio del capitolo che conta i fallimenti di una ricerca e provalo su una knowledge base tua.
5. **Debug di un goal.** Un goal non trova risposte: come usi la traccia per capire dove la ricerca fallisce? Scrivi la sequenza che ti aspetti.
6. **Eventi come dati.** La traccia produce valori: scrivi un programma che filtri gli eventi per tipo e ne conti ciascuna categoria.
7. **Traccia o `println`?** Perché per un programma logico una traccia è meglio di `println` di debug? Rispondi con un esempio in cui l'ordine dei tentativi conta.
8. **Prevedere l'ordine.** Dato un goal con due soluzioni e una clausola che fallisce, scrivi a mano l'ordine atteso degli eventi e verifica con `with trace`.
9. **Profondità e traccia. [Full]** Aggiungi `depth` a una query tracciata: che cosa cambia negli eventi osservati?

10. **Trace e budget. [Full]** Unisci `trace` e `steps` in una query: come si riconosce nell'emissione degli eventi l'esaurimento del budget?

---

## 15. Effetti dichiarati

Le funzioni sono **pure per impostazione predefinita**. Gli effetti si dichiarano nella firma, dopo i parametri. **[Full]**

«Puro per impostazione predefinita» è una scelta di sicurezza: se dimentichi una dichiarazione, la funzione non può fare più del previsto, e l'errore arriva quando l'effetto viene tentato. In questo senso un effetto è una **capacità**, non un'annotazione descrittiva: `!io` dice che la funzione ha il permesso di stampare, e il controllo dinamico verifica il permesso al momento dell'esecuzione.

```
function saluta(nome::String) !io
    println("ciao, ", nome)
end

function impara(x) !state
    fact visto($x)
end
```

| effetto | operazioni |
|---|---|
| `io` | `println`, `print` |
| `state` | `fact` / `rule` eseguiti dentro una funzione (modificano la knowledge base) |

Il controllo è **dinamico**. Una funzione viene eseguita solo con gli effetti che dichiara, e solo se il chiamante li permette. Se una funzione pura chiama `println`, o chiama una funzione `!io`, si ottiene `Err(:effect)`; `data` contiene `fn`, `caller` e `missing`.

La purezza per impostazione predefinita significa che una funzione può calcolare e restituire valori senza modificare lo stato esterno né produrre I/O; non significa che il runtime analizzi staticamente ogni possibile percorso prima dell'esecuzione. Il controllo dinamico osserva gli effetti mentre avvengono e li confronta con il contesto attivo. Le mutazioni locali e quelle di liste non sono classificate come effetti dichiarati in questa specifica: `!state` si riferisce alla modifica della knowledge base tramite `fact` e `rule`, non a qualunque assegnamento.

- Il top level permette tutti gli effetti.
- Le lambda ereditano gli effetti del contesto in cui vengono eseguite.
- Macro e contratti sono puri.
- Leggere la knowledge base (`solve`, chiamare un predicato) è puro.
- La mutazione locale di dati (`push!` su una lista, assegnamento a variabili locali) **non** è un effetto: modifica solo valori raggiungibili dall'ambiente locale.

```
function rumorosa()
    println("faccio rumore")
end

function tranquilla()
    rumorosa()
end
```

```
e = try tranquilla() end
println(e.kind)
```

**Output (golden Full):**

```
effect
```

`rumorosa` non dichiara `!io`, quindi il suo `println` è un effetto non permesso; `tranquilla` la chiama da un contesto puro e riceve lo stesso errore. `data` contiene `fn` (chi ha tentato l'effetto), `caller` (da dove) e `missing` (quale effetto mancava): sono le tre informazioni che servono per correggere la firma. Il controllo è dinamico, quindi un ramo che non viene eseguito non produce errori di effetto, e non c'è un'analisi statica che ti avvisa prima: la dichiarazione è un patto che il runtime verifica mentre il codice gira.

## Esercizi

1. **Pura di default. [Full]** Scrivi una funzione che stampi senza dichiarare `!io`: quale errore ottieni e da chi viene segnalato?
2. **Dichiarare `!io`.** Aggiungi la dichiarazione alla funzione dell'esercizio 1 e verifica che ora funziona.
3. **`!state`. [Full]** Scrivi una funzione che aggiunga un fatto (`fact visto($x)`): quale effetto richiede e perché `fact` è un effetto mentre un assegnamento locale no?
4. **Chiamata indiretta.** Come nell'esempio `tranquilla` → `rumorosa`: chi riceve l'errore? Scrivi un caso con tre livelli di chiamata.
5. **`fn`, `caller`, `missing`. [Full]** Costruisci un `Err(:effect)` e ispeziona i tre campi di `data`: a che cosa serve ciascuno per correggere il codice?

6. **Al top level.** Perché al top level si può stampare senza dichiarazioni? Che cosa significa che il top level «permette tutti gli effetti»?
7. **Lambda ed effetti.** Da dove una lambda eredita gli effetti? Scrivi un esempio con una lambda creata dentro una funzione `!io`.
8. **Puro leggere, impuro scrivere.** Perché `solve` è puro mentre `fact` e `rule` eseguiti dentro una funzione no?
9. **Mutazione locale.** Perché `push!` e gli assegnamenti su variabili locali non sono effetti dichiarati? Dove passa il confine secondo la specifica?
10. **Progetta una firma.** Data una funzione che stampa una relazione e aggiunge un fatto alla knowledge base, scrivi la firma corretta e giustifica ogni effetto dichiarato.

---

## 16. Logica

Triad integra la programmazione logica **nella sintassi a blocchi**: i fatti e le regole sono costrutti di primo livello, non un linguaggio incorporato. **[Full]**

Fatti e regole formano la **knowledge base**, che il programma costruisce mentre gira. Un fatto asserisce che una relazione vale; una regola dice che vale a condizione che valgano altre cose. `solve` pone una domanda alla base e restituisce le risposte come valori: non c'è un interprete separato da invocare, perché la logica sta dentro la stessa sintassi e lo stesso motore del resto del linguaggio.

### 16.1 Fatti, regole, termini

```
fact parent(:tom, :bob)
fact parent(:bob, :jim)

rule ancestor(?a, ?d) :- parent(?a, ?d)
rule ancestor(?a, ?d) :- parent(?a, ?x), ancestor(?x, ?
d)
```

Dentro un termine:

- `:atomo` è un simbolo; `?x` è una variabile logica; numeri, stringhe e liste sono se stessi;
- `f(t1, t2)` è un termine composto. Anche `?x + 1` è un termine ( `+(?x, 1)` ) e **non viene valutato**;
- un **nome nudo** è un valore dell'ambiente corrente, letto al momento in cui viene eseguito `fact`, `rule` o `solve`: `chi = :bob; solve parent(chi, ?c)`;
- `$(espr)` inserisce esplicitamente un valore calcolato: `fact eta(:bob, $(2026 - 1975))`;
- `[a, b]` e `[?h | ?t]` sono liste e forme che unificano con le liste;
- il nome di una struct con argomenti, come `Point(?x, ?y)`, unifica con le istanze di quella struct.

Le variabili di una clausola vengono **rinominate a ogni uso** (standardizzazione a parte). L'unificazione fa l'**occurs-check**: `solve ?x = f(?x)` non ha soluzioni.

Per leggere una query occorre distinguere i **termini** dai normali valori calcolati. Un simbolo come `:alice` è un termine costante; `?who` è una variabile logica che cerca un valore; una forma come `parent(:alice,`

`?who)` descrive un goal da dimostrare. Le variabili logiche non sono variabili locali Triad assegnate con `=`: il motore cerca associazioni che rendano compatibile il goal con fatti e regole. In particolare, `?x + 1` dentro un termine non si calcola da sé; l'operatore `is` esprime esplicitamente la richiesta di valutare un'espressione usando variabili già legate.

L'unificazione costruisce progressivamente un insieme di associazioni. Se si confrontano `parent(:alice, ?who)` e `parent(:alice, :bob)`, gli argomenti corrispondenti coincidono e il motore può legare `?who` a `:bob`; se un argomento è diverso, quel tentativo fallisce. Quando più clausole possono soddisfare un goal, il backtracking torna al punto di scelta e prova la clausola successiva. Ogni soluzione è una fotografia delle variabili richieste dalla query; una lista vuota significa "nessuna dimostrazione trovata", non un errore di esecuzione. I limiti `limit`, `depth` e `steps` controllano quante soluzioni o quanti passi di ricerca sono ammessi.

## 16.2 I goal

Un goal è la descrizione di ciò che vuoi che sia vero: il motore cerca legami per le variabili che lo rendono dimostrabile. La tabella raccoglie le forme che un goal può assumere; le prime due sono le più importanti, perché `p(t…)` è l'unico modo per interrogare la knowledge base e la congiunzione è l'unico modo per chiedere più cose insieme.

| goal | significato |
|---|---|
| `p(t…)` | predicato della knowledge base, risolto con SLD e backtracking, clausole in ordine |
| `g1 && g2`, `(g1, g2)` | congiunzione; nelle `rule` anche con `,` |
| `g1 \|\| g2` | disgiunzione |

| goal | significato |
|---|---|
| `!g`, `not(g)` | negazione per fallimento |
| `not(b)` *fuori* da un goal | è una funzione: nega un `Bool` (`not(true) == false`) |
| `a = b` | unificazione |
| `?y is espr` | valuta `espr` (le variabili devono essere legate) e unifica con `?y` |
| `a < b`, `==`, `!=`, `<=`, `>`, `>=` | confronto tra valori, valutati |
| `f(t…)` con `f` funzione Triad | chiama la funzione con gli argomenti valutati; deve restituire `Bool` |
| `true`, `false` | successo, fallimento |

Un predicato senza clausole con quell'arità, che non sia nemmeno una funzione, dà `Err(:undef, "predicato sconosciuto: p/2")`. Una variabile non legata dentro un'espressione dà `Err(:instantiation)`.

## 16.3 `solve` è un valore

`solve` **restituisce una lista** di `Solution`. Non stampa nulla: è un valore come un altro.

```
fact parent(:alice, :bob)
fact parent(:alice, :carol)
fact parent(:bob, :dave)

rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y,
?z)

sols = solve grandparent(:alice, ?who) limit 5
```

```
println(sols)
println(sols[1].who)
println(length(sols))
```

**Output (golden Full):**

```
[{who = :dave}]
dave
1
```

Ogni soluzione lega le variabili della query **non anonime**, nell'ordine in cui compaiono. Le variabili rimaste libere restano `?x#n`. `vars(sol)` restituisce i nomi legati.

`Solution` è un record con i nomi della query: `sols[1].who` legge il legame, e `vars(sol)` elenca i nomi. Il risultato di `solve` è una lista normale, quindi puoi filtrare, ordinare o contare le soluzioni con gli strumenti del resto del linguaggio: la logica non produce un formato speciale, produce valori.

Opzioni di `solve`:

- `limit n` — al massimo `n` soluzioni;
- `depth d` — i rami più profondi di `d` vengono tagliati (falliscono);
- `steps s` — al massimo `s` passi di risoluzione, altrimenti `Err(:budget)`; il valore predefinito è `1 000 000`;
- `trace` — vedi cap. 14.

Le opzioni non sono equivalenti: `limit` ferma la ricerca quando hai abbastanza soluzioni, `depth` taglia i rami troppo profondi e li considera falliti, `steps` limita il lavoro complessivo e segnala l'esaurimento con

`Err(:budget)`. Distinguerle conta perché solo l'ultima è un errore: le altre due producono un risultato parziale ma legittimo.

## 16.4 Un predicato usato come funzione

Chiamare un predicato dove ci si aspetta un valore restituisce un `Bool`: dice se il goal è dimostrabile.

```
if grandparent(:alice, :dave)
    println("alice è nonna di dave")
end
```

## 16.5 Esempi

Percorso con costo:

```
fact edge(:a, :b, 2)
fact edge(:b, :c, 3)
rule path(?x, ?y, ?c) :- edge(?x, ?y, ?c)
rule path(?x, ?y, ?c) :- edge(?x, ?z, ?c1), path(?z, ?
y, ?c2), ?c is ?c1 + ?c2
println((solve path(:a, :c, ?costo))[1].costo)
```

**Output (golden Full):**

```
5
```

Il costo si accumula per composizione: `path(:a, :c, ?costo)` dimostra il goal unendo il fatto `edge(:a, :b, 2)` alla regola ricorsiva; il secondo passo trova `edge(:b, :c, 3)` e `?c is ?c1 + ?c2`

calcola `2 + 3`. `is` è necessario perché `?c1 + ?c2` dentro un termine resterebbe un termine non valutato; `is` è il punto in cui la logica chiede al valutatore di fare l'aritmetica.

Append in stile Prolog:

```
rule app([], ?l, ?l)
rule app([?h | ?t], ?l, [?h | ?r]) :- app(?t, ?l, ?r)
println((solve app([1, 2], [3], ?r))[1].r)
println(length(solve app(?a, ?b, [1, 2, 3])))
```

**Output (golden Full):**

```
[1, 2, 3]
4
```

`app([1, 2], [3], ?r)` unifica la lista concatenata e restituisce `[1, 2, 3]`. La seconda query non fissa il risultato ma chiede tutte le coppie di liste che, concatenate, danno `[1, 2, 3]`: sono quattro, perché il primo pezzo può essere la lista vuota, `[1]`, `[1, 2]` o `[1, 2, 3]`. Il numero non è un errore: è il motore che enumera le alternative, e `limit` serve proprio a fermarlo quando non ti interessano tutte.

Negazione per fallimento:

```
fact parent(:tom, :bob)
persone = [:tom, :bob]
foglie = filter(p -> !parent(p, ?_), persone)
println(foglie)
```

**Output (golden Full):**

```
[:bob]
```

`!parent(p, ?_)` è la negazione per fallimento: `p` passa il filtro solo se il goal `parent(p, ?_)` non è dimostrabile. `:tom` ha un figlio, quindi non passa; `:bob` non ne ha, quindi passa. La variabile anonima `?_` è quella che non compare nel risultato: ogni sua occorrenza è una variabile nuova, così non devi inventare un nome per una variabile che non ti interessa.

## Esercizi

1. **Fatti. [Full]** Dichiara i fatti `parent(:tom, :bob)`, `parent(:bob, :jim)` e altri due tuoi, e interroga la base con `solve parent(:tom, ?c)`.
2. **Regole. [Full]** Scrivi `rule ancestor(?a, ?d)` nelle due forme del capitolo e verifica che `ancestor(:tom, ?jim)` sia dimostrabile.
3. **`solve` è un valore. [Full]** Raccogli le soluzioni in una variabile, stampa `length(sols)` e leggi un legame con `sols[1].who`.
4. **Variabili anonime. [Full]** Usa `?_` in una query dove non ti interessa un legame e spiega perché ogni occorrenza è una variabile nuova.
5. **`is` valuta. [Full]** Perché in `path` il costo si calcola con `?c is ?c1 + ?c2` e non con `?c = ?c1 + ?c2`? Verifica su una knowledge base con costi.
6. **`app` in stile Prolog. [Full]** Riscrivi le regole di `app` e spiega perché `solve app(?a, ?b, [1, 2, 3])` ha quattro soluzioni.
7. **Occurs-check. [Full]** `solve ?x = f(?x)`: quante soluzioni? Perché l'unificazione si rifiuta?

8. **Predicato come funzione. [Full]** Usa `grandparent(:alice, :dave)` come condizione di un `if`: che valore produce?
9. **Negazione per fallimento. [Full]** Con `!parent(p, ?_)` seleziona le «foglie» di una knowledge base familiare, come nell'esempio.
10. **Le opzioni. [Full]** Costruisci una query con molte soluzioni e applica `limit`, `depth` e `steps` separatamente: che risultato ottieni in ciascun caso?

---

## 17. Codice come dato

Il parser produce **un solo tipo di nodo**, `Node(head, args)`. È lo stesso albero che il kernel esegue, che `:( … )` restituisce come valore e che `triad --tree` stampa come S-expression. **[Full]**

L'idea che sta dietro è radicale: il programma non è una sequenza di comandi che il valutatore interpreta a mano, è un **dato**. Il parser produce l'albero, il valutatore esegue l'albero, le macro ricevono pezzi di albero e ne restituiscono altri. Poiché esiste un solo tipo di nodo, chi scrive codice che manipola il programma non deve conoscere dieci rappresentazioni diverse: conosce `Node(head, args)` e basta.

La tabella mostra come le forme che hai già incontrato diventano nodi: leggila come un dizionario tra la sintassi che scrivi e il dato che il programma può ispezionare.

| sintassi | nodo |
|---|---|
| `f(a, b)`, `a + b` | `(call f a b)`, `(call + a b)` |
| `a && b`, `a \|\| b` | `(and a b)`, `(or a b)` |
| `x = e` | `(assign x e)` |

| sintassi | nodo |
|---|---|
| `:alice` | `(sym alice)` |
| `a.x`, `xs[i]` | `(dot a x)`, `(index xs i)` |
| `[a, b]`, `[h \| t]` | `(list a b)`, `(cons (list h) t)` |
| `[e for x in xs if c]` | `(comprehension e x xs c)` |
| `x -> e` | `(lambda (tuple x) e)` |
| `x::T` | `(typed x T)` |
| `"a $x"` | `(string "a " x)` |
| `if c a else b end`, `c ? a : b` | `(if c (block a) (block b))` |
| `while`, `for`, `let`, `begin` | `(while c body)`, `(for x iter body)`, `(let (tuple …) body)`, `(block …)` |
| `function f(a::T)::R !io … end` | `(function f (tuple (typed a T)) R (tuple io) body)` |
| `struct P; x::Float; end` | `(struct P (tuple (typed x Float)))` |
| `macro m(a) … end`, `@m(e)` | `(macro m (tuple a) body)`, `(macrocall m e)` |
| `:(e)`, `quote … end`, `$e` | `(quasi e)`, `(quasi (block …))`, `(unquote e)` |
| `fact t`, `rule h :- g1, g2` | `(fact t)`, `(rule h g1 g2)` |

| sintassi | nodo |
|---|---|
| `solve g limit 5 trace` | `(solve g (tuple (assign limit 5) (assign trace true)))` |
| `try b catch e h end` | `(try b e h)` |
| `with budget(steps = N), trace … end` | `(with (tuple (call budget (assign steps N)) trace) body)` |

## 17.1 Quote, unquote, eval

```
c = :(a.x + f(b, 2))
println(c.head)
println(sexpr(c))

n = 7
quadrato = :($n * $n)
println(eval(quadrato))
```

**Output (golden Full):**

```
call
(call + (dot a x) (call f b 2))
49
```

Dentro un quote, `$x` e `$(e)` inseriscono il **valore** di un'espressione: se è codice viene innestato, se è un numero o una lista diventa una costante, se è un Symbol diventa un identificatore. `eval(c)` esegue codice **nello scope globale**. Gli altri builtin del gruppo "codice" sono `parse`, `unparse`, `sexpr`, `Expr` e `gensym`.

Quotare e valutare sono operazioni opposte ma non intercambiabili. Il parser normalmente trasforma il sorgente in un albero e il valutatore esegue quell'albero; `:(...)` ferma il processo al confine tra le due fasi e restituisce l'albero come valore `Expr`. `eval(expr)` lo consegna di nuovo al valutatore, nello scope globale. L'unquote `$x` dentro una quote inserisce il valore calcolato di `x` nel nuovo albero: se è un `Expr`, innesta codice; se è un numero, crea un nodo costante. `sexpr` serve a ispezionare la struttura e non esegue il programma.

L'esempio mostra il viaggio completo: `:(a.x + f(b, 2))` restituisce un `Expr` senza eseguirlo, `c.head` vale `call` e `sexpr` lo scrive in forma lineare. `:($n * $n)` con `n = 7` non crea una funzione: crea un albero che contiene già la costante `7`, e `eval` lo esegue. La differenza fra `eval` e una chiamata normale è lo scope: `eval` parte dallo scope globale, quindi non vede le variabili locali di chi lo invoca. È il motivo per cui `eval` si usa per costruire codice completo, non per leggere valori del contesto.

## Esercizi

1. **Quotare. [Full]** Scrivi `:(1 + 2)` e osserva il valore prodotto: è un calcolo o un albero?
2. **Unquote. [Full]** Inserisci con `$` un valore calcolato dentro una quote e verifica che venga valutato prima di entrare nell'albero.
3. **`eval`. [Full]** Valuta una struttura quotata con `eval` e confronta il risultato con quello della forma non quotata.
4. **Round-trip. [Full]** Con `parse` e `unparse` fai il round-trip di una piccola espressione: il testo cambia?
5. **Albero uniforme.** Perché `1`, `:x`, `[1, 2]` e `:(f(1))` sono tutti nodi `Node(head, args)`? Scrivi per ciascuno head e args.

6. **Codice che costruisce codice. [Full]** Scrivi una funzione che, dato un nome, produca la quote di una funzione che lo stampa.
7. **`gensym`. [Full]** A cosa serve generare nomi freschi? Scrivi un esempio in cui un nome fisso creerebbe un conflitto.
8. **Quotare un blocco. [Full]** Quota un `function … end` completo e riscrivilo in testo con `unparse`.
9. **Dati o codice?** Riconosci quali di questi sono valori e quali sono codice quotato: `1`, `:(1)`, `:(x)`, `:x`, `"x"`.
10. **`--tree`.** Usa `triad --tree FILE` su un programma piccolo e confronta l'S-expression con la struttura `Node` vista nel capitolo.

---

## 18. Macro

Una macro riceve gli argomenti **come codice**, non valutati, e restituisce codice, che viene poi eseguito al posto della chiamata `@nome(...)` nello scope del chiamante. **[Full]**

La differenza rispetto a una funzione non è solo tecnica: una funzione riceve valori già calcolati, quindi non può decidere se valutare un argomento una volta, due volte o mai. Una macro riceve la forma sorgente, e questa è la sua unica vera ragione d'essere: si usa quando il comportamento dipende dal codice, non dal valore degli argomenti.

```
macro unless(c, body)
    :(if !$c; $body; end)
end

macro swap(a, b)
    t = gensym(:tmp)
    quote
```

```
        $t = $a
        $a = $b
        $b = $t
    end
end

x = 1
y = 2
@swap(x, y)
println(x, " ", y)
```

**Output (golden Full):**

```
2 1
```

L'espansione è **pura** (§15) e **non è igienica**: per i nomi temporanei si usa `gensym()`.

Questa distinzione è importante per gli effetti. Una funzione riceve normalmente valori già calcolati; una macro riceve la forma sorgente e può quindi decidere se inserirla una volta, più volte o non inserirla. Le macro di Triad non sono igieniche: un nome temporaneo introdotto dalla macro può collidere con un nome del chiamante. `gensym` crea un simbolo fresco per i temporanei, ma non rende automaticamente sicuro ogni riferimento libero. Per questo conviene limitare le macro a trasformazioni piccole e ispezionare l'albero espanso prima di attribuire al risultato un comportamento inatteso.

## 18.1 Codice che genera codice

```
function poly(coeffs)
    acc = :(0)
```

```
    i = 0
    for c in coeffs
        acc = :($acc + $c * x ^ $i)
        i += 1
    end
    acc
end

p = poly([1, 2, 3])
x = 2
println(eval(p))
```

**Output (golden Full):**

```
17
```

`poly` costruisce un albero a ogni giro: `:(0)` è il punto di partenza, e ogni coefficiente innesta l'albero precedente con `$acc`, la costante `$c` e l'esponente `$i`. Il `for` qui lavora su valori normali; solo i quote e gli unquote costruiscono codice. Con `[1, 2, 3]` l'albero finale rappresenta `1 + 2 * x + 3 * x^2`, e con `x = 2` vale `1 + 4 + 12 = 17`. `eval(p)` cerca `x` nello scope globale: se `x` fosse stato locale a una funzione, l'albero non l'avrebbe visto.

## Esercizi

1. **`@misura`. [Full]** Scrivi la macro `@misura(label, espr)` che stampa etichetta e valore e restituisce il valore.
2. **Macro o funzione?** Perché una macro riceve **codice** e non valori? Scrivi un esempio in cui la differenza si vede (per esempio un costrutto che non deve essere valutato subito).

3. `@due_volte`. **[Full]** Scrivi una macro che esegua due volte la sua espressione e provala con una `println`.
4. **Unquote dei parametri.** Usa `$` per inserire ciascun parametro della macro nella quote: che cosa succede se lo dimentichi?
5. **Igiene. [Full]** Costruisci un esempio in cui una macro con un nome fisso «calpesta» una variabile del chiamante, poi correggila con `gensym`.
6. **Codice che genera codice. [Full]** Scrivi una macro che, dato un nome, definisca una funzione che lo restituisca.
7. **Macro pure.** Perché macro e contratti sono puri? Che cosa succede se una macro provasse a stampare durante l'espansione?
8. **Guardare l'espansione. [Full]** Osserva il codice prodotto da una macro (con `unparse` o `--tree`) e confrontalo con la tua aspettativa.
9. **Ordine di espansione.** Con due macro annidate, in quale ordine avviene l'espansione? Verifica con un esempio.
10. **Quando non serve.** Scrivi un caso in cui una funzione basta e una macro sarebbe solo complicazione, e spiega la scelta.

---

## 19. Builtin

Un **builtin** è una funzione che il linguaggio fornisce senza che tu la definisca. Alcuni sono primitive vere e proprie (l'aritmetica), altri sono comodità costruite sopra il nucleo (le funzioni di ordine superiore in Full). La tabella elenca i builtin di Triad-Full, raggruppati per ambito; sotto trovi la parte che esiste anche in Core.

| ambito | funzioni |
|---|---|
| aritmetica e confronto | `+ - * / ^ % == != < <= > >= !`, `abs sqrt floor ceil round exp log sin cos div mod min max`; `*` concatena le stringhe |
| liste | `length isempty first last rest reverse sort sum push! pop! append! vcat collect`, `a:b`, `x in xs` |
| ordine superiore | `map filter reduce any all count` |
| stringhe e output | `string repr println print` |
| tipi | `typeof`, `isa`, `Int Float String Symbol` |
| errori | `err iserr throw error` |
| codice | `eval parse unparse sexpr Expr gensym` |
| logica | `vars(sol)` |

Due dettagli che la tabella nasconde. Primo, gli **operatori sono funzioni**: in Full `+` è un nome come gli altri, quindi `reduce(+, xs)` funziona e puoi aggiungere metodi a un operatore (cap. 7). Secondo, builtin e funzioni utente non sono mondi separati: in Full puoi aggiungere metodi a un nome che è anche un builtin, come `+` su `Point`, e il builtin resta la scelta per i valori che non hanno un metodo più specifico.

**Il sottoinsieme di Core** è più piccolo: `+ - * % ^`, i confronti, `&& || !`, `length`, `push!`, la costruzione `[a, b]` e l'indicizzazione `xs[i]`, `*` sulle stringhe, `Int`, `string`, `Symbol`, `print`, `println`, `typeof` (`SPEC-CORE §7`). La riduzione è per progetto: un interprete di bootstrap deve poter eseguire se stesso, e ogni builtin in più è

codice che quel bootstrap deve fornire. Per questo in Core `map`, `filter`, `reduce`, `sum`, `sort`, `div` e `mod` **non esistono**: si scrivono a mano (cap. 22.1).

## Esercizi

1. **Core o Full?** Dividi la tabella dei builtin in due colonne: quali esistono anche in Core e quali no? Confronta con `SPEC-CORE §7`.
2. **Operatori come funzioni. [Full]** Perché `reduce(+, xs)` funziona? Che cosa implica per la possibilità di aggiungere metodi a `+`?
3. **`map` scritto a mano.** Implementa `mappa(f, xs)` in Core (cap. 22.1) e confronta il risultato con `map` **[Full]**.
4. **`reduce` scritto a mano.** Implementa la riduzione con un ciclo e usala per somma e prodotto.
5. **La famiglia `first last rest`. [Full]** Usali su una lista; poi riscrivi `first` e `rest` con l'indicizzazione e `length`.
6. **`div` e `mod`. [Full]** Confronta `div(7, 2)`, `mod(7, 2)`, `7 / 2` e `7 % 2`: quali differenze di tipo e di risultato?
7. **Produrre e gestire errori. [Full]** Con `err`, `iserr`, `throw` e `error` scrivi un esempio per ciascun ruolo e spiega quando usare l'uno o l'altro.
8. **Il gruppo «codice». [Full]** Fai un esempio per ciascuno: `eval`, `parse`, `unparse`, `sexpr`, `gensym`.
9. **`typeof` e `isa`.** Scrivi un esempio in cui danno informazioni diverse e spiega la differenza.
10. **Estendere un builtin. [Full]** Aggiungi un metodo `*` per una struct `Stringa` ... meglio: aggiungi un metodo `+` a una struct e verifica che `+` continui a funzionare per gli `Int`.

## 20. Riga di comando e REPL

Un programma Triad non vive solo dentro un file: lo standard definisce anche come si invoca l'interprete. L'interfaccia di `triad` è normativa (`SPEC §14`), cioè fa parte del linguaggio tanto quanto la sintassi; se due implementazioni leggono gli argomenti in modo diverso, i programmi non sono portabili.

```
triad                           # REPL
triad FILE [ARG...]             # esegue FILE; ARGS ==
[ARG...]
triad -l FILE                   # carica FILE e apre
il REPL
triad -i FILE                   # esegue FILE e poi
apre il REPL
triad -e CODE                   # esegue CODE
triad -E CODE                   # esegue CODE e stampa
il valore
triad --tree FILE               # stampa l'albero
uniforme come S-expression
triad -q                        # REPL senza banner
triad -h, --help
triad -v, --version
triad -                         # legge lo script da
stdin
```

Le opzioni coprono tre esigenze diverse. `-e` e `-E` servono per provare un frammento da terminale, con la differenza che la seconda stampa anche il valore; `-l` e `-i` combinano un file e la sessione interattiva, secondo che il file vada caricato o anche eseguito; `--tree` mostra

l'albero uniforme del capitolo 17, utile quando vuoi vedere come il parser ha raggruppato il codice. Il trattino finale legge lo script da stdin, così Triad può stare in una pipeline.

- L'estensione `.triad` / `.td` si può omettere; lo shebang `#!` è ignorato.
- Il programma vede gli argomenti in `ARGS`, lista di stringhe, senza il nome del programma.
- Codici di uscita: `0` successo, `1` errore non gestito (su stderr), `2` uso non valido.

Questa interfaccia è normativa, ma non tutte le implementazioni la coprono: il seed Windows oggi esegue un file e basta, senza REPL né opzioni (cap. 29). Per eseguire gli esempi di questo manuale basta la forma `triad FILE`, oppure l'`node seed/dev/triad_core.cjs` mostrato in apertura.

## 20.1 Il REPL

Il REPL è una sessione interattiva: scrivi un'espressione, vedi il valore, e le definizioni restano attive tra un comando e l'altro. È utile per esplorare, per provare un goal logico o per capire perché un'espressione dà un certo risultato, senza creare un file ogni volta.

Il REPL è in stile Prolog:

```
triad> 1 + 2
3
triad> ?- ancestor(:tom, ?d)
?d = :bob ;
?d = :liz.
triad> parent(:jim, ?x)
false.
```

Le risposte mostrano i due modi del REPL: `1 + 2` è un'espressione Triad normale e stampa il valore; `?- ancestor(...)` è una query e stampa le soluzioni una per una, con `;` per chiedere la successiva e `.` per fermarsi. `false.` non è un errore: significa che non ci sono altre soluzioni.

- I blocchi restano aperti finché non sono chiusi da `end`.
- `?- goal` (il `.` finale è facoltativo) esegue una query con le opzioni di `solve`: `limit`, `depth`, `steps`, `trace`.
- Un'istruzione che è un goal su un predicato della knowledge base è trattata come query.
- Dopo ogni soluzione: `;`, `n` o spazio per la successiva, `a` per tutte le rimanenti, invio o `.` per fermarsi. `false.` significa nessun'altra soluzione.
- Comandi: `:load FILE…` (`:l`), `:reload` (`:r`), `:listing [nome]` (`:ls`), `:tree CODICE`, `:reset`, `:help` (`:h`), `:quit` (`:q`, `halt`, Ctrl-D).
- Ctrl-C interrompe il calcolo in corso; un errore non gestito è stampato e il REPL continua.

## Esercizi

1. **Quale opzione?** Indica l'opzione giusta per: eseguire un frammento, eseguirlo e stampare il valore, caricare un file nel REPL, eseguire un file e poi aprire il REPL, vedere l'albero.
2. **`ARGS`.** Scrivi un programma che stampi i suoi argomenti e provalo con `triad programma.triad a b c`.
3. **Codici di uscita.** Quali codici restituiscono: un programma riuscito, un errore non gestito, un uso non valido del comando? Verifica con `$?` / `%ERRORLEVEL%`.
4. **REPL stile Prolog.** Che cosa fanno `;`, `a`, `.` e `false.` durante una query al REPL?

5. **I comandi del REPL.** Carica un file con `:load`, ricaricalo con `:reload` e ispeziona una definizione con `:listing`.
6. **Query interattive.** Esegui `?- ancestor(:tom, ?d)` con `limit` e poi con `trace`: che cosa cambia nella risposta?
7. **Dalla pipeline.** Usa `triad -` per leggere uno script da stdin e componilo con altri comandi del terminale.
8. **Estensione omessa.** `triad programma` senza `.triad`: funziona? E con uno shebang `#!` in testa al file?
9. **`-e` contro `-E`.** Scrivi un frammento il cui valore non sia `nothing` e confronta le due forme.
10. **Oggi, in pratica.** Perché il seed Windows non supporta queste opzioni? Con quale comando esegui oggi gli esempi di questo capitolo?

---

## 21. Primitivi di bootstrap

Cinque primitive servono a S1 per leggere e analizzare il sorgente Triad. **Non fanno parte di Triad-Full** ( `SPEC-CORE §8` ): usale solo nel codice di bootstrap.

Sono primitive e non funzioni perché stanno **sotto** il linguaggio: servono a un interprete scritto in Triad per leggere il proprio sorgente come sequenza di byte. Per questo non fanno parte di Triad-Full: un programma applicativo non dovrebbe dipendere da loro, e il codice che le usa non è portabile fuori dal bootstrap. La loro presenza nel profilo Core spiega perché una stringa è una sequenza di byte e non una sequenza di caratteri (cap. 10).

| primitiva | semantica |
|---|---|
| `readfile(path)` | contenuto del file come `String`; errore se non leggibile |
| `strlen(s)` | numero di byte |
| `strbyte(s, i)` | byte (`Int` 0–255) alla posizione `i`, da 1 |
| `strsub(s, a, b)` | sottostringa dai byte `a` a `b` inclusi, da 1 |
| `exit(n)` | termina il processo con codice `n` |

Le cinque primitive sono volutamente minime: nessuna conosce i caratteri, nessuna conosce la sintassi. `strbyte` restituisce un `Int` fra 0 e 255 e `strsub` lavora su intervalli di byte inclusivi, quindi un programma che analizza sorgente deve fare da sé il confronto con i codici dei caratteri che gli interessano.

```
s = "abc 123 xyz"
cifre = 0
i = 1
while i <= strlen(s)
    b = strbyte(s, i)
    if b >= 48 && b <= 57
        cifre += 1
    end
    i += 1
end
println(cifre)
fine = 1
while fine <= strlen(s) && strbyte(s, fine) != 32
    fine += 1
```

```
end
println(strsub(s, 1, fine - 1))
```

**Output:**

```
3
abc
```

Il primo ciclo conta le cifre confrontando il byte con `48` e `57`, che sono i codici ASCII di `0` e `9`: è il modo in cui un analizzatore lessicale distingue un numero da un nome senza avere funzioni sulle stringhe (cap. 23, esercizio 6). Il secondo ciclo cerca il primo spazio e si ferma, poi `strsub(s, 1, fine - 1)` estrae i byte dalla posizione 1 all'ultimo prima dello spazio: `abc`. Nota l'ordine della condizione nel `while`: prima si controlla di essere dentro la stringa, poi si legge il byte; invertirle produrrebbe un accesso fuori intervallo.

Un programma può leggere se stesso con `readfile(ARGS[1])`: è il primo passo di un interprete self-hosted.

## Esercizi

1. **Una per ciascuna.** Scrivi un programma minimo per ogni primitiva: `readfile`, `strlen`, `strbyte`, `strsub`, `exit`.
2. **Byte, non caratteri.** Verifica `strlen("è") == 2`: perché una stringa è una sequenza di byte? Che conseguenza ha per gli analizzatori di testo?
3. **Contare le cifre.** Riscrivi l'esempio del capitolo e provalo su tre stringhe diverse.
4. **La prima parola.** Estrai la prima parola di una stringa con `strsub`, come nell'esempio, e generalizzala in una funzione.

5. **Codici ASCII.** Conta le maiuscole di una stringa (byte da 65 a 90) senza funzioni sui caratteri.
6. **`exit(n)`.** Termina un programma con codice di uscita 3 e verificalo dal terminale.
7. **Leggere sé stessi.** Con `readfile(ARGS[1])` fai stampare a un programma il proprio sorgente: quale riga del capitolo rende possibile questo?
8. **Solo cifre.** Scrivi un programma che dica se una stringa rappresenta un intero (tutti byte cifra, con gestione del segno).
9. **Perché non fanno parte di Full.** Scrivi due righe sul perché queste primitive vanno usate solo nel codice di bootstrap.
10. **Sottostringhe.** Con `strsub(s, a, b)` scrivi una funzione che stampi i primi `n` byte di una stringa e gestisca il caso `n > strlen(s)`.

---

# 22. Stile e pattern idiomatici

Questo capitolo non introduce costrutti nuovi: mostra come combinare quelli dei capitoli precedenti nei casi che ricorrono sempre — trasformare una lista, rappresentare un record, tenere uno stato condiviso, costruire una stringa. Sono pattern, non regole del linguaggio, e li riconoscerai nel codice del progetto.

## 22.1 Funzioni di ordine superiore scritte in Core

In Core non esistono `map` / `filter` / `reduce`: si scrivono una volta, con `while` e lambda, e poi si usano ovunque.

La differenza rispetto alle versioni Full non è solo nel nome: qui il ciclo è esplicito, quindi vedi l'indice, la condizione di fine e il punto in cui l'elemento viene aggiunto. Se un giorno una di queste funzioni dà un risultato inatteso, il difetto è visibile in tre righe.

```
function mappa(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end

function filtra(p, xs)
    out = []
    i = 1
    while i <= length(xs)
        if p(xs[i])
            push!(out, xs[i])
        end
        i += 1
    end
    out
end

function riduci(f, acc, xs)
    i = 1
    while i <= length(xs)
        acc = f(acc, xs[i])
        i += 1
    end
    acc
end
```

```
xs = [1, 2, 3, 4, 5, 6]
println(mappa(x -> x * x, xs))
println(filtra(x -> x % 2 == 0, xs))
println(riduci((a, b) -> a + b, 0, xs))
```

**Output:**

```
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
21
```

`mappa` e `filtra` restituiscono liste nuove, `riduci` restituisce un valore; in tutti e tre i casi la funzione ricevuta è un parametro normale, quindi il comportamento si può cambiare senza toccare il ciclo. Nota che `riduci` accumula in `acc` e non muta `xs`: la lista in ingresso resta la stessa.

## 22.2 Record come liste taggate

In Core non c'è `struct`: un record è una lista con un simbolo come tag in prima posizione. La convenzione è importante perché senza struct non esiste un tipo nominale: il significato del record sta interamente nel tag e nell'ordine dei campi, che il programma deve rispettare da solo.

```
punto(x, y) = [:Punto, x, y]
px(p) = p[2]
py(p) = p[3]
somma_punti(a, b) = punto(px(a) + px(b), py(a) + py(b))
p = somma_punti(punto(1, 2), punto(10, 20))
println(p, " ", p[1] == :Punto)
```

**Output:**

```
[:Punto, 11, 22] true
```

L'esempio è costruito per mostrare il patto: `punto` è il costruttore, `px` / `py` sono gli accessori e nessuna funzione tocca gli elementi a mano tranne loro. Se un giorno cambi l'ordine dei campi, cambi solo il costruttore e gli accessori; chi usa `px` e `py` non se ne accorge. È lo stesso principio di incapsulamento di una struct, ottenuto con convenzioni invece che con il tipo.

## 22.3 Ambiente come lista di associazioni

È la struttura usata dagli interpreti in `selfhost/`: una lista piatta `[nome1, valore1, nome2, valore2, …]`.

```
function env_get(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    nothing
end

function env_set(env, nome, valore)
    i = 1
    while i < length(env)
        if env[i] == nome
            env[i + 1] = valore
            return env
        end
```

```
        i += 2
    end
    push!(env, nome)
    push!(env, valore)
end

env = []
env_set(env, :x, 10)
env_set(env, :y, 20)
env_set(env, :x, 11)
println(env, " ", env_get(env, :x), " ", env_get(env,
:z))
```

**Output:**

```
[:x, 11, :y, 20] 11 nothing
```

È una lista piatta per non pagare un livello di annidamento: le coppie sono affiancate e `env[i+1]` legge il valore del nome in posizione `i`. Il costo è che ogni ricerca è lineare e che le due funzioni devono avanzare di due in due; il vantaggio è che la struttura è una lista normale, quindi si stampa e si ispeziona senza conversioni. È lo stesso schema che un interprete usa per l'ambiente dei nomi.

## 22.4 Stato mutabile condiviso

Poiché una funzione nominata non può riassegnare un globale, lo stato condiviso si tiene in una lista mutabile o in una chiusura.

```
stato = [0]
function tick()
    stato[1] += 1
```

```
end
tick()
tick()
println(stato[1])
```

**Output:**

```
2
```

`tick` non riassegna `stato`: muta l'elemento della lista che `stato` indica. È la differenza del capitolo 3 fra cambiare un legame e mutare un oggetto, e qui è ciò che permette a una funzione nominata di aggiornare uno stato esterno senza violare la regola 8.3 sull'assegnamento locale. La lista fa da contenitore: il legame `stato` resta lo stesso, il contenuto cambia. Se lo stato deve avere più campi, si allunga la lista (o si usa un record taggato come in 22.2).

## 22.5 Costruire stringhe

Due funzioni che tornano sempre: ripetere una stringa e unire una lista con un separatore. In Core non c'è una funzione che unisca una lista con un separatore, quindi il separatore va inserito a mano, e la posizione è tutto: va messo davanti a ogni elemento tranne il primo. Ricorda che ogni `*` crea una stringa nuova (cap. 10).

```
function ripeti(s, n)
    out = ""
    while n > 0
        out = out * s
        n -= 1
    end
    out
```

```
end

function unisci(xs, sep)
    out = ""
    i = 1
    while i <= length(xs)
        if i > 1
            out = out * sep
        end
        out = out * string(xs[i])
        i += 1
    end
    out
end

println(ripeti("ab", 3))
println(unisci([1, 2, 3], ", "))
```

**Output:**

```
ababab
1, 2, 3
```

`unisci` mette il separatore prima di ogni elemento tranne il primo: per questo il controllo `i > 1` viene prima delle due concatenazioni. Se lo mettessi dopo l'elemento, l'ultima concatenazione lascerebbe il separatore in fondo alla stringa.

## 22.6 Regole di stile

- Funzioni piccole con nomi chiari; caso base esplicito in ogni ricorsione; per profondità grandi preferisci `while`.

- Cicli con contatori espliciti ( `i = 1 … while i <= length(xs)` ).
- Non usare `fact` , `rule` , `solve` , `struct` e le altre parole chiave di Full come nomi: il codice deve restare valido anche in Triad-Full.
- Scrivi `!io` sulle funzioni che stampano: Core lo ignora, Full lo richiede.
- Non spezzare un'espressione su più righe: l'interprete di riferimento chiude l'istruzione a ogni a capo anche dentro le parentesi (cap. 1.6). Una istruzione per riga.

Queste regole non sono gusti personali: ognuna evita un problema concreto — un nome che collide in Full, un effetto non dichiarato, un'ambiguità di lettura, una riga spezzata che il runtime rifiuta.

## Esercizi

1. **HOF in Core.** Scrivi `mappa(f, xs)` e `filtra(f, xs)` con un ciclo, senza builtin di ordine superiore.
2. **Record come liste taggate.** Costruisci un record `[:Persona, "Ada", 36]` e scrivi le funzioni di accesso ai campi per posizione.
3. **Ambiente come associazioni.** Riscrivi `env_get` , `env_set` e `env_rimuovi` e provali con tre coppie.
4. **Stato condiviso.** Scrivi un contatore basato su una lista mutabile condivisa e spiega perché funziona anche senza assegnamenti condivisi.
5. **Costruire stringhe.** Riscrivi l'esempio che costruisce una riga con uno spazio fra i valori e usalo per stampare una lista di interi.
6. **Le regole di stile.** Prendi un tuo programma e riscrivilo applicando tutte le regole del capitolo 22.6: cosa è cambiato

nella leggibilità?

7. **`get` non è standard.** Perché `get(xs, i)` non si usa nei programmi portabili? Riscrivi un programma che lo usi con l'indicizzazione standard.
8. **Una funzione, un compito.** Dividi una funzione lunga in tre funzioni piccole collegate e verifica che il programma resti corretto.
9. **Nomi che aiutano.** Rinomina i nomi di un programma ( `x` , `y` , `tmp` ) secondo le convenzioni del capitolo e commenta le scelte.
10. **Sottoinsieme comune.** Riscrivi un programma perché giri sia sull'interprete JS sia sul seed, usando solo le funzionalità comuni elencate nel capitolo 29.

---

# Parte II — Esercizi

Ogni esercizio ha una soluzione verificata. Prova prima, poi apri il riquadro «Soluzione».

Gli esercizi della prima serie sono programmi Triad-Core e si eseguono con l'interprete di riferimento; quelli della seconda usano costrutti Full, quindi hanno risultati attesi ma non sono eseguibili oggi. In entrambi i casi la soluzione non è l'unico programma giusto: è un programma che funziona, ed è utile confrontarla con la tua per capire quale delle due è più chiara.

## 23. Esercizi guidati (Triad-Core)

### Esercizio 1 — Potenza e somme

Senza usare `^`, scrivi `potenza(b, e)` ricorsiva e `somma_fino(n)` (somma `1 + … + n`) con `while`. Stampa `potenza(3, 13)`, `potenza(2, 10)` e `somma_fino(100)`.

▼ Soluzione

```
potenza(b, e) = e == 0 ? 1 : b * potenza(b, e - 1)

function somma_fino(n)
    s = 0
    i = 1
    while i <= n
        s += i
        i += 1
    end
    s
end

println(potenza(3, 13), " ", potenza(2, 10), " ",
somma_fino(100))
```

**Output:**

```
1594323 1024 5050
```

### Esercizio 2 — Numeri primi

Stampa la lista dei numeri primi minori di 30.

▼ Soluzione

```
function primo(n)
    if n < 2
        return false
    end
    d = 2
    while d * d <= n
        if n % d == 0
            return false
        end
        d += 1
    end
    true
end

primi = []
n = 2
while n < 30
    if primo(n)
        push!(primi, n)
    end
    n += 1
end
println(primi)
```

**Output:**

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

### Esercizio 3 — Inverti una lista

Scrivi `inverti(xs)` che restituisca una nuova lista con gli elementi in ordine inverso.

▼ Soluzione

```
function inverti(xs)
    out = []
    i = length(xs)
    while i >= 1
        push!(out, xs[i])
        i -= 1
    end
    out
end

println(inverti([1, :due, "tre"]))
```

**Output:**

```
[tre, :due, 1]
```

### Esercizio 4 — Ordinamento per inserzione

Ordina in loco `[5, 2, 9, 1, 7]`.

▼ Soluzione

```
function ordina!(xs)
    i = 2
    while i <= length(xs)
        k = xs[i]
```

```
        j = i - 1
        while j >= 1 && xs[j] > k
            xs[j + 1] = xs[j]
            j -= 1
        end
        xs[j + 1] = k
        i += 1
    end
    xs
end

println(ordina!([5, 2, 9, 1, 7]))
```

**Output:**

```
[1, 2, 5, 7, 9]
```

Il `!` finale nel nome segnala, per convenzione, una funzione che muta il suo argomento.

## Esercizio 5 — Contatore di parole

Conta le parole (separate da spazi) di `"il linguaggio Triad "` usando `strbyte`.

▼ Soluzione

```
function conta_parole(s)
    n = 0
    dentro = false
    i = 1
    while i <= strlen(s)
        spazio = strbyte(s, i) == 32
```

```
        if !spazio && !dentro
            n += 1
        end
        dentro = !spazio
        i += 1
    end
    n
end

println(conta_parole("il  linguaggio Triad "))
```

**Output:**

```
3
```

## Esercizio 6 — Un mini-analizzatore lessicale

Trasforma `"12+x*3"` in una lista di token `[tipo, testo]`, con tipi `:num`, `:id`, `:op`. È il primo passo di un interprete.

▼ Soluzione

```
cifra(b) = b >= 48 && b <= 57
lettera(b) = (b >= 97 && b <= 122) || (b >= 65 && b <=
90) || b == 95

function tokenizza(s)
    toks = []
    i = 1
    while i <= strlen(s)
        b = strbyte(s, i)
        if cifra(b)
```

```
            j = i
            while j <= strlen(s) && cifra(strbyte(s,
j))
                j += 1
            end
            push!(toks, [:num, strsub(s, i, j - 1)])
            i = j
        elseif lettera(b)
            j = i
            while j <= strlen(s) && (lettera(strbyte(s,
j)) || cifra(strbyte(s, j)))
                j += 1
            end
            push!(toks, [:id, strsub(s, i, j - 1)])
            i = j
        else
            push!(toks, [:op, strsub(s, i, i)])
            i += 1
        end
    end
    toks
end

println(tokenizza("12+x*3"))
```

**Output:**

```
[[:num, 12], [:op, +], [:id, x], [:op, *], [:num, 3]]
```

### Esercizio 7 — Valutare un'espressione

Usando i token dell'esercizio 6, scrivi un valutatore a discesa ricorsiva per `+` e `*` con precedenza, dove le variabili vengono da una lista di associazioni. Suggerimento: tieni la posizione corrente in una lista `pos = [1]`, così le funzioni possono aggiornarla.

▼ Soluzione

```
cifra(b) = b >= 48 && b <= 57
lettera(b) = (b >= 97 && b <= 122) || b == 95

function tokenizza(s)
    toks = []
    i = 1
    while i <= strlen(s)
        b = strbyte(s, i)
        if cifra(b)
            j = i
            while j <= strlen(s) && cifra(strbyte(s,
j))
                j += 1
            end
            push!(toks, [:num, strsub(s, i, j - 1)])
            i = j
        elseif lettera(b)
            j = i
            while j <= strlen(s) && lettera(strbyte(s,
j))
                j += 1
            end
            push!(toks, [:id, strsub(s, i, j - 1)])
            i = j
```

```
        else
            push!(toks, [:op, strsub(s, i, i)])
            i += 1
        end
    end
    toks
end

function cerca(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    0
end

function primario(toks, pos, env)
    t = toks[pos[1]]
    pos[1] += 1
    if t[1] == :num
        Int(t[2])
    else
        cerca(env, t[2])
    end
end

function prodotto(toks, pos, env)
    v = primario(toks, pos, env)
    while pos[1] <= length(toks) && toks[pos[1]][2] ==
```

```
"*"
        pos[1] += 1
        v = v * primario(toks, pos, env)
    end
    v
end

function somma(toks, pos, env)
    v = prodotto(toks, pos, env)
    while pos[1] <= length(toks) && toks[pos[1]][2] ==
"+"
        pos[1] += 1
        v = v + prodotto(toks, pos, env)
    end
    v
end

valuta(s, env) = somma(tokenizza(s), [1], env)
println(valuta("12+x*3", ["x", 10]))
println(valuta("2*3+4*5", []))
```

**Output:**

```
42
26
```

## Esercizio 8 — Un ambiente a dizionario

Estendi l'esercizio 7 (il valutatore con ambiente) con la rimozione di un nome, e scrivi `ambiente_da_elenco` che costruisce un ambiente da coppie `[nome, valore]`. Verifica con `env_get` ed `env_set` del capitolo 22.3.

▼ Soluzione

```
function env_get(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    nothing
end

function env_set(env, nome, valore)
    i = 1
    while i < length(env)
        if env[i] == nome
            env[i + 1] = valore
            return env
        end
        i += 2
    end
    push!(env, nome)
    push!(env, valore)
end

function env_rimuovi(env, nome)
    out = []
    i = 1
    while i < length(env)
        if env[i] == nome
            i += 2
```

```
        else
            push!(out, env[i])
            push!(out, env[i + 1])
            i += 2
        end
    end
    out
end

function ambiente_da_elenco(coppie)
    env = []
    i = 1
    while i < length(coppie)
        env_set(env, coppie[i], coppie[i + 1])
        i += 2
    end
    env
end

env = ambiente_da_elenco([:x, 10, :y, 20])
println(env, " ", env_get(env, :x))
env = env_rimuovi(env, :x)
println(env, " ", env_get(env, :x))
```

**Output:**

```
[:x, 10, :y, 20] 10
[:y, 20] nothing
```

### Esercizi supplementari

1. **Somma dei quadrati.** Calcola `1² + 2² + … + n²` per `n = 10` con un ciclo.
2. **Prodotti pari.** Calcola il prodotto di tutti i numeri pari da 2 a 12.
3. **Ricerca lineare.** Dati una lista e un valore `v`, restituisci la posizione del primo elemento uguale a `v` (0 se non c'è), con un ciclo.
4. **Rotazione.** Sposta il primo elemento di una lista in fondo e stampa il risultato.
5. **Diagonale.** Data una lista di liste 3×3, stampa gli elementi della diagonale principale.
6. **Palindromo.** Verifica se una lista di interi è uguale alla sua inversione (senza `reverse`).
7. **Ordinamento per selezione.** Ordina una lista di cinque interi con l'algoritmo di selezione, una coppia alla volta.
8. **Intersezione.** Costruisci la lista degli elementi comuni a due liste, senza duplicati.

## 24. Esercizi su Triad-Full

Questi esercizi usano i costrutti dello standard completo. Sono definiti dalla specifica e hanno risultati attesi in `conformance/cases/`, ma **non sono ancora eseguibili** dal runtime corrente.

Studiarli serve comunque: le soluzioni sono codice normativo, e vederle accanto agli esercizi Core mostra che i costrutti Full si innestano nella stessa sintassi, non in un linguaggio separato. L'output riportato è un valore atteso conservato dalla suite, non una verifica eseguita oggi.

### Esercizio 9 — Dispatch su struct

Definisci `struct Vec2` con due campi `Float` e tre metodi di `area`: uno per `Vec2`, uno per `Number` e uno per `Any`. Verifica che il metodo più specifico vinca.

▼ Soluzione

```
struct Vec2
    x::Float
    y::Float
end

area(v::Vec2) = v.x * v.y
area(n::Number) = n * n
area(x::Any) = 0

println(area(Vec2(3, 4)))
println(area(5))
println(area("ciao"))
```

**Output (golden Full):**

```
12.0
25
0
```

### Esercizio 10 — Una base di conoscenza

Scrivi fatti e regole per `parent` e `sorella`, e usa `solve` per trovare le sorelle di `:bob` e i nipoti di `:alice`.

▼ Soluzione

```
fact parent(:alice, :bob)
fact parent(:alice, :carol)
fact parent(:bob, :dave)
fact parent(:carol, :erin)

rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y,
?z)

rule sorella(?a, ?b) :- parent(?p, ?a), parent(?p, ?b),
?a != ?b

println([s.b for s in solve sorella(:bob, ?b)])
println([s.z for s in solve grandparent(:alice, ?z)])
```

**Output (golden Full):**

```
[:carol]
[:dave, :erin]
```

## Esercizio 11 — Contratti in pratica

Scrivi `dividi(a, b)` con `requires b != 0` e `ensures abs(result * b - a) < 1e-9`, e usa `try` per distinguere il caso di contratto violata dal caso di successo.

▼ Soluzione

```
function dividi(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end
```

```
r = try dividi(1, 0) end
if iserr(r)
    println("contratto: ", r.data.clause)
else
    println("valore: ", r)
end
```

**Output (golden Full):**

```
contratto: requires
```

## Esercizio 12 — Una macro che misura

Scrivi la macro `@misura(label, espr)` che valuta l'espressione e stampa etichetta e valore, restituendo il valore.

▼ Soluzione

```
macro misura(label, e)
    :(begin
        r = $e
        println($label, " -> ", r)
        r
    end)
end

totale = @misura("somma", 1 + 2 + 3)
println(totale)
```

**Output (golden Full):**

```
somma -> 6
6
```

## Esercizi supplementari

1. **Struct e dispatch. [Full]** Definisci `Quadrato` e `Cerchio` con un metodo `area` per ciascuno e chiama `area` su entrambi.
2. **`for` e comprehension. [Full]** Con una comprehension costruisci la lista delle coppie `(i, j)` con `i` da 1 a 3 e `j` da 1 a 3.
3. **`try` / `catch`. [Full]** Cattura un `Err(:type)` e rilancialo o trasformalo in un messaggio personalizzato.
4. **Contratto. [Full]** Aggiungi `requires` e `ensures` a una funzione che calcoli la media di una lista (con `/` **[Full]**) e prova una violazione.
5. **Budget. [Full]** Limita con `with budget` una ricerca ricorsiva e gestisci `Err(:budget)`.
6. **Knowledge base. [Full]** Con `fact`, `rule` e `solve` descrivi un piccolo albero genealogico e trova tutti i nonni.
7. **Macro. [Full]** Scrivi `@due_volte(e)` che esegua due volte la sua espressione.
8. **Destrutturazione. [Full]** Con `[a, b] = [1, 2]` e un match su una lista `[h | t]` riscrivi `app` senza usare `length`.

---

# Parte III — Riferimento

Questa parte è pensata per la consultazione, dopo la lettura della prima: raccoglie la grammatica, le schede rapide, la tabella degli errori, il glossario e lo stato reale delle implementazioni. Non è materiale nuovo: se un punto non torna, il rimando al capitolo della Parte I ti riporta alla spiegazione. Tieni presente che le sezioni di riferimento descrivono lo **standard**; lo stato di ciò che è eseguibile oggi è raccolto nell'ultimo capitolo.

## 25. Grammatica

La grammatica è la forma più compatta in cui una specifica dice che cosa è un programma valido. Si legge come una ricetta: `=` significa «è fatto di», `{ … }` significa «zero o più», `[ … ]` «opzionale». Le due grammatiche che seguono non descrivono due linguaggi: la prima è il sottoinsieme Core, la seconda è Full, e le produzioni che Core condivide compaiono in Full con la stessa forma.

### 25.1 Triad-Core (EBNF normativa)

```
program  = { sep } { stmt sep { sep } } ;
sep      = NEWLINE | ";" ;
stmt     = funcdef | "return" [ expr ] | "break" |
"continue" | expr ;
funcdef  = "function" NAME "(" [ NAME { "," NAME } ]
")" block "end"
         | NAME "(" [ NAME { "," NAME } ] ")" "=" expr
;
expr     = assegnamento ;
assegnamento = NAME ( "=" | "+=" | "-=" | "*=" ) expr |
ternario ;
```

```
ternario = or_expr [ "?" expr ":" expr ] ;
or_expr  = and_expr { "||" and_expr } ;
and_expr = cmp_expr { "&&" cmp_expr } ;
cmp_expr = sum_expr { ( "==" | "!=" | "<" | "<=" | ">"
| ">=" ) sum_expr } ;
sum_expr = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr = pow_expr { ( "*" | "%" ) pow_expr } ;
pow_expr = unary { "^" unary } ;                    (*
destra *)
unary    = ( "-" | "!" ) unary | postfix ;
postfix  = primary { "(" args ")" | "[" expr "]" } ;
primary  = INT | STRING | SYMBOL | "true" | "false" |
"nothing" | NAME
         | "(" expr ")"
         | "[" [ expr { "," expr } ] "]"
         | "(" NAME { "," NAME } ")" "->" expr
         | "if" expr block { "elseif" expr block } [
"else" block ] "end"
         | "while" expr block "end"
         | "let" [ NAME "=" expr { "," NAME "=" expr }
] block "end"
         | "begin" block "end"
         | "function" "(" [ NAME { "," NAME } ] ")"
block "end" ;
block    = { stmt sep { sep } } ;                   (*
fino a end/else/elseif *)
```

La grammatica Core è volutamente piccola: niente `for`, niente range, niente tipi nelle firme. Non descrive però ogni dettaglio che l'esecuzione accetta: l'assegnamento a un elemento (`xs[i] = e`, `xs[i] += e`), che gli esempi del capitolo 9 usano per mutare le liste, non compare nella

produzione `assegnamento`. È una lacuna del testo della specifica, non del linguaggio: l'interprete di riferimento lo supporta, e il codice che lo usa è portabile fra le implementazioni Core.

## 25.2 Triad-Full (EBNF normativa)

```
program    = { sep } { stmt sep { sep } } ;
sep        = NEWLINE | ";" ;
block      = { sep } { stmt sep { sep } } ;

stmt       = funcdef | structdef | macrodef | fact |
rule
           | "requires" expr | "ensures" expr
           | "return" [ expr ] | "break" | "continue"
           | "(" NAME { "," NAME } ")" "=" expr
           | expr ;

funcdef    = "function" fname "(" [ params ] ")" [ "::"
TYPE ] [ "!" EFFECT { "," EFFECT } ]
             block "end"
           | NAME "(" [ params ] ")" "=" expr ;
fname      = NAME | "+" | "-" | "*" | "/" | "%" | "^" |
"==" | "<" | "<=" | ">" | ">=" ;
params     = param { "," param } ;
param      = NAME [ "::" TYPE ] | "(" NAME { "," NAME }
")" ;
structdef  = "struct" NAME sep { NAME [ "::" TYPE ] sep
} "end" ;
macrodef   = "macro" NAME "(" [ NAME { "," NAME } ] ")"
block "end" ;

fact       = "fact" term ;
```

```
rule       = "rule" term [ ":-" goal { "," goal } ] ;

expr       = (* Pratt: precedenze dalla più bassa alla
più alta *)
  1  a = b   a += b   a -= b   a *= b
(destra)
  2  x -> e   (a, b) -> e   c ? a : b
(destra)
  3  ||
  4  &&
  5  ==  !=  <  <=  >  >=  in  isa  is
  6  a:b
(range, senza spazi)
  7  +  -
  8  *  /  %
  9  -x  !x  +x  $x
(unari)
 10  ^
(destra)
 11  e::T
(asserzione di tipo)
 12  f(args)  xs[i]  a.campo
(postfissi, senza spazio prima di ( e [)

primary    = NUMBER | STRING | SYMBOL | LVAR | NAME |
"true" | "false" | "nothing"
           | "(" expr ")" | "(" expr "," ... ")"
(* tupla: parametri di lambda *)
           | "[" [ expr { "," expr } [ "|" expr ] ] "]"
           | "[" expr "for" NAME "in" expr [ "if" expr
] "]"
           | ":(" expr ")" | "quote" block "end"
```

```
            | "@" NAME "(" args ")"
            | "if" expr block { "elseif" expr block } [
"else" block ] "end"
            | "while" expr block "end"
            | "for" NAME "in" expr block "end"
            | "for" "(" NAME { "," NAME } ")" "in" expr
block "end"
            | "let" [ NAME "=" expr { "," NAME "=" expr
} ] block "end"
            | "begin" block "end"
            | "try" block [ "catch" [ NAME ] block ]
"end"
            | "with" modifier { "," modifier } block
"end"
            | "solve" goal { "limit" expr | "depth" expr
| "steps" expr | "trace" }
            | "function" "(" params ")" block "end" ;
modifier   = "budget" "(" "steps" "=" expr ")" |
"trace" ;
```

La grammatica Full elenca i livelli di precedenza nella forma numerata che il parser usa: è la stessa tabella del capitolo 4, e la produzione `primary` dice quali forme possono stare alla base di un'espressione. Le produzioni `stmt` e `block` dicono invece che cosa può contenere un blocco, e quindi in quali punti `fact`, `rule`, `requires` e `ensures` sono ammessi.

## Esercizi

1. **Si legge o no?** Usando la grammatica, decidi quali di queste righe sono programmi validi: `x = 1`, `x =`, `x 1 =`, `x = 1 2`.

2. **Dove sta la precedenza.** Nelle regole EBNF individua dove compaiono il ternario, `->` e `^`: come si collegano ai livelli del capitolo 4?
3. **Separatori.** Perché newline e `;` sono separatori e non terminatori? Scrivi un esempio in cui un a capo in più cambia il programma.
4. **Commenti.** Dove ammette la grammatica i commenti `#` e `#= =#`? Prova a metterli in tre posizioni insolite (dentro una chiamata, tra un `if` e la sua condizione...).
5. **Le stringhe.** Scrivi la regola degli escape con un esempio per ciascun escape ammesso e un esempio di escape rifiutato.
6. **Chi richiede `end`?** Elenca dalla grammatica tutti i costrutti che aprono un blocco chiuso da `end`.
7. **Regole ricorsive.** Scegli una regola ricorsiva della grammatica e spiega perché lo è, con un esempio di albero di derivazione.
8. **Grammatica e parser.** Come si collegano le regole della grammatica ai dodici livelli del parser Pratt del capitolo 4?
9. **Range e due punti.** Perché `1:5` è un range e `1 : 5` no? Quale regola della grammatica lo dice?
10. **Token.** Scrivi la sequenza dei token di un programma piccolo e confrontala con ciò che `triad --tree` mostra dell'albero.

---

## 26. Schede rapide

Le schede rapide non sostituiscono i capitoli: servono a ricordare la forma esatta di un costrutto quando sai già che cosa vuoi scrivere. Sono divise per tipo di problema — un valore, un controllo di flusso, una funzione — e ogni riga rimanda, quando serve, al capitolo che spiega la scelta.

## 26.1 Valori

| vuoi | scrivi |
|---|---|
| un intero | `42`, `1_000` |
| un float **[Full]** | `3.14`, `1e-9` |
| un booleano | `true`, `false` |
| il vuoto | `nothing` |
| una stringa | `"testo"` |
| un simbolo | `:alice` |
| una lista | `[1, 2, 3]` |
| una lambda | `x -> x + 1` |
| una funzione | `function (x) x end` |
| un record Core | `[:Punto, 1, 2]` |
| una struct **[Full]** | `Point(3, 4)` |
| codice come dato **[Full]** | `:(a + b)`, `quote … end` |

## 26.2 Controllo di flusso

| vuoi | scrivi |
|---|---|
| una condizione | `if c … elseif d … else … end` |
| una condizione breve | `c ? a : b` |
| un ciclo | `while c … end` **[Core]** · `for x in xs … end` **[Full]** |
| uscire dal ciclo | `break` |
| saltare al passo dopo | `continue` |

| vuoi | scrivi |
|---|---|
| uscire dalla funzione | `return e` |
| uno scope nuovo | `let x = 1 … end` |
| un blocco senza scope | `begin … end` |
| una lista costruita **[Full]** | `[x * x for x in xs]` |
| catturare un errore **[Full]** | `try b catch e h end` |

## 26.3 Funzioni

| vuoi | scrivi |
|---|---|
| una funzione | `f(x) = x + 1` |
| un corpo multi-istruzione | `function f(x) … end` |
| un tipo di ritorno **[Full]** | `function f(x)::Float … end` |
| un effetto **[Full]** | `function f(x) !io … end` |
| un predicato in un goal **[Full]** | `function evena(n) n % 2 == 0 end` |
| una precondizione **[Full]** | `requires cond` |
| una postcondizione **[Full]** | `ensures cond` |

## Esercizi

1. **Valori.** Con la sola scheda 26.1, scrivi un programma che usi tutti i tipi di base.

2. **Controllo di flusso.** Con la scheda 26.2 scrivi un `while` con `break` e un `if` / `elseif`.
3. **Funzioni.** Con la scheda 26.3 scrivi una funzione blocco con `return` e una breve.
4. **Senza guardare.** Scrivi in due minuti un programma con ternario, `+=` e un confronto: controlla con la scheda solo alla fine.
5. **Lambda.** Dalla scheda, scrivi una lambda, assegnala e chiamala.
6. **Liste.** Costruisci una lista, accedi al primo e ultimo elemento, aggiungi con `push!`.
7. **Stringhe.** Costruisci una frase con `*`, `string()` e `Int()`.
8. **Errori attesi.** Usando la scheda e la tabella del capitolo 27, scrivi tre programmi che producano tre errori diversi annunciati.
9. **Core o Full?** Con le schede decidi per ciascuno se è eseguibile oggi: `for`, `map`, `/`, `solve`, `lambda`, `try`.
10. **Sfida a tempo.** In cinque minuti scrivi un programma che usi almeno cinque voci diverse delle schede rapide, senza consultare i capitoli.

---

## 27. Tabella degli errori

La tabella raccoglie i messaggi che l'interprete di riferimento stampa oggi in Triad-Core, con la causa tipica. I messaggi sono dell'implementazione, non dello standard: la categoria dell'errore è normativa (cap. 11.3), il testo no, quindi non scrivere programmi che dipendono da queste stringhe.

## 27.1 In Core

Core non ha `try` / `catch` né valori `Err`: un errore **termina il processo** con exit code `1` e un messaggio diagnostico su stderr. Quello già stampato su stdout resta.

| messaggio | causa |
|---|---|
| `parse error: unexpected token …` / `parse error: expected …` | sintassi non Core (`1.5`, `/`, `() -> …`, `end` mancante) |
| `error: undefined variable NOME` | nome mai assegnato, o funzione chiamata prima di essere definita |
| `error: not a function` | si chiama un valore che non è una funzione (`x = 5; x(1)`) |
| `error: arity mismatch` | numero di argomenti diverso dai parametri |
| `error: index out of range` | `xs[i]` con `i < 1` o `i > length(xs)` |
| `error: + type error` (e simili) | operandi di tipo sbagliato (`1 + "a"`) |
| `error: if cond` / `while cond` / `ternary cond` / `&& expects bool` | condizione non `Bool` |
| `error: div by zero` / `negative exponent` | `x % 0`, `2 ^ -1` |
| `error: unknown function NOME` | builtin riservato usato in modo non supportato |
| `cannot read file: …` | file inesistente o non leggibile |

| messaggio | causa |
|---|---|
| `usage: triad FILE [ARG...]` (exit 2) | nessun file passato |

> ***Nota di implementazione.*** *La tabella descrive i messaggi dell'interprete di riferimento. Il seed in codice macchina termina* ***senza messaggi****: codice `1` per ogni fallimento del parser o dell'esecuzione, codice `127` per `%` per zero (eccezione hardware), codice `2` per l'uso errato della riga di comando. Come nell'interprete, ciò che è già stato stampato su stdout resta.*

## 27.2 In Triad-Full

Gli stessi fallimenti arrivano come `Err` con un `kind` (cap. 11.3), invece di terminare il processo. Un errore non gestito al top level stampa `riga N: …` e esce con codice `1`.

## Esercizi

1. **Quattro errori Core.** Scrivi un programma minimo per ciascuno: `:syntax`, `:undef`, `:type`, `:bounds`.
2. **Cinque errori Full. [Full]** Un esempio per ciascuno: `:method`, `:ambiguous`, `:contract`, `:budget`, `:effect`.
3. **Dal messaggio alla categoria.** Dato il messaggio `error: index out of range` (Core) e `Err(:bounds)` (Full), spiega la relazione fra i due.
4. **Core o Full?** Quali categorie di errore si possono incontrare in un programma Core puro? Giustifica la risposta.
5. **`:host`.** Che cosa significa la categoria `:host`? Scrivi un caso in cui potrebbe comparire.

6. **Due modi di fallire.** Confronta il comportamento di un errore in Core (exit code 1) e in Full ( `Err` come valore) con due programmi paralleli.
7. **Catturare. [Full]** Scrivi un programma che catturi due categorie diverse e reagisca in modo diverso a ciascuna.
8. **Diagnosi da riga.** Un programma con un errore segnala la riga: scrivi un programma con tre errori potenziali e determina quale viene segnalato per primo.
9. **Ordine di scoperta.** In un programma con un errore di sintassi e uno di esecuzione, quale viene scoperto per primo? Perché?
10. **La tua tabella.** Aggiungi alla tabella un esempio personale per due categorie e verificalo eseguendolo.

---

## 28. Glossario

Il glossario raccoglie i termini che il manuale usa con un significato preciso e che ricorrono in capitoli diversi. Le definizioni sono brevi apposta: il rimando al capitolo serve quando la parola da sola non basta.

| termine | significato |
|---|---|
| **Albero uniforme** | l'unico tipo di nodo prodotto dal parser, `Node(head, args)` ; è anche ciò che il kernel esegue |
| **Ambiente** | l'insieme dei legami di nomi a valori visibile in un punto del programma |
| **Backtracking** | il motore logico torna sui legami e prova la clausola successiva |
| **Budget** | limite di passi eseguibili; esaurito produce `Err(:budget)` |

| termine | significato |
|---|---|
| **Clausola** | un `fact` o una `rule` della knowledge base |
| **Chiusura** | funzione che cattura l'ambiente in cui è stata creata |
| **Dispatch** | scelta del metodo in base ai **tipi** degli argomenti |
| **Effetto** | capacità dichiarata di una funzione: `io`, `state` |
| **Err** | l'errore come dato: `kind`, `msg`, `data` |
| **Goal** | una domanda alla knowledge base: predicato, congiunzione, disgiunzione, unificazione |
| **Knowledge base** | l'insieme dei fatti e delle regole |
| **Macro** | una funzione che riceve e restituisce codice |
| **Occurs-check** | la regola che impedisce a `?x` di unificarsi con un termine che contiene `?x` |
| **Porta (di Byrd)** | uno dei quattro eventi `:call`, `:exit`, `:redo`, `:fail` che descrivono un passo della ricerca logica (cap. 14) |
| **Precedenza** | l'ordine in cui gli operatori si raggruppano (12 livelli in Full, 10 in Core) |
| **Profilo** | un sottoinsieme normativo dello standard: Core o Full |
| **Scope lessicale** | un nome è visibile dove è stato legato e all'interno, mai all'esterno |
| **Soluzione** | una `Solution`: i legami prodotti da un goal risolto |
| **Standardizzazione a parte** | le variabili di una clausola vengono rinominate a ogni uso |

| termine | significato |
|---|---|
| **Termine** | un dato della logica: costante, variabile logica o forma composta, non valutata finché non serve |
| **Tracciato** | il valore `Traced(result, trace)` restituito da `with trace` o `solve … trace` |
| **Unificazione** | l'uguaglianza logica: due termini si rendono identici legando le variabili |
| **Valori di prima classe** | funzioni e operatori usabili come qualsiasi altro valore |

## Esercizi

1. **Definizioni tue.** Definisci con le tue parole: *unificazione*, *backtracking*, *occurs-check*, *dispatch*, *chiusura*.
2. **Tre lingue, idee proprie.** Collega ciascuna influenza (Julia, Prolog, Lisp) a due idee che Triad ha ripreso e a una che non ha ripreso.
3. **Parlare di `solve`.** Quali termini del glossario servono per descrivere `solve` e la sua ricerca? Scrivi una frase che li usi tutti.
4. **Vero o falso.** Valuta: «il budget misura il tempo», «`ensures` è una precondizione», «le macro ricevono valori», «una soluzione logica è una lista di legami», «`Err` è un tipo».
5. **Coppie che confondono.** Distingui con un esempio: *effetto* e *contratto*; *budget* e *limit*; *parametro* e *argomento*; *simbolo* e *variabile logica*.
6. **Albero uniforme.** Scrivi la tua definizione di «albero uniforme» e falla vedere su un esempio `:(f(1))`.
7. **Termini in inglese.** Quali termini tecnici restano in inglese nel libro e perché? Scrivi la traduzione che useresti per tre di essi.

8. **Il glossario di un programma.** Prendi un programma Full del capitolo 16 e annota almeno sei termini del glossario che servono per descriverlo.
9. **Cinque domande.** Scrivi cinque domande di ripasso sul glossario e rispondi senza guardare.
10. **Aggiungi una voce.** Scegli un concetto del libro non presente nel glossario e scrivi la voce che aggiungeresti, nello stesso stile.

---

## 29. Cosa gira oggi

Questa sezione non fa parte del linguaggio: è lo stato delle implementazioni, utile per orientarsi prima di scrivere codice. Serve a rispondere a una domanda che un manuale rischia di lasciare implicita: quello che hai appena letto è **definito** dallo standard, ma che cosa puoi **eseguire** oggi? Le due cose non coincidono, e confonderle porta a due errori opposti: credere che un esempio Full non funzioni perché è sbagliato, o credere che un programma Core funzioni su ogni runtime.

| profilo | stato |
|---|---|
| **Triad-Core** | **eseguibile**: interprete di riferimento JavaScript `seed/dev/triad_core.cjs` (37/37 casi Core) e, in parte, il seed Windows `seed/windows-x86_64/seed.exe` |
| **Triad-Full** | **non eseguibile**: struct, dispatch, logica, macro, contratti, budget, tracce, effetti, destrutturazione, `for`, `try`, `Float` sono specifica; nella conformance Full passa solo `errore` |

Il profilo Full non è «quasi pronto»: è definito per intero nello standard, e i suoi esempi hanno risultati attesi conservati dalla suite, ma nessuno dei runtime del repository lo esegue oggi. Quando nel libro vedi `**Output`

`(golden Full):**`, quindi, leggi un valore atteso, non una verifica appena fatta. Sul versante Core la situazione è diversa: l'interprete JavaScript passa l'intera suite, mentre il seed Windows copre solo una parte dei casi — in particolare le liste restano parziali.

Anche restando in Core, «eseguibile» non significa «identico ovunque»: esistono due runtime, e non coprono lo stesso sottoinsieme. La tabella elenca le differenze che contano per chi scrive codice destinato a girare su entrambi; leggerla prima di scegliere un costrutto evita di scoprire il problema solo quando il programma arriva sul seed.

Differenze note fra i due runtime Core:

| funzionalità | interprete JS | seed Windows |
|---|---|---|
| `return`, `break`, `continue` | sì | non elencati nel README del seed |
| lambda | più parametri | **un solo parametro** |
| chiusure | condivise | **per valore**, solo l'ultimo legame del frame |
| `string(x)` | tutti i tipi | solo `Int` e `String` |
| liste | eterogenee, `xs[i]`, `length`, `push!` | solo interi; costruzione, stampa, `push!` |
| `get(xs, i)` | non esiste | funzione ausiliaria **non standard**, indici **da 0** |
| `ARGS`, primitivi di bootstrap | sì | non elencati nel README del seed |
| messaggi di errore | su stderr | solo exit code |

Le righe più importanti sono le prime tre. Un `return` o un `break` che l'interprete JS esegue può non essere supportato dal seed; una lambda con due parametri funziona su uno e non sull'altro; una chiusura che aggiorna uno stato condiviso si comporta diversamente. Sono differenze di implementazione, non di standard: la semantica corretta è quella dell'interprete JS e della specifica, e il seed è il runtime più limitato. Se un programma deve girare su entrambi, scegli il sottoinsieme comune e non contare su ciò che una sola colonna dichiara.

Regola pratica: **scrivi codice che rispetta `SPEC-CORE` e verificalo con l'interprete JS**; se deve girare anche sul seed, restringiti alle funzionalità elencate nella colonna di destra e verificalo su Windows.

Due limiti noti che il lettore deve conoscere prima di scrivere codice:

- **ogni a capo chiude l'istruzione**, anche dentro `( )`, `[ ]` e negli argomenti di una chiamata. L'eccezione prevista da `SPEC §1` per Triad-Full (a capo dentro le parentesi, o subito dopo un operatore binario) non è implementata nell'interprete di riferimento. Scrivi una istruzione per riga;
- **il seed Windows non implementa l'indicizzazione standard `xs[i]`**: lì c'è la funzione ausiliaria non standard `get(xs, i)`, con indici **da 0**.

A questi due limiti va aggiunto un dettaglio di piattaforma: il seed Linux è allo stadio E0 e stampa solo un messaggio di prova, quindi per eseguire programmi su Linux oggi si usa l'interprete JavaScript (`node seed/dev/triad_core.cjs`). Il seed Windows è il runtime in codice macchina più completo, ma non è l'unico modo di eseguire Triad.

Che cosa farne, in pratica: per studiare e per scrivere codice usa l'interprete JS; per verificare la portabilità verso il bootstrap prova sul seed; per gli esempi Full leggi e confronta con i golden, sapendo che

nessun runtime li esegue ancora. Il manuale non promette ciò che il repository non sa fare, e questa sezione esiste per rendere esplicito il confine.

## Esercizi

1. **Sottoinsieme comune.** Scrivi un programma che gira su entrambi i runtime Core usando solo le funzionalità della colonna destra della tabella delle differenze.
2. **`get` non è portabile.** Perché `get(xs, i)` non si può usare in codice portabile? Riscrivi un programma che lo usi con `xs[i]`.
3. **Lambda a due parametri.** Scrivi una lambda con due parametri e spiega su quale runtime non girerebbe.
4. **Chiusure diverse.** Costruisci un esempio in cui la cattura per valore del seed e quella condivisa dell'interprete JS danno risultati diversi.
5. **`string(x)` limitato.** Perché `string(true)` non gira sul seed? Che cosa succede invece sull'interprete JS?
6. **Due runtime, un test.** Esegui lo stesso caso di conformance con l'interprete JS e con il seed Windows e confronta gli output.
7. **Limiti del seed.** Elenca cinque limiti del seed Windows e per ciascuno scrivi la variante di programma che funziona ugualmente.
8. **Limiti noti.** Spiega i due limiti noti del capitolo (ogni a capo chiude l'istruzione; `xs[i]` non implementato) con un programma che li incontri.
9. **Golden, non verificati.** Perché gli esempi Full hanno «Output (golden Full)»? Che cosa puoi e non puoi dedurne?

10. **La tua regola.** Scrivi la checklist personale che userai prima di pubblicare un programma Core destinato a girare su entrambi i runtime.

---

# Appendice A — Influenze dichiarate

Triad nasce da idee di Julia, Prolog e Lisp, ma **non è un dialetto di nessuno dei tre**: la definizione del linguaggio è la specifica.

Conoscere le influenze non è curiosità: se vieni da uno di questi linguaggi, sapere che cosa Triad ha ripreso e che cosa ha cambiato ti aiuta a non importare abitudini sbagliate. La tabella non è un elenco di parentele, è un avviso: le somiglianze sono di idea, non di comportamento.

| da | idea ripresa |
|---|---|
| Julia | blocchi `function … end`, `struct`, tipi con `::`, multiple dispatch, operatori-funzione, `if` / `for` / `while` / `let` |
| Prolog | `fact` / `rule` come costrutti, unificazione con occurs-check, backtracking, `solve`, variabili logiche `?x` |
| Lisp | codice come dato, albero uniforme, `:( … )` / `quote` con `$`, macro |

**Propri di Triad**, non presi in prestito: errori come dati, contratti, budget, tracce ed effetti dichiarati come parte del linguaggio, e l'integrazione dei tre modelli in una sintassi a blocchi.

---

# Appendice B — Dove guardare

Questa appendice elenca dove approfondire. Sono materiali **facoltativi**: il manuale è autosufficiente, e questi rimandi servono a chi vuole risalire alla fonte normativa, controllare un caso limite o vedere il codice del progetto. Il percorso di studio per capitoli e la visita al codice sono un complemento, non un prerequisito.

| vuoi | leggi |
|---|---|
| la definizione normativa completa | `SPEC.md` |
| il sottoinsieme di bootstrap | `SPEC-CORE.md` |
| il percorso di studio per capitoli e il codice del progetto | `TUTORIAL.md` |
| gli esempi completi con output | `examples/` |
| il contratto eseguibile, con i golden | `conformance/` |
| lo stato del bootstrap | `STATUS.md` · `BOOTSTRAP.md` |
| il piano di lavoro | `plan.md` · `TASK.md` |
| gli strumenti per gli editor | `editors/` |
| la licenza | `LICENSE` — MIT |

---

*Manuale scritto per lo standard Triad `0.2.0`. Le sezioni normative sono in `SPEC.md` e `SPEC-CORE.md`; in caso di divergenza, prevale la specifica.*




# Triad 2.0 — Tutorial: imparare il linguaggio e il codice del progetto

Questo tutorial è il percorso guidato con cui impari Triad partendo da zero. Non dà per scontato che tu conosca il linguaggio: dà per scontato solo che tu sappia programmare in qualche altro linguaggio. Se sai già scrivere cicli, funzioni e liste, hai tutto il necessario; il resto — la sintassi di Triad, il comportamento dei valori, la distinzione tra il sottoinsieme eseguibile e lo standard completo — è spiegato qui, nell'ordine in cui conviene impararlo.

Il percorso porta a tre risultati. Primo, **scrivere programmi Triad-Core che girano davvero**, cioè programmi completi, eseguibili oggi con l'interprete di riferimento. Secondo, **conoscere Triad-Full**, il linguaggio completo definito dallo standard, comprese le parti che il runtime attuale non esegue ancora. Terzo, **saper leggere il codice del progetto**: l'interprete di riferimento, il seed in codice macchina, la suite di conformità e gli strumenti di sviluppo. Il tutorial non rimanda altrove per capire i concetti: la specifica (`SPEC.md`, `SPEC-CORE.md`) e i casi di `conformance/` sono approfondimenti facoltativi, non prerequisiti.

## Che cosa impari e come è organizzato

Il tutorial è un libro in cinque parti, pensate per essere lette in ordine ma abbastanza autonome da poter essere consultate separatamente.

- **Parte I — Primi passi.** Che cos'è Triad, come si installa e si esegue, il primo programma spiegato riga per riga.
- **Parte II — Triad-Core passo per passo.** Il sottoinsieme eseguibile oggi, costrutto per costrutto: lessico, valori, variabili, espressioni, controllo di flusso, funzioni, chiusure, liste, stringhe, primitivi di bootstrap, errori e differenze tra i runtime. Gli esercizi guidati in fondo alla parte servono a fissare quello che hai letto.
- **Parte III — Triad-Full (lo standard completo).** Tipi dichiarati, multiple dispatch, struct, `for` e comprehension, logica con fatti e regole, macro, contratti, budget, tracce ed effetti. È il linguaggio definito dalla specifica: qui lo studi, ma il runtime attuale non lo esegue ancora.
- **Parte IV — Il codice del progetto.** Una visita guidata al repository: l'interprete di riferimento, il seed, S1, i casi di conformità, la CLI, il benchmark, il flusso di contributo. Serve a capire come il linguaggio è implementato, non solo come si usa.
- **Parte V — Riferimento.** Grammatica, schede rapide, messaggi di errore, glossario, stato del progetto e FAQ: capitoli da consultare, più che da leggere di seguito.

**Che cosa significa «eseguibile oggi».** Lo standard Triad definisce due profili di conformità. Un esempio è *eseguibile oggi* quando gira, senza modifiche, su un'implementazione che esiste già nel repository; è *definito*

*dallo standard* quando la specifica lo descrive ma nessun runtime attuale lo esegue. Il tutorial distingue sempre i due casi, e ogni blocco di codice dichiara il profilo a cui appartiene.

| profilo | dove è definito | eseguibile oggi? |
|---|---|---|
| **Triad-Core** | `SPEC-CORE.md` | **sì**: interprete di riferimento JavaScript `seed/dev/triad_core.cjs` (37/37 casi Core) e, in parte, il seed Windows `seed/windows-x86_64/seed.exe` (33 casi Core verificati in CI) |
| **Triad-Full** | `SPEC.md` | **no**: struct, dispatch, logica, macro, contratti, budget, tracce, effetti, destrutturazione, `for`, `try`, `Float` sono specifica; nella conformance Full passa solo `errore` |

Due precisazioni sul contesto, utili per non confondere il libro con il progetto. La versione dello standard è la `0.2.0`, registrata in `VERSION`. Triad non dipende da Julia: quella vecchia implementazione è stata rimossa, e gli output Triad-Full che leggi nel libro sono i **golden**, cioè i risultati attesi prodotti a suo tempo da quell'implementazione e congelati in `conformance/cases/`; non sono stati riprodotti dal runtime attuale. Al contrario, ogni esempio marcato **Output:** è un programma Triad-Core verificato con `node seed/dev/triad_core.cjs`; gli esempi Full sono marcati **Output (golden Full):** oppure non riportano output.

---

## Come usare questo tutorial

Non c'è un solo modo di leggere il libro: dipende da quello che ti serve. La tabella qui sotto riassume i percorsi principali. Chi non conosce Triad, però, dovrebbe cominciare dalla Parte I: i capitoli successivi danno per acquisito il lessico del linguaggio.

| vuoi… | leggi |
|---|---|
| scrivere programmi Triad che girano oggi | Parte I → Parte II (capitoli 4–17, con esercizi) |
| conoscere il linguaggio completo dello standard | Parte III (capitoli 18–32) |
| contribuire all'interprete JS, al seed, a S1, alla CLI o alla conformance | Parte IV (capitoli 33–43) |
| consultare grammatica, schede rapide, errori, glossario | Parte V (capitoli 44–49) |

Alcune convenzioni rendono più facile seguire gli esempi. I comandi sono mostrati per Linux/macOS (`sh`) e, quando sono diversi, per Windows (`cmd`/PowerShell). I blocchi `triad` contengono codice Core, normalmente eseguibile con l'interprete JS; i frammenti specifici del seed Windows sono indicati nel testo. I blocchi `triad-full` mostrano codice definito dallo standard Full, non ancora eseguibile dal runtime corrente. Con **interprete JS** si intende `seed/dev/triad_core.cjs`, l'interprete di riferimento Triad-Core usato su Linux/macOS e dal runner di conformità; con **seed** si intende `seed/windows-x86_64/seed.exe`, l'interprete Triad-Core in codice macchina (S0). Dove i due si comportano diversamente lo dice il capitolo 15. I sorgenti si scrivono in file `.triad` (estensione canonica) o `.td` (alias storico).

# Parte I — Primi passi

## 1. Cos'è Triad

Triad è un linguaggio di programmazione autonomo che riunisce tre paradigmi — imperativo, logico e metaprogrammatorio — in un'unica sintassi a blocchi. L'idea di fondo è che questi paradigmi non vadano tenuti in linguaggi separati: un programma Triad può definire funzioni e cicli come un linguaggio imperativo, dichiarare fatti e regole come Prolog e trattare il codice come un valore costruibile e trasformabile come in Lisp, e le tre cose convivono nello stesso file senza strati né interfacce.

Le influenze dichiarate sono tre, e per ciascuna il libro mostrerà via via che cosa Triad ha ripreso:

| influenza | idea ripresa in Triad |
|---|---|
| Julia | `function … end`, `struct`, tipi con `::`, multiple dispatch, operatori come funzioni, `if` / `for` / `while` / `let` |
| Prolog | `fact`, `rule`, unificazione con occurs-check, backtracking, `solve` con variabili logiche `?x` |
| Lisp | codice come dato (`Node`), `:( … )` / `quote` con `$`, macro |

A questi si aggiungono, come parte del linguaggio e non di una libreria: **errori come dati** (`Err`), **contratti** (`requires` / `ensures`), **budget** (`with budget`), **tracce** (`with trace`) ed **effetti dichiarati** (`!io`, `!state`).

La distinzione tra «parte del linguaggio» e «libreria» non è nominale, e vale la pena capirla subito. Un errore che è un valore può essere restituito, ispezionato e composto come qualsiasi altro dato, invece di interrompere il programma; un contratto scritto nella firma è visibile all'implementazione, che può controllarlo; un effetto dichiarato permette al runtime di verificare che una funzione pura non stampi. Sono tutte

funzionalità che un linguaggio può offrire solo se le conosce, e per questo Triad le definisce nello standard anziché lasciarle a convenzioni tra programmatori.

Da Julia arriva l'ossatura della sintassi: blocchi `function … end`, tipi con `::`, struct e multiple dispatch. Da Prolog arriva il motore logico: fatti, regole, unificazione con occurs-check, backtracking e `solve` con variabili logiche. Da Lisp arriva l'idea che il codice sia un albero uniforme, quindi un valore come gli altri, e la macchina delle macro che su quell'idea si appoggia. Tenere insieme i tre modelli è la parte propriamente nuova: non esiste un «linguaggio del lunedì» e un «linguaggio del martedì», esiste un solo programma con una sola semantica di espressioni e di blocchi.

## 1.1 Due profili: Core e Full

Lo standard definisce due profili di conformità (SPEC §16.1):

- **Triad-Core** ( `SPEC-CORE.md` ): interi, Bool, stringhe, simboli, `nothing`, liste, funzioni, lambda e chiusure, `if` / `elseif` / `else`, `while`, `let`, `begin`, ternario, `return` / `break` / `continue`, più alcuni primitivi di bootstrap. È abbastanza piccolo da essere implementato in codice macchina scritto a mano e abbastanza espressivo da scriverci l'interprete di tutto Triad.
- **Triad-Full** ( `SPEC.md` ): tutto il linguaggio.

La separazione risponde a un vincolo pratico, non a una scelta di comodo. Il primo interprete di Triad deve esistere prima che esista un compilatore Triad, quindi va scritto a mano in codice macchina: più il linguaggio è piccolo, più quel compito è realistico. Allo stesso tempo il sottoinsieme deve bastare a scrivere l'interprete completo nella seconda fase. Triad-Core è esattamente il punto di equilibrio tra le due esigenze, e per questo è definito da una specifica a sé.

Ogni programma Triad-Core (esclusi i primitivi di bootstrap) è anche un programma Triad-Full valido e deve comportarsi allo stesso modo. Il contrario non vale: un programma che usa `struct`, `for` o `float` è Full, e il runtime Core non lo eseguirà. È la distinzione che ritroverai in tutto il libro: la Parte II insegna il profilo che gira oggi, la Parte III aggiunge il resto.

## 1.2 La catena di bootstrap

Triad non vuole dipendere da nessun linguaggio ospite, ma ogni linguaggio deve pur partire da qualcosa. La catena di bootstrap risolve il problema in tappe: si comincia con un interprete scritto a mano in codice macchina, lo si usa per eseguire un interprete scritto in Triad, e così via fino a un compilatore che compila se stesso. Il piano (`BOOTSTRAP.md`):

```
S0  seed: interprete Triad-Core in codice macchina x86-
64 (dump esadecimale nel repo)
S1  interprete di tutto Triad scritto in Triad-Core
(selfhost/, in corso)
S2  compilatore nativo scritto in Triad
(non iniziato)
S3  compilatore self-hosted: S2 compilato da S2
(non iniziato)
S4  Triad completo: REPL, CLI, tooling, parità Linux
(non iniziato)
```

Il seed S0 non è distribuito come eseguibile ma come dump esadecimale: è il modo più semplice per versionare un binario in un repository testuale, e si materializza con lo strumento di decodifica del sistema operativo (capitolo 2). Gli stadi S1–S4 sono la direzione del progetto, non ciò che serve per leggere questo libro: per seguire il tutorial basta il seed S0 o l'interprete JS.

Gli strumenti in Node.js (`seed/dev/*.js`, `conformance/run.mjs`, `cli/`) sono **strumenti di sviluppo**: aiutano a generare e verificare, ma non fanno parte della catena S0→S4. La Parte IV del libro li presenta uno per uno, spiegando che cosa fanno e perché sono esterni al bootstrap.

## Esercizi

1. **Core o Full?** Classifica: `while`, `fact`, `struct`, `lambda`, `for`, `macro`, `push!`, `solve`, `try`, `println`.
2. **Perché due profili.** Spiega con le tue parole perché il linguaggio ha un profilo Core «di bootstrap» e uno Full «standard».
3. **La catena di bootstrap.** Ordina e descrivi le tappe S0–S4: quale è realizzata oggi?
4. **Cosa gira oggi.** Che cosa può fare il seed Linux attuale? E il seed Windows?
5. **Core dentro Full.** Scrivi un programma Core e spiega perché è anche un programma Full valido, e che cosa gli sarebbe richiesto in più.
6. **Le fonti normative.** Quali file del repository definiscono il linguaggio? Trovali e apri la sezione sugli operatori.
7. **I runtime.** Individua nel repository l'interprete JS di riferimento e il generatore del seed Windows: in quali file stanno?
8. **Golden e verifiche.** Perché per gli esempi Full si parla di output golden? Che cosa distingue un output verificato da uno atteso?
9. **Tre cose proprie.** Elenca tre aspetti di Triad che non vengono da Julia, Prolog o Lisp.

10. **Il tuo primo Core.** Scrivi un programma che usi variabile, `if` e `while`, eseguilo con l'interprete JS e conferma che sia Core puro.

---

## 2. Installare ed eseguire

«Installare» Triad qui non significa installare un pacchetto: significa avere il repository e uno dei due runtime disponibili. Non esiste ancora un compilatore da aggiungere al sistema, né un gestore di pacchetti del linguaggio. Il capitolo risponde quindi a due domande concrete: come si esegue un file e da quale runtime conviene partire.

La scelta consigliata per chi comincia è l'**interprete JS**: si avvia con Node.js, non richiede di decodificare binari ed è quello con cui vengono verificati i casi di conformità. Il seed Windows è interessante quando vuoi vedere il linguaggio eseguito senza runtime ospiti; i suoi limiti sono elencati nel capitolo 15. Per seguire il libro puoi usare solo l'interprete JS, e passare al seed quando un capitolo lo richiede.

### 2.1 Requisiti

- **Node.js ≥ 18** (la CI usa Node 20) per l'interprete JS, il runner di conformance e la CLI.
- Su Windows, nient'altro per il seed: `seed.exe` si ottiene decodificando `seed.hex` con `certutil`.
- Git per clonare il repository: `git clone https://github.com/GPuntobit/triad2.0.git`.

I requisiti sono pochi di proposito: Triad non vuole dipendenze da altri linguaggi per eseguire i programmi, e l'unica dipendenza dello strumentario di sviluppo è Node.js. Se hai Node.js, hai già tutto ciò che serve per eseguire gli esempi della Parte II.

## 2.2 Eseguire un file

Un programma Triad è un file di testo che il runtime legge, analizza ed esegue. La tabella mostra come avviarlo sulle due piattaforme e come passargli lo script da stdin:

| piattaforma | comando |
|---|---|
| Linux / macOS | `node seed/dev/triad_core.cjs file.triad arg1 arg2` |
| Windows (seed) | `seed\windows-x86_64\seed.exe file.triad` oppure `bin\triad.cmd file.triad` |
| qualsiasi (stdin, interprete JS) | `echo 'println(7 * 6)' \| node seed/dev/triad_core.cjs -` |

Esempi pronti che girano oggi: tutti i `conformance/core/cases/*/program.td`, cioè i casi che il libro cita come riferimento normativo in fondo a ogni capitolo.

```
node seed/dev/triad_core.cjs
conformance/core/cases/ricorsione/program.td   # 120,
55
```

```
bin\triad.cmd
conformance\core\cases\ricorsione\program.td
```

Il commento `# 120, 55` non fa parte del comando: indica l'output atteso, cioè quello che dovresti vedere comparire. Il trattino `-` al posto del nome del file dice all'interprete JS di leggere il programma dallo standard input. Lo stesso file può essere eseguito dai due runtime, ma non sempre con lo stesso risultato: la differenza è l'argomento del capitolo 15.

### 2.3 Materializzare il seed

Il sorgente del seed è il dump esadecimale; l'eseguibile si ottiene con un decodificatore di sistema. Serve solo se vuoi provare il seed: l'interprete JS non richiede questo passaggio.

```
Remove-Item seed\windows-x86_64\seed.exe -ErrorAction
SilentlyContinue
certutil -decodehex seed\windows-x86_64\seed.hex
seed\windows-x86_64\seed.exe
```

```
# Linux: seed ELF allo stadio E0 (stampa solo "Triad
seed Linux OK")
xxd -r -p seed/linux-x86_64/seed.hex seed/linux-
x86_64/seed && chmod +x seed/linux-x86_64/seed
```

Il comando Windows cancella prima l'eseguibile precedente, se c'è, e poi lo ricrea dal dump; su Linux il risultato è un ELF minimale, lo stadio E0, che per ora si limita a stampare un messaggio di avvio: non è ancora un interprete. Non è un errore di installazione, è lo stato del port descritto in `STATUS.md`.

### 2.4 Launcher `bin/triad` e `bin/triad.cmd`

Un launcher serve a non dover ricordare ogni volta quale binario usare e in quale cartella si trova. Non è obbligatorio, ma è comodo: quando `bin/` è nel `PATH` basta `triad file.triad`, come con un linguaggio installato nel sistema. La ricerca del runtime segue un ordine preciso, dal runtime migliore a quello disponibile:

1. `bin/triad-native` (eseguibile standalone, **non ancora prodotto**: arriverà con S2–S3);

2. `seed/<piattaforma>/seed` (su Windows `seed\windows-x86_64\seed.exe`).

Su Linux il launcher trova il seed ELF E0, che **non interpreta ancora programmi**: per eseguire codice su Linux usa direttamente l'interprete JS. `triad --build` è un residuo della vecchia implementazione Julia: stampa un messaggio che spiega perché la build non è più supportata ed esce con codice 2. Quando nessun runtime è disponibile, il launcher non fallisce in silenzio: stampa le istruzioni per materializzare il seed.

## 2.5 Codici di uscita

I codici di uscita sono parte del contratto del linguaggio, non un dettaglio di comodità: la suite di conformità confronta anche il codice, quindi un programma che stampa l'output giusto ma esce con il codice sbagliato non passa.

| codice | significato |
|---|---|
| `0` | successo |
| `1` | errore di parsing o di esecuzione (messaggio su stderr), file non leggibile |
| `2` | uso non valido (es. nessun file) |

Le opzioni di SPEC §14 (`-e`, `-E`, `-l`, `-i`, `--tree`, `--help`, REPL) **non sono ancora implementate**: l'interprete JS le tratta come un nome di file, e il seed Windows, che documenta solo l'uso `seed.exe FILE`, non le riconosce. Per questo, quando un esempio non funziona, la prima domanda da farsi è quale dei tre codici hai ottenuto: un `2` indica un problema nel comando, un `1` un problema nel programma.

## 2.6 La CLI di sviluppo in breve

La CLI è facoltativa: raccoglie in un solo comando le operazioni che altrimenti faresti a mano nel repository. Le righe seguenti mostrano le più usate, ciascuna con il commento che dice che cosa fa.

```
bin/triad-cli status                 # ambiente e seed
trovato (Windows: bin\triad-cli.cmd)
bin/triad-cli test conformance       # conformance Core
bin/triad-cli ai verify -v           # verifica le
soluzioni di ai/tasks/
bin/triad-cli seed generate windows  # rigenera
seed.hex e stampa lo SHA-256
node conformance/run.mjs             # runner di
conformance diretto (Core)
```

I launcher `bin/triad-cli*` eseguono `npm install` in `cli/` al primo avvio. In alternativa si usa l'eseguibile standalone `triad-cli.exe` / `triad-cli` allegato alla release beta. Dettagli nel capitolo 39. Per seguire la Parte II la CLI non serve: gli esempi si eseguono direttamente con `node`.

### Esercizi

1. **Requisiti.** Verifica `node --version` e la presenza del repository: quale versione serve?
2. **Eseguire un file.** Esegui un programma con `node seed/dev/triad_core.cjs file.triad` e con l'estensione `.td`: entrambe funzionano?
3. **Materializzare il seed.** Segui la procedura del paragrafo 2.3 e verifica il seed ottenuto.

4. **Il launcher.** Prova `bin/triad` (e `bin/triad.cmd` su Windows): quale runtime usano?
5. **Codici di uscita.** Produci programmi con codice `0`, `1` e `2` e verificali dal terminale.
6. **La CLI di sviluppo.** Esegui `node cli/triad-cli.mjs test conformance` e leggi il riepilogo.
7. **File mancante.** Esegui un programma inesistente: quale errore e quale codice di uscita?
8. **Percorsi.** Scrivi lo stesso comando di esecuzione nella forma Windows e nella forma Linux/macOS: che cosa cambia?
9. **Due runtime, un programma.** Esegui lo stesso programma con `bin/triad` e con l'interprete JS: gli output coincidono?
10. **Dalla cartella giusta.** Esegui un programma da un'altra cartella usando percorsi relativi e assoluti: che cosa insegna sui percorsi?

---

## 3. Il primo programma

Un programma Triad non richiede cerimonie: non c'è un file di configurazione, non ci sono import da dichiarare, non c'è una funzione `main` da invocare. Il primo programma è un file di testo che contiene le istruzioni nell'ordine in cui devono essere eseguite.

Crea `ciao.triad`:

```
#!/usr/bin/env triad
# il mio primo programma Triad
nome = "Triad"
println("ciao, " * nome)
println(6 * 7)
```

Output:

```
ciao, Triad
42
```

Eseguilo:

```
node seed/dev/triad_core.cjs ciao.triad          # Linux/macOS
```

```
seed\windows-x86_64\seed.exe ciao.triad          # Windows
```

Cosa notare:

- la prima riga `#!` (shebang) è ignorata: inizia con `#`, quindi è un commento;
- `println` stampa i suoi argomenti **senza separatori** e aggiunge un a capo;
- `*` tra stringhe è la **concatenazione** (non `+`);
- non c'è una funzione `main`: le istruzioni al top level si eseguono in ordine.

Puoi seguire l'esecuzione riga per riga. La riga `nome = "Triad"` valuta la stringa e associa quel valore al nome `nome`. Nella riga seguente il runtime legge `nome`, concatena i due valori stringa con `*` e passa il risultato a `println`. Nell'ultima riga, `6 * 7` viene valutato prima della chiamata: a `println` arriva il valore intero `42`, non il testo del calcolo. Ogni istruzione viene eseguita una sola volta e nell'ordine in cui compare; una funzione chiamata può naturalmente eseguire il proprio corpo prima che il programma passi all'istruzione successiva.

La riga `#!` non è una direttiva necessaria per l'interprete: in questi esempi è una forma convenzionale utile quando il sistema operativo può avviare direttamente un file eseguibile. Il runtime Triad la ignora perché la tratta come commento. Se un esempio non funziona, verifica separatamente che il file sia salvato con estensione `.triad` o `.td`, che il comando punti al runtime giusto e che il sorgente usi soltanto costrutti supportati dal profilo dichiarato.

## 3.1 Argomenti da riga di comando

Gli argomenti dopo il nome del file sono in `ARGS`, una lista di stringhe (senza il nome del programma). Esempio con l'interprete JS:

```
printf 'println(ARGS)\nprintln(length(ARGS))\n' >
args.triad
node seed/dev/triad_core.cjs args.triad uno due      #
[uno, due]  poi  2
```

> *Il seed Windows documenta solo l'uso `seed.exe FILE`: `ARGS` non è tra le funzionalità elencate in `seed/windows-x86_64/README.md`. Usa l'interprete JS per i programmi con argomenti.*

## 3.2 Il ciclo di lavoro consigliato

1. scrivi il programma usando solo Triad-Core (Parte II);
2. eseguilo con l'interprete JS (`node seed/dev/triad_core.cjs`);
3. se deve girare anche sul seed Windows, controlla la tabella del capitolo 15;

4. se è un comportamento del linguaggio da fissare, trasformalo in un caso di conformance (capitolo 38).

Lo stesso ciclo vale per gli esercizi del capitolo 17: prima si scrive il programma, poi si confronta l'output con quello atteso, e solo alla fine, se serve, si guarda la soluzione.

## Esercizi

1. **`ARGS`.** Scrivi un programma che stampi i suoi argomenti e provalo con zero, uno e tre argomenti.
2. **Senza argomenti.** Che cosa vede `ARGS` quando il programma non riceve nulla? Verifica stampandolo.
3. **Ciclo di lavoro.** Applica il ciclo consigliato del paragrafo 3.2 a un programma nuovo di tua invenzione.
4. **Una modifica alla volta.** Applica tre modifiche successive a un programma, rieseguendo dopo ciascuna: quale modifica ha introdotto l'errore?
5. **Commentare.** Commenta riga per riga un tuo programma: quale riga era la meno chiara?
6. **Output di tre righe.** Produci un output di tre righe esatte con `println` e verificalo carattere per carattere.
7. **Il nome conta.** Che cosa succede con un file `prova.triad.txt`? E con `prova.td`?
8. **Errore volontario.** Introduci un errore di sintassi, leggi il messaggio e correggilo in meno di un minuto.
9. **Mini-conformance.** Scrivi il tuo output atteso su un foglio prima di eseguire, poi confronta: hai previsto tutto?
10. **`ARGS` utile.** Scrivi un programma che, dato un numero `n` come argomento, stampi i numeri da 1 a `n`.

# Parte II — Triad-Core passo per passo (eseguibile oggi)

Ogni capitolo di questa parte usa solo Triad-Core. Gli esempi con **Output:** sono stati eseguiti con l'interprete JS; il caso di conformance indicato in fondo a ogni capitolo è il riferimento normativo da leggere e far girare.

La struttura dei capitoli è sempre la stessa, così sai sempre dove guardare: prima la spiegazione del costrutto e del problema che risolve, poi la sintassi e le regole, quindi un esempio completo con il suo output, e infine il caso di conformance da cui l'esempio deriva. Quando una regola nasconde una trappola, il testo la segnala esplicitamente.

## 4. Lessico

Il lessico è l'insieme dei mattoni con cui si scrive un programma: che cosa è un nome, come si scrive un numero o una stringa, che cosa il runtime considera separatore tra un'istruzione e l'altra. Non è un elenco da imparare a memoria: serve a capire perché un certo testo viene letto in un modo e non in un altro, e quindi perché un programma che sembra giusto produce un `parse error`.

La tabella raccoglie gli elementi di Triad-Core, con la forma ammessa e un esempio per ciascuno:

| elemento | forma | esempi |
|---|---|---|
| identificatore | lettera o `_`, poi lettere/cifre/`_`; un `!` finale è ammesso | `x`, `somma_2`, `push!` |

| elemento | forma | esempi |
|---|---|---|
| intero | cifre decimali, `_` come separatore; il segno è l'operatore unario | `42`, `1_000_000`, `-5` |
| booleano | `true`, `false` | |
| nothing | `nothing` | |
| stringa | `"..."` con escape `\n \t \" \\`; **nessuna interpolazione** | `"ciao\n"` |
| simbolo | `:` seguito da un nome o da un operatore | `:alice`, `:+` |
| commento | `#` fino a fine riga; `#= ... =#` su più righe | |
| separatore | a capo oppure `;` | `println(1); println(2)` |

Operatori di Core: `+ - * % ^ == != < <= > >= && || ! = += -= *= -> ?:`.

Parole riservate usate da Core: `function end if elseif else while let begin return break continue true false nothing`.

Non fanno parte di Core: letterali `Float` (`1.5`), variabili logiche `?x`, macro `@nome`, interpolazione `$x`, l'operatore `/`.

L'operatore `->` costruisce una lambda, mentre `?` e `:` formano il condizionale: sono token del lessico, ma il loro significato è argomento del capitolo 7 e del capitolo 10. Le parole riservate non possono essere usate come nomi di variabile: il parser le riconosce come costruzioni, e un assegnamento come `let = 3` produce un errore di sintassi. L'elenco

delle parole riservate è breve perché Core è un linguaggio piccolo: tutto ciò che non figura qui appartiene a Triad-Full, e usarlo in un programma Core non è «codice eseguibile oggi».

Un'altra distinzione da tenere a mente è quella tra spazi e separatori. Spazi e tabulazioni separano i token ma non le istruzioni; il punto e virgola e il ritorno a capo sì. Per questo `println(1); println(2)` mette due istruzioni sulla stessa riga, mentre una riga vuota non produce un'istruzione. Il punto e virgola, insomma, ha lo stesso ruolo del ritorno a capo: è un separatore, non una fine di blocco.

```
#= commento
   su più righe =#
x = 1_000   # commento di riga
println(x); println("tab:\t|", " virgolette: \"", "
backslash: \\")
println(:alice, " ", :+)
```

Output:

```
1000
tab:     | virgolette: " backslash: \
alice +
```

> ***Attenzione.*** *`"x = $x"` in Core stampa letteralmente `x = $x`: l'interpolazione è solo Triad-Full. In Core si concatena: `"x = " * string(x)`.*

> *I commenti* `#= … =#` ***non si annidano*** *nell'interprete JS: il commento si chiude al primo* `=#` *incontrato. Non contare sull'annidamento; se devi commentare codice che contiene già un blocco* `#= … =#` *, usa commenti di riga.*

I due avvertimenti hanno la stessa radice: in Core il lessico è volutamente minimo, e ciò che in un altro linguaggio è «solo un dettaglio» qui può non esistere. Il testo di un programma Core non va interpretato con le abitudini di un altro linguaggio: se un escape o un simbolo non compare nella tabella, il runtime non lo riconosce.

Riferimento: `conformance/core/cases/aritmetica`, `simboli_base`, `stringhe_base`.

## Esercizi

1. **Identificatori.** Quali nomi sono validi? `_x`, `x!`, `push!`, `2x`, `x2`, `x-y`, `x_y`. Provali come nomi di variabili.
2. **Numeri.** Scrivi un milione in tre forme ammesse (`1000000`, `1_000_000`…) e un intero negativo: come vengono stampati?
3. **Stringhe ed escape.** Stampa una stringa con `\n`, `\t`, `\"`, `\\`: ricrea a mano l'output atteso prima di eseguire.
4. **Simboli.** Scrivi `:ok`, `:nome` e il simbolo `:+`: che cosa cambia rispetto alle stringhe con lo stesso testo?
5. **Commenti.** Scrivi un programma con `#` e `#= ... =#` in tre posizioni diverse: il comportamento cambia mai?
6. **Separatori.** Due istruzioni su una riga con `;`, poi la stessa coppia su due righe: è lo stesso programma per il parser?
7. **Operatori Core.** Elenca tutti gli operatori Core e scrivi un esempio per ciascun gruppo (aritmetica, confronto, logica, assegnamento, scelta).

8. **Lettura del codice.** Dividi in token questa riga: `x = n + 1 # avanzamento` e spiega quale pezzo il parser ignora.
9. **Refusi.** Scrivi apposta `printn`, `elsa`, `whlie`: quale errore produce ciascuno?
10. **Con la grammatica.** Confronta la sintassi di questo capitolo con la grammatica del capitolo 44: quale regola produce i numeri con `_`?

---

## 5. Valori, tipi e stampa

Triad è un linguaggio a tipi dinamici: ogni valore ha un tipo ben preciso, ma le variabili no. Una stessa variabile può contenere un intero e poi una stringa, e il tipo viene controllato al momento dell'operazione, non della dichiarazione. Da qui deriva una conseguenza pratica: per sapere che cosa contiene un valore devi chiederlo al runtime con `typeof`, e per scrivere codice robusto devi pensare ai tipi che possono arrivare, non solo a quelli che intendevi passare.

La tabella elenca i tipi che un programma Core può incontrare:

| tipo | valori | `typeof(v)` |
|---|---|---|
| `Int` | interi con segno a 64 bit, aritmetica con wrap (complemento a due) | `Int` |
| `Bool` | `true`, `false` | `Bool` |
| `String` | sequenza immutabile di byte UTF-8 | `String` |
| `Symbol` | `:nome`; uguali se hanno lo stesso nome | `Symbol` |
| `List` | lista eterogenea **mutabile**, indici da 1 | `List` |

| tipo | valori | `typeof(v)` |
|---|---|---|
| `Function` | funzione nominata, lambda o chiusura | `Function` |
| `Nothing` | `nothing` | `Nothing` |

`typeof` restituisce il **nome del tipo come simbolo**.

Il dettaglio del simbolo non è pedanteria: `typeof(1)` vale `:Int`, non `"Int"`. Confrontare il risultato con una stringa non funziona, e il confronto corretto è `typeof(v) == :Int`. In generale, in Triad un tipo si nomina con un simbolo; la stringa serve a chi stampa un messaggio, non a chi programma un controllo.

I tipi non sono un ornamento: da essi dipende che cosa un'operazione accetta. `Int` è a 64 bit perché deve stare in una parola di macchina, che è l'unità con cui il seed lavora; `String` è immutabile perché il linguaggio non offre modi per modificarla in loco; `Symbol` è pensato per i nomi e gli identificatori interni; `List` è mutabile perché è la struttura dati di lavoro del linguaggio; `Nothing` rappresenta l'assenza di un valore utile, non un errore.

## 5.1 Regole di stampa di `print` / `println`

- concatenano gli argomenti senza separatori; `println` aggiunge `\n`;
- `Int` in decimale, `Bool` come `true` / `false`, `String` così com'è, `Nothing` come `nothing`;
- un `Symbol` passato direttamente è stampato **senza** `:`; dentro una lista **con** `:`;

- una `List` è stampata come `[a, b]`; le stringhe dentro una lista sono stampate senza virgolette.

```
xs = [1, :a, "s", true, nothing, [2, 3]]
println(xs)
println(:alice)
println(typeof(1), " ", typeof("s"), " ", typeof(:a), "
", typeof(xs))
println(typeof(nothing), " ", typeof(true), " ",
typeof(x -> x))
print("senza ", "a capo")
println()
```

Output:

```
[1, :a, s, true, nothing, [2, 3]]
alice
Int String Symbol List
Nothing Bool Function
senza a capo
```

## 5.2 Uguaglianza

`==` e `!=` confrontano numeri, stringhe e simboli **per valore**, le liste **elemento per elemento** (anche annidate), le funzioni **per identità**.

```
println([1, [2, [3]]] == [1, [2, [3]]])
println("ab" == "a" * "b", " ", :a == :a, " ", 1 == 2)
```

Output:

```
true
true true false
```

Il valore stampato non è sempre una rappresentazione da riutilizzare come sorgente. Per esempio, un simbolo passato direttamente a `println` appare senza i due punti (`alice`), mentre dentro una lista viene mostrato come `:alice`; le stringhe dentro una lista sono mostrate senza virgolette. `typeof` è quindi il modo corretto per verificare il tipo del valore, mentre la forma visiva di `println` serve soprattutto alla lettura umana. Analogamente, `==` confronta il contenuto per numeri, stringhe, simboli e liste, ma le funzioni per identità: due funzioni scritte con lo stesso testo non sono per questo lo stesso oggetto funzione.

La rappresentazione UTF-8 è misurata in byte, non necessariamente in lettere visibili. Una lettera accentata può occupare più byte; perciò `length` su una stringa non è sempre il conteggio dei caratteri percepiti da una persona. In Core non ci sono operazioni di indicizzazione dei caratteri: i primitivi di bootstrap lavorano esplicitamente sui byte e vanno usati solo per il compito di lettura e analisi del sorgente.

Riferimento: `conformance/core/cases/valori`, `nothing_base`, `confronti`.

## Esercizi

1. **`print` o `println`?** Stampa tre parole su una sola riga con `print` e chiudi con `println()`: quale differenza con tre `println`?
2. **Argomenti multipli.** `println("a", 1, true)`: come vengono concatenati? Prevedi e verifica.
3. **Simboli in due modi.** Spiega perché `println(:a)` stampa `a` e `println([:a])` stampa `[:a]`.

4. **Uguaglianza di liste.** Valuta `[1, [2, [3]]] == [1, [2, [3]]]` e `[1, 2] == [2, 1]`: quale regola si applica?
5. **Funzioni uguali.** Con `f(x) = x` e `g(x) = x`, che cosa valgono `f == f` e `f == g`? Perché?
6. **`typeof` completo.** Stampa `typeof` per `42`, `"s"`, `true`, `:a`, `[1]`, `nothing`.
7. **`42` e `"42"`.** Perché `42 + 1` funziona e `"42" + 1` no? Quale regola di `+` si applica?
8. **Stampare `nothing`.** Che cosa produce `println(nothing)`? E `println(typeof(nothing))`?
9. **Formattazione esatta.** Producì l'output `a b c` (una riga, spazi esatti) usando almeno due tecniche diverse.
10. **Il caso di conformance.** Esegui il caso di conformance da cui deriva questo capitolo e confronta byte per byte il tuo output con il golden.

---

## 6. Variabili e assegnamento

Una variabile, in Triad, è un nome legato a un valore. Non la dichiari e non ne annunci il tipo: il primo assegnamento crea il legame, i successivi lo aggiornano. Questo rende il codice breve, ma sposta l'attenzione su due domande che in un linguaggio con le dichiarazioni sarebbero automatiche: quale valore c'è dentro un nome in questo punto del programma, e in quale ambiente quel nome è visibile.

Le regole di base sono queste:

- `nome = espr` lega o aggiorna un nome; l'assegnamento è un'espressione e vale il valore assegnato.
- `+=`, `-=`, `*=` aggiornano una variabile esistente (il lato sinistro è valutato una volta).

- Usare un nome mai assegnato è un errore di esecuzione (`undefined variable`).
- Non ci sono dichiarazioni né tipi: le variabili sono dinamiche.

Che l'assegnamento sia un'espressione, e non un'istruzione, è la regola meno ovvia. Significa che `x = 42` non «fa qualcosa e basta»: produce il valore `42`, e quel valore può essere usato dove compare l'assegnamento. Nella pratica serve quando un assegnamento compare dentro un blocco che restituisce un valore, ma è anche il motivo per cui non trovi una distinzione rigida tra «istruzione» e «espressione» nel resto del libro. L'assegnamento composto (`+=`, `-=`, `*=`) è zucchero per una lettura e una scrittura della stessa variabile, con la garanzia che il lato sinistro venga valutato una sola volta: se contiene un indice, quell'indice non viene calcolato due volte.

Un nome usato prima di essere assegnato non vale `nothing` e non vale zero: è un errore che interrompe il programma. È una scelta voluta, perché un valore «vuoto» implicito nasconderebbe un errore di logica; il capitolo 14 spiega come si riconosce questo caso e che cosa si vede sul terminale.

```
x = 42
y = x + 8
x = x + 1
z = (x + y) * 2
println(x, " ", y, " ", z)
n = 10
n += 5
n -= 3
n *= 2
println(n)
```

Output:

```
43 50 186
24
```

Le regole di **visibilità** (dentro funzioni, `let`, lambda) sono nel capitolo 10.

Un modo utile di seguire il programma è tenere una piccola tabella dei legami correnti:

| istruzione conclusa | `x` | `y` | `z` |
|---|---|---|---|
| `x = 42` | 42 | — | — |
| `y = x + 8` | 42 | 50 | — |
| `x = x + 1` | 43 | 50 | — |
| `z = (x + y) * 2` | 43 | 50 | 186 |

Questa tabella mostra perché l'assegnamento non è una formula reattiva: `y` conserva il risultato calcolato quando la sua riga viene eseguita, anche dopo che `x` cambia. Per contro, una lista è mutabile e può essere raggiunta da più nomi; modificare un suo elemento agisce sull'oggetto condiviso, non solo su una singola associazione nome-valore. Il capitolo 11 mostra come riconoscere questa differenza nel codice.

Riferimento: `conformance/core/cases/variabili`.

## Esercizi

1. **L'assegnamento produce un valore.** Che cosa stampa `println(x = 7)`? Perché?
2. **Le scorciatoie.** Riscrivi `n += 3`, `n -= 2`, `n *= 4` in forma esplicita: il risultato cambia? Su quale runtime delle due

le scorciatoie complete non sono disponibili?

3. **Nome indefinito.** `println(k)` senza assegnamento: quale errore? E quale su un programma dove `k` è assegnato solo in un ramo di un `if`?
4. **Tabella dei valori.** Traccia `a = 1`, `b = a + 1`, `a = a * 10`, `c = a + b` e conferma l'output con `println(a, " ", b, " ", c)`.
5. **Riassegnare o mutare.** Con `xs = [1, 2]` e `alias = xs`: che cosa vede `alias` dopo `xs[1] = 9`? E dopo `xs = [5]`?
6. **Nomi.** Perché `eta` e `Eta` sono diverse? Quale convenzione di nomenclatura usa il libro?
7. **Cambio di tipo.** `x = 1` poi `x = "ciao"`: permesso, ma saggio? Scrivi la variante con due nomi diversi.
8. **`xs[i] += 1`.** Perché il lato sinistro viene valutato una volta sola? Costruisci un caso in cui una doppia valutazione darebbe un risultato diverso.
9. **Contatore.** Senza guardare, scrivi il ciclo che conta da 1 a 5 con `+=` e un `while`.
10. **Il caso di conformance.** Esegui il caso del capitolo e spiega perché usa solo assegnamenti su nomi preesistenti.

---

## 7. Operatori ed espressioni

Un'espressione è un modo di combinare valori per produrne un altro. Triad non ha una distinzione rigida tra espressioni e istruzioni: quasi tutto è un'espressione, e anche i costrutti di controllo hanno un valore. Per leggere un'espressione senza ambiguità servono due informazioni: quanto «lega forte» ogni operatore e in che direzione si raggruppano gli operatori dello stesso livello.

## 7.1 Precedenza (dalla più bassa alla più alta)

| livello | operatori | associatività |
|---|---|---|
| 1 | `=` `+=` `-=` `*=` | destra |
| 2 | `c ? a : b` | destra |
| 3 | `\|\|` | sinistra, cortocircuito |
| 4 | && | sinistra, cortocircuito |
| 5 | `==` `!=` `<` `<=` `>` `>=` | sinistra |
| 6 | `+` `-` | sinistra |
| 7 | `*` `%` | sinistra |
| 8 | `^` | **destra** |
| 9 | unari `-` `!` | prefissi |
| 10 | `f(args)` `xs[i]` | postfissi |

La tabella va letta come una classifica: il livello 1 lega meno, il livello 10 di più. In `2 + 3 * 4` convivono il livello 6 (`+`) e il livello 7 (`*`); vince il più alto, quindi si calcola prima `3 * 4`. Quando due operatori stanno sullo stesso livello decide l'associatività: `-` è a sinistra, quindi `8 - 3 - 2` è `(8 - 3) - 2` e non `8 - (3 - 2)`. L'eccezione più importante è `^`, che è a destra: `2 ^ 3 ^ 2` significa `2 ^ (3 ^ 2)` e vale 512, non 64.

Questa tabella descrive Triad-Core. Triad-Full aggiunge altri operatori e altri livelli — il range `a:b`, `in` / `isa` / `is`, l'asserzione di tipo `e::T`, la lambda `x -> e` — descritti nella specifica (SPEC §2). Chi scrive solo Core può ragionare con questa tabella; chi passa a Full deve consultare quella completa.

## 7.2 Aritmetica intera

- `+ - * ^` su `Int` con **wrap a 64 bit**;
- `%` è il **resto con il segno del dividendo** (come `rem` di Julia): `-7 % 3 == -1`;
- `^` con esponente negativo è un errore; `%` per zero è un errore;
- **non esiste** `/` in Core (la divisione è di Triad-Full).

```
println(2 + 3 * 4, " ", (2 + 3) * 4, " ", 2 ^ 3 ^ 2)
println(7 % 3, " ", -7 % 3, " ", 10 % 4)
println(9223372036854775807 + 1)
```

Output:

```
14 20 512
1 -1 2
-9223372036854775808
```

Il risultato `-9223372036854775808` è la faccia visibile del wrap: il massimo intero più uno non produce un errore, produce il minimo. Triad-Core non ha un tipo «intero grande» né controlli di overflow, perché deve restare implementabile con poche istruzioni di macchina; se un calcolo deve restare nel dominio, il controllo va scritto nel programma.

Il resto negativo segue la stessa logica: `-7 % 3` vale `-1` perché il segno lo dà il dividendo, non il divisore. È il comportamento di `rem` in Julia e va tenuto presente quando si usa `%` per calcolare posizioni cicliche: con valori negativi serve una correzione esplicita.

### 7.3 Logica e condizioni

- Le condizioni ( `if` , `while` , ternario, `&&` , `||` , `!` ) accettano **solo `Bool`** : non c'è "truthiness" ( `if 1` è un errore).
- `&&` e `||` cortocircuitano.
- I confronti non si concatenano: `1 < 2 < 3` è un errore (si confronta `Bool < Int` ); scrivi `1 < 2 && 2 < 3` .

```
println(!false, " ", true && false, " ", false || true)
println(1 < 2 && 2 < 3)
x = 3
println(x > 2 ? "grande" : "piccolo")
```

Output:

```
true false true
true
grande
```

Il **cortocircuito** è osservabile: con `a && b` , se `a` è `false` , `b` non viene valutato; con `a || b` , se `a` è `true` , `b` non viene valutato. Non è solo un'ottimizzazione: impedisce anche che un calcolo non necessario produca un errore. Un controllo del tipo `i <= length(xs) && xs[i] > 0` può quindi verificare il limite prima di accedere alla lista, purché `i` sia un intero valido. Entrambi gli operandi valutati devono comunque essere booleani.

La scelta di accettare solo `Bool` evita una classe intera di ambiguità: in molti linguaggi `0` , `""` e `nothing` sono «falsi» per convenzione, e ogni programmatore arriva con una convenzione diversa. In Triad non ci sono convenzioni da ricordare: una condizione è vera o falsa, e qualsiasi

altro tipo è un errore. Per questo `1 < 2 < 3` non funziona: i confronti si associano a sinistra, quindi il parser legge `(1 < 2) < 3`, cioè un `Bool` confrontato con un `Int`.

Una trappola collegata riguarda `=` e `==`: `if x = 0` non confronta `x` con zero, assegna zero e produce `0` come valore della condizione; il runtime rifiuta quel valore perché non è un `Bool`. Il confronto è `==`, e vale la pena prendere l'abitudine di scriverlo sempre.

La tabella di precedenza descrive il raggruppamento: `a + b * c` equivale a `a + (b * c)`. Quando due operatori hanno lo stesso livello, entra in gioco l'associatività. In caso di dubbio, usa parentesi: rendono esplicita la struttura che vuoi e aiutano a distinguere un errore di raggruppamento da un errore nel valore degli operandi.

Riferimento: `conformance/core/cases/aritmetica`, `operatori`, `interi`, `confronti`.

## Esercizi

1. **Precedenza.** Calcola a mano `2 + 3 * 4`, `(2 + 3) * 4`, `2 ^ 3 ^ 2`, `1 + 2 == 3` e verifica con un programma.
2. `^` **associativo a destra.** Scrivi con le parentesi il raggruppamento di `2 ^ 3 ^ 2` e spiega il risultato (`512`).
3. **Resto.** Valuta `7 % 3`, `10 % 5`, `-7 % 3`: quale definizione di `%` usa Triad?
4. **A 64 bit, con wrap.** Che cosa stampa `println(9_223_372_036_854_775_807 + 1)`? Verifica e spiega con l'aritmetica in complemento a due del volume.
5. **Cortocircuito.** Dimostra che in `false && <secondo>` il secondo operando non viene valutato (per esempio con una chiamata che stamperebbe qualcosa).

6. **Ternario.** Riscrivi un `if`/`else` come `cond ? a : b` e viceversa: quando conviene quale?
7. **`=` e `==`.** Che cosa produce `if x = 0`? Perché il linguaggio segnala l'errore invece di comportarsi in modo misterioso?
8. **Confronti combinati.** Scrivi un'espressione che sia vera solo per i numeri fra 10 e 20 inclusi, e provala con quattro valori.
9. **Solo `Bool`.** Perché `if 1` e `while "s"` sono errori? Quale regola del volume difende questa scelta?
10. **Il caso di conformance.** Esegui il caso del capitolo, prevedi ogni riga dell'output e poi confronta.

---

## 8. Controllo di flusso

Il controllo di flusso decide quali istruzioni eseguire e quante volte. Triad ha un vincolo che semplifica il modello mentale: **tutto è un'espressione**, e anche `if`, `while`, `let` e `begin` hanno un valore (quello dell'ultima istruzione eseguita, `nothing` se non ne esegue nessuna). Non esistono quindi due mondi separati — istruzioni che non valgono niente ed espressioni che valgono qualcosa — ma un solo meccanismo, e i blocchi sono espressioni che contengono altre espressioni.

### 8.1 `if` / `elseif` / `else`

```
function classifica(x)
    if x < 0
        "negativo"
    elseif x == 0
        "zero"
    else
        "positivo"
    end
```

```
end
println(classifica(-3), " ", classifica(0), " ",
classifica(7))
v = if 2 > 1 :si else :no end
println(v)
```

Output:

```
negativo zero positivo
si
```

L'`if` valuta una condizione alla volta, dalla prima all'ultima, ed esegue solo il ramo corrispondente al primo valore `true`; se nessuna condizione è vera ed esiste un `else`, esegue quello, altrimenti non esegue niente e vale `nothing`. `elseif` non è uno zucchero per un `else` che contiene un altro `if`: è un ramo della stessa catena, e questo evita di accumulare `end` a ogni livello. L'ultima riga dell'esempio mostra che un `if` può stare a destra di un assegnamento: il suo valore è quello del ramo scelto, qui il simbolo `:si`.

Due trappole ricorrenti: dimenticare l'`end` (che chiude `if`, `while`, `function` e `let`) e scrivere `else if` al posto di `elseif`: in Core `elseif` è un unico ramo della catena, mentre `else if` annida un secondo `if` e richiede un `end` in più. Il `? :` del capitolo 7 è la versione compatta dello stesso schema, comoda quando i due rami sono espressioni brevissime.

## 8.2 `while`, `break`, `continue`

```
i = 0
somma_dispari = 0
while i < 10
```

```
    i += 1
    if i % 2 == 0
        continue
    end
    somma_dispari += i
end
println(somma_dispari)
while true
    break
end
println(:fine)
```

Output:

```
25
fine
```

Il `while` è l'unico ciclo di Triad-Core: non c'è `for`, e l'iterazione su una lista si scrive con un contatore e un limite, come farà il capitolo 11. La scelta è coerente con il bootstrap: un solo costrutto di ciclo è più facile da implementare in codice macchina, e `for` può essere aggiunto dopo, in S1, senza cambiare la semantica del `while`.

La trappola classica del `while` è l'ordine delle istruzioni dentro il corpo: se `continue` salta l'aggiornamento del contatore, il ciclo non termina. Nell'esempio il contatore viene incrementato prima di ogni `continue`, e non è un caso: la condizione di uscita deve essere raggiungibile lungo tutti i percorsi del corpo. `break` e `continue` agiscono sempre sul `while` più interno, quindi in cicli annidati vanno letti con attenzione.

### 8.3 `let` e `begin`

`let` introduce un nuovo scope con legami locali; `begin … end` raggruppa istruzioni senza creare legami nuovi.

```
x = 1
let x = 10
    let x = 20
        println(x)
    end
    println(x)
end
println(x)
r = begin
    a = 2
    a * 21
end
println(r)
```

Output:

```
20
10
1
42
```

Un blocco `begin … end` raggruppa istruzioni e vale l'ultima: serve a usare più istruzioni dove ne serve una sola, per esempio a destra di un assegnamento. `let` invece apre un ambiente nuovo, e i nomi che lega lì dentro possono nascondere quelli esterni senza toccarli: è il meccanismo

con cui Triad-Core isola un calcolo. L'esempio mostra i tre legami di `x` (`20`, `10`, `1`) e poi il valore del `begin` (`42`), che arriva all'assegnamento come qualsiasi altro valore.

La differenza tra i due costrutti è la stessa che c'è tra «raggruppare» e «isolare»: usa `begin` quando vuoi solo tenere insieme istruzioni, `let` quando un nome locale deve restare locale.

Riferimento: `conformance/core/cases/controllo`, `if_else`, `if_elseif`, `if_nested`, `if_no_else`, `while_loop`, `let_annidato`, `begin_block`.

## Esercizi

1. **Classificare.** Con `if` / `elseif` / `else` classifica un intero in `negativo`, `zero`, `piccolo` (1–9), `grande` (≥ 10).
2. **Contatore.** Scrivi un `while` che stampi i numeri pari da 2 a 20.
3. **`break`.** Trova il primo numero la cui somma con 10 supera 50, interrompendo il ciclo.
4. **`continue` e la sua variante.** Stampa i dispari da 1 a 9 con `continue`; poi riscrivilo senza `continue` perché giri anche sul seed Windows.
5. **Due cicli.** Con una variabile di stato, interrompi due `while` annidati appena `i * j` supera 12.
6. **`let` e visibilità.** Che cosa succede a una variabile nata dentro un `let` dopo il blocco? Verifica con un accesso dall'esterno.
7. **`begin`.** Costruisci un esempio in cui `begin … end` serve per raggruppare più istruzioni dove ci vuole un'espressione.
8. **`while` come espressione.** `x = while … end`: gira sull'interprete JS? E sul seed? Confronta con il capitolo 15.

9. **Condizione non `Bool`.** Un `while` con condizione intera: che cosa fanno i due runtime? Scrivi il programma che lo dimostra e annota la divergenza.
10. **Il caso di conformance.** Esegui il caso del capitolo e spiega ogni istruzione del golden.

---

## 9. Funzioni

Una funzione è un pezzo di programma a cui dai un nome, con dei parametri, che restituisce un valore. È il mattone con cui si costruisce tutto il resto: in Triad-Core non ci sono struct né metodi, quindi la funzione è anche l'unico modo per dare un nome a un'operazione e riusarla.

### 9.1 Tre modi di definire una funzione

```
# forma breve: il corpo è un'espressione
quadrato(x) = x * x

# forma blocco: il valore è quello dell'ultima
espressione
function calcola(n)
    m = n + 1
    m * 2
end

# funzione anonima assegnata a un nome
moltiplica = function (a, b)
    a * b
end
```

```
println(quadrato(4), " ", calcola(3), " ",
moltiplica(6, 7))
```

Output:

```
16 8 42
```

Quando `calcola(3)` viene chiamata, `n` riceve `3`; il corpo calcola `m = 4`, poi valuta `m * 2` e restituisce `8` perché è l'ultima espressione. Il legame `m` appartiene alla chiamata e non diventa automaticamente una variabile globale. Con la forma breve, l'espressione a destra di `=` è direttamente il corpo. La funzione anonima può essere assegnata a un nome, passata come argomento o restituita da un'altra funzione: il nome è un riferimento utile, non una proprietà necessaria del valore funzione.

## 9.2 `return`, ricorsione, arità

- `return e` esce subito dalla funzione; `return` da solo restituisce `nothing`.
- Una funzione nominata è visibile nel proprio corpo: la ricorsione diretta funziona.
- Il numero di argomenti deve coincidere con i parametri (altrimenti `arity mismatch`).

```
function taglia(n)
    if n > 100
        return :grande
    end
    :piccola
end
fattoriale(n) = n < 2 ? 1 : n * fattoriale(n - 1)
fib(n) = n < 2 ? n : fib(n - 1) + fib(n - 2)
```

```
mcd(a, b) = b == 0 ? a : mcd(b, a % b)
println(taglia(500), " ", taglia(5))
println(fattoriale(10), " ", fib(15), " ", mcd(48, 18))
```

Output:

```
grande piccola
3628800 610 6
```

La ricorsione si comprende seguendo una catena di chiamate, ciascuna con il proprio parametro. In `fattoriale(4)`, per esempio, la chiamata riduce `n` a 3, 2 e 1; arrivata al caso base restituisce 1, poi le chiamate sospese riprendono e moltiplicano i risultati per 2, 3 e 4. Il caso base è quindi una condizione di terminazione, non un semplice esempio aggiuntivo. Se il parametro non si avvicina a un caso base, la funzione continua a chiamarsi; nel runtime Core il budget di esecuzione non è un costrutto del linguaggio, quindi è responsabilità del programma evitare ricorsioni illimitate.

Senza `return`, il valore della funzione è l'ultima espressione eseguita, anche quando il corpo termina dentro un `if`. Una funzione è un valore legato a un nome quando l'istruzione di definizione viene eseguita: per questo chiamarla prima della sua riga di definizione produce `undefined variable`, non una risoluzione anticipata. Non ci sono argomenti di default né variadici: l'arità deve coincidere esattamente con i parametri.

## 9.3 Annotazione di effetto `!io`

In Triad-Full una funzione che stampa deve dichiarare `!io` (capitolo 28). Core accetta l'annotazione e la ignora: scriverla rende il codice già pronto per Full.

```
function mostra() !io
    println("hello from block")
    20 + 22
end
println(mostra())
```

Output:

```
hello from block
42
```

L'annotazione fa parte della firma in Triad-Full: lì una funzione che stampa senza `!io` viola il controllo degli effetti e il chiamante ottiene `Err(:effect)` (capitolo 28). In Core il controllo non esiste, quindi l'annotazione viene accettata e ignorata: il programma gira, ma non hai alcuna garanzia in più. Scriverla fin da subito resta comunque utile, perché il codice sarà valido quando passerà a Full e perché documenta l'intenzione di chi l'ha scritto.

Riferimento: `conformance/core/cases/funzioni`, `funzioni_semplici`, `funzioni_blocco`, `funzioni_io`, `ricorsione`, `gcd`, `formula`.

## Esercizi

1. **Tre forme.** Scrivi `quadrato` in forma breve, blocco e anonima: quale usi e quando?
2. **L'ultima espressione.** Cosa restituisce una funzione il cui corpo termina con `println(x)`? Verifica usando il risultato.
3. **`return`.** Scrivi `descrizione(n)` con due `return` anticipati e l'ultima espressione.

4. **Arity.** Quali errori ottieni chiamando `f()`, `f(1, 2)` per `f(x)`? E per `f(x, y)`?
5. **Ricorsione.** Scrivi `conta(n)` che stampa da `n` a 1 e poi `via!`; individua caso base e passo ricorsivo.
6. **`!io`.** A cosa serve l'annotazione `!io` in una firma? Scrivi una funzione `!io` che stampi e una pura che non può.
7. **Funzione come valore.** Assegna una funzione a una variabile e usala dove servirebbe una lambda.
8. **Ordine di definizione.** Cosa succede a chiamare una funzione prima di definirla? Verifica e spiega.
9. **Comporre.** Scrivi `fattoriale` che usi `prodotto(a, b)`: quante funzioni ci sono e chi chiama chi?
10. **Il caso di conformance.** Esegui il caso del capitolo e confronta con il tuo programma.

---

## 10. Lambda, chiusure e scope

Una lambda è una funzione che non ha bisogno di un nome: la scrivi dove serve, e il suo valore è la funzione stessa. Serve ogni volta che vuoi passare un comportamento a un'altra funzione — «applica questa trasformazione a ogni elemento», «chiama questo controllo su ogni valore» — senza inventare un nome che userai una volta sola. Una chiusura è una lambda che ricorda l'ambiente in cui è nata; è il meccanismo che rende possibile costruire funzioni parametriche senza ricorrere a variabili globali.

### 10.1 Lambda

```
doppio = x -> x * 2
somma = (a, b) -> a + b
```

```
applica = f -> f(3)
println(doppio(5), " ", somma(1, 2), " ", applica(x ->
x * x))
```

Output:

```
10 3 9
```

Una lambda con corpo a più istruzioni si scrive con `begin … end` o con `function (…) … end`. Le lambda senza parametri (`() -> …`) **non** sono supportate: usa `function () … end`.

## 10.2 Chiusure

Una lambda cattura l'ambiente in cui è creata. Secondo SPEC-CORE §5 le variabili catturate sono **condivise**: le mutazioni sono visibili.

```
function add(n)
    x -> x + n
end
f = add(10)
g = add(100)
println(f(5), " ", g(5))

function contatore()
    v = 0
    x -> begin
        v += x
        v
    end
end
acc = contatore()
```

```
acc(5)
println(acc(10))
```

Output:

```
15 105
15
```

> *Il seed Windows cattura **per valore** e copre solo l'ultimo legame del frame: un accumulatore come `contatore` può non funzionare sul seed (capitolo 15).*

La differenza tra una funzione ordinaria e una chiusura è il contesto esterno che resta accessibile. `add(10)` termina, ma la lambda restituita conserva il legame `n = 10`; un'altra chiamata ad `add` crea un ambiente distinto. Nel contatore, invece, la chiusura non si limita a leggere `v`: aggiorna il legame condiviso. L'interprete JS segue il contratto Core in cui questa mutazione è visibile alle chiamate successive. Il seed Windows ha una cattura più limitata, quindi un programma che dipende da questo caso va provato con l'interprete di riferimento e non inferito dal solo fatto che il seed accetti la sintassi.

## 10.3 Regole di scope

1. `function`, lambda, `let` e `while` introducono ambienti; un nome è visibile nell'ambiente in cui è legato e in quelli interni (scope lessicale).
2. **Dentro una funzione nominata** l'assegnamento lega nel frame locale: una funzione **non modifica i globali**.
3. **Dentro una lambda** l'assegnamento a un nome già legato fuori aggiorna quel legame (è così che funziona

l'accumulatore sopra).

4. **Al top level** i cicli aggiornano i globali.
5. `let x = …` crea un nuovo `x`; assegnare `x` dentro il `let` non tocca l'`x` esterno.

```
x = 1
function f()
    x = 2
    x
end
println(f(), " ", x)
c = 0
function incrementa()
    c += 1
end
incrementa()
println(c)
```

Output:

```
2 1
0
```

Il punto delicato è che le regole 2 e 3 sembrano in contraddizione: una funzione nominata non modifica un globale, una lambda sì. La differenza non è capricciosa, dipende da come vengono catturati gli ambienti. La lambda porta con sé il riferimento all'ambiente in cui è stata creata, quindi l'assegnamento trova il legame esterno e lo aggiorna. La funzione nominata è pensata per essere richiamabile da sola, senza effetti nascosti

sui nomi dello scope globale: l'assegnamento crea un legame locale, e il globale resta quello che era. L'esempio lo mostra con `x` (che resta `1`) e con `c` (che resta `0` anche dopo `incrementa()`).

Se ti serve comunicare un risultato, ci sono due strade coerenti con questo modello: restituire il valore con `return`, oppure mettere lo stato in una lista, che è mutabile e quindi condivisa anche da una funzione nominata (capitolo 11).

Riferimento: `conformance/core/cases/lambda`, `closure`, `chiusure`, `chiusure_local`, `chiusura_in_if`, `let_chiusura`, `scope`.

## Esercizi

1. **Lambda.** Scrivi `x -> x * 2`, assegnala a un nome e chiamala.
2. **Quanti parametri?** Una lambda con due parametri: gira sull'interprete JS? E sul seed Windows (cap. 15)? Provala.
3. **Chiusure diverse.** Scrivi un contatore con una chiusura e osserva la differenza fra cattura per valore (seed) e condivisa (interprete JS) descritta nel capitolo 15.
4. **Cosa vede una funzione.** Una funzione legge le variabili del programma che la contiene? E il programma quelle della funzione? Verifica entrambe.
5. **Shadowing.** Con `x = 1` esterno e `let x = 2` interno, che cosa stampa dentro e fuori il blocco?
6. **Cattura e mutazione.** Una chiusura che cattura una lista vede le modifiche fatte con `push!`? Verifica.
7. **Lambda come argomento.** Passa una lambda a una funzione di ordine superiore scritta in Core (cap. 16.1).

8. **Anonima o lambda.** Riscrivi `function (x) x + 1 end` in forma lambda e viceversa.
9. **Parametri locali.** Modificare un parametro dentro la funzione cambia l'argomento del chiamante? Prova con un `Int` e con una lista.
10. **Il caso di conformance.** Esegui il caso del capitolo e spiega l'output atteso.

---

## 11. Liste

Una lista è una sequenza ordinata di valori, raggiungibili per posizione. In Triad-Core è la struttura dati di lavoro: non ci sono struct né dizionari, quindi liste, liste di liste e liste taggate sono il modo in cui si rappresentano record, ambienti e alberi. Capire come si comporta una lista — che è mutabile e che gli indici partono da 1 — è il passaggio che apre tutta la programmazione non banale in Core.

Le operazioni di base sono poche, e vale la pena leggerle pensando a che cosa fanno alla lista come oggetto, non solo al valore che restituiscono:

- Letterale: `[1, 2, 3]`, `[]`, eterogenee e annidabili.
- Indici **da 1**: `xs[1]` è il primo elemento; fuori range è un errore.
- Assegnamento per indice: `xs[i] = v`, anche `xs[i] += v`.
- `length(xs)` numero di elementi; `push!(xs, v)` aggiunge in coda, **muta** la lista e la restituisce.
- Le liste sono riferimenti: due nomi possono indicare la stessa lista.

```
xs = [1, 2, 3]
println(xs[1], " ", length(xs))
push!(xs, 4)
xs[1] += 100
println(xs)
alias = xs
push!(alias, 5)
println(xs)
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
```

Output:

```
1 3
[101, 2, 3, 4]
[101, 2, 3, 4, 5]
115
```

Core non ha `map`, `filter`, `reduce`, `sum`, `sort`: si scrivono in Core (vedi capitolo 16).

> *Sul seed Windows: liste solo di interi, `push!` sì, `xs[i]` e `length(xs)` **no**; c'è invece l'helper non standard `get(xs, i)` con indici **da 0** (capitolo 15).*

Riferimento: `conformance/core/cases/liste`, `liste_base`, `liste_push`.

Ricorda il modello di alias: dopo `alias = xs`, i nomi non identificano due copie, ma la stessa lista. Cambiare `xs[1]` o aggiungere un elemento attraverso `alias` modifica un oggetto osservato da entrambi. Se hai bisogno di un valore indipendente, costruisci esplicitamente una nuova lista e copi gli elementi con un ciclo. Questo comportamento è diverso da una variabile intera: assegnare `y = x` copia il valore numerico e una successiva riassegnazione di `x` non aggiorna `y`.

Negli algoritmi con indici, la coppia `i = 1` e `i <= length(xs)` esprime l'intervallo completo delle posizioni Core. Prima di leggere `xs[i]`, assicurati che `i` sia dentro quell'intervallo; un indice fuori limite è un errore, non un elemento `nothing`. Il caso di conformance `liste` verifica proprio le operazioni standard, mentre l'helper `get` del seed ha indici 0-based e non deve essere confuso con `xs[i]`.

## Esercizi

1. **Costruire e leggere.** Crea `[7, 2, 9]`, stampa primo e ultimo elemento e la lunghezza.
2. **Indici da 1.** Che cosa producono `xs[0]` e `xs[length(xs) + 1]`? Quale errore?
3. **`push!` sul seed.** Perché sul seed si usa la forma `xs = push!(xs, v)`? Verifica la differenza con `push!(xs, v)` da solo.
4. **Giro completo.** Stampa tutti gli elementi con un `while` e un contatore, senza guardare l'esempio.
5. **Somma e massimo.** Scrivi `somma(xs)` e `massimo(xs)` e provalle su una lista vuota: che cosa devono restituire?

6. **Alias.** Dimostra che `alias = xs` non crea una copia e che `xs[1] = 9` è visibile ad `alias`.
7. **Copia manuale.** Crea una copia vera di una lista con un ciclo e prova a dimostrare che è indipendente.
8. **Annidate.** Costruisci `[[1, 2], [3, 4]]` e stampa il primo elemento interno.
9. **Interi soltanto.** Perché sul seed le liste sono solo di interi? Che cosa succede con `[1, "due"]`?
10. **Il caso `liste`.** Esegui il caso di conformance `liste` con l'interprete JS e con il seed: perché sul seed è escluso di default?

---

## 12. Stringhe, simboli e conversioni

Stringhe e simboli sembrano parenti stretti — entrambi contengono testo — ma hanno ruoli diversi. Una stringa è un dato che si costruisce, si concatena e si stampa; un simbolo è un nome che il linguaggio tratta come valore atomico, adatto a etichette, chiavi, tag di record e nomi di tipo. Le conversioni tra i due, e tra testo e numeri, sono il ponte che serve quando un programma legge input, costruisce messaggi o confronta valori che arrivano come testo.

La tabella riassume le operazioni disponibili in Core:

| operazione | significato |
|---|---|
| `a * b` | concatenazione di stringhe |
| `length(s)` | numero di **byte** UTF-8 (`length("é") == 2`) |

| operazione | significato |
|---|---|
| `string(x)` | rappresentazione testuale di qualsiasi valore ( `string(:a) == "a"` , `string([1, 2]) == "[1, 2]"` ) |
| `Int(s)` | da stringa a intero ( `Int("42") == 42` ); da Bool: `0` / `1` |
| `Symbol(s)` | da stringa a simbolo ( `Symbol("ciao") == :ciao` ) |
| `typeof(x)` | tipo come simbolo |

```
nome = "Triad"
saluto = "ciao " * nome
println(saluto, " (", length(saluto), " byte)")
println("risposta: " * string(40 + 2))
println(Int("41") + 1, " ", Symbol("bob"), " ",
string(true))
println(:ok == Symbol("ok"))
```

Output:

```
ciao Triad (10 byte)
risposta: 42
42 bob true
true
```

Per accedere ai singoli byte di una stringa servono i primitivi di bootstrap del capitolo successivo.

Tre cose meritano attenzione. Primo, le stringhe sono **immutabili**: `saluto = "ciao " * nome` non modifica `"ciao "`, costruisce una nuova stringa. Questo è coerente con il resto del linguaggio, dove solo le liste sono mutabili, e rende innocuo passare una stringa a una funzione: chi la riceve non può alterarla. Secondo, la concatenazione usa `*` e non `+`: `+` resta l'addizione numerica, e la scelta evita di dover indovinare il tipo degli operandi in un'espressione come `a + b`. Terzo, `length` conta i **byte**, non i caratteri percepiti: una lettera accentata può occupare due byte, quindi in Core «lunghezza di una stringa» significa «dimensione in byte UTF-8», e le operazioni sui singoli caratteri si fanno a livello di byte con i primitivi del capitolo 13.

`string(x)` è una rappresentazione pensata per la lettura, non una promessa che il risultato si possa rileggere come sorgente: `string(:a)` produce `"a"` (senza i due punti), e la stampa di una lista non è necessariamente codice Triad valido. Per i messaggi va benissimo; per ricostruire un valore servono le conversioni esplicite. Anche la conversione non è una validazione: `Int("42")` presuppone che il testo sia nel formato giusto, e che cosa faccia `Int` su un testo che non è un numero **non è specificato**; il programma deve controllare prima, non sperare in un risultato utile.

L'ultima trappola riguarda i simboli. `println(:ok)` stampa `ok`, ma il valore è ancora un simbolo: `:ok == "ok"` è `false`, e il confronto corretto è con `:ok` oppure con `Symbol("ok")`. L'aspetto stampato di un valore non è il valore.

Riferimento: `conformance/core/cases/stringhe`, `stringhe_concat`, `stringhe_multi`, `simboli_base`.

## Esercizi

1. **Concatenare.** Costruisci `ciao, Ada!` con `*` e almeno tre pezzi.

2. **Byte.** Verifica `length("ciao")` e `length("è")`: su quale runtime il secondo caso cambia comportamento e perché?
3. **Convertire.** Con `string()` e `Int()` trasforma `42` in testo, `"42"` in numero e ritorno.
4. **Escape.** Stampa `prima\ndopo` con una sola `println`: che cosa succede sul seed (cap. 15)?
5. **Simboli.** Confronta `:ok` con `"ok"`: uguaglianza, stampa e uso come etichette.
6. **Conversioni impermesse.** `Int("ciao")` e `"42" + 1`: quale errore per ciascuno?
7. **Frasi composte.** Dati `nome = "Ada"` e `anni = 36`, costruisci e stampa la frase completa.
8. **Immutabili.** Dimostra che ogni concatenazione produce una stringa nuova.
9. **Primi `n` byte.** Scrivi una funzione che stampi i primi `n` byte di una stringa (servono le primitive del capitolo 13).
10. **Il caso `stringhe`.** Esegui il caso di conformance `stringhe`: quali parti funzionano sul seed e quali no?

---

## 13. Primitivi di bootstrap

Servono a S1 per leggere e analizzare sorgente Triad. **Non** fanno parte di Triad-Full (SPEC-CORE §8): usali solo nel codice di bootstrap.

Il nome «bootstrap» spiega la loro esistenza. Per scrivere in Triad un interprete di Triad serve la capacità di leggere un file, di guardare i singoli byte di quel file e di estrarne pezzi: sono operazioni che il linguaggio non offre con i tipi normali, perché le stringhe sono immutabili e non

indicizzabili. I primitivi colmano questa lacuna al livello più basso possibile, quello dei byte, e restano fuori dal linguaggio normale perché una libreria di stringhe completa arriverà con S1, scritta in Triad.

La tabella elenca i cinque primitivi e la loro semantica:

| primitiva | semantica |
|---|---|
| `readfile(path)` | contenuto del file come `String`; errore se non leggibile |
| `strlen(s)` | numero di byte |
| `strbyte(s, i)` | byte (`Int` 0–255) alla posizione `i`, da 1 |
| `strsub(s, a, b)` | sottostringa dai byte `a` a `b` inclusi, da 1 |
| `exit(n)` | termina il processo con codice `n` |

Tutte le posizioni sono **1-based**, come gli indici delle liste, e `strsub` include entrambi gli estremi: `strsub(s, 1, 3)` restituisce i primi tre byte. `strbyte` restituisce un intero tra 0 e 255, non un carattere: il programma deve sapere quali codici gli interessano. Un indice fuori intervallo è un errore di esecuzione, non un byte zero; per questo chi analizza un sorgente controlla sempre i limiti prima di leggere.

Esempio: contare le cifre di una stringa e separare la prima parola.

```
s = "abc 123 xyz"
cifre = 0
i = 1
while i <= strlen(s)
    b = strbyte(s, i)
    if b >= 48 && b <= 57
        cifre += 1
```

```
    end
    i += 1
end
println(cifre)
fine = 1
while fine <= strlen(s) && strbyte(s, fine) != 32
    fine += 1
end
println(strsub(s, 1, fine - 1))
```

Output:

```
3
abc
```

Il primo ciclo conta le cifre confrontando il byte con i codici ASCII `48`–`57`: `strbyte` non dice «questo è un carattere», dice «questo è il numero 48», e il programma deve sapere che 48 è la cifra `0`. Il secondo ciclo cerca il primo spazio (codice `32`) e poi `strsub` estrae i byte fino a quello precedente; nota che `fine - 1` può valere `0` se la stringa comincia con uno spazio, ed è un caso che un tokenizer reale deve gestire esplicitamente.

Un programma può leggere se stesso o un altro sorgente con `readfile(ARGS[1])`: è esattamente il primo passo di un interprete self-hosted (capitolo 37). Vale però la pena ribadire il confine: un programma che usa `readfile`, `strlen`, `strbyte`, `strsub` o `exit` **non** è un programma Triad-Full valido. È una delle poche eccezioni alla regola «Core è un sottoinsieme di Full», ed è il motivo per cui questi primitivi compaiono in un capitolo a parte invece che tra le operazioni normali sulle stringhe.

### Esercizi

1. **Le cinque primitive.** Scrivi un programma minimo per ciascuna: `readfile`, `strlen`, `strbyte`, `strsub`, `exit`.
2. **Byte e caratteri.** Verifica `strlen("è") == 2` e spiega perché una `String` è una sequenza di byte.
3. **Codici ASCII.** Conta le cifre di una stringa confrontando i byte con 48–57, come nell'esempio.
4. **La prima parola.** Trova il primo spazio e estrai la prima parola con `strsub`: gestisci il caso in cui lo spazio non c'è.
5. **Maiuscole.** Conta i byte nell'intervallo 65–90 di una frase.
6. **`exit(n)`.** Termina un programma con codice 3 e verificalo dal terminale.
7. **Leggere sé stessi.** Con `readfile(ARGS[1])` fai stampare a un programma il proprio sorgente: perché è il primo passo di un interprete self-hosted?
8. **Solo cifre.** Scrivi un programma che dica se una stringa è un intero scritto in decimale.
9. **Perché «primitive».** Perché queste cinque non fanno parte di Triad-Full e non vanno usate nei programmi applicativi?
10. **Analizzatore.** Costruisci un piccolo programma che, data una stringa, ne stampi il numero di parole (parole = gruppi separati da spazi, byte 32).

---

## 14. Errori di esecuzione

Un errore di esecuzione è un punto in cui il programma non può più andare avanti: un nome che non esiste, un tipo sbagliato, un indice fuori intervallo. In Triad-Full gli errori sono valori e si possono gestire; in

Triad-Core no, e la differenza è importante: qui un errore **termina il processo con exit code 1** e stampa un messaggio diagnostico su stderr. Quello che era già stato stampato su stdout resta.

Poiché non c'è modo di catturare l'errore, la strategia di un programma Core non è «gestirlo» ma «non provocarlo»: controllare i limiti prima di un accesso, convertire i valori prima di usarli, verificare le ipotesi all'inizio di una funzione. Il messaggio del runtime serve a capire dove e perché il programma si è fermato.

```
println("prima")
println(non_esiste)
println("dopo")
```

```
stdout:  prima
stderr:  error: undefined variable non_esiste
exit:    1
```

Messaggi dell'interprete JS (stderr) e cause tipiche:

| messaggio | causa |
|---|---|
| `parse error: unexpected token …` / `parse error: expected …` | sintassi non Core (es. `1.5`, `/`, `() -> …`, `end` mancante) |
| `error: undefined variable NOME` | nome mai assegnato, o funzione chiamata prima di essere definita |
| `error: not a function` | si chiama un valore che non è una funzione (`x = 5; x(1)`) |
| `error: arity mismatch` | numero di argomenti diverso dai parametri |

| messaggio | causa |
|---|---|
| `error: index out of range` | `xs[i]` con `i < 1` o `i > length(xs)` |
| `error: + type error` (e simili per `- * % < …`) | operandi di tipo sbagliato (`1 + "a"`) |
| `error: if cond` / `while cond` / `ternary cond` / `&& expects bool` | condizione non `Bool` |
| `error: div by zero` / `negative exponent` | `x % 0`, `2 ^ -1` |
| `error: unknown function NOME` | builtin riservato usato in modo non supportato |
| `cannot read file: …` | file inesistente o non leggibile |
| `usage: triad FILE [ARG...]` (exit 2) | nessun file passato |

Il seed Windows non stampa messaggi dettagliati: esce con codice `1` (sintassi, variabile o funzione non definita, arità, lettura) o `2` (nessun argomento).

Per diagnosticare un errore, separa il **livello** dal **punto del programma**. Un `parse error` indica che il testo non è stato riconosciuto come grammatica Core: controlla prima il profilo, parentesi e `end`, poi gli operatori non Core. Un `undefined variable` o un errore di tipo arriva invece durante la valutazione: controlla l'ordine delle definizioni, l'ambiente in cui nasce il nome e il tipo effettivo degli argomenti. Se il programma stampa una prima riga e poi fallisce, quell'output precedente non viene annullato; riproduci il caso minimo e confronta stdout, stderr e codice di uscita separatamente.

Non copiare automaticamente il testo dell'errore prodotto da un runtime in un test. La suite Core confronta stdout e codice di uscita, non stderr: il contratto importante è la classe di comportamento dichiarata dallo standard. I dettagli diagnostici possono cambiare tra runtime, mentre un test del valore restituito o dell'uscita deve restare ripetibile.

## 14.1 Errori come valori, in Core

Per propagare errori senza terminare il processo (come farà S1 per i programmi Full) si usa una convenzione con liste: `[:Err, kind, msg]`.

```
err(kind, msg) = [:Err, kind, msg]
is_err(v) = typeof(v) == :List && length(v) == 3 &&
v[1] == :Err
function div_intera(a, b)
    if b == 0
        return err(:domain, "divisione per zero")
    end
    q = 0
    while a >= b
        a -= b
        q += 1
    end
    q
end
r = div_intera(7, 0)
if is_err(r)
    println("errore ", r[2], ": ", r[3])
end
println(div_intera(17, 5))
```

Output:

```
errore domain: divisione per zero
3
```

La convenzione `[:Err, kind, msg]` non è un tipo del linguaggio: è un protocollo tra funzioni, scritto con i mezzi che Core offre (liste e simboli). Chi rileva un errore lo costruisce con `err`; chi lo riceve deve controllare il tag prima di leggere i campi, ed è per questo che `is_err` esiste. Il vantaggio è che l'errore viaggia come qualsiasi altro valore: può essere restituito, salvato, confrontato, stampato. Il limite è che nulla obbliga una funzione a rispettare il protocollo: se restituisce una lista normale, il controllo del tag fallisce e il programma prosegue con il valore sbagliato.

Quando S1 eseguirà Triad-Full, il linguaggio avrà i valori `Err` veri (capitolo 24) e questa convenzione diventerà superflua; nel frattempo è il modo di far convivere «errore come dato» con un runtime che non lo prevede.

Nota: `&&` cortocircuita, quindi `v[1]` è valutato solo se `v` è una lista di 3 elementi.

## Esercizi

1. **Errori classici.** Produci volontariamente: `undefined variable`, `+ type error`, `index out of range`, `arity mismatch`, `if cond`, `not a function`.
2. **`parse error` o `error`?** Classifica ciascuno dei casi dell'esercizio 1 nelle due famiglie del capitolo 14.
3. **Codice di uscita.** Verifica che ogni programma sbagliato termini con codice `1` e che il messaggio finisca su stderr.
4. **Errori come liste taggate.** Scrivi `err(kind, msg) = [:Err, kind, msg]` e `iserr(x)` e usali per restituire un

errore da una funzione.

5. **Gestire l'errore.** Con `iserr` trasforma il risultato di una funzione in un messaggio leggibile.
6. **Categorie tue.** Inventati tre categorie di errore per un programma che lavori su liste e usale nel codice.
7. **Diagnosi.** Un programma stampa `0` invece della somma: applica il metodo del capitolo 12 delle Fondamenta e trova il bug.
8. **Errori del seed.** Perché sul seed Windows non vedi il messaggio ma solo il codice di uscita? Come diagnostichi in quel caso?
9. **Errori e output.** Un programma che stampa l'output giusto ma fallisce alla fine: quale codice di uscita? Perché è corretto così?
10. **Il caso `errore`.** Esegui il caso di conformance `errore` e spiega golden e codice atteso.

---

## 15. Interprete JS e seed Windows: differenze

Triad-Core ha due implementazioni, e non sono la stessa cosa. L'interprete JS è lo strumento di sviluppo: gira ovunque ci sia Node.js, è facile da modificare ed è il riferimento usato per verificare i casi di conformità. Il seed Windows è il primo passo del bootstrap: un interprete in codice macchina, senza runtime ospiti, più limitato perché scritto a mano. Sapere che cosa funziona su entrambi è essenziale se un programma deve girare anche sul seed, come il codice di S1.

Il contratto è `conformance/core/`: entrambi i runtime devono produrre gli stessi golden. Oggi l'interprete JS li passa tutti, il seed Windows i 33 casi elencati nel job `seed-windows` della CI. Fuori da quei casi i due runtime differiscono:

| funzionalità | interprete JS (`seed/dev/triad_core.cjs`) | seed Windows (`seed.exe`) |
|---|---|---|
| interi, Bool, operatori, confronti, ternario | ✅ | ✅ |
| `if` / `elseif` / `else`, `while`, `let`, `begin` | ✅ | ✅ |
| funzioni forma breve e blocco, ricorsione | ✅ | ✅ (max 4 parametri) |
| `return`, `break`, `continue` | ✅ | non elencati in `seed/windows-x86_64/README.md` |
| lambda | ✅ più parametri | ✅ **un solo parametro** |
| chiusure | condivise (mutazioni visibili, SPEC-CORE §5) | **per valore**, solo l'ultimo legame del frame |
| stringhe, `*`, `length(s)` | ✅ | ✅ |
| `string(x)` | ✅ tutti i tipi | solo `Int` e `String` |
| simboli, `nothing` | ✅ | ✅ |
| liste | eterogenee, `xs[i]`, `xs[i] = v`, `length`, `push!` | solo interi; costruzione, stampa e `push!` (max 64 elementi) |
| `get(xs, i)` | non esiste | helper **non standard**, indici **da 0** |

| funzionalità | interprete JS (`seed/dev/triad_core.cjs`) | seed Windows (`seed.exe`) |
|---|---|---|
| `-=`, `*=`, `Int`, `Symbol`, `typeof` | ✅ | non elencati nel README del seed |
| `ARGS`, primitivi di bootstrap | ✅ | non elencati nel README del seed |
| stampa di una funzione | `#lambda(x)` | `fun` |
| `triad -` (stdin) | ✅ | no |
| messaggi d'errore | su stderr | solo exit code |

La tabella va letta come una mappa di portabilità: la colonna di sinistra è il comportamento che un programma Core corretto può usare, la destra dice che cosa il seed sa fare oggi. «Non elencato nel README» non significa automaticamente «non funziona», ma significa che non è un comportamento su cui puoi contare senza provarlo. Le differenze più insidiose sono quelle semantiche, non quelle sintattiche: il seed può accettare una lambda e catturarla in modo diverso, quindi un programma che viene eseguito non è detto che dia lo stesso risultato.

Regola pratica: **scrivi codice che rispetta SPEC-CORE e verificalo con l'interprete JS**; se deve girare sul seed (come `selfhost/`), restringiti alla colonna di destra e verificalo su Windows.

## Esercizi

1. **Le differenze che contano.** Elenca a memoria le prime tre righe della tabella delle differenze fra i due runtime.

2. **Lambda a un parametro.** Scrivi una lambda con due parametri e verifica dove non gira: quale alternativa usi?
3. **Chiusure.** Costruisci un esempio in cui la cattura per valore del seed e quella condivisa dell'interprete JS danno risultati diversi.
4. **`string(x)` limitato.** `string(true)` e `string(:a)`: su quale runtime falliscono?
5. **Liste del seed.** Scrivi un programma sulle liste che giri su entrambi i runtime: cosa devi evitare?
6. **`get` non standard.** Perché non si usa nei programmi portabili? Riscrivi un programma che lo usi con `xs[i]`.
7. **`return`, `break`, `continue`.** Quali di questi sono rischiosi per la portabilità? Scrivi la variante sicura.
8. **Programma portabile.** Scrivi un programma Core che giri identico su entrambi i runtime e verificalo davvero su entrambi.
9. **Semantica giusta.** Quale delle due colonne della tabella descrive la semantica corretta? Perché?
10. **La tua checklist.** Scrivi la checklist di controllo che userai prima di dichiarare un programma «portabile».

---

## 16. Pattern idiomatici Triad-Core

I capitoli precedenti hanno spiegato i singoli costrutti; questo li mette insieme in alcuni schemi ricorrenti. Non sono l'unico modo di scrivere Triad-Core, ma sono i modi in cui il linguaggio viene effettivamente usato nel repository, compreso il codice di S1 in `selfhost/`. Riconoscerli aiuta a leggere il codice altrui e a scegliere una struttura quando non ne hai già una in mente.

### 16.1 Funzioni di ordine superiore scritte in Core

Core non ha `map` / `filter` / `reduce`: si scrivono una volta con `while` e lambda.

```
function mappa(f, xs)
    out = []
    i = 1
    while i <= length(xs)
        push!(out, f(xs[i]))
        i += 1
    end
    out
end
function filtra(p, xs)
    out = []
    i = 1
    while i <= length(xs)
        if p(xs[i])
            push!(out, xs[i])
        end
        i += 1
    end
    out
end
function riduci(f, acc, xs)
    i = 1
    while i <= length(xs)
        acc = f(acc, xs[i])
        i += 1
    end
    acc
```

```
end
xs = [1, 2, 3, 4, 5, 6]
println(mappa(x -> x * x, xs))
println(filtra(x -> x % 2 == 0, xs))
println(riduci((a, b) -> a + b, 0, xs))
```

Output:

```
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
21
```

Questi tre schemi mostrano che le funzioni di ordine superiore non richiedono supporto sintattico: una funzione che riceve un'altra funzione è già possibile, e `while` basta per scorrere la lista. `mappa` costruisce una lista nuova, `filtra` decide elemento per elemento, `riduci` porta avanti un accumulatore. Sono anche il punto di partenza quando il tuo programma deve trasformare dati senza le comodità di Triad-Full.

## 16.2 Record come liste taggate

Core non ha `struct`: un record è una lista con un simbolo come tag nella prima posizione.

```
punto(x, y) = [:Punto, x, y]
px(p) = p[2]
py(p) = p[3]
somma_punti(a, b) = punto(px(a) + px(b), py(a) + py(b))
p = somma_punti(punto(1, 2), punto(10, 20))
println(p, " ", p[1] == :Punto)
```

Output:

```
[:Punto, 11, 22] true
```

Il tag non è decorazione: è il discriminante che permette di riconoscere il tipo di record a runtime e di scrivere funzioni come `px` e `py` che assumono una forma precisa. In Full lo stesso ruolo sarà dello `struct` (capitolo 20), con il vantaggio di un controllo dei campi; in Core la lista taggata dà la stessa struttura senza quel controllo, e sta al programmatore rispettare la convenzione.

## 16.3 Ambiente come lista di associazioni

È la struttura usata dagli interpreti in `selfhost/`: una lista piatta `[nome1, valore1, nome2, valore2, …]` (in Core puoi anche usare coppie annidate).

```
function env_get(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    nothing
end
function env_set(env, nome, valore)
    i = 1
    while i < length(env)
        if env[i] == nome
            env[i + 1] = valore
            return env
        end
```

```
        i += 2
    end
    push!(env, nome)
    push!(env, valore)
end
env = []
env_set(env, :x, 10)
env_set(env, :y, 20)
env_set(env, :x, 11)
println(env, " ", env_get(env, :x), " ", env_get(env,
:z))
```

Output:

```
[:x, 11, :y, 20] 11 nothing
```

La lista piatta alterna nome e valore: `env_get` avanza di due posizioni, `env_set` aggiorna se trova il nome e altrimenti aggiunge in coda. Il `while i < length(env)` (non `<=`) è deliberato: l'ultima posizione utile è `length(env) - 1`, perché `env[i + 1]` deve restare dentro la lista. È la stessa struttura che rende possibile un interprete scritto in Core: gli ambienti sono dati come gli altri, non un meccanismo nascosto del linguaggio.

## 16.4 Stato mutabile condiviso

Poiché una funzione nominata non può riassegnare un globale, lo stato condiviso si tiene in una lista (mutabile) o in una chiusura.

```
stato = [0]
function tick()
    stato[1] += 1
```

```
end
tick()
tick()
println(stato[1])
```

Output:

```
2
```

La lista fa da cella condivisa: la funzione nominata non può riassegnare `stato`, ma può modificare l'oggetto a cui `stato` punta, e la mutazione resta visibile fuori. È il modo più semplice di ottenere stato mutabile in Core senza chiusure; la chiusura (capitolo 10) è l'alternativa quando lo stato deve restare privato.

## 16.5 Costruire stringhe

```
function ripeti(s, n)
    out = ""
    while n > 0
        out = out * s
        n -= 1
    end
    out
end
function unisci(xs, sep)
    out = ""
    i = 1
    while i <= length(xs)
        if i > 1
            out = out * sep
        end
```

```
        out = out * string(xs[i])
        i += 1
    end
    out
end
println(ripeti("ab", 3))
println(unisci([1, 2, 3], ", "))
```

Output:

```
ababab
1, 2, 3
```

Costruire una stringa per concatenazioni successive è corretto ma costa: ogni `*` crea una nuova stringa, quindi il ciclo costruisce tante stringhe intermedie quante sono le iterazioni. In Core non ci sono alternative migliori — non c'è una struttura di appoggio — e per i programmi di studio va bene; in un interprete reale conviene accumulare i pezzi in una lista e unirli una volta sola alla fine.

## 16.6 Stile

Le regole seguenti non sono estetiche: nascono dal fatto che lo stesso codice potrebbe dover passare a Triad-Full e che il repository è scritto in italiano.

- Funzioni piccole con nomi chiari (in italiano, come nel resto del repository).
- Caso base esplicito in ogni ricorsione; per profondità grandi preferisci `while`.
- Cicli con contatori espliciti (`i = 1 … while i <= length(xs)`).

- Non usare `fact`, `rule`, `solve`, `struct` e le altre parole chiave di Full come nomi: il codice deve restare valido anche in Triad-Full.
- Scrivi `!io` sulle funzioni che stampano: Core lo ignora, Full lo richiede.

Sono convenzioni, non regole del linguaggio: il runtime non le controlla. Ma un programma che le rispetta resta valido sia in Core sia in Full, e questa è la qualità che rende il codice del progetto riutilizzabile negli stadi successivi del bootstrap.

## Esercizi

1. **HOF in Core.** Scrivi `mappa(f, xs)`, `filtra(f, xs)` e `riduci(f, acc, xs)` con un ciclo.
2. **Record taggato.** Rappresenta `(:persona, "Ada", 36)` come lista taggata e scrivi le funzioni di accesso.
3. **Ambiente.** Riscrivi `env_get`, `env_set` e `env_rimuovi` e provali con quattro coppie.
4. **Stato condiviso.** Scrivi un contatore basato su una lista mutabile e spiega perché funziona senza assegnamenti condivisi.
5. **Costruire stringhe.** Stampa una lista di interi nel formato `[1, 2, 3]` costruendo la stringa pezzo per pezzo.
6. **Stile.** Riscrivi un tuo programma applicando le regole di stile del capitolo 16.6.
7. **Predicati come funzioni.** Scrivi `tutti(f, xs)` e `qualcuno(f, xs)` con i pattern del capitolo.
8. **Una funzione, un compito.** Dividi una funzione da venti righe in tre funzioni piccole e verifica che l'output non cambi.

9. **Nomi con `!`.** Perché `push!` ha il punto esclamativo? Scrivi una tua funzione mutante e nominala secondo la convenzione.
10. **Sottoinsieme comune.** Riscrivi un programma del capitolo perché giri anche sul seed Windows, mantenendo lo stesso output.

---

## 17. Esercizi guidati

Gli esercizi mettono in pratica i capitoli 4–16 e, letti di seguito, raccontano un percorso: si parte da calcolo e ricorsione, si passa a liste e stringhe, e si finisce con un piccolo tokenizer e un valutatore di espressioni, cioè con i primi due pezzi di un interprete. Ogni esercizio ha una soluzione verificata, nascosta in un riquadro: aprila solo dopo averci provato, perché il punto non è leggere la soluzione ma scriverne una che produca lo stesso output.

Per controllare la tua soluzione, salvala in un file ed eseguila con `node seed/dev/triad_core.cjs`; confronta poi l'output con quello mostrato nella soluzione. Se l'output coincide ma il codice è diverso, va bene: gli esercizi descrivono un comportamento, non un'unica scrittura possibile.

### Esercizio 1 — Potenza e somme

Senza usare `^`, scrivi `potenza(b, e)` ricorsiva e `somma_fino(n)` (somma `1 + … + n`) con `while`. Stampa `potenza(3, 13)`, `potenza(2, 10)` e `somma_fino(100)`.

▼ Soluzione

```
potenza(b, e) = e == 0 ? 1 : b * potenza(b, e - 1)
function somma_fino(n)
    s = 0
```

```
    i = 1
    while i <= n
        s += i
        i += 1
    end
    s
end
println(potenza(3, 13), " ", potenza(2, 10), " ",
somma_fino(100))
```

Output:

```
1594323 1024 5050
```

### Esercizio 2 — Numeri primi

Stampa la lista dei numeri primi minori di 30.

▼ Soluzione

```
function primo(n)
    if n < 2
        return false
    end
    d = 2
    while d * d <= n
        if n % d == 0
            return false
        end
        d += 1
    end
    true
```

```
end
primi = []
n = 2
while n < 30
    if primo(n)
        push!(primi, n)
    end
    n += 1
end
println(primi)
```

Output:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

### Esercizio 3 — Inverti una lista

Scrivi `inverti(xs)` che restituisce una nuova lista con gli elementi in ordine inverso.

▼ Soluzione

```
function inverti(xs)
    out = []
    i = length(xs)
    while i >= 1
        push!(out, xs[i])
        i -= 1
    end
    out
end
println(inverti([1, :due, "tre"]))
```

Output:

```
[tre, :due, 1]
```

### Esercizio 4 — Ordinamento per inserzione

Ordina in loco `[5, 2, 9, 1, 7]`.

▼ Soluzione

```
function ordina!(xs)
    i = 2
    while i <= length(xs)
        k = xs[i]
        j = i - 1
        while j >= 1 && xs[j] > k
            xs[j + 1] = xs[j]
            j -= 1
        end
        xs[j + 1] = k
        i += 1
    end
    xs
end
println(ordina!([5, 2, 9, 1, 7]))
```

Output:

```
[1, 2, 5, 7, 9]
```

Il `!` finale nel nome segnala, per convenzione, una funzione che muta il suo argomento.

### Esercizio 5 — Contatore di parole

Conta le parole (separate da spazi) di `"il  linguaggio Triad "` usando `strbyte`.

▼ Soluzione

```
function conta_parole(s)
    n = 0
    dentro = false
    i = 1
    while i <= strlen(s)
        spazio = strbyte(s, i) == 32
        if !spazio && !dentro
            n += 1
        end
        dentro = !spazio
        i += 1
    end
    n
end
println(conta_parole("il  linguaggio Triad "))
```

Output:

```
3
```

### Esercizio 6 — Un mini-tokenizer

Trasforma `"12+x*3"` in una lista di token `[tipo, testo]`, con tipi `:num`, `:id`, `:op`. È il primo passo di un interprete (vedi `selfhost/tokenizer.td`).

▼ Soluzione

```
cifra(b) = b >= 48 && b <= 57
lettera(b) = (b >= 97 && b <= 122) || (b >= 65 && b <=
90) || b == 95
function tokenizza(s)
    toks = []
    i = 1
    while i <= strlen(s)
        b = strbyte(s, i)
        if cifra(b)
            j = i
            while j <= strlen(s) && cifra(strbyte(s,
j))
                j += 1
            end
            push!(toks, [:num, strsub(s, i, j - 1)])
            i = j
        elseif lettera(b)
            j = i
            while j <= strlen(s) && (lettera(strbyte(s,
j)) || cifra(strbyte(s, j)))
                j += 1
            end
            push!(toks, [:id, strsub(s, i, j - 1)])
            i = j
        else
            push!(toks, [:op, strsub(s, i, i)])
            i += 1
        end
    end
    toks
```

```
end
println(tokenizza("12+x*3"))
```

Output:

```
[[:num, 12], [:op, +], [:id, x], [:op, *], [:num, 3]]
```

### Esercizio 7 — Valutare un'espressione

Usando i token dell'esercizio 6, scrivi un valutatore a discesa ricorsiva per `+` e `*` con precedenza, dove le variabili vengono da una lista di associazioni. Suggerimento: tieni la posizione corrente in una lista `pos = [1]` per poterla aggiornare dalle funzioni.

▼ Soluzione

```
cifra(b) = b >= 48 && b <= 57
lettera(b) = (b >= 97 && b <= 122) || b == 95
function tokenizza(s)
    toks = []
    i = 1
    while i <= strlen(s)
        b = strbyte(s, i)
        if cifra(b)
            j = i
            while j <= strlen(s) && cifra(strbyte(s,
j))
                j += 1
            end
            push!(toks, [:num, strsub(s, i, j - 1)])
            i = j
        elseif lettera(b)
```

```
            j = i
            while j <= strlen(s) && lettera(strbyte(s,
j))
                j += 1
            end
            push!(toks, [:id, strsub(s, i, j - 1)])
            i = j
        else
            push!(toks, [:op, strsub(s, i, i)])
            i += 1
        end
    end
    toks
end
function cerca(env, nome)
    i = 1
    while i < length(env)
        if env[i] == nome
            return env[i + 1]
        end
        i += 2
    end
    0
end
function primario(toks, pos, env)
    t = toks[pos[1]]
    pos[1] += 1
    if t[1] == :num
        Int(t[2])
    else
        cerca(env, t[2])
    end
```

```
end
function prodotto(toks, pos, env)
    v = primario(toks, pos, env)
    while pos[1] <= length(toks) && toks[pos[1]][2] ==
"*"
        pos[1] += 1
        v = v * primario(toks, pos, env)
    end
    v
end
function somma(toks, pos, env)
    v = prodotto(toks, pos, env)
    while pos[1] <= length(toks) && toks[pos[1]][2] ==
"+"
        pos[1] += 1
        v = v + prodotto(toks, pos, env)
    end
    v
end
valuta(s, env) = somma(tokenizza(s), [1], env)
println(valuta("12+x*3", ["x", 10]))
println(valuta("2*3+4*5", []))
```

Output:

```
42
26
```

## Esercizi supplementari

1. **Contatore di vocali.** Conta le vocali di una stringa usando i byte (`strbyte`) o i confronti sui caratteri.

2. **Massimo comune divisore.** Implementa l'algoritmo di Euclide con il solo operatore `%`.
3. **Divisori.** Stampa tutti i divisori propri di un intero e la loro somma.
4. **Crivello.** Trova i primi numeri primi fino a 50 con un crivello su lista booleana.
5. **Merge.** Unisci due liste ordinate in una terza lista ordinata, senza `sort`.
6. **Torre di Hanoi.** Stampa le mosse per spostare `n` dischi: ricorsione pura.
7. **Cifrario di Cesare.** Sposta di 3 posizioni i byte alfabetici di una stringa (attenzione a restare nell'intervallo delle lettere).
8. **Gioco del numero.** Simula «indovina il numero» con una sequenza fissa di tentativi in una lista, stampando `troppo alto` / `troppo basso`.

---

# Parte III — Triad-Full (lo standard completo)

---

Questa parte descrive il linguaggio definito da `SPEC.md`. **Non è ancora eseguibile** dal runtime attuale: serve per conoscere lo standard, scrivere codice pronto per S1 e capire i golden di `conformance/cases/`. Ogni capitolo indica la sezione di SPEC da cui deriva. Gli output marcati **Output (golden Full):** provengono dai golden congelati.

## 18. Lessico e tipi Full (SPEC §1, §4)

Full estende il lessico di Core con:

| elemento | forma | esempi |
|---|---|---|
| identificatore | come Core, anche lettere non ASCII; `!` finale se non seguito da `=` | `età`, `push!` |
| variabile logica | `?` **attaccato** a un nome | `?x`, `?who` |
| variabile anonima | `?_` (ogni occorrenza è nuova) | `parent(?x, ?_)` |
| `Float` | `3.14`, `1e-9` | |
| interpolazione | `"x = $x"`, `"$(a + b)"`; escape aggiuntivo `\$` | |
| commenti annidati | `#= a #= b =# c =#` | |
| macro | `@nome(...)` | `@unless(c, e)` |

**Parole chiave:** `function struct macro end if elseif else for in while let begin return break continue fact rule solve quote try catch with requires ensures true false nothing isa is`. Contestuali dopo `solve`: `limit depth steps trace`.

**Regole dei due punti `:`**

- `1:n`, `xs[2:end]`: `:` attaccato a un valore, senza spazi → range;
- `f(:a)`, `x = :a`: `:` non preceduto da un valore e seguito da un nome → simbolo;
- `:( … )` → codice quotato;
- `c ? a : b`: `:` circondato da spazi → ternario.

**Regola del `?:`** `?x` attaccato = variabile logica; `? ` con spazio = ternario.

Gli a capo chiudono un'istruzione, tranne dentro `( )`, `[ ]` e `:( )`, e subito dopo un operatore binario o una virgola di `rule`.

**Tipi Full** (oltre a quelli di Core):

| tipo | valori |
|---|---|
| `Int`, `Float` (sottotipi di `Number`) | `42`, `3.5` |
| `Expr` | codice quotato (`Node`) |
| `Term`, `Cons` | termini e pattern lista logici |
| `Solution` | `{who = :dave}` |
| `Err` | errore come dato |
| `Type` | `typeof(3) == Int` |
| struct utente | `Point(3.0, 4.0)` |
| `Any` | supertipo di tutto, usato nelle firme |

**Variabili tipate:**

```
x::Int = 5
x = 6          # bene
x = 5.5        # Err(:type): il nome mantiene il tipo
dichiarato
y::Float = 1.0
y = 2          # 2.0: conversione Int → Float
```

Il vincolo nasce con la dichiarazione e vive nell'ambiente in cui il nome è stato creato.

## Esercizi

1. **Novità di lessico.** Quali forme introduce Triad-Full rispetto al Core (float, interpolazione, tipi dichiarati)? Scrivi un esempio per ciascuna.
2. **Interpolazione. [Full]** Riscrivi con `"ciao $nome"` e `"$(x + 1)"` un saluto che in Core richiedeva `*` e `string()`.
3. **Tipi dichiarati. [Full]** Scrivi `x :: Int = 5` e prova assegnamenti successivi di tipi diversi: quando scatta l'errore?
4. **Gerarchia. [Full]** Disegna la gerarchia fra `Any`, `Number`, `Int`, `Float` e spiega cosa significa per il dispatch del capitolo 19.
5. **Conversioni. [Full]** Con `Int(x)`, `Float(x)`, `String(x)`, `Symbol(x)` converti un valore per ciascun tipo.
6. **`isa` e `typeof`. [Full]** Confronta i due operatori con esempi e spiega quando usare l'uno o l'altro.
7. **Escape Full.** Quali escape sono ammessi in Full (`\n \t \" \\ \$`)? Che cosa succede con uno sconosciuto?
8. **Float. [Full]** Scrivi un programma con `1.5` e `3.0`: come si comportano `+` e la stampa rispetto agli `Int`?
9. **Core dentro Full.** Prendi un programma Core del capitolo 7 e verifica che sia valido anche come programma Full.
10. **Golden.** Verifica gli output del capitolo confrontandoli con i golden della conformance: quali sono verificati e quali attesi?

---

## 19. Funzioni tipate e multiple dispatch (SPEC §5)

```
sq(x) = x * x
area(n::Number) = :number
area(n::Int) = :int

function dist(a::Point, b::Point)::Float
    requires a.x != b.x || a.y != b.y
    sqrt((a.x - b.x)^2 + (a.y - b.y)^2)
end

function +(a::Point, b::Point)      # gli operatori
sono funzioni ridefinibili
    Point(a.x + b.x, a.y + b.y)
end
```

Regole:

- vince il metodo **più specifico** tra gli applicabili ( `area(1) == :int` , `area(1.5) == :number` );
- nessun metodo applicabile → `Err(:method)` (o fallback builtin per gli operatori numerici);
- più metodi ugualmente specifici → `Err(:ambiguous)` ;
- il tipo di ritorno `::R` è controllato all'uscita (con conversione Int→Float);
- funzioni e operatori sono valori: `reduce(+, xs)` , `map(sq, xs)` ;
- lambda tipate: `function (x::Int) x end` → `f(2.0)` è `Err(:type)` .

Per prevedere quale implementazione verrà chiamata, controlla prima il numero di argomenti e poi il tipo di ciascuno. Con i metodi `area(n::Number)` e `area(n::Int)`, l'intero `1` è compatibile con entrambi, ma `Int` è più specifico di `Number`; il `Float` `1.5` è compatibile solo con il metodo su `Number`. Nei metodi con più parametri il confronto è per tupla: una firma può essere più specifica sul primo parametro ma più generale sul secondo. Se nessuna firma domina tutte le altre, non si indovina l'intenzione: il risultato è l'errore `:ambiguous`, che richiede una firma più specifica o una revisione delle firme esistenti.

Il controllo del tipo di ritorno avviene dopo il corpo. Per questo `::Float` non converte gli argomenti in anticipo e non garantisce che ogni operazione intermedia sia valida; richiede che il valore prodotto alla fine sia convertibile o compatibile con il tipo dichiarato. Le annotazioni di parametro e di ritorno sono quindi parti distinte del contratto del metodo.

## Esercizi

1. **Firme tipate. [Full]** Definisci `area(x::Int)` e `area(x::Float)` e verifica `area(1) == :int` e `area(1.5) == :number` come nell'esempio del capitolo.
2. **Più specifico vince. [Full]** Aggiungi un metodo su `Number`: quale viene scelto per `1.5` e perché?
3. **Confronto per tupla. [Full]** Con `f(a::Int, b::Number)` e `f(a::Number, b::Int)`, costruisci una chiamata ambigua e una univoca.
4. **Nessun metodo. [Full]** Chiama una funzione con un tipo senza metodi: quale errore e quali informazioni contiene?
5. **Tipo di ritorno. [Full]** In `f(x)::Int` il risultato viene convertito: scrivi un caso in cui la conversione è silenziosa e uno in cui fallisce.

6. **Dispatch e gerarchia. [Full]** Disegna l'albero dei metodi di una funzione con tre firme e indica quale scatta per quattro argomenti diversi.
7. **Operatori-funzione. [Full]** Aggiungi un metodo `+` a una struct e prova la somma di due istanze.
8. **Funzione generica. [Full]** Scrivi una funzione generica con tre metodi e spiega che cosa succede quando ne ridefinisci uno.
9. **Parametri e ritorno.** Perché il tipo di ritorno non partecipa alla scelta del metodo? Costruisci un esempio che lo dimostri.
10. **Golden.** Verifica l'output golden dell'esempio `area` e spiega perché è «atteso» e non «eseguito».

---

## 20. Struct (SPEC §4)

```
struct Point
    x::Float
    y::Float
end

p = Point(3, 4)      # costruttore posizionale, Int → Float
p.x                  # 3.0
typeof(p) == Point   # true
p isa Point          # true
```

- Immutabili; campi tipizzati.
- Campo inesistente → `Err(:field)`; tipo sbagliato → `Err(:type)`.

- In Core lo stesso concetto si rappresenta con una lista taggata (§16.2).

Una struct fornisce un record **nominato**: il nome del tipo distingue `Point` da un altro record, mentre i nomi dei campi (`x`, `y`) evitano di ricordare posizioni numeriche. La costruzione controlla i valori rispetto ai tipi dei campi; l'accesso `p.x` risolve il campo per nome. La struttura è immutabile nello standard, quindi i campi descrivono lo stato dell'istanza creata e non si aggiornano con un'assegnazione successiva. La lista taggata Core è più flessibile, ma sposta queste garanzie sul codice che costruisce, legge e modifica il record.

## Esercizi

1. **Definire una struct. [Full]** Definisci `Punto` con campi `x::Int` e `y::Int` e costruisci un'istanza.
2. **Campi. [Full]** Accedi e modifica un campo; poi prova a leggere un campo inesistente: quale errore?
3. **Costruttore.** Perché il nome della struct usato come funzione costruisce un valore? Prova con argomenti di tipo sbagliato.
4. **Struct e logica. [Full]** Usa `Point(?x, ?y)` come termine in un `fact` e unificalo con un'istanza.
5. **Struct e dispatch. [Full]** Definisci due struct con un metodo comune e chiama la funzione su entrambe.
6. **Stampa.** Come viene stampata una struct? Confronta con la stampa di una lista taggata (Manuale, cap. 22.2).
7. **Annidate. [Full]** Costruisci una struct che contiene un'altra struct e accedi ai campi interni.
8. **Confronto. [Full]** Che cosa valgono `p1 == p2` per due istanze con gli stessi campi? Verifica.
9. **Struct o lista taggata?** Quando conviene una struct e quando una lista taggata? Scrivi la stessa entità in entrambi i modi.

10. **Golden.** Verifica l'output golden dell'esempio sulle struct e annota le differenze con il comportamento attuale dei runtime.

---

## 21. Controllo di flusso Full: `for`, range, comprehension, `try` (SPEC §2)

```
s = 0
for i in 1:10
    s += i
end                                  # s == 55

xs = [1, 2, 3]
xs[end]                              # 3
xs[2:end]                            # [2, 3]
[x^2 for x in 1:4 if x > 2]          # [9, 16]
x = 3; x > 2 ? :big : :small         # :big

e = try div(1, 0) end                # l'errore diventa
il valore
r = try div(1, 0) catch err; 0 end   # con catch: err è
l'Err, il valore è quello del ramo
```

Altri valori attesi (dai vecchi test dell'implementazione Julia):

```
x = 2; y = x * 21; y                              # 42
7 / 2                                             # 3.5
"ab" * "cd"                                       # "abcd"
let a = 1, b = a + 1; a + b; end                  # 3
if 1 > 2; :a; elseif 2 > 1; :b; else; :c; end     # :b
sq(x) = x * x; map(sq, [1, 2, 3])                 # [1, 4,
```

```
9]
reduce(+, [1, 2, 3, 4])                          # 10
filter(x -> x % 2 == 0, 1:6)                     # [2, 4,
6]
function fact_(n::Int)::Int
    if n <= 1; return 1; end
    n * fact_(n - 1)
end; fact_(10)                                   #
3628800
```

`requires c` al top level è un'asserzione; `ensures c` è ammesso solo dentro `function`.

Nel ciclo, il range `1:10` rappresenta i valori interi inclusivi da 1 a 10: non è una lista letterale che devi scrivere a mano. `for` associa a `i` ciascun valore successivo, esegue il corpo per quell'associazione e prosegue fino alla fine dell'iterabile. Una comprehension combina lo stesso schema con la costruzione del risultato: valuta il filtro per ogni elemento e aggiunge all'output solo le espressioni corrispondenti. L'ordine di lettura è quindi "iteratore, filtro, espressione", non una valutazione simultanea.

`try` delimita invece il punto in cui un errore può essere trasformato in risultato. Senza `catch`, l'espressione restituisce direttamente l'`Err`; con `catch err`, il nome locale `err` identifica l'errore e il valore finale è quello del blocco di recupero. Questo rende possibile gestire un caso senza confondere un errore con un valore di successo dello stesso tipo.

## Esercizi

1. **`for` e range. [Full]** Riscrivi un `while` che somma 1..10 con `for i in 1:10`.

2. **Range. [Full]** Scrivi `a:b` con `a > b`: che cosa produce? E con `a == b`?
3. **Comprehension. [Full]** Costruisci la lista dei quadrati da 1 a 5 e poi quella dei soli dispari con un filtro.
4. **Comprehensione annidata. [Full]** Genera le coppie `(i, j)` di una matrice 3×3.
5. **`try` / `catch`. [Full]** Cattura `Int("ciao")` e gestisci l'errore per categoria.
6. **`try` come valore. [Full]** `e = try f(x) end`: che cosa contiene `e` quando tutto va bene e quando c'è un errore?
7. **`x in xs`. [Full]** Usa `in` come filtro di una comprehension.
8. **Da Full a Core.** Riscrivi un `for` con comprehension in un `while` Core equivalente e confronta gli output.
9. **Ordine di valutazione.** In una comprehension con più generatori, in quale ordine vengono prodotte le coppie? Verifica.
10. **Golden.** Verifica gli output golden degli esempi del capitolo su `for`, comprehension e `try`.

---

## 22. Logica: fatti, regole, `solve` (SPEC §6)

```
fact parent(:alice, :bob)
fact parent(:alice, :carol)
fact parent(:bob, :dave)

rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y,
?z)
rule ancestor(?a, ?d) :- parent(?a, ?d)
```

```
rule ancestor(?a, ?d) :- parent(?a, ?x), ancestor(?x, ?
d)
```

`solve` restituisce una **lista di** `Solution` (è un valore, non stampa nulla):

```
sols = solve grandparent(:alice, ?who) limit 5
[s.who for s in sols]                        # [:dave]
[s.d for s in solve ancestor(:alice, ?d)]
```

- Opzioni: `limit n`, `depth d`, `steps s` (default 1 000 000), `trace`.
- Un predicato usato come funzione è un `Bool`: `grandparent(:alice, :dave)` → `true`; si può usare in `if`.
- Goal: `p(t…)`, `g1 && g2`, `(g1, g2)`, `g1 || g2`, `!g` / `not(g)`, `a = b` (unificazione), `?y is espr`, confronti, funzione Triad che restituisce `Bool`, `true` / `false`.
- Occurs-check attivo: `solve ?x = f(?x)` non ha soluzioni (non va in ciclo).
- Liste stile Prolog nei pattern: `[?h | ?t]`.
- `vars(s)` restituisce i nomi legati in una `Solution`; si accede con `s.nome` o `s[:nome]`.

Per leggere una regola, separa la **testa** dal **corpo**. In `grandparent(?x, ?z) :- parent(?x, ?y), parent(?y, ?z)`, la testa descrive quali valori la regola può dimostrare; il corpo richiede due prove in sequenza. Il motore cerca un `parent` per il primo goal, conserva il legame trovato per `?y`, poi usa quel legame nel secondo goal. Se il secondo goal

fallisce, torna alla scelta precedente e prova il prossimo fatto compatibile: è il backtracking. Se entrambi riescono, `?z` viene restituito nella soluzione.

Le variabili con lo stesso nome in clausole diverse non condividono accidentalmente il loro stato: a ogni uso della regola vengono rinominate. L'occurs-check impedisce legami ricorsivi impossibili come `?x = f(?x)`. Il limite `depth` taglia i rami troppo profondi; `steps` limita il lavoro totale; `limit` limita quante soluzioni vengono raccolte. Sono controlli differenti, e una lista vuota ottenuta entro i limiti non è equivalente a una ricerca interrotta da `Err(:budget)`.

Esempi:

```
# percorso con costo
fact edge(:a, :b, 2)
fact edge(:b, :c, 3)
rule path(?x, ?y, ?c) :- edge(?x, ?y, ?c)
rule path(?x, ?y, ?c) :- edge(?x, ?z, ?c1), path(?z, ?
y, ?c2), ?c is ?c1 + ?c2
(solve path(:a, :c, ?costo))[1].costo      # 5

# append alla Prolog
rule app([], ?l, ?l)
rule app([?h | ?t], ?l, [?h | ?r]) :- app(?t, ?l, ?r)
(solve app([1, 2], [3], ?r))[1].r          # [1, 2, 3]
length(solve app(?a, ?b, [1, 2, 3]))       # 4

# negazione per fallimento
fact parent(:tom, :bob)
persone = [:tom, :bob]
```

```
foglie = filter(p -> !parent(p, ?_), persone)   #
[:bob]
```

## Esercizi

1. **Fatti e regole. [Full]** Costruisci una knowledge base familiare con `fact` e la regola `ancestor` del capitolo.
2. **`solve` è un valore. [Full]** Raccogli le soluzioni in una variabile, contale e leggi un legame.
3. **Percorso con costo. [Full]** Riscrivi `path` / `edge` e spiega perché serve `is`.
4. **`app`. [Full]** Riscrivi le regole di append con `[?h | ?t]` e spiega le quattro soluzioni di `solve app(?a, ?b, [1, 2, 3])`.
5. **Negazione per fallimento. [Full]** Con `!parent(p, ?_)` seleziona le foglie di una knowledge base.
6. **Occurs-check. [Full]** `solve ?x = f(?x)`: quante soluzioni e perché?
7. **Le opzioni. [Full]** Applica `limit`, `depth` e `steps` alla stessa query e confronta i tre risultati.
8. **Predicato come funzione. [Full]** Usa un predicato come condizione di un `if` e spiega che valore produce.
9. **Variabile anonima. [Full]** Usa `?_` dove non ti interessa il legame e spiega la regola delle occorrenze.
10. **Golden.** Verifica l'output golden degli esempi logici del capitolo e annota la differenza tra «atteso» e «eseguito oggi».

## 23. Destrutturazione (SPEC §2)

Pattern di lista con **arità esatta**; le parentesi attorno al pattern sono obbligatorie.

```
(a, b) = [1, 2]                 # a == 1, b == 2
e = try (a, b) = [1] end        # e.kind == :bounds
e = try (a, b) = [1, 2, 3] end  # e.kind == :bounds

xs = []
for (k, v) in [[1, 2], [3, 4]]
    push!(xs, k + v)
end                             # xs == [3, 7]
```

- Un elemento non lista o di arità diversa nel `for` → `Err(:bounds)` o `Err(:type)`.
- `function f((a, b))` definisce **due parametri** `a` e `b` (i parametri sono appiattiti): `f(10, 20) == 30`. Per destrutturare un singolo argomento lista usa `(a, b) = pair` nel corpo.
- I pattern annidati ricorsivi non sono ancora previsti.

### Esercizi

1. **Coppie. [Full]** Con `[a, b] = [1, 2]` assegna due nomi in una riga e usali.
2. **Testa e coda. [Full]** Con `[h | t] = [1, 2, 3]` che cosa leggono `h` e `t`? E con la lista vuota?
3. **In una regola. [Full]** Riscrivi `app` con i pattern `[?h | ?t]` e confronta con la versione senza destrutturazione.
4. **Scambio. [Full]** Scambia due variabili con `[a, b] = [b, a]`: funziona? Perché?

5. **Lunghezze diverse. [Full]** Che cosa succede destrutturando `[a, b]` su una lista di tre elementi? E su una lista vuota?
6. **Pattern annidati. [Full]** Destruttura `[[1, 2], [3, 4]]` per estrarre il secondo elemento della seconda coppia.
7. **Parametri. [Full]** Una funzione può ricevere una lista e destrutturarla nel corpo? Scrivi un esempio.
8. **`unparse` . [Full]** Osserva con `unparse` la forma `:destructure` di una destrutturazione: che cosa mostra il round-trip?
9. **Riscrivere senza pattern.** Riscrivi un esercizio di destrutturazione usando solo indicizzazione e `length`, e confronta la leggibilità.
10. **Golden.** Verifica l'output golden degli esempi di destrutturazione del capitolo.

---

## 24. Errori come dati (SPEC §8)

Un errore è un valore `Err(kind, msg, data)`; non diventa un'eccezione dell'host salvo che non sia gestito al top level (allora il processo esce con codice 1 e stdout vuoto per l'istruzione fallita: è il caso di conformance `errore`).

```
e = try div(1, 0) end     # Err(:domain, ...)
e.kind                    # :domain
e.msg                     # messaggio
iserr(e)                  # true
7.0 / 0                   # Inf: la divisione Float non
è un errore
throw(err(:mio, "ops", 42))
error("boom")             # Err(:error, ...)
```

```
describe(e::Err) = string("errore ", e.kind, ": ",
e.msg)
describe(x::Any) = string("valore ", x)
for r in [try div(10, d) end for d in [2, 0, 5]]
    println(describe(r))
end
```

Kinds standard: `:syntax`, `:undef`, `:type`, `:method`, `:ambiguous`, `:field`, `:bounds`, `:domain`, `:instantiation`, `:contract`, `:budget`, `:effect`, `:host`.

`Err(:syntax)` porta in `data` la posizione (`e.data.line`, `e.data.col`).

La gestione degli errori separa l'operazione fallita dal punto in cui il programma decide come reagire. `try` non ripara automaticamente lo stato: converte l'esito in un valore `Err` oppure passa quel valore al blocco `catch`. Il chiamante può quindi scegliere tra mostrare un messaggio, provare un'alternativa o propagare l'errore restituendolo. Se nessun `try` lo intercetta, il runtime lo segnala al livello superiore e il processo termina con codice non riuscito. In questo modo la gestione è esplicita, ma un errore non diventa un successo solo perché è rappresentato da un valore.

## Esercizi

1. **La forma `Err`. [Full]** Costruisci un `Err` con categoria e messaggio e ispeziona `kind` e `data`.
2. **Produrre o propagare. [Full]** Scrivi una funzione che restituisce un `Err` e una che lo lancia con `throw`: quando usare l'una o l'altra?

3. **Catturare. [Full]** Con `try` / `catch` gestisci un errore e stampa la sua categoria.
4. **Le categorie.** Associa a ciascuna categoria (`:type`, `:method`, `:bounds`, `:contract`, `:budget`, `:effect`) un programma che la produce.
5. **I dati dell'errore. [Full]** Per un `Err(:contract)` leggi `data.clause` e `data.fn`: che cosa ti dicono?
6. **Core e Full a confronto.** Riscrivi la convenzione `[:Err, kind, msg]` del Core con il tipo `Err` di Full e confronta i due programmi.
7. **Non gestito. [Full]** Lascia un errore non gestito al top level: codice di uscita, messaggio e riga segnalata.
8. **Un messaggio per ogni categoria. [Full]** Scrivi una funzione che traduca gli errori in messaggi italiani a seconda della categoria.
9. **Errori e contratti.** Perché la violazione di un contratto è un `Err` e non un arresto del programma? Scrivi un programma che continua dopo un errore.
10. **Il caso `errore`.** Esegui il caso di conformance `errore` e spiega perché è l'unico caso Full verde oggi.

---

## 25. Contratti (SPEC §9)

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
    a / b
end
```

```
safe_div(6, 3)            # 2.0
e = try safe_div(1, 0) end
e.kind                    # :contract
e.data.clause             # :requires
e.data.fn                 # :safe_div
```

- `requires c` è controllata dove compare (inizio del corpo, oppure al top level come asserzione).
- `ensures c` è controllata all'uscita, con `result` legato al valore restituito, dopo il controllo del tipo di ritorno.
- Violazione → `Err(:contract, …)` con `data.clause`, `data.fn`, `data.cond`, `data.result`.
- I contratti sono **puri**: un effetto dentro un contratto → `Err(:effect)`.
- `ensures` fuori da `function` → `Err(:syntax)`.

Pensa a `requires` come alla condizione d'ingresso dell'operazione e a `ensures` come alla condizione d'uscita. Una precondizione specifica ciò che il chiamante deve garantire; una postcondizione specifica ciò che la funzione promette quando termina correttamente. Le clausole non sostituiscono il controllo dei tipi: una funzione può ricevere argomenti del tipo giusto e violare comunque una precondizione di dominio, oppure restituire un valore tipato che viola la postcondizione. Poiché i contratti sono puri, la loro valutazione non può stampare o modificare la base di conoscenza: il controllo non deve cambiare il programma che sta verificando.

## Esercizi

1. **Precondizione. [Full]** Aggiungi `requires` a una funzione che divida per un intero e provala con input validi e non validi.

2. **Postcondizione. [Full]** Aggiungi `ensures` con `result` a una funzione che calcoli una media.
3. **Leggere la violazione. [Full]** Stampa `e.kind`, `e.data.clause`, `e.data.fn` dopo una violazione: che cosa ti dicono?
4. **Ordine dei controlli.** Perché il tipo di ritorno viene controllato prima di `ensures`? Costruisci un caso in cui la differenza si vede.
5. **Condizioni pure. [Full]** Metti un `println` dentro un contratto: quale errore?
6. **Al top level. [Full]** `requires` fuori da una funzione: che ruolo ha? Usalo come asserzione.
7. **Contratto su una ricorsione. [Full]** Aggiungi contratti a `fattoriale` e verifica che valgano a ogni livello di ricorsione.
8. **Controllo o prova?** Scrivi un contratto che non basti a garantire la correttezza e spiega il confine fra controllo e dimostrazione.
9. **Effetti vietati.** Perché le condizioni dei contratti sono pure? Che cosa succede se violi questa regola?
10. **Golden.** Verifica l'output golden dell'esempio `safe_div` del capitolo.

---

## 26. Budget (SPEC §10)

```
r = with budget(steps = 10_000)
    while true end
end
```

```
r.kind                    # :budget — il ciclo infinito
diventa un valore
```

- Un passo = una chiamata, un'iterazione di ciclo, un passo di risoluzione.
- I budget si annidano: vale il più stretto; solo il budget del `with` corrente diventa valore, quello esterno si propaga.
- `solve … steps N` è il budget di una singola query.
- La ricorsione infinita non fa stack overflow: finisce in `Err(:budget)` (stack esplicito).

Il budget misura unità di lavoro definite dallo standard, non secondi di CPU e non memoria allocata. È quindi un limite riproducibile sul numero di passi del modello di esecuzione, non un timeout generale del sistema operativo. Usalo per rendere limitate le computazioni potenzialmente lunghe, in particolare la risoluzione logica o un ciclo che potrebbe non terminare. Il risultato `Err(:budget)` è distinto dal risultato valido del corpo e può essere gestito con `try` o con il controllo del valore restituito dal `with`.

## Esercizi

1. **Budget minimo. [Full]** Avvolgi un `while true` in `with budget(steps = 10_000)` e verifica `Err(:budget)`.
2. **Che cos'è un passo.** Elenca tre costrutti che consumano passi e spiega perché il budget misura lavoro e non tempo.
3. **Annidati. [Full]** Un budget di 100 dentro uno di 1000: quale scatta e che cosa restituisce il blocco interno?
4. **`solve … steps`. [Full]** Confronta `steps` di `solve` con `with budget`: quale vince quando valgono entrambi?
5. **Ricorsione sinistra. [Full]** `f(n) = f(n + 1)` con budget: quale errore invece dello stack overflow?

6. **Riproducibilità.** Perché due esecuzioni con lo stesso budget consumano gli stessi passi? Che cosa renderebbe instabile un timeout sull'orologio?
7. **Gestire l'esaurimento. [Full]** Scrivi un programma che, ricevuto `Err(:budget)`, stampi una diagnosi e riprovi con un budget maggiore.
8. **Vuoto o esaurito. [Full]** Distingui una query senza soluzioni da una terminata per budget con due esempi.
9. **`limit`, `depth`, `steps`.** Riassumi in tabella le tre opzioni e il tipo di risultato che producono.
10. **Golden.** Verifica l'output golden dell'esempio del capitolo.

---

## 27. Tracce (SPEC §11)

La traccia è un **valore**, non output.

```
fib(n::Int) = n < 2 ? n : fib(n - 1) + fib(n - 2)
t = with trace
    fib(3)
end
t.result      # 2
t.trace       # [Event(port=:call, fn=:fib, args=[3],
depth=0), …]

t = solve path(:a, :c) trace
t.trace       # porte Byrd: :call, :exit, :redo, :fail,
:depth
```

Gli eventi sono record con `port`, `goal` / `fn`, `depth` (e `value` / `args` per le funzioni): si filtrano come qualsiasi valore.

La traccia registra la storia del calcolo, non solo il suo risultato. In una traccia di funzione `call` indica l'ingresso e `exit` l'uscita; in una query logica compaiono anche `redo` e `fail`, perché la ricerca può provare più alternative. Il valore `result` resta disponibile accanto agli eventi: puoi confrontare il risultato finale con il percorso seguito. La traccia non è un log stampato automaticamente; decide il programma quali eventi filtrare, contare o presentare.

## Esercizi

1. **Prima traccia. [Full]** Avvolgi un piccolo `solve … trace` o `with trace` e leggi gli eventi prodotti.
2. **Le porte di Byrd.** Spiega `:call`, `:redo`, `:exit`, `:fail` con un esempio per ciascuna.
3. **Contare i fallimenti.** Riscrivi l'esempio che conta i fallimenti di una ricerca e provalo su una knowledge base tua.
4. **Eventi come dati. [Full]** Filtra gli eventi di traccia per categoria e contali.
5. **Ordine atteso.** Dato un goal con due soluzioni, scrivi a mano la sequenza attesa degli eventi e verifica.
6. **Debug di un goal.** Un goal non trova risposte: come usi la traccia per localizzare il fallimento?
7. **Traccia o `println`.** Perché per la logica una traccia è meglio di `println` di debug? Rispondi con un esempio.
8. **Trace e limit. [Full]** Unisci `trace` e `limit` e osserva dove la ricerca si ferma.
9. **Trace e budget. [Full]** Con `trace` e `steps` insieme, come si riconosce negli eventi l'esaurimento del budget?
10. **Golden.** Verifica l'output golden dell'esempio tracciato del capitolo.

## 28. Effetti dichiarati (SPEC §12)

Le funzioni sono pure per default. Effetti: `io` (`println`, `print`) e `state` (`fact` / `rule` dentro una funzione).

```
function saluta(nome::String) !io
    println("ciao, ", nome)
end

function impara(x) !state
    fact visto($x)
end
```

- Controllo dinamico: una funzione pura che chiama una `!io` o `println` → `Err(:effect)`.
- Il top level permette tutti gli effetti; le lambda ereditano quelli del contesto.
- Macro e contratti sono puri; leggere la KB (`solve`, predicati) è puro.
- Mutare liste o variabili locali non è un effetto.

La dichiarazione di effetto è una verifica dinamica dei permessi di esecuzione, non un'annotazione puramente descrittiva. Il runtime controlla la catena delle chiamate quando una funzione tenta un'operazione con effetto: una funzione `!io` può stampare solo se il contesto chiamante lo consente. Il top level è il contesto più permissivo; una funzione pura chiamata da lì non diventa per questo autorizzata a stampare. Elencare `!io` nella firma documenta il requisito per i chiamanti e permette al runtime di rifiutare l'effetto nel punto in cui violerebbe la dichiarazione.

## Esercizi

1. **Pura di default. [Full]** Scrivi una funzione che stampi senza `!io`: quale errore?
2. **Dichiarare `!io`.** Correggi la firma dell'esercizio 1 e verifica che ora funziona.
3. **`!state`. [Full]** Scrivi una funzione che aggiunga un fatto alla knowledge base: quale effetto richiede?
4. **Chiamate indirette. [Full]** Costruisci una catena di tre funzioni in cui l'effetto è tentato alla terza: chi riceve l'errore?
5. **`fn`, `caller`, `missing`. [Full]** Ispeziona i tre campi di `data` in un `Err(:effect)` e spiega come guidano la correzione.
6. **Top level.** Perché al top level si può stampare senza dichiarazioni?
7. **Lambda.** Da dove una lambda eredita gli effetti? Scrivi un esempio.
8. **Puro e impuro.** Perché `solve` è puro mentre `fact` eseguito in una funzione no?
9. **Mutazione locale.** Perché `push!` e gli assegnamenti locali non sono effetti dichiarati?
10. **Golden.** Verifica l'output golden dell'esempio del capitolo (`effect`).

---

# 29. Codice come dato, albero uniforme e macro (SPEC §3, §7)

## 29.1 L'albero uniforme

Il parser produce un solo tipo di nodo, `Node(head, args)`: è lo stesso albero che il kernel esegue, che `:( … )` restituisce e che `--tree` stampa come S-expression.

| sintassi | nodo |
|---|---|
| `f(a, b)`, `a + b` | `(call f a b)`, `(call + a b)` |
| `a && b`, `a \|\| b` | `(and a b)`, `(or a b)` |
| `x = e` | `(assign x e)` |
| `:alice` | `(sym alice)` |
| `a.x`, `xs[i]` | `(dot a x)`, `(index xs i)` |
| `[a, b]`, `[h \| t]` | `(list a b)`, `(cons (list h) t)` |
| `[e for x in xs if c]` | `(comprehension e x xs c)` |
| `x -> e` | `(lambda (tuple x) e)` |
| `x::T` | `(typed x T)` |
| `"a $x"` | `(string "a " x)` |
| `if c a else b end`, `c ? a : b` | `(if c (block a) (block b))` |
| `while`, `for`, `let`, `begin` | `(while c body)`, `(for x iter body)`, `(let (tuple …) body)`, `(block …)` |

| sintassi | nodo |
|---|---|
| `function f(a::T)::R !io … end` | `(function f (tuple (typed a T)) R (tuple io) body)` |
| `struct P; x::Float; end` | `(struct P (tuple (typed x Float)))` |
| `macro m(a) … end`, `@m(e)` | `(macro m (tuple a) body)`, `(macrocall m e)` |
| `:(e)`, `quote … end`, `$e` | `(quasi e)`, `(quasi (block …))`, `(unquote e)` |
| `fact t`, `rule h :- g1, g2` | `(fact t)`, `(rule h g1 g2)` |
| `solve g limit 5 trace` | `(solve g (tuple (assign limit 5) (assign trace true)))` |
| `try b catch e h end` | `(try b e h)` |
| `with budget(steps = N), trace … end` | `(with (tuple (call budget (assign steps N)) trace) body)` |

Leggi una riga della tabella come una mappa tra **sintassi visibile** e **rappresentazione interna**. Per esempio `a + b` non richiede un tipo di nodo speciale per la somma: è una chiamata al nome `+` con due argomenti. Al contrario `&&` diventa un nodo dedicato perché il valutatore deve poter evitare di valutare il secondo operando. La forma dell'albero quindi non è una decorazione: codifica le decisioni che il runtime deve prendere per rispettare la semantica.

Questo modello è utile anche quando si legge un errore: se `sexpr` mostra un albero che non corrisponde al raggruppamento atteso, il problema nasce nel parsing; se l'albero è corretto ma il risultato è

sbagliato, il problema è più probabilmente nella valutazione. La distinzione tra forma del programma e valore del programma è il fondamento della metaprogrammazione.

## 29.2 Quote, unquote, eval

```
c = :(a.x + f(b, 2))
c.head        # :call
sexpr(c)      # "(call + (dot a x) (call f b 2))"

n = 7
quadrato = :($n * $n)
eval(quadrato)                  # 49
parse("if x > 0; :pos; else; :neg; end")   # albero
unparse(parse_program("1:n"))   # "1:n"
Expr(:call, :f, 1)              # costruisce un Node
```

## 29.3 Macro

Le macro ricevono codice e restituiscono codice; l'espansione è pura e **non igienica**: usa `gensym()` per nomi freschi.

```
macro unless(c, body)
    :(if !$c; $body; end)
end

macro swap(a, b)
    tmp = gensym(:tmp)
    quote
        $tmp = $a
        $a = $b
        $b = $tmp
```

```
    end
end

macro time_it(label, e)
    :(begin
        r = $e
        println($label, " -> ", r)
        r
    end)
end

@swap(x, y)
@unless(x < y, println("x non è minore di y"))
@time_it("somma", reduce(+, 1:100))     # stampa "somma
-> 5050"
```

Per valutare una macro, confronta l'argomento sorgente con il codice che essa costruisce. In `@unless(c, body)`, `c` e `body` arrivano come nodi; la macro li inserisce in un `if` con la condizione negata. Il corpo verrà eseguito solo se la condizione originale è falsa, proprio come se il programmatore avesse scritto quel codice manualmente. Se la macro usasse una funzione ordinaria, gli argomenti verrebbero calcolati prima della chiamata e non sarebbe possibile controllare se `body` viene valutato. Questa capacità è potente, ma rende più difficile leggere l'esecuzione: l'espansione esplicita e i nomi temporanei generati sono strumenti essenziali per evitare sorprese.

## 29.4 Codice che genera codice

```
function poly(coeffs)
    acc = :(0)
    i = 0
```

```
    for c in coeffs
        acc = :($acc + $c * x ^ $i)
        i += 1
    end
    acc
end
p = poly([1, 2, 3])
x = 2
eval(p)        # 17
```

## Esercizi

1. **Albero uniforme. [Full]** Scrivi head e args di `Node` per `1`, `:x`, `[1, 2]` e `:(f(1))`.
2. **Quote e unquote. [Full]** Costruisci `:(x + $n)` con `n` calcolato: che cosa ottieni?
3. **`eval`. [Full]** Valuta una quote con `eval` e confronta con la forma non quotata.
4. **Round-trip. [Full]** `unparse(parse("1 + 2"))` (o l'equivalente): il testo è lo stesso?
5. **`triad --tree`.** Stampa l'albero uniforme di un programma piccolo e riconosci i nodi.
6. **Una macro. [Full]** Scrivi `@due_volte(e)` che esegua due volte la sua espressione.
7. **Igiene. [Full]** Costruisci un conflitto di nomi in una macro e correggilo con `gensym`.
8. **Codice che genera codice. [Full]** Scrivi una macro che definisca una funzione, come nell'esempio 29.4.
9. **Macro o funzione.** Spiega con un esempio quando serve una macro (codice non valutato subito) e quando basta una

funzione.

10. **Golden.** Verifica l'output golden dell'esempio sulle macro del capitolo.

---

## 30. Builtin Full (SPEC §13)

| ambito | funzioni |
|---|---|
| aritmetica e confronto | `+ - * / ^ % == != < <= > >= !`, `abs sqrt floor ceil round exp log sin cos div mod min max`; `*` concatena stringhe |
| liste | `length isempty first last rest reverse sort sum push! pop! append! vcat collect`, `a:b`, `x in xs` |
| ordine superiore | `map filter reduce any all count` |
| stringhe e output | `string repr println print` |
| tipi | `typeof`, `isa`, `Int Float String Symbol` |
| errori | `err iserr throw error` |
| codice | `eval parse unparse sexpr Expr gensym` |
| logica | `vars(sol)` |

Di questi, Core fornisce solo `+ - * % ^`, confronti, `&& || !`, `length`, `push!`, indicizzazione, `*` su stringhe, `Int`, `string`, `Symbol`, `print`, `println`, `typeof` (SPEC-CORE §7).

### Esercizi

1. **Aritmetica completa. [Full]** Prova `abs sqrt floor ceil round exp log sin cos div mod min max`: quali convertono il risultato e in quale tipo?
2. **Liste. [Full]** Usa `isempty first last rest reverse sort sum pop! append! vcat collect` su una lista di cinque elementi.
3. **Ordine superiore. [Full]** Con `map filter reduce any all count` risolvi tre piccoli problemi (quadrati, maggiori di 3, somma).
4. **`x in xs` e `a:b`. [Full]** Usa `in` e il range in un programma solo e spiega il tipo dei risultati.
5. **Stringhe. [Full]** Confronta `string` e `repr` su un intero, una stringa e un simbolo.
6. **Errori. [Full]** Fai un esempio per ciascuno: `err`, `iserr`, `throw`, `error`.
7. **Codice. [Full]** Fai un esempio per ciascuno: `eval`, `parse`, `unparse`, `sexpr`, `gensym`.
8. **Tipi. [Full]** Con `typeof`, `isa` e i costruttori `Int Float String Symbol` converti un valore per tipo.
9. **`vars(sol)`. [Full]** Dopo una `solve`, elenca i nomi legati con `vars`.
10. **Core e Full.** Riscrivi un programma Full del capitolo usando solo i builtin Core e confronta la lunghezza dei due sorgenti.

---

## 31. Riga di comando e REPL (SPEC §14)

Interfaccia normativa di `triad` (oggi il runtime implementa solo `triad FILE [ARG...]` e, nell'interprete JS, `triad -`):

```
triad                              # REPL
triad FILE [ARG...]                # esegue FILE; ARGS ==
[ARG...]
triad -l FILE                      # carica FILE e apre
il REPL
triad -i FILE                      # esegue FILE e poi
apre il REPL
triad -e CODE                      # esegue CODE
triad -E CODE                      # esegue CODE e stampa
il valore
triad --tree FILE                  # stampa l'albero
uniforme come S-expression
triad -q                           # REPL senza banner
triad -h, --help
triad -v, --version
triad -                            # legge lo script da
stdin
```

- L'estensione `.triad` / `.td` si può omettere; lo shebang `#!` è ignorato.
- Exit: `0` successo, `1` errore non gestito (stampato su stderr), `2` uso non valido.

**REPL** (stile Prolog):

```
triad> 1 + 2
3
triad> ?- ancestor(:tom, ?d)
?d = :bob ;
?d = :liz.
```

```
triad> parent(:jim, ?x)
false.
```

- I blocchi aperti restano in attesa fino a `end`.
- `?- goal` (o un goal su un predicato della KB) esegue una query; dopo ogni soluzione `;` / `n` /spazio = successiva, `a` = tutte, invio/ `.` = stop.
- Comandi: `:load FILE…` ( `:l` ), `:reload` ( `:r` , `make` ), `:listing [nome|nome/arità]` ( `:ls` ), `:tree CODICE` , `:reset` , `:help` ( `:h` ), `:quit` ( `:q` , `halt` , Ctrl-D).

## Esercizi

1. **Eseguire con argomenti.** `triad FILE a b` : come li vede il programma? Verifica con `ARGS` .
2. **`-e` e `-E` .** Confronta le due forme con un frammento il cui valore non sia `nothing` .
3. **`-l` e `-i` .** Prepara un file di definizioni e provalo con entrambe le opzioni: quale differenza?
4. **`--tree` .** Stampa l'albero uniforme di un espressione complessa e leggilo.
5. **Dalla pipeline.** Usa `triad -` per leggere uno script da stdin.
6. **Query al REPL.** Esegui `?- goal` con `limit` e con `trace` e leggi le risposte.
7. **I comandi del REPL.** Prova `:load` , `:reload` , `:listing` , `:tree` , `:reset` , `:quit` .
8. **Codici di uscita.** Produci `0` , `1` , `2` con tre programmi diversi e verificali.
9. **Estensioni e shebang.** `triad file` senza estensione e un file con `#!` : che cosa succede?

10. **Oggi, in pratica.** Quale di queste opzioni supporta il seed Windows? Con quale comando esegui oggi questi esempi?

---

## 32. Esempi Full con output golden

I file sono in `examples/` (eseguibili quando S1 sarà pronto); gli output sono i golden congelati in `conformance/cases/`.

| file | copre | caso golden |
|---|---|---|
| `examples/assaggio.td` | struct, dispatch, logica, macro, contratti | `assaggio` |
| `examples/famiglia.td` | fatti/regole, ricorsione, liste Prolog, negazione | `famiglia` |
| `examples/garanzie.td` | errori, contratti, budget, tracce, effetti | `garanzie_core` |
| `examples/macro.td` | quote, eval, macro, gensym | `macro` |
| `examples/tour.td` | tutti i paradigmi | `tour` |
| `examples/hello_shebang.td` | shebang, `ARGS`, effetti | `hello_shebang` |
| `examples/benvenuto.triad` | estensione canonica, `Float`, `solve` | — |

### 32.1 `assaggio.td`

```
struct Point
    x::Float
    y::Float
end
```

```
function dist(a::Point, b::Point)::Float
    requires a.x != b.x || a.y != b.y
    sqrt((a.x - b.x)^2 + (a.y - b.y)^2)
end

fact parent(:alice, :bob)
fact parent(:bob, :dave)
rule grandparent(?x, ?z) :- parent(?x, ?y), parent(?y,
?z)

result = solve grandparent(:alice, ?who) limit 5
println("nipoti: ", result)     # [{who = :dave}]
println("primo: ", result[1].who) # :dave
```

Output (golden Full) del file completo:

```
nipoti di alice: [{who = :dave}]
primo: dave
quanti: 1
dist(p, q) = 5.0
dist(p, p) -> Err(:contract, "requires violata in dist:
a.x != b.x || a.y != b.y", Info(fn = :dist, clause =
:requires, cond = :(a.x != b.x || a.y != b.y)))
tipo: contract
almeno un nipote
alice è nonna di dave? true
```

### 32.2 `famiglia.td`

Output (golden Full):

```
discendenti di tom: [:bob, :liz, :ann, :pat, :jim]
fratelli di ann: [:pat]
madri: ["pat -> jim"]
senza figli: [:liz, :ann, :jim]
over 50: [:tom, :bob]
modi di spezzare [1, 2, 3]:
  [] ++ [1, 2, 3]
  [1] ++ [2, 3]
  [1, 2] ++ [3]
  [1, 2, 3] ++ []
figli di bob: [:ann, :pat]
```

### 32.3 `garanzie.td`

Output (golden Full, traccia abbreviata):

```
2.5
violazione: contract — requires violata in safe_div: b
!= 0
clausola: requires, funzione: safe_div
valore 5
errore domain: divisione intera per zero
valore 2
valore 42
errore input: età non valida
catturato mio con dati [1, 2, 3]
ciclo infinito -> budget
solve infinito -> budget
fib(3) = 2 in 10 eventi
chiamate: [[3], [2], [1], [0], [1]]
  call path(:a, :c) @0
```

```
  call edge(:a, :c) @1
  fail edge(:a, :c) @1
  …
ciao, Triad
effetto non dichiarato -> println richiede !io ma
rumorosa non lo dichiara
imparato: [:gatto]
```

### 32.4 `macro.td`

Output (golden Full):

```
codice:  :(a.x + f(b, 2))
head:    call
args:    [:+, :(a.x), :(f(b, 2))]
sexpr:   (call + (dot a x) (call f b 2))
:(7 * 7) = 49
parse:   (if (call > x 0) (block (sym pos)) (block (sym
neg)))
unparse: if x > 0; :pos; else; :neg; end
dopo @swap: x = 2, y = 1
x non è minore di y
somma -> 5050
polinomio: :(0 + 1 * x ^ 0 + 2 * x ^ 1 + 3 * x ^ 2)
p(2) = 17
```

### 32.5 Contratti e budget insieme

```
function safe_div(a::Number, b::Number)::Float
    requires b != 0
    ensures abs(result * b - a) < 1e-9
```

```
    a / b
end
e = try safe_div(1, 0) end
e.kind          # :contract
r = with budget(steps = 1000)
    while true end
end
r.kind          # :budget
```

## Esercizi

1. **`assaggio.td`.** Leggi l'esempio 32.1 e annota i costrutti Full che usa; confronta l'output con il golden.
2. **`famiglia.td`.** Nell'esempio 32.2 individua i metodi e spiega quale scatta per ogni chiamata.
3. **`garanzie.td`.** Nell'esempio 32.3 quali garanzie sono contratti e quali budget? Spiega l'ordine dei controlli.
4. **`macro.td`.** Espandi a mano la macro dell'esempio 32.4 e confronta con il golden.
5. **Contratti e budget insieme.** Nell'esempio 32.5 quale dei due interviene per primo e perché?
6. **Golden o verificati?** Perché questi output si chiamano golden? Che cosa si può dire e che cosa no del programma che li produce?
7. **Dove stanno i golden.** Individua nella suite di conformance i file di output atteso di questi esempi.
8. **Tradurre in Core.** Riscrivi un esempio Full in un programma Core equivalente e confronta le due versioni.
9. **Un esempio tuo.** Scrivi un programma che usi tre costrutti Full insieme e redigine l'output atteso come se fosse un

golden.

10. **Dal golden al test.** Spiega come si potrebbe verificare automaticamente il tuo esempio quando il runtime Full sarà disponibile.

---

# Parte IV — Il codice del progetto

Questa parte spiega come è fatto il repository e come si scrive (o si modifica) ogni tipo di codice che contiene.

## 33. Mappa del repository

| percorso | linguaggio | cosa contiene |
|---|---|---|
| `SPEC.md`, `SPEC-CORE.md`, `VERSION`, `CHANGELOG.md` | Markdown | lo standard (Full e Core), versione, storia |
| `BOOTSTRAP.md`, `STATUS.md`, `TASK.md`, `plan.md` | Markdown | piano S0 → S4, stato, backlog e task |
| `conformance/` | Triad + Node | casi golden ( `cases/` Full, `core/cases/` Core) e runner `run.mjs` |
| `seed/windows-x86_64/` | codice macchina | `seed.hex` (sorgente), `seed.exe` (decodificato), README con layout |

| percorso | linguaggio | cosa contiene |
|---|---|---|
| `seed/linux-x86_64/` | codice macchina | seed ELF E0 ( `seed.hex` , `seed` ) |
| `seed/dev/` | Node.js | generatori del seed ( `gen_seed.js` , `gen_seed_linux.js` ) e interprete di riferimento `triad_core.cjs` |
| `selfhost/` | Triad-Core | S1: interprete Triad scritto in Triad-Core (in corso) |
| `examples/` | Triad-Full | programmi di esempio dello standard |
| `cli/` | Node.js (ESM) | `triad-cli` : CLI di sviluppo (commander, execa, chalk) |
| `bin/` | sh / cmd | launcher `triad` , `triad.cmd` , `triad-cli` , `triad-cli.cmd` |
| `ai/` | Triad + Node | benchmark di generazione codice per LLM, prompt, task con golden |
| `editors/` | JSON / Vim script | grammatica VS Code (TextMate), sintassi Vim, voce Linguist |
| `docs/` | HTML/CSS/JS | sito GitHub Pages e materiali pubblicati dal repository |
| `obsidian-triad/` | Markdown | vault Obsidian con indice dei file e grafo del progetto |
| `.github/workflows/` | YAML | CI ( `ci.yml` ) e pubblicazione del sito ( `pages.yml` , `static.yml` ) |

| percorso | linguaggio | cosa contiene |
|---|---|---|
| `probes/test_*.td` | Triad-Core | piccoli probe usati durante lo sviluppo del seed e nella CI (es. `probes/test_get.td`) |

Convenzioni comuni:

- documentazione, commenti e messaggi sono in **italiano**;
- fine riga **LF** ovunque, tranne `*.cmd` (CRLF), imposto da `.gitattributes`;
- `.gitignore` esclude `*.out` con l'eccezione `!conformance/**/expected.out`.

## Esercizi

1. **I luoghi del progetto.** Dove si trovano: lo standard, l'interprete JS, il seed, la suite di conformance? Apri ciascun percorso.
2. **Gli esempi.** Scegli tre file da `examples/` e prevedi l'output prima di eseguirli.
3. **Il benchmark.** Individua `ai/` e apri un task: in quale file sta il problema?
4. **Gli editor.** Quali file definiscono il riconoscimento di `.triad` per VS Code, Vim e GitHub Linguist?
5. **Il self-hosting.** Dove sta il codice S1 e quale file lo mette alla prova?
6. **Gli strumenti.** Distingui gli strumenti di sviluppo (`seed/dev/`, `cli/`, `conformance/`) dalla catena di bootstrap.

7. **La documentazione.** In quale cartella stanno il Tutorial, il Manuale e questo volume?
8. **I test.** Dove stanno i programmi di prova ( `tests/` )? Ne esegui uno a scelta.
9. **Il sito.** Quale cartella genera il sito del progetto e quale file lo costruisce?
10. **Mappa tua.** Disegna la mappa del repository a memoria e poi verificala con `ls` .

---

## 34. L'interprete di riferimento JS ( `seed/dev/triad_core.cjs` )

È un interprete Triad-Core completo in un solo file CommonJS (~1000 righe, nessuna dipendenza). È lo strumento con cui si verifica la conformance Core su Linux/macOS ed è il modo più rapido per capire la semantica di Core leggendo codice.

### 34.1 Architettura

```
sorgente ──tokenize()──▶ token ──Parser──▶ AST {kind: …} ──evalStmt/evalExpr──▶ effetti su stdout
```

1. **Valori** (in cima al file): oggetti `{t, v}` costruiti da `I` (Int, `BigInt` con `wrap()` a 64 bit), `B` , `S` , `Y` (Symbol), `L` (List), `F` (funzione: `params` , `body` , `env` , `isLambda` ) e la costante `NOTHING` . I builtin sono `{t:'builtin', name}` .
2. **`tokenize(src)`** : produce token `SEP` (a capo o `;` ), `NUM` , `STR` , `SYM` , `IDENT` , `OP` , `LPAREN` / `RPAREN` , `LBRACKET` / `RBRACKET` , `COMMA` , `QMARK` , `COLON` , `EOF` . Commenti e spazi sono scartati qui.

3. `class Parser` : discesa ricorsiva che segue la precedenza del §7.1: `parseProgram → parseStmt → parseExpr → parseAssign → parseTernary → … → parsePostfix → parsePrimary` , più `parseBlock(terminators)` per i corpi fino a `end` / `else` / `elseif` . Nodi prodotti: `program` , `funcdef` , `return` , `break` , `continue` , `exprstmt` , `num` , `str` , `sym` , `bool` , `nothing` , `var` , `list` , `lambda` , `assign` , `indexassign` , `binary` , `unary` , `ternary` , `call` , `index` , `if` , `while` , `let` , `begin` .
4. **Ambienti**: `makeFrame(parent, owner, isLambda)` crea un frame con una `Map` di legami; `lookup` risale la catena. `writeVar` implementa le regole di scope del §10.3: aggiorna un legame dello stesso "proprietario" (funzione), oppure, dentro una lambda, un legame esterno; altrimenti crea un locale.
5. **Valutazione**: `evalStmt` , `evalExpr` → `evalExprBinary` → `evalExprRest` → `evalExprBlocks` . `return` , `break` , `continue` sono eccezioni JS ( `ReturnSignal` , `BreakSignal` , `ContinueSignal` ) catturate dalla chiamata o dal `while` .
6. **Builtin**: insieme `BUILTIN_NAMES` + `callBuiltin(name, args, env)` ; stampa con `printValue` (simboli senza `:` solo al top level) e conversione con `stringOf` .
7. `main()` : legge il file (o stdin con `-` ), lega `ARGS` , esegue; errori di parsing → `parse error: …` , errori di esecuzione → `error: …` , entrambi con exit 1; nessun file → exit 2.

Questo flusso separa responsabilità che conviene diagnosticare una alla volta. Se il token prodotto da `tokenize` è sbagliato, il problema è prima del parser; se il parser genera la forma errata, l'albero AST non corrisponde alla grammatica; se l'albero è giusto ma il risultato no, il

difetto è nell'ambiente, nella valutazione o in un builtin. Un caso di conformance piccolo permette di fissare il comportamento osservato mentre si corregge il livello responsabile. Quando si modifica sintassi o semantica, il test, l'interprete e la specifica devono restare coerenti: aggiungere un ramo al parser senza la corrispondente valutazione crea una forma riconosciuta ma non eseguibile.

## 34.2 Esempio: aggiungere un builtin

Supponiamo di voler sperimentare un builtin `abs` (non è in Core: va prima proposto nello standard, vedi §43). Servono due modifiche:

```
// 1. registrare il nome
const BUILTIN_NAMES = new
Set(['print','println','length','push!','Int','string',
'Symbol',

'typeof','readfile','strlen','strbyte','strsub','exit',
'abs']);

// 2. implementarlo in callBuiltin, prima del throw
finale
  if (name === 'abs') {
    if (args.length !== 1 || args[0].t !== 'int') throw
new Error('abs');
    return I(args[0].v < 0n ? -args[0].v : args[0].v);
  }
```

Poi: un caso in `conformance/core/cases/` (§38) e `node conformance/run.mjs`. Ricorda che gli `Int` sono `BigInt` (`0n`, `1n`) e che i risultati vanno costruiti con gli helper (`I`, `B`, `S`, `Y`, `L`, `NOTHING`).

## 34.3 Esempio: aggiungere sintassi

1. se serve un nuovo token, estendi `tokenize` (es. un operatore in `twoOps`);
2. aggiungi il livello di precedenza nel `Parser` producendo un nuovo `kind`;
3. gestisci il `kind` in `evalExpr*`;
4. aggiorna `SPEC-CORE.md` (grammatica §4 e semantica §5) e aggiungi un caso Core.

Regola: l'interprete JS deve restare **fedele a SPEC-CORE**; non aggiungere estensioni che il seed e S1 non dovrebbero avere.

## Esercizi

1. **Architettura.** Leggi il paragrafo 34.1 e indica dove si collocano lexer, parser e valutatore.
2. **Un builtin in più.** Segui il paragrafo 34.2 e aggiungi un builtin (per esempio `doppio`): provalo e poi rimuovilo.
3. **Nuova sintassi.** Segui il paragrafo 34.3 per aggiungere una piccola forma sintattica e verifica che il parser la accetti.
4. **La conformance dopo la modifica.** Esegui la suite dopo ogni modifica: quali casi proteggono il tuo cambiamento?
5. **Trovare il codice.** Dove sono implementati `push!`, `length` e `println`? Individua le funzioni.
6. **`ARGS`.** Come arriva `ARGS` al programma? Traccia il percorso nel codice.
7. **Un errore da vicino.** Individua dove viene prodotto `error: index out of range` e cosa contiene il messaggio.
8. **Debug dell'interprete.** Inserisci una stampa di debug nel valutatore e osserva l'ordine di valutazione di un'espressione; poi rimuovila.

9. **Perché è fuori dal bootstrap.** Spiega perché l'interprete JS è uno strumento di sviluppo e non parte della catena S0 → S4.
10. **Refactor sicuro.** Scegli una piccola funzione dell'interprete, rinominala ovunque e verifica la suite: perché i test sono il tuo contratto?

---

# 35. Il seed S0 Windows e il suo generatore

## 35.1 Cos'è

`seed/windows-x86_64/seed.exe` è un PE32+ x86-64 di **25.088 byte** che importa solo `KERNEL32.dll` (`GetStdHandle`, `GetCommandLineA`, `CreateFileA`, `ReadFile`, `WriteFile`, `CloseHandle`, `ExitProcess`, `VirtualAlloc`). Non usa libc né runtime. La **fonte** è il dump `seed.hex` (SHA-256 verificato dalla CI: `3A75A21E2F47D8DC4C88D4BC2580A85D5349BCDF7550E5FFA16E2C0AC7CA8C11`).

Il seed **non costruisce un AST**: interpreta direttamente il testo sorgente, con un cursore (`rsi`) che scorre i byte; i corpi di funzioni e lambda sono puntatori nel sorgente che vengono rieseguiti a ogni chiamata.

## 35.2 Convenzioni interne

| registro | ruolo |
|---|---|
| `r14` | base della memoria `VirtualAlloc` (sorgente a `+0`, ambiente a `+0x10000`, funzioni a `+0x20000`, heap a `+0x30000`) |
| `r15` | ambiente variabili (entry da 24 byte: nome ptr, len, valore) |

| registro | ruolo |
|---|---|
| `rbx` | contatore variabili |
| `r13` | tabella funzioni (entry da 128 byte) |
| `r12` | handle di stdout |
| `rsi` | cursore nel sorgente |

Valori taggati a 2 bit: `Int = n << 2`, `false = 1`, `true = 5`, chiusura `= ptr | 2`, oggetti heap (stringhe `[tipo][len][byte…]`) con tag 3. Il layout completo (header PE, IAT, mappa delle etichette `parse_program`, `st_if`, `parse_primary`, `env_find`, …) è in `seed/windows-x86_64/README.md`.

## 35.3 Il generatore `seed/dev/gen_seed.js`

Scrivere migliaia di byte a mano è impraticabile: il generatore è un **assembler-lite** in JavaScript che emette i byte e risolve le etichette. Sezioni del file:

| sezione | cosa |
|---|---|
| `emitter` | `B(...)` emette byte, `B32(v)` un dword, `L(nome)` definisce un'etichetta, `reloc` / `je` / `jne` / `jmp` / `callL` emettono salti con patch relative, `callIat(i)` chiama l'import `i` |
| helper di istruzione | funzioni con il nome dell'istruzione (`incRsi`, `cmpByteRsiImm('(')`, `testRax`, `leaRdxR14(d)`, …) che emettono i byte x86-64 corrispondenti |

| sezione | cosa |
|---|---|
| `PE/data layout` | costanti `IAT = 0x4000`, `SEC_RAW`, `HEADERS` |
| `main` | prologo, lettura della riga di comando, apertura/lettura del file |
| `parser` | il cuore dell'interprete: statement, espressioni, funzioni, lambda |
| `ambiente` | `env_find`, `env_store`, `env_assign` |
| `exit labels` | `fail`, `read_fail`, `open_fail`, `usage` |
| `PE builder` | scrive header DOS/PE/COFF, sezione `.text`, tabelle import, e il dump esadecimale |

Esempio di stile (dal `main`): saltare il nome del programma nella riga di comando.

```
cmpByteRsiImm(0x22); jne('skip_plain');   // il path
inizia con " ?
L('q_loop');
incRsi();
cmpByteRsiImm(0); je('no_arg');           // fine
stringa: nessun argomento → exit 2
cmpByteRsiImm(0x22); jne('q_loop');
```

## 35.4 Flusso per modificare il seed

1. modifica `seed/dev/gen_seed.js` (aggiungi un'etichetta, un ramo nel dispatcher, …);
2. rigenera il dump: `node seed/dev/gen_seed.js seed/windows-x86_64/seed.hex` (oppure `bin/triad-cli seed generate windows`, che stampa anche lo SHA-256);
3. decodifica su Windows: `certutil -decodehex -f seed\windows-x86_64\seed.hex seed\windows-x86_64\seed.exe`;
4. esegui i casi Core sul seed e il probe `test_get.td`;
5. aggiorna **insieme**: hash in `.github/workflows/ci.yml` (job `seed-windows`, step `Hash check`), `seed/windows-x86_64/README.md`, `seed/dev/README.md`, `STATUS.md`; se un nuovo caso Core passa sul seed, aggiungilo all'array `$cases` del job.

Trappole note (da `STATUS.md`): il codice cresce e può sovrapporsi ai dati (i nomi import ora partono da RVA `0x5000`); registri usati prima di essere inizializzati (bug `pbs_int`); ordine degli argomenti sullo stack nelle chiamate builtin (`get`). Defender può segnalare il PE come falso positivo (la CI aggiunge un'esclusione).

## Esercizi

1. **Che cos'è il seed.** Spiega con le tue parole che cos'è S0 e perché è scritto in codice macchina.
2. **Dump esadecimale.** Perché il seed è versionato come `seed.hex` e non come binario? Quale comando lo materializza?

3. **Convenzioni interne.** Leggi il paragrafo 35.2 e annota tre convenzioni del generatore.
4. **`gen_seed.js`.** Rigenera il seed con il generatore e verifica che l'hash corrisponda a quello documentato.
5. **Il flusso di modifica.** Descrivi i passi del paragrafo 35.4 per modificare il seed senza romperlo.
6. **Verificare sul codice macchina.** Esegui tre casi Core sul seed Windows e confronta con l'interprete JS.
7. **Limiti misurati.** Scegli tre limiti del seed (lambda, `string`, liste…) e scrivi un programma per ciascuno che li incontri.
8. **`continue` mancante.** Riscrivi un programma con `continue` nella forma equivalente con `break` e verifica che giri sul seed.
9. **`-=` e `*=`.** Perché conviene usare solo `+=` nel codice destinato al seed? Riscrivi un esempio.
10. **Hash e CI.** Perché la CI controlla l'hash del seed? Che cosa garantirebbe un hash diverso?

---

## 36. Il seed Linux (E0)

`seed/linux-x86_64/seed` è un ELF64 statico di **173 byte** generato da `seed/dev/gen_seed_linux.js`: entry `0x400078`, un solo `PT_LOAD` R|X, syscall dirette `write`/`exit`. Stampa `Triad seed Linux OK` ed esce con 0. È il primo passo della milestone M7; E1/E2/E3 (lettura file, interprete) sono da fare, riusando la logica del seed Windows con un layer `os_*` diverso.

```
node seed/dev/gen_seed_linux.js seed/linux-
x86_64/seed.hex   # verifica magic, entry, round-trip
xxd -r -p seed/linux-x86_64/seed.hex seed/linux-
```

```
x86_64/seed && chmod +x seed/linux-x86_64/seed
./seed/linux-x86_64/seed
# Triad seed Linux OK
```

## Esercizi

1. **Che fa oggi.** Esegui il seed Linux e descrivi esattamente il suo comportamento.
2. **Materializzare l'ELF.** Genera il binario Linux a partire dal dump e verificane l'hash.
3. **Confronto fra seed.** Metti a confronto seed Windows e seed Linux: quali funzionalità ha ciascuno?
4. **Perché E0 non basta.** Che cosa manca al seed Linux per essere un interprete? Elabora una lista di requisiti.
5. **Il prossimo passo.** Quale sarebbe il primo costrutto da implementare per un E1 utile? Giustifica.
6. **Su Linux oggi.** Perché su Linux si usa l'interprete JS? Scrivi i comandi di esecuzione.
7. **Hash e verifica.** Verifica l'hash del seed Linux come fa la CI e spiega cosa protegge.
8. **Programmi portabili.** Scrivi un programma che un futuro seed Linux dovrà eseguire: quale sottoinsieme usi?
9. **Stato del bootstrap.** Dove si documenta lo stato del seed (`STATUS.md`, `TASK.md`)? Che cosa dicono oggi?
10. **Pianifica una milestone.** Scrivi cosa dovrebbe contenere una milestone «seed Linux esegue il primo caso Core».

---

## 37. Self-hosting: S1 in `selfhost/`

S1 è l'interprete di tutto Triad scritto in Triad-Core ed eseguito dal seed. Oggi è allo stadio di prototipi separati, ognuno con un piccolo test in fondo al file:

| file | cosa prova |
|---|---|
| `main.td` | ambiente inline come lista piatta `[chiave, valore]`; smoke test in CI (atteso `Value: 10`) |
| `value.td` | valori taggati a 2 bit simulati con interi (`make_int`, `make_bool`, `as_int`) |
| `tokenizer.td` | `next_token(chars, pos)` su un sorgente codificato come lista `[lunghezza, byte…]`; token `[tipo, valore, pos]` |
| `parser.td` | riconoscimento delle keyword `if` / `end` dai codici ASCII |
| `eval.td`, `program.td` | valutazione di token numerici, identificatori, assegnamenti e `println` |
| `env.td`, `heap.td`, `ast.td`, `interpreter.td` | stub dell'ambiente, dell'allocatore bump e dell'AST |

### 37.1 Come si scrive codice S1 oggi

Il codice S1 deve girare sul **seed Windows**, quindi segue le restrizioni della colonna destra del capitolo 15:

- usa `get(xs, i)` con indici **da 0** invece di `xs[i]` (non standard: andrà sostituito con l'indicizzazione Core quando il seed la supporterà);

- liste solo di interi: le stringhe sorgente si rappresentano come liste di codici ASCII, con la lunghezza nella posizione 0;
- `push!` restituisce la lista: si scrive `env = push!(env, v)`;
- costanti per i tipi di token (`0` EOF, `1` NUMBER, `2` IDENTIFIER, …) documentate in un commento in testa al file;
- funzioni con al massimo 4 parametri, lambda a un parametro, niente cattura multi-binding.

```
# stile S1: ambiente piatto e lookup con get (0-based)
env = []
env = push!(env, 1)      # chiave
env = push!(env, 10)     # valore
if get(env, 0) == 1
  println("Value: " * string(get(env, 1)))
end
```

Su Linux questi file **non girano** con l'interprete JS (che non ha `get`): vanno provati su Windows con `seed\windows-x86_64\seed.exe selfhost\main.td`.

## 37.2 Dove va S1

Le task S1.3–S1.15 sono in `TASK.md`: valori taggati, tokenizer e parser inline, eval di espressioni, `let`, programmi completi, funzioni e chiamate, chiusure, e infine far girare la conformance sopra seed+S1. Gli esercizi 6 e 7 del capitolo 17 sono una versione in Core standard degli stessi passi.

## Esercizi

1. **Che cos'è S1.** Definisci S1 con le tue parole: linguaggio, forma e ruolo nel bootstrap.

2. **Lo stile S1.** Perché `selfhost/` usa `get(xs, i)` con indici da 0? Dove si trova questo helper?
3. **Smoke test.** Esegui `selfhost/main.td` con il runtime indicato dal capitolo e leggi l'output.
4. **Primitive di bootstrap.** Quali primitive del capitolo 13 servono a S1 e perché?
5. **Leggere un pezzo.** Scegli una funzione di `selfhost/` e spiegala riga per riga.
6. **Cosa manca.** Quanto dista S1 dalla conformance? Elenca i passi rimanenti.
7. **Verso S2.** Descrivi cosa farà S2 e che cosa deve avere S1 per produrlo.
8. **Portabilità di S1.** Su quale runtime gira oggi S1 e perché non gira sull'interprete JS?
9. **Verifica in CI.** Quale comando della CI mette alla prova S1? Eseguilo in locale.
10. **Perché self-hosting.** Scrivi un paragrafo sul perché il self-hosting è la prova definitiva di un linguaggio autonomo.

---

# 38. Conformance: leggere, scrivere e verificare casi

## 38.1 Formato

```
conformance/core/cases/<nome>/      (oppure
conformance/cases/<nome>/ per Full)
  program.td        programma Triad eseguito come
script
  args.txt          opzionale: un argomento per riga
```

```
(ARGS)
  expected.out      stdout atteso, byte per byte, fine
riga LF
  expected.golden   copia di expected.out (fallback del
runner)
  expected.code     exit code atteso
```

Un caso passa se stdout e codice di uscita coincidono; stderr non è confrontato. Niente stdin né input interattivo.

## 38.2 Il runner `conformance/run.mjs`

```
node conformance/run.mjs                              #
Core (default conformance/core/cases)
node conformance/run.mjs --cases conformance/cases    #
Full (oggi passa solo errore)
node conformance/run.mjs --skip liste,valori          #
salta casi (default: liste)
node conformance/run.mjs --update                     #
RIGENERA i golden dal runtime corrente
```

Runtime scelto, in ordine: variabile `TRIAD_SEED`; su Linux/macOS l'interprete JS; su Windows `bin/triad-native.exe`, poi `seed/windows-x86_64/seed.exe`. Esempio per provare il seed con un percorso esplicito: `TRIAD_SEED=/percorso/seed node conformance/run.mjs`.

Su Linux/macOS il runner usa l'interprete JS e verifica tutti i 37 casi. Su Windows usa il seed e salta per impostazione predefinita il caso `liste`, perché il seed non implementa l'indicizzazione standard `xs[i]`: il runner stampa quindi 36 casi. Per provare il caso escluso con l'interprete JS:

```
node seed/dev/triad_core.cjs
conformance/core/cases/liste/program.td
```

Messaggi:

- `PASS nome` / `FAIL nome (exit X, atteso Y, stdout N byte, attesi M)`;
- `golden mancante`: il checkout non ha `expected.out` (di solito basta `git pull`).

Un golden è un'aspettativa versionata, non la fonte normativa della semantica. Il file `program.td` definisce l'input, `args.txt` gli argomenti, `expected.out` l'output byte per byte e `expected.code` il codice d'uscita. Il runner confronta questi risultati con il runtime scelto; così uno stesso caso può essere eseguito da più interpreti e rendere visibili le divergenze. Se il comportamento del runtime è in dubbio, confronta prima il caso con SPEC-CORE e poi correggi il golden solo quando la specifica conferma il nuovo valore atteso. L'opzione `--update` copia il comportamento corrente nei risultati attesi: usarla senza revisione trasformerebbe un bug in una pretesa di correttezza.

## 38.3 Aggiungere un caso Core

1. scegli un nome descrittivo in italiano (`liste_annidate`, `stringhe_vuote`, …);
2. scrivi `program.td` usando **solo** SPEC-CORE §2–§7 (niente primitivi di bootstrap: devono girare anche su Full);
3. scrivi a mano `expected.out` (con LF finale) ed `expected.code` (`0`), ragionando sulla specifica — **non** generarli alla cieca con `--update`, altrimenti fissi i bug del runtime;
4. copia `expected.out` in `expected.golden`;

5. `node conformance/run.mjs` deve stampare `PASS <nome>`;
6. se il caso passa anche sul seed Windows, aggiungilo a `$cases` nel job `seed-windows` di `.github/workflows/ci.yml`.

```
mkdir conformance/core/cases/somma_lista
cat > conformance/core/cases/somma_lista/program.td
<<'TD'
xs = [3, 4, 5]
tot = 0
i = 1
while i <= length(xs)
    tot += xs[i]
    i += 1
end
println(tot)
TD
printf '12\n' >
conformance/core/cases/somma_lista/expected.out
cp conformance/core/cases/somma_lista/expected.out
conformance/core/cases/somma_lista/expected.golden
printf '0' >
conformance/core/cases/somma_lista/expected.code
node conformance/run.mjs | grep somma_lista          #
PASS  somma_lista
```

I golden Full (`conformance/cases/`) sono **congelati**: non vanno rigenerati finché non esiste un'implementazione Full.

## Esercizi

1. **Il formato.** Quali file compongono un caso di conformance e che cosa dichiara ciascuno?
2. **Il runner.** Esegui `node conformance/run.mjs` (o il comando della CLI) e leggi il riepilogo.
3. **Un caso alla volta.** Esegui un singolo caso Core e confronta output e codice con i golden.
4. **Aggiungi un caso.** Segui il paragrafo 38.3 e crea un caso Core tuo (programma, output atteso, codice atteso).
5. **Golden mancante.** Cosa succede quando manca un `expected.out`? Provocalo e osserva.
6. **Casi che falliscono.** Aggiungi un caso il cui programma termini con codice 1: come si esprime il codice atteso?
7. **Core e Full.** Quali sono le due suite e perché solo pochi casi Full sono verdi?
8. **Portabilità.** Esegui la suite con il seed Windows: quali casi escono e perché?
9. **Un golden è un contratto.** Spiega perché i golden vanno versionati e non rigenerati alla leggera.
10. **La tua routine.** Scrivi la sequenza di comandi che eseguirai prima di ogni cambiamento al linguaggio.

---

# 39. La CLI di sviluppo `triad-cli`

## 39.1 Uso

| comando | cosa fa |
|---|---|
| `triad-cli status` | versione Node, piattaforma, seed trovato, root del progetto |
| `triad-cli test conformance [-c DIR] [-u]` | esegue `conformance/run.mjs` (default Core, `--skip liste`) |
| `triad-cli test run` | alias della conformance Core (i vecchi test Julia non esistono più) |
| `triad-cli ai verify [-t DIR] [-v]` | esegue le `solution.triad` di `ai/tasks/` sul seed nativo e le confronta con i golden |
| `triad-cli seed generate windows\|linux [-o FILE]` | esegue il generatore del seed e stampa lo SHA-256 del dump |
| `triad-cli build standalone` | non disponibile: stampa un messaggio |

Avvio: `node cli/triad-cli.mjs …` (dopo `cd cli && npm install`), i launcher `bin/triad-cli` / `bin\triad-cli.cmd` (installano le dipendenze al primo avvio e si fermano se `npm install` fallisce), oppure l'eseguibile standalone della release.

## 39.2 Struttura del codice

```
cli/
  triad-cli.mjs            definizione dei comandi con
```

```
commander
  lib/utils.mjs            findProjectRoot, findSeed,
runSeed, runNode, projectPath, …
  lib/commands/test.mjs    handleTestRun,
handleTestConformance
  lib/commands/ai.mjs      handleAIVerify
  lib/commands/seed.mjs    handleSeedGenerate
(windows/linux)
  lib/commands/build.mjs   handleBuildStandalone
  build-exe.mjs            build dell'eseguibile Node
SEA (esbuild + postject)
```

- `findProjectRoot()` cerca la root (una cartella con `conformance/run.mjs` e `seed/`) partendo da `TRIAD_HOME`, dalla cartella del modulo, da quella dell'eseguibile e dalla cartella corrente.
- Ogni handler restituisce `true` / `false`; `triad-cli.mjs` lo traduce in exit code 0/1.

## 39.3 Aggiungere un comando

1. crea `cli/lib/commands/<nome>.mjs` con una funzione `handle<Nome>(options)` che restituisce un booleano e usa `projectPath`, `runNode`, `runSeed` da `utils.mjs`;
2. registralo in `triad-cli.mjs`:

```
import { handleLint } from './lib/commands/lint.mjs';

program
  .command('lint')
  .description('Controlla i file .triad')
  .option('-v, --verbose', 'Verbose output')
```

```
  .action(async (options) => {
    const success = await handleLint(options);
    process.exit(success ? 0 : 1);
  });
```

3. aggiungi la riga in `status` e documentalo in `cli/README.md`.

## 39.4 Eseguibile standalone

```
cd cli && npm install
npm run build:exe                          # dist/triad-
cli per la piattaforma corrente
node build-exe.mjs path/to/win/node.exe    # dist/triad-
cli.exe per Windows, anche da Linux
```

Il binario Node passato determina la piattaforma; deve avere la stessa versione del Node che esegue la build.

## Esercizi

1. **I comandi.** Esegui `node cli/triad-cli.mjs --help` (o il paragrafo 39.1) e l'elenco dei comandi disponibili.
2. **Test di conformance.** Lancia la suite dalla CLI e interpreta il riepilogo.
3. **Verifica AI.** Usa il comando `ai verify` su una soluzione corretta e su una sbagliata.
4. **Codici di uscita.** Che codice restituisce la CLI quando un test fallisce? Verifica.
5. **CLI o launcher?** Spiega la differenza fra `triad-cli` (strumento di sviluppo) e `bin/triad` (launcher del runtime).

6. **Un comando tuo.** Scrivi uno script di shell che esegua un caso, il runner e la stampa del risultato in una riga.
7. **Errori di uso.** Invoca la CLI con argomenti sbagliati: che messaggi ottieni?
8. **Automatizzare.** Come faresti a eseguire solo i casi il cui nome contiene `list`?
9. **Dentro la CLI.** Apri il sorgente della CLI e indica dove vengono lanciati i runner.
10. **Documentazione.** Scrivi la riga di aiuto che vorresti trovare per un comando che manca.

---

## 40. Benchmark AI (`ai/`)

Misura quanto bene un LLM scrive Triad-Core.

- `ai/SYSTEM_PROMPT.md`: prompt di sistema con il sottoinsieme ammesso (prudente: vieta liste, `string()`, `elseif`, `return` perché il seed non li supportava tutti);
- `ai/tasks/NN_nome/`: `task.md` (richiesta), `expected.out`, `solution.triad` (soluzione di riferimento);
- verifica: `node cli/triad-cli.mjs ai verify` (o `--tasks ai/tasks/01_somma`): `PASS` / `FAIL`, `SKIP` se manca la soluzione;
- `ai/jev_audit.mjs`, `ai/jev_prompt.md`: audit separato (vedi `PROMPT_JEV.md`).

Aggiungere un task: crea `ai/tasks/06_nome/` con `task.md` (output atteso descritto con precisione), `expected.out` e una `solution.triad` di riferimento che passa. Esegui il codice generato da un modello solo in un ambiente isolato.

### Esercizi

1. **Leggere un task.** Apri `ai/tasks/01_sum`: quali parti compongono un task e dove sta l'output atteso?
2. **Risolvere un task.** Scrivi la soluzione di un task e verificala con il comando della CLI.
3. **PASS e FAIL.** Perché il verdetto è il confronto con l'output atteso? Costruisci una soluzione che stampi il risultato giusto ma in un formato sbagliato.
4. **pass@1 e pass@k.** Con tre tentativi di cui il secondo giusto: pass@1 e pass@3 che valore hanno?
5. **Lo `SYSTEM_PROMPT`.** Che cosa richiede `ai/SYSTEM_PROMPT.md` al modello? Perché «solo un blocco di codice»?
6. **Tentativi.** Simula cinque tentativi di uno stesso task e calcola pass@1 e pass@5.
7. **Conformance e benchmark.** Spiega la differenza fra un caso di conformance e un task del benchmark.
8. **Un task tuo.** Scrivi un task nuovo (problema + output atteso) con la struttura di `ai/tasks/`.
9. **Perché misurare.** Che cosa misura il benchmark e che cosa non misura? Scrivi due righe.
10. **Senza guardare.** Risolvi un task senza leggere la soluzione, poi confronta i due programmi.

---

## 41. Editor, launcher e sito

- **VS Code** (`editors/vscode/`): `package.json` dichiara il linguaggio `triad` (estensioni `.triad`, `.td`); `syntaxes/triad.tmLanguage.json` è la grammatica

TextMate ( `source.triad` ) con le regole `keywords` , stringhe con interpolazione, numeri, `:-` , shebang; `language-configuration.json` gestisce commenti e parentesi. Pacchetto: `cd editors/vscode && npx @vscode/vsce package` .

- **Vim/Neovim** ( `editors/vim/` ): `ftdetect/triad.vim` e `syntax/triad.vim` .
- **Linguist** ( `editors/linguist/` ): voce `languages.yml` pronta per una PR a GitHub Linguist.
- Se aggiungi una parola chiave allo standard, aggiornala in **tutte** le grammatiche.
- **Launcher** ( `bin/` ): `triad` (POSIX, risolve i symlink e calcola `TRIAD_HOME` ) e `triad.cmd` (Windows, file CRLF); vedi §2.4.
- **Sito** ( `docs/` ): HTML statico pubblicato su GitHub Pages; `docs/assets/js/app.js` carica i Markdown del repository con `marked` (es. `docs/pages/tutorial.html` carica questo file). Le ancore dell'indice usano `<a id="…">` espliciti perché funzionino anche lì.

## Esercizi

1. **L'editor.** Apri un file `.triad` in VS Code (o Vim) e verifica l'evidenziazione della sintassi.
2. **Le grammatiche.** Individua i file della grammatica per VS Code e Vim: quale regola colora le parole chiave?
3. **GitHub Linguist.** Che cosa serve perché GitHub riconosca `.triad` e `.td` ? Trova la voce.
4. **Il launcher.** Esegui un programma con `bin/triad` e con `bin/triad.cmd` : quale runtime scelgono?

5. **Il sito generato.** Esplora `tools/site-gen/` : perché i generatori sono scritti in Triad?
6. **Due runtime, un output.** Che cosa controlla `build.mjs` quando genera il sito? Perché richiede output identici?
7. **Il lint.** Che proibisce `lint.mjs` e perché? Prova a violare una regola e osserva l'errore.
8. **Le pagine.** Quale file del sito carica questo Tutorial? Come funzionano le ancore?
9. **Aggiungere una pagina.** Descrivi i passi per aggiungere una pagina al sito generato.
10. **La demo.** Spiega come funziona la demo di battitura del sito e da quale interprete gira.

---

## 42. Integrazione continua

`.github/workflows/ci.yml` , a ogni push e pull request:

| job | cosa verifica |
|---|---|
| `conformance-core` (Ubuntu, Node 20) | `node conformance/run.mjs --cases conformance/core/cases --skip liste` con l'interprete JS |
| `seed-windows` | decodifica `seed.hex` , esegue 33 casi Core sul PE confrontando stdout, probe `test_get.td` (atteso `20` ), smoke test `selfhost/main.td` (atteso `Value: 10` ), hash di `seed.hex` |
| `seed-linux` | decodifica l'ELF E0, lo esegue e ne verifica l'hash |

`pages.yml` e `static.yml` pubblicano `docs/` su GitHub Pages a ogni push su `main`.

Il profilo Full **non è in CI** perché non è implementato: non presentarlo come verde.

## Esercizi

1. **I job.** Leggi `.github/workflows/ci.yml` e l'elenco dei job: cosa verifica ciascuno?
2. **`conformance-core`.** Perché su Ubuntu il job salta alcuni casi? Quali?
3. **`seed-windows`.** Che cosa controlla il job Windows e perché confronta l'hash di `seed.hex`?
4. **`seed-linux`.** Che cosa fa il job Linux oggi? Perché non può ancora eseguire la suite?
5. **Site-gen.** Perché la generazione del sito è in CI e cosa richiede ai due runtime?
6. **Smoke test.** Che cosa verifica lo smoke test di `selfhost/main.td` e perché è in CI?
7. **Locale e CI.** Esegui in locale gli stessi comandi della CI: quali risultati ottieni?
8. **Un fallimento.** Immagina un cambiamento che rompa un caso: quale job fallisce per primo e come diagnostichi?
9. **Hash e riproducibilità.** Spiega il rapporto fra hash del seed, golden e riproducibilità delle build.
10. **Prima di una PR.** Scrivi la checklist di ciò che deve essere verde prima di aprire una modifica.

---

## 43. Flusso di contributo

Checklist per tipo di modifica:

| modifica | da fare |
|---|---|
| comportamento del linguaggio | aggiorna `SPEC.md` / `SPEC-CORE.md`, `CHANGELOG.md` e, se normativa, `VERSION`; aggiungi o aggiorna casi di conformance |
| interprete JS | resta fedele a SPEC-CORE; `node conformance/run.mjs` verde |
| seed Windows | rigenera `seed.hex`, aggiorna hash in CI e nei README, prova su Windows |
| S1 (`selfhost/`) | gira su `seed.exe`; se cambia l'output di `main.td`, aggiorna lo smoke test in CI |
| CLI | handler che restituisce un booleano; `cli/README.md`; ricostruisci l'eseguibile se serve |
| editor | grammatiche VS Code e Vim allineate alle parole chiave di SPEC §1 |
| documentazione | `README.md`, `STATUS.md` e i testi del repository descrivono ciò che gira davvero |

Regole generali: non dichiarare funzionante ciò che non è verificato; i golden si scrivono ragionando sulla specifica; i nuovi strumenti di sviluppo non devono entrare nella catena S0 → S4.

### Esercizi

1. **Cosa aggiornare.** Quali documenti vanno aggiornati quando il lavoro cambia lo stato del progetto?

2. **Conformance prima di tutto.** Scrivi la frase che riassume la regola operativa 3 e applicala a un caso concreto.
3. **Modifiche al linguaggio.** Quali passi servono per proporre una modifica allo standard? Elencali in ordine.
4. **Niente linguaggi host.** Perché il percorso di bootstrap non può dipendere da runtime esterni? Dove si applica la regola?
5. **Output deterministico.** Perché i golden includono newline finali e codici di uscita? Costruisci un caso che lo dimostri.
6. **Test con il cambiamento.** Immagina una modifica al parser: quali test aggiungeresti?
7. **Documentare.** Scrivi la voce di CHANGELOG che descriverebbe l'aggiunta degli esercizi a un capitolo del libro.
8. **Review.** Elabora la checklist per revisionare una patch di un altro agente/autore.
9. **Il benchmark.** Quale regola vale quando si generano soluzioni per `ai/`?
10. **Contributo end-to-end.** Simula un contributo completo: modifica, test, documentazione, descrizione del cambiamento.

---

# Parte V — Riferimento

## 44. Grammatica Triad-Core (EBNF)

Da `SPEC-CORE.md` §4:

```
program  = { sep } { stmt sep { sep } } ;
stmt     = funcdef | "return" [ expr ] | "break" |
"continue" | expr ;
```

```
funcdef  = "function" NAME "(" [ NAME { "," NAME } ]
")" block "end"
         | NAME "(" [ NAME { "," NAME } ] ")" "=" expr
;
expr     = assegnamento ;
assegnamento = NAME ( "=" | "+=" | "-=" | "*=" ) expr |
ternario ;
ternario = or_expr [ "?" expr ":" expr ] ;
or_expr  = and_expr { "||" and_expr } ;
and_expr = cmp_expr { "&&" cmp_expr } ;
cmp_expr = sum_expr { ( "==" | "!=" | "<" | "<=" | ">"
| ">=" ) sum_expr } ;
sum_expr = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr = pow_expr { ( "*" | "%" ) pow_expr } ;
pow_expr = unary { "^" unary } ;                    (*
destra *)
unary    = ( "-" | "!" ) unary | postfix ;
postfix  = primary { "(" args ")" | "[" expr "]" } ;
primary  = INT | STRING | SYMBOL | "true" | "false" |
"nothing" | NAME
         | "(" expr ")"
         | "[" [ expr { "," expr } ] "]"
         | "(" NAME { "," NAME } ")" "->" expr
         | "if" expr block { "elseif" expr block } [
"else" block ] "end"
         | "while" expr block "end"
         | "let" [ NAME "=" expr { "," NAME "=" expr }
] block "end"
         | "begin" block "end"
         | "function" "(" [ NAME { "," NAME } ] ")"
block "end" ;
```

```
block    = { stmt sep { sep } } ;                    (*
fino a end/else/elseif *)
```

La grammatica Full (Pratt, 12 livelli) è in `SPEC.md` §2.

## Esercizi

1. **Si legge o no?** Con la grammatica EBNF decidi quali righe sono programmi Core validi: `x = 1`, `x =`, `f(1, 2)`, `f(1,,2)`.
2. **Derivazione.** Scrivi la derivazione di `x = 1 + 2 * 3` a partire dalla regola delle espressioni.
3. **Terminali e non terminali.** Nel frammento EBNF del capitolo, elenca tre terminali e tre non terminali e spiega la differenza.
4. **Separatori.** Perché newline e `;` sono separatori? Costruisci un caso in cui un a capo in più cambia il programma.
5. **Blocchi ed `end`.** Quali regole della grammatica aprono un blocco? Scrivi un programma con tre livelli di annidamento.
6. **Stringhe e escape.** Dalla grammatica, elenca gli escape ammessi e scrivi una stringa per ciascuno.
7. **Operatori.** Come descrive la grammatica la precedenza? Confronta con la tabella del capitolo 7.
8. **Tokenizzare.** Scrivi la sequenza dei token di `xs[i] += 1` e commentala.
9. **EBNF a mano.** Scrivi in EBNF una piccola sotto-grammatica (per esempio i commenti) e verificala su tre esempi.
10. **Grammatica e parser.** Spiega perché una grammatica EBNF da sola non basta a descrivere la precedenza e come il parser Pratt la risolve.

## 45. Schede rapide Core e Full

**Triad-Core** (eseguibile oggi):

```
assegnamento:  x = e    x += e    x -= e    x *= e
xs[i] = e
funzione:      f(a, b) = e
               function f(a, b) ... end        (anche
!io, ignorato)
anonima:       x -> e    (a, b) -> e    function (a)
... end
condizione:    if c ... elseif c ... else ... end    c
? a : b
ciclo:         while c ... end    break    continue
blocchi:       let a = 1, b = 2 ... end    begin ...
end
uscita:        return e
liste:         [1, 2]    xs[i] (da 1)    length(xs)
push!(xs, v)
stringhe:      "a" * "b"    length(s)    string(x)
conversioni:   Int("42")    Symbol("s")    typeof(x)
I/O:           print(...)    println(...)    ARGS
bootstrap:     readfile(p)  strlen(s)  strbyte(s, i)
strsub(s, a, b)  exit(n)
```

**Triad-Full** (specifica):

```
struttura:   struct Nome; x::T; y::T; end
funzione:    function f(a::T)::R !eff ... end
fatto:       fact p(:a, :b)
regola:      rule p(?x, ?y) :- q(?x), r(?y)
query:       solve p(?x) limit N depth D steps S trace
```

```
destruttura: (a, b) = [1, 2]        for (k, v) in
[[1,2],[3,4]] ... end
ciclo for:   for v in it ... end    1:n    xs[2:end]
[e for x in xs if c]
try/catch:   try b catch e h end / try b end
contratto:   requires c / ensures c
budget:      with budget(steps = N) ... end
traccia:     with trace ... end / solve p trace
effetto:     function f() !io ... end
macro:       macro m(a) ... end / @m(e)
quote:       :(espr) / quote ... end / $espr
eval:        eval(codice)
```

## Esercizi

1. **Scheda valori.** Con la sola scheda 45 scrivi un programma con tutti i tipi Core.
2. **Scheda controllo.** Scrivi un `while` con `break` e un `if` / `elseif` / `else` senza guardare i capitoli.
3. **Scheda funzioni.** Scrivi una funzione breve, una blocco e una lambda.
4. **Scheda liste.** Costruisci, indicizza, aggiungi e attraversa una lista.
5. **Scheda stringhe.** Concatena, converti con `string()` e `Int()`, misura con `length`.
6. **Core o Full.** Dalla scheda, decidi per `for`, `solve`, `map`, `try`, `/` se sono Core o Full.
7. **Sfida a tempo.** In cinque minuti scrivi un programma che usi almeno cinque voci della scheda.
8. **Dalla scheda al codice.** Scegli cinque righe della scheda e trasformale in un programma funzionante.

9. **Errori rapidi.** Produci tre errori della scheda errori e riconoscili senza guardare il capitolo 46.
10. **La tua scheda.** Scrivi la scheda rapida «dei tuoi errori più comuni» con la correzione.

---

## 46. Debug e problemi comuni

**Core (oggi):**

| sintomo | causa probabile | soluzione |
|---|---|---|
| `parse error: expected RPAREN` | `1.5`, `/`, `() -> …` o altra sintassi Full | resta nel sottoinsieme Core (§4) |
| `"x = $x"` stampato letteralmente | l'interpolazione è solo Full | `"x = " * string(x)` |
| `error: if cond` | condizione non `Bool` | usa un confronto (`x != 0`) |
| `error: < type error` su `a < b < c` | confronti concatenati | `a < b && b < c` |
| una funzione non aggiorna un globale | le funzioni nominate assegnano in locale | restituisci il valore o usa una lista (§16.4) |
| `error: undefined variable f` | funzione chiamata prima della definizione | definiscila più in alto |
| `FAIL … (golden mancante …)` | checkout senza `expected.out` | `git pull`; controlla `.gitignore` |
| il seed esce con 1 senza messaggi | funzionalità non supportata dal seed | prova con l'interprete JS; vedi §15 |

| sintomo | causa probabile | soluzione |
|---|---|---|
| il seed Windows viene bloccato | falso positivo dell'antivirus | escludi `seed\windows-x86_64` |

**Full (specifica):**

| sintomo | causa probabile | soluzione |
|---|---|---|
| `Err(:syntax)` "atteso fine istruzione" | manca `end` o un separatore | aggiungi `end` o `;` |
| `Err(:undef)` | nome non definito | definiscilo, o usa `:nome` per un simbolo |
| `Err(:method)` / `Err(:ambiguous)` | nessun metodo / più metodi ugualmente specifici | aggiungi o ridisegna le firme |
| `Err(:bounds)` in destrutturazione | arità diversa dal pattern | liste di lunghezza esatta |
| `Err(:effect)` | `!io` / `!state` chiamata da funzione pura | dichiara l'effetto nel chiamante |
| `Err(:budget)` in `solve` | troppi passi | aumenta `steps` o usa `limit` / `depth` |

Strumenti Full previsti: `triad --tree file`, `sexpr(codice)`, `parse` / `unparse`, `with trace … end`, `solve … trace`, `try … end` con `.kind` / `.msg` / `.data`.

### Esercizi

1. **Diagnosi guidata.** Dato un programma che non parte, applica la sequenza: nome file, cartella, comando, Node.js.
2. **`parse error`.** Scrivi tre programmi con errori di sintassi diversi e riconosci la causa dal messaggio.
3. **`error:` di esecuzione.** Produci `undefined variable`, `type error`, `index out of range` e spiega ciascuno.
4. **Ciclo infinito.** Scrivi un ciclo che gira all'infinito, interrompilo e correggilo.
5. **Due runtime.** Un programma gira sull'interprete JS e no sul seed: individua la causa nella tabella delle differenze.
6. **UTF-8 e nomi.** Verifica la codifica di un file e l'estensione `.triad` / `.td` in un caso problematico.
7. **Messaggi del runtime.** Traduci in italiano tre messaggi d'errore del runtime.
8. **Debug con `println`.** Trova il bug di un programma dato usando solo stampe di controllo.
9. **Problemi comuni.** Prendi i problemi elencati nel capitolo e scrivi per ciascuno un programma che li riproduca.
10. **La tua casella degli attrezzi.** Scrivi la checklist di debug che userai prima di chiedere aiuto.

---

## 47. Glossario

- **S0 / seed**: interprete Triad-Core in codice macchina ( `seed/` ).
- **S1**: interprete di tutto Triad scritto in Triad-Core ( `selfhost/` ).
- **S2 / S3 / S4**: compilatore nativo, self-hosting, runtime completo (vedi `BOOTSTRAP.md` ).

- **E0 / E1 / …**: stadi del port Linux del seed (milestone M7).
- **Triad-Core / Triad-Full**: i due profili di conformità.
- **Interprete JS**: `seed/dev/triad_core.cjs`, riferimento Core per sviluppo e CI.
- **Golden**: output atteso committato (`expected.out`, `expected.code`).
- **Primitivi di bootstrap**: `readfile`, `strlen`, `strbyte`, `strsub`, `exit` (solo Core).
- **Lista taggata**: record Core del tipo `[:Tag, campo1, …]`.
- **Node**: l'unico tipo di albero di Triad Full (`head`, `args`): codice e dati coincidono.
- **Solution**: `{nome = valore, …}` restituita da `solve`; `s.nome` o `s[:nome]`.
- **Err**: errore come dato (`kind`, `msg`, `data`).
- **Traced**: `(result, trace)` restituito da `with trace` / `solve … trace`.
- **Occurs-check**: l'unificazione rifiuta `?x = f(?x)`.
- **Multiple dispatch**: scelta del metodo più specifico tra gli applicabili.
- **Effetti dichiarati**: `!io`, `!state` nella firma; controllo dinamico.
- **Contratti**: `requires` (pre) / `ensures` (post, con `result`).
- **Budget**: `with budget(steps = N)`; l'esaurimento è `Err(:budget)`.
- **Destrutturazione**: `(a, b) = lista`, `for (k, v) in lista_di_liste`.

### Esercizi

1. **Definizioni.** Definisci con le tue parole: *unificazione*, *backtracking*, *dispatch*, *chiusura*, *albero uniforme*.
2. **Coppie che confondono.** Distingui con un esempio: budget/limit, effetto/contratto, simbolo/variabile logica, parametro/argomento.
3. **Parlare di logica.** Scrivi una frase che usi: goal, soluzione, knowledge base, occurs-check.
4. **Parlare di macro.** Scrivi una frase che usi: quote, unquote, espansione, igiene.
5. **Vero o falso.** Valuta cinque affermazioni tratte dal glossario e correggi quelle false.
6. **Dalle lingue madri.** Quali parole arrivano da Julia, Prolog e Lisp? Collega ciascuna alla sua origine.
7. **Tradurre.** Scegli cinque voci in inglese e scrivi la spiegazione che daresti a un principiante italiano.
8. **Il glossario di un programma.** Annota un tuo programma con almeno sei termini del glossario.
9. **Domande di ripasso.** Scrivi cinque domande sul glossario e rispondi senza guardare.
10. **Una voce nuova.** Aggiungi al glossario un termine che manca, nello stile delle voci esistenti.

---

## 48. Stato, limiti noti e fix storici

**Stato** (dettagli in `STATUS.md`):

- Core: 37/37 casi con l'interprete JS; 33 casi sul seed Windows in CI.

- Full: 12 casi golden congelati, passa solo `errore`.
- S1: prototipi in `selfhost/`, smoke test in CI; non ancora conformance.
- S2–S4: non iniziati. Seed Linux: E0.
- CLI `triad`: solo esecuzione di file (REPL e opzioni di SPEC §14 da fare).

**Limiti dello standard** (SPEC §15): argomenti con nome (salvo `budget(steps = …)`), struct mutabili, tipi astratti utente e parametrici, macro senza parentesi, quote annidate a più livelli, cut di Prolog non sono previsti; il controllo degli effetti è dinamico.

**Fix storici** applicati alla vecchia implementazione Julia; il comportamento "dopo" è quello richiesto da SPEC.md per ogni implementazione:

| # | fix | dopo |
|---|---|---|
| 1 | round-trip del range | `unparse(1:n)` produce `"1:n"` (senza spazi) |
| 2 | `unparse` di `:destructure` / `:dfor` | hanno rami propri e fanno round-trip |
| 3 | `xs[end] = v` | `end` è legato anche in scrittura: `xs[end] = 7` funziona |
| 4 | escape sconosciuti | solo `\n \t \" \\ \$`; altri → `Err(:syntax)` con riga e colonna |
| 5 | commenti `#=` annidati | la profondità è contata; `#=` non chiuso → errore `incomplete` |
| 6 | lambda con parametri tipati | `function (x::Int) x end` controlla i tipi: `f(2.0)` → `Err(:type)` |

### Esercizi

1. **Verificare un fix.** Scegli uno dei fix storici della tabella (per esempio `xs[end] = v`) e scrivi il programma che lo mette alla prova.
2. **Round-trip.** Verifica `unparse(1:n)` produce `"1:n"` e spiega perché contano i dettagli di forma.
3. **Escape sconosciuti.** Prova `"\q"` e verifica che l'errore riporti riga e colonna.
4. **Commenti annidati.** Prova `#=` annidati e un `#=` non chiuso: quali comportamenti?
5. **Lambda tipata.** Verifica che `function (x::Int) x end` controlli il tipo degli argomenti.
6. **Limiti noti.** Scrivi un programma per ciascuno dei limiti noti del capitolo e annota la divergenza fra i runtime.
7. **Documentare una divergenza.** Trova un comportamento diverso fra i due runtime e scrivi la voce che aggiungeresti allo stato del progetto.
8. **Prima e dopo.** Immagina il comportamento di un programma prima di un fix e dopo: scrivi entrambi gli output attesi.
9. **Lo stato del progetto.** Dove si tengono aggiornati limiti e fix (`STATUS.md`, `TASK.md`, `CHANGELOG.md`)? Che cosa va scritto in ciascuno?
10. **Regressioni.** Perché ogni fix storico merita un caso di conformance? Scrivi il caso per un fix a scelta.

---

## 49. FAQ

**Serve Julia?** No. L'implementazione Julia è stata rimossa; Triad gira sul seed in codice macchina (Windows) o sull'interprete JS di sviluppo.

**Perché i miei esempi con `struct` o `fact` non girano?** Sono Triad-Full: il runtime attuale esegue solo Triad-Core. Vedi la tabella dei profili all'inizio del Tutorial.

**`.td` o `.triad`?** `.triad` è canonica; `.td` è accettata come alias (ed è anche l'estensione di LLVM TableGen, quindi per il riconoscimento esterno conviene `.triad`).

**Come verifico che un programma sia Core "puro"?** Eseguilo con l'interprete JS e controlla di non usare primitivi di bootstrap se deve valere anche per Full; per i casi normativi aggiungilo alla conformance Core.

**Perché `selfhost/*.td` falliscono con l'interprete JS?** Usano `get(xs, i)`, helper non standard del seed Windows; vanno eseguiti con `seed.exe`.

**Dove propongo una modifica al linguaggio?** Nello standard (`SPEC.md` / `SPEC-CORE.md`) con voce in `CHANGELOG.md` e casi di conformance; poi nelle implementazioni.

**Cosa sono i «tentativi» di successo e fallimento (PASS/FAIL, `pass@k`) del benchmark `ai/`?** Un task è un piccolo esercizio (somma, fattoriale, stringhe…) con un output atteso; la soluzione si verifica eseguendola e confrontando l'output: **PASS** se coincide, **FAIL** altrimenti. Un *tentativo* è una generazione della soluzione: `pass@1` conta solo il primo tentativo riuscito, `pass@k` accetta che basti uno dei k tentativi per task («il primo che riesce conta»). Il riferimento di successo/fallimento è dunque l'output atteso del task, non il linguaggio. Le stesse parole nella programmazione logica (SPEC §6, capitolo 22) indicano invece l'esito di un goal: `:call` / `:redo` / `:exit` / `:fail` sono i tentativi del motore registrati da `solve … trace`. Vedasi anche le Fondamenta del volume, capitolo 16.

## Esercizi

1. **No Julia.** Perché il programma non ha più bisogno di Julia? Quale runtime esegue oggi Triad?
2. **Perché non gira.** Prendi un esempio con `struct` o `fact` e spiega perché non viene eseguito oggi.
3. **`.td` o `.triad`.** Qual è l'estensione canonica? Prova a eseguire un file con ciascuna.
4. **Codice Core puro.** Verifica con l'interprete JS che un tuo programma sia Core «puro»: su cosa ti fidi?
5. **`get` in `selfhost/`.** Perché `selfhost/*.td` gira con `seed.exe` e non con l'interprete JS? Verifica.
6. **Proporre una modifica.** Descrivi i passi per proporre una modifica al linguaggio secondo la FAQ.
7. **PASS, FAIL e pass@k.** Spiega a voce il significato delle tre espressioni con un esempio.
8. **Successo e fallimento in logica.** Spiega la seconda faccia di `success` / `fail` con le porte di Byrd.
9. **Le tue FAQ.** Scrivi tre domande che avresti voluto trovare nella FAQ con la risposta.
10. **Domande senza risposta.** Scegli una domanda aperta sul progetto e scrivi come la investigheresti.

---

Gli esempi Core marcati **Output:** si verificano eseguendoli con `node seed/dev/triad_core.cjs`.

# Il libro di Triad

Triad 2.0 è un linguaggio di programmazione autonomo, definito da una specifica normativa indipendente da qualsiasi implementazione. Non è un dialetto di Julia, Prolog o Lisp, pur avendo preso da ciascuno di essi idee fondamentali: blocchi `function … end` e tipi con `::` da Julia, unificazione con occurs-check, backtracking e `solve` con variabili logiche `?x` da Prolog, codice come dato e macro da Lisp. Ciò che è proprio di Triad, e non in prestito, è l'integrazione dei tre modelli — imperativo, logico, metaprogrammatorio — in una sola sintassi a blocchi, accompagnata da errori come dati (`Err`), contratti (`requires` / `ensures`), budget (`with budget`), tracce (`with trace`) ed effetti dichiarati (`!io`, `!state`) come parte del linguaggio e non di una libreria.

Il linguaggio si presenta in due profili di conformità. Triad-Core è il sottoinsieme di bootstrap: valori, funzioni, chiusure, controllo di flusso, liste, stringhe, I/O. Triad-Full è lo standard completo: tipi dichiarati, multiple dispatch, logica integrata, macro, contratti, budget, tracce, effetti. Ogni programma Core è anche un programma Full valido e deve comportarsi allo stesso modo. L'interprete di riferimento JavaScript esegue Triad-Core; il seed S0 Windows è un interprete in codice macchina x86-64 distinto e più limitato. Il seed Linux attuale è ancora un test di avvio E0, non un interprete. Il piano di bootstrap prevede S1 (un interprete Triad scritto in Triad-Core), S2 (compilatore nativo), S3 (self-hosting) e S4 (parità cross-platform).

Questo libro è il percorso di studio del linguaggio per come è definito dallo standard. Le Fondamenta insegnano la programmazione da zero usando Triad, per chi non ha mai scritto un programma. Il Manuale e il Tutorial alternano spiegazioni concettuali, semantica tecnica, esempi annotati, esercizi e riferimenti alla specifica. Ciascun testo presenta i concetti nel punto in cui servono e distingue i costrutti eseguibili oggi da quelli definiti dallo standard ma non ancora supportati. Il Tutorial accompagna prima il lettore nei costrutti Core eseguibili oggi, poi presenta Triad-Full come standard da studiare e infine mostra come leggere le implementazioni, il bootstrap e la suite di conformance nel repository. Le etichette dei blocchi distinguono il codice eseguibile dal codice definito dallo standard ma non ancora supportato dal runtime.

Il progetto è rilasciato sotto licenza MIT. La specifica normativa è in SPEC.md (standard 0.2.0), il sottoinsieme di bootstrap in SPEC-CORE.md, la suite di conformance in conformance/ con golden verificabili. Il repository include anche un benchmark per la generazione di codice Triad da LLM (ai/), grammatiche per VS Code e Vim, e un sito con la curva di apprendimento. L'obiettivo è un linguaggio autonomo e riconosciuto: standard proprio, conformance eseguibile, estensione .triad, tooling editor, bootstrap senza linguaggi ospiti e, infine, self-hosting.

La suite di conformance è il contratto eseguibile del linguaggio. L'interprete di riferimento JavaScript passa tutti i 37 casi Core; il seed Windows passa i casi supportati e lascia fuori per default il caso `liste`, che richiede l'indicizzazione standard. Il seed Linux è ancora allo stadio E0 e stampa solo un messaggio di avvio. Il profilo Full è definito dallo standard, ma il runtime corrente non implementa ancora il linguaggio completo; gli

output etichettati golden sono risultati attesi conservati nei file della suite. Ogni caso Core dichiara output e codice di uscita, così che un'implementazione possa essere confrontata con un contratto riproducibile, non con un esempio isolato.

Gli strumenti per gli editor includono grammatiche per VS Code e Vim/Neovim e una voce per GitHub Linguist, così che i file `.triad` (canonica) e `.td` (alias storico) siano riconosciuti ovunque. Il sito del progetto espone la curva di apprendimento, i grafici delle fasi e una demo animata della battitura di un programma Triad-Core eseguito dall'interprete. Il repository è organizzato per essere esplorato: esempi progressivi in `examples/`, task guidati in `ai/tasks/`, benchmark LLM in `ai/`. Ogni modifica è accompagnata da test e da aggiornamenti a STATUS.md, BOOTSTRAP.md e CHANGELOG.md.

Questo libro è scritto per lo standard 0.2.0. In caso di divergenza tra manuale e specifica, prevale la specifica: SPEC.md e SPEC-CORE.md sono la fonte di verità normativa, e questo volume ne è il percorso di studio, con spiegazioni, esempi ed esercizi.